AUTOMIND



Documentação

Autoload V4

REV. 01

27/05/2026

Português (BR)

Sumário

I. Visão Geral

1. O que é o Autoload V4?
2. Problemas que o Produto Resolve
3. Impacto nas Operações e Negócios
4. Público-Alvo
5. Casos de Uso
6. Diferenciais Competitivos
7. Conclusão

II. Arquitetura e Software

1. Visão Geral da Arquitetura
2. Diagrama de Arquitetura
3. Componentes Principais
4. Tecnologias Utilizadas
5. Estrutura do Projeto
6. Padrões Arquiteturais
7. Fluxo de Dados e Comunicação
8. Segurança
9. Escalabilidade e Performance
10. Conclusão

III. Aplicação

1. Visão Geral da Aplicação
2. Fluxo de Funcionamento Operacional
3. Arquitetura Tecnica
4. Principais Módulos da Aplicação
5. Regras de Negócio
6. Maquinas de Estado
7. Integração entre Módulos
8. Infraestrutura e Deployment

IV. Integrações

1. Visão Geral das Integrações
2. Autenticação e Segurança
3. Integrações com Serviços Externos
4. Versionamento da API
5. Documentação OpenAPI (Swagger/ReDoc)
6. Principais Endpoints
7. Resumo de Contadores

V. Drivers

1. Visão Geral de Drivers
2. Tipos de Drivers

3. Hierarquia de Classes e Interfaces
4. Comunicação entre Sistema e Drivers
5. Arquitetura de Threads do EdgeGateway
6. Configurações dos Drivers
7. Ciclo de Vida dos Drivers
8. Drivers Mock para Testes
9. Conclusão

VI. Banco de Dados

1. Visão Geral do Banco de Dados
2. Hierarquia de Entidades Base
3. Padrões Arquiteturais do Banco de Dados
4. AutoloadContext — Banco de Dados Global
5. TenantContext — Domínio Commons (13 tabelas)
6. TenantContext — Domínio Management (18 tabelas)
7. TenantContext — Domínio Operations: Infraestrutura (18 tabelas)
8. TenantContext — Domínio Operations: Dispositivos e Gateways (11 tabelas)
9. TenantContext — Domínio Operations: Programações (10 tabelas)
10. TenantContext — Domínio Operations: Operações (11 tabelas)
11. TenantContext — Domínio Operations: Transações (2 tabelas)
12. Fluxos de Dados Integrados
13. Conclusão

VII. Segurança

1. Visão Geral da Segurança
2. Autenticação
3. Autorização
4. Boas Práticas de Segurança
5. Fluxos de Segurança Integrados
6. Referência de Arquivos
7. Conclusão

VIII. Deploy e Ambientes

1. Visão Geral de Deploy
2. Configuração por Ambiente
3. Arquitetura de Rede e Comunicação
4. Git Flow e Branching Strategy
5. Pipelines de CI/CD
6. Configuração do Ambiente Local (Setup)
7. Variáveis de Ambiente
8. Configurações de Rede e Sistema
9. Processo de Deploy por Módulo
10. Troubleshooting
11. Conclusão

I. Visão Geral

Resumo do produto, público e problemas que resolve.

1. O que é o Autoload V4?

1.1 O que é o Autoload V4?

O **Autoload V4** é uma plataforma integrada de gestão e automação para terminais de granéis líquidos, desenvolvida para otimizar todas as etapas das operações de carregamento e descarregamento de produtos como combustíveis, óleos, produtos químicos e outros líquidos. Trata-se de uma solução completa que abrange desde o agendamento inteligente de veículos até o controle em tempo real das operações de carregamento através de automação industrial.

A plataforma é composta por **dois módulos de negócio principais** que trabalham de forma integrada:

- **Autoload Management**: Centraliza e consolida informações operacionais e de estoque, proporcionando uma visão integrada e estratégica para clientes e gestores
- **Autoload Operations**: Garante execução autônoma das operações dos terminais, com integração total aos sistemas de automação, permitindo completo controle na execução das operações multimodais

Estes módulos são suportados por **quatro componentes técnicos**:

- **Backend (.NET 8)**: Sistema de gestão centralizado que gerencia todo o ecossistema
- **Frontend Web (Next.js 14)**: Interface administrativa e portal de autoatendimento para clientes
- **Edge Gateway (.NET 8)**: Sistema de automação industrial que atua nas instalações físicas
- **Kiosk (React/Vite)**: Interface simplificada para operações no terminal

1.2 Principais Funcionalidades

Gestão e Agendamento

Agendamento Inteligente de Veículos

- Sistema de slots configuráveis (15, 30 ou 60 minutos)
- Regras automáticas de alocação baseadas em múltiplos critérios
- Calendário visual com disponibilidade em tempo real
- Validação automática de aptidões (veículo, motorista, produtos)
- Notificações por email e SMS
- Portal de autoatendimento para transportadoras

Gestão de Frota e Motoristas

- Cadastro completo de veículos e composições
- Registro de motoristas com validação de documentação
- Controle de aptidões e certificações
- Rastreamento de histórico de operações
- Gestão de transportadoras e organizações

Controle de Inventário

- Monitoramento em tempo real de estoques
- Gestão de produtos autorizados por veículo
- Controle de tanques e compartimentos

- Rastreamento de movimentações
- Integração com programações de carregamento

Gestão Operacional

- Programação de operações de carga e descarga
- Controle de baias e circuitos
- Gestão de áreas operacionais
- Checklists de segurança e conformidade
- Auditoria completa de operações

Automação e Controle Industrial

Edge Gateway - Controle em Tempo Real

- Integração com equipamentos de carregamento (preset loaders)
  - Accuload, Danload, Fusion4, Microload, TCC
- Comunicação com protocolos industriais
  - OPC UA para automação industrial
  - Modbus para equipamentos legados
- Controle de periféricos
  - Unidades de Controle de Acesso (UCA 330)
  - Balanças industriais (Toledo)
  - Displays industriais (Spider)
- Execução de processos scriptados em Lua
- Telemetria e monitoramento via MQTT
- Operação offline com sincronização automática

Monitoramento e Telemetria

- Dashboards em tempo real
- Widgets customizáveis de performance
- Métricas operacionais (ocupação, tempo médio, desvios)
- Alertas e notificações automáticas
- Histórico completo de operações

Plataforma e Integrações

Arquitetura Multi-Tenant

- Gestão de múltiplos terminais em uma única plataforma
- Isolamento completo de dados entre tenants
- Hierarquia de permissões configurável
- Dashboard consolidado para super administradores

Integrações e APIs

- REST API documentada (OpenAPI)
- SignalR para comunicação em tempo real
- MQTT para IoT e edge computing
- Webhooks para eventos do sistema
- Integração com sistemas externos (TMS, WMS, ERP)

Internacionalização e Acessibilidade

- Suporte multi-idioma (Português-BR, Inglês)

- Interface responsiva (desktop, tablet, mobile)
- Tema claro/escuro
- Conformidade com padrões de acessibilidade

Segurança e Compliance

- Autenticação JWT Bearer + Windows Authentication (Negotiate)
- Controle de acesso baseado em roles (RBAC)
- Auditoria completa de ações
- Rastreamento de erros com Sentry
- Backup e recuperação de dados
- Conformidade com regulamentações do setor

1.3 Posicionamento no Mercado

O Autoload V4 se posiciona como uma **solução enterprise completa** para o segmento de terminais de granéis líquidos, diferenciando-se por:

Integração Total: Diferentemente de soluções pontuais, o Autoload V4 integra gestão administrativa, agendamento, controle operacional e automação industrial em uma única plataforma.

Tecnologia Moderna: Construído com stack tecnológico de ponta (.NET 8, Next.js 14, PostgreSQL), garantindo escalabilidade, performance e evolução contínua.

Edge Computing: Capacidade única de operar no edge, mantendo operações críticas funcionando mesmo com falhas de conectividade, essencial para ambientes industriais.

Multi-Tenancy: Arquitetura que permite gestão de múltiplos terminais, ideal para grupos econômicos e distribuidoras com várias unidades.

Ecossistema Aberto: APIs abertas e integrações facilitadas permitem conexão com sistemas existentes, criando um ecossistema conectado desde fornecedores até clientes finais.

Foco em Experiência: Não apenas automatiza processos, mas transforma a experiência de todos os stakeholders (operadores, motoristas, transportadoras, gestores).

---

2. Problemas que o Produto Resolve

2.1 Problemas dos Gestores de Terminal

Ineficiência Operacional

Picos de Demanda e Filas

- **Problema**: Chegada descontrolada de veículos gera filas extensas, reclamações e tempo ocioso
- **Solução**: Agendamento inteligente distribui a demanda uniformemente ao longo do dia
- **Impacto**: Redução de até 80% no tempo médio de espera

Subutilização de Recursos

- **Problema**: Períodos de ociosidade resultam em desperdício de capacidade instalada
- **Solução**: Sistema de slots otimiza a ocupação das baias através de regras automáticas
- **Impacto**: Aumento de 30-50% na taxa de utilização de ativos

Falta de Previsibilidade

- **Problema**: Impossibilidade de planejar operações e alocar recursos adequadamente
- **Solução**: Visão antecipada de toda a programação com confirmações automáticas

- **Impacto**: Redução de 60% em horas extras não programadas

Problemas de Controle e Conformidade

Rastreabilidade Limitada

- **Problema**: Dificuldade em validar documentação e rastrear histórico de operações
- **Solução**: Auditoria completa com registro de todas as transações e documentos
- **Impacto**: Conformidade regulatória e redução de riscos legais

Falta de Segurança Operacional

- **Problema**: Dificuldade em garantir que apenas veículos e motoristas aptos operem
- **Solução**: Validação automática de aptidões, certificações e documentações
- **Impacto**: Redução de 90% em não conformidades de segurança

Ausência de KPIs

- **Problema**: Gestão baseada em intuição sem métricas concretas
- **Solução**: Dashboards em tempo real com indicadores de performance
- **Impacto**: Decisões baseadas em dados, melhoria contínua mensurável

Impacto Financeiro

Perdas por Ineficiência

- **Problema**: Custos elevados com ociosidade, horas extras e reclamações
- **Solução**: Otimização automática reduz desperdícios operacionais
- **Impacto**: Redução de 20-30% nos custos operacionais

Multas e Penalidades

- **Problema**: Não conformidades resultam em multas regulatórias
- **Solução**: Compliance automático e auditoria preventiva
- **Impacto**: Eliminação de multas evitáveis

2.2 Problemas dos Clientes (Transportadoras e Motoristas)

Experiência Negativa

Incerteza e Imprevisibilidade

- **Problema**: Ausência de garantias de horário gera custos extras e frustração
- **Solução**: Slots confirmados com horários confiáveis
- **Impacto**: Planejamento eficiente de rotas e turnos

Atrasos Frequentes

- **Problema**: Longas filas custam tempo e dinheiro para transportadoras
- **Solução**: Agendamento elimina espera não programada
- **Impacto**: Redução de 70% no tempo total de operação

Falta de Visibilidade

- **Problema**: Ausência de informações em tempo real sobre status de operações
- **Solução**: Portal de autoatendimento e notificações automáticas
- **Impacto**: Transparência total do processo

Rigidez Operacional

Processo Inflexível

- **Problema**: Dificuldade para reagendar em casos de necessidade
- **Solução**: Reagendamento self-service com regras configuráveis
- **Impacto**: Flexibilidade sem perda de controle

Comunicação Deficiente

- **Problema**: Canais de comunicação ineficientes com o terminal
- **Solução**: Notificações automáticas multi-canal (email, SMS, app)
- **Impacto**: Comunicação proativa e em tempo real

2.3 Problemas Operacionais Técnicos

Automação Industrial

Dependência de Operadores

- **Problema**: Operações manuais propensas a erros humanos
- **Solução**: Automação completa através do Edge Gateway
- **Impacto**: Redução de 95% em erros operacionais

Integração de Equipamentos

- **Problema**: Dificuldade em integrar diferentes marcas e protocolos
- **Solução**: Drivers universais para múltiplos protocolos (OPC, Modbus, proprietários)
- **Impacto**: Flexibilidade para trabalhar com equipamentos existentes

Conectividade Instável

- **Problema**: Falhas de rede podem interromper operações críticas
- **Solução**: Edge computing com operação offline e sincronização automática
- **Impacto**: 99.9% de disponibilidade operacional

Gestão de Dados

Informações Fragmentadas

- **Problema**: Dados espalhados em múltiplos sistemas não integrados
- **Solução**: Plataforma unificada com fonte única de verdade
- **Impacto**: Eliminação de inconsistências e duplicações

Falta de Inteligência

- **Problema**: Dados históricos não utilizados para otimização
- **Solução**: Analytics e preparação para machine learning preditivo
- **Impacto**: Insights acionáveis e otimização contínua

---

3. Impacto nas Operações e Negócios

3.1 Impactos Operacionais Mensuráveis

Métrica	Antes	Depois	Melhoria
Tempo médio de espera	2-4 horas	15-30 min	-80%
Taxa de utilização de baias	40-60%	70-90%	+50%
Operações simultâneas	Limitado	Otimizado	+40%
Erros operacionais	5-10%	<0.5%	-95%
Tempo de processamento	45-60 min	20-30 min	-50%
Não conformidades	15-20%	<2%	-90%

3.2 Impactos Financeiros

Redução de Custos

- Redução de 20-30% nos custos operacionais totais
- Eliminação de horas extras não programadas
- Redução de desperdícios e retrabalho
- Diminuição de multas e penalidades

Aumento de Receita

- Maior capacidade de atendimento (+40% de throughput)
- Redução de perda de clientes por má experiência
- Possibilidade de precificação dinâmica (roadmap futuro)
- Serviços premium baseados em agendamento

ROI Típico

- Retorno do investimento em 12-18 meses
- Payback acelerado em terminais de alto volume
- Benefícios crescentes com escala (multi-tenant)

3.3 Impactos Estratégicos

Competitividade

- Diferenciação no mercado através de tecnologia
- Capacidade de oferecer serviços de valor agregado
- Atração e retenção de clientes premium

Escalabilidade

- Plataforma preparada para crescimento
- Adição de novos terminais sem replicar infraestrutura
- Arquitetura cloud-native

Sustentabilidade

- Redução de emissões através de menor tempo ocioso
- Otimização de recursos energéticos
- Menor pegada ambiental das operações

Compliance e Segurança

- Conformidade automática com regulamentações
- Auditoria completa e rastreabilidade
- Redução de riscos operacionais e legais

Transformação Digital

- Base para inovações futuras (IA, ML, IoT)
- Dados estruturados para business intelligence
- Integração com ecossistema digital da cadeia logística

---

4. Público-Alvo

4.1 Perfis de Clientes Primários

Terminais de Granéis Líquidos

Terminais de Combustíveis

- Distribuidoras de combustíveis (postos de abastecimento)
- Terminais de importação e exportação
- Bases de distribuição regionais
- Volume típico: 50-500 operações/dia

Terminais Químicos

- Plantas petroquímicas
- Distribuidoras de produtos químicos
- Terminais especializados em químicos líquidos
- Volume típico: 20-200 operações/dia

Terminais de Óleos e Lubrificantes

- Plantas de envase de lubrificantes
- Distribuidoras de óleos industriais
- Terminais de óleos vegetais
- Volume típico: 30-150 operações/dia

Características Comuns

- Necessidade de controle rigoroso de qualidade
- Regulamentação intensa (ANP, ANVISA, etc.)
- Operações 24/7 ou em múltiplos turnos
- Alto custo de não conformidade
- Pressão por eficiência operacional

Grupos Econômicos e Holdings

Distribuidoras com Múltiplas Bases

- Grupos com 3+ terminais
- Necessidade de visão consolidada
- Padronização de processos entre unidades
- Gestão centralizada com operação descentralizada

Benefícios Específicos

- Arquitetura multi-tenant nativa
- Dashboard consolidado de todas as operações
- Economia de escala na implementação
- Transferência de melhores práticas

4.2 Perfis de Usuários do Sistema

Usuários Administrativos (Terminal)

Super Administradores (Plataforma)

- Gestão de múltiplos tenants
- Configurações globais
- Monitoramento de saúde do sistema
- Visão consolidada cross-tenant

Administradores de Terminal

- Configuração de slots e regras
- Gestão de usuários e permissões
- Configuração de parâmetros operacionais
- Análise de KPIs e relatórios

Operadores de Torre

- Aprovação de agendamentos
- Monitoramento de operações em tempo real
- Reagendamento de urgência
- Controle de filas e exceções

Operadores de Baia

- Execução de checklists
- Controle de carregamento/descarregamento
- Validação de documentação
- Interface com sistemas de automação

Usuários Externos

Gestores de Transportadora

- Visão de frota e programação
- Agendamento em lote
- Análise de performance (SLA, tempo médio)
- Gestão de motoristas e veículos

Despachantes de Transportadora

- Criação e gestão de agendamentos
- Acompanhamento em tempo real
- Reagendamentos
- Comunicação com terminal

Motoristas (via Kiosk ou futuro app mobile)

- Check-in na chegada ao terminal
- Visualização de programação
- Confirmação de operações
- Recebimento de instruções

4.3 Segmentos de Mercado

Por Porte

Grandes Terminais

- 200 operações/dia
- Múltiplas baias e produtos
- Automação completa
- Equipe dedicada de operação
- **Valor**: Eficiência máxima, ROI rápido

Médios Terminais

- 50-200 operações/dia
- 3-10 baias
- Mix de automação e processo manual
- Equipe enxuta
- **Valor**: Otimização de recursos, profissionalização

Pequenos Terminais/Bases

- <50 operações/dia
- 1-3 baias
- Operação simplificada
- Poucos operadores
- **Valor**: Organização, controle, agendamento básico

Por Região Geográfica

Brasil

- Mercado primário inicial
- Regulamentação ANP
- Idioma português
- Infraestrutura típica brasileira

América Latina

- Expansão natural
- Similaridade regulatória
- Suporte a espanhol (roadmap)
- Adaptação cultural

Mercados Internacionais

- Oportunidade de longo prazo
- Adaptação regulatória local
- Múltiplos idiomas
- Padrões internacionais

4.4 Perfil do Cliente Ideal (ICP - Ideal Customer Profile)

Características Demográficas

- Terminal de combustíveis ou químicos
- 100-300 operações/dia
- 5-15 baias de carregamento
- Localização: Grandes centros ou portos
- Faturamento: R$ 50M - R$ 500M/ano

Características Tecnográficas

- Infraestrutura básica de TI
- Conectividade estável (mas tolerante a falhas)
- Sistemas de automação industrial existentes
- Interesse em transformação digital

Características Psicográficas

- Gestão profissionalizada
- Foco em eficiência e conformidade
- Abertura para inovação
- Visão de médio/longo prazo
- Valoriza tecnologia como diferencial

Dores Mais Críticas

- Filas excessivas gerando reclamações
- Dificuldade em prever demanda
- Não conformidades frequentes
- Custos operacionais elevados
- Falta de controle e visibilidade

---

5. Casos de Uso

5.1 Agendamento de Veículos

Cenário: Transportadora Agenda Carregamento

Contexto

- Transportadora TransLog precisa carregar 3 caminhões com diesel
- Operação para o dia seguinte
- Terminal opera das 7h às 19h

Fluxo de Uso

1. **Acesso ao Portal**
    - Despachante acessa portal web do Autoload V4
    - Login com credenciais da transportadora
    - Visualiza dashboard com agendamentos ativos
2. **Criação do Agendamento**

- Clica em "Novo Agendamento"
- Seleciona **Terminal**: Terminal Santos
- Escolhe **Tipo de Operação**: Carga (expedição)
- Indica **Destino**: Posto Bandeirante - Zona Sul
- Confirma **Transportadora**: TransLog Ltda

3. **Seleção de Recursos**
   - Seleciona **Motorista**: João Silva (CNH validada)
   - Escolhe **Composição**: Cavalo ABC-1234 + Tanque XYZ-5678
   - Sistema valida automaticamente:
     - □ CNH do motorista válida
     - □ Veículo certificado para diesel
     - □ Tanque limpo e compartimentado
   - Define **Volume Total**: 40.000 litros
     - Compartimento 1: 15.000L diesel
     - Compartimento 2: 15.000L diesel
     - Compartimento 3: 10.000L diesel
4. **Escolha de Horário**
   - Sistema mostra calendário visual
   - Slots disponíveis em verde: 8:00, 8:30, 9:00, 14:00, 14:30
   - Slots ocupados em vermelho
   - Slots restritos (privativo) em amarelo
   - Seleciona slot **9:00 - 9:30**
5. **Confirmação**
   - Sistema valida regras:
     - □ Capacidade da baia suficiente
     - □ Produto em estoque
     - □ Não excede limite de agendamentos simultâneos
   - Status: **PENDENTE** (aguarda aprovação do terminal)
   - Notificação enviada por email para João Silva
6. **Aprovação pelo Terminal**
   - Operador de torre recebe notificação
   - Revisa agendamento
   - Aprova a solicitação
   - Status muda para **APROVADO**
   - Notificações enviadas à transportadora e motorista
7. **Confirmação Automática**
   - 24 horas antes, sistema confirma automaticamente
   - Status muda para **CONFIRMADO**
   - Lembrete enviado ao motorista via SMS

Resultado

- Agendamento confirmado para 9:00 do próximo dia
- Motorista chega no horário sem fila
- Operação completa em 25 minutos
- Cliente satisfeito, terminal eficiente

Benefícios Demonstrados

- □ Processo self-service
- □ Validação automática de aptidões
- □ Visibilidade total de disponibilidade
- □ Notificações automáticas multi-canal
- □ Eliminação de filas
- □ Previsibilidade para todos

5.2 Operação de Carregamento Automatizado

Cenário: Carregamento de Combustível com Preset Loader

Contexto

- Caminhão agendado chega ao Terminal Santos às 9:00
- Operação de carga de 40.000L de diesel
- Terminal equipado com preset Microload e automação completa

Fluxo de Uso

1. **Chegada ao Terminal**
   - Motorista João Silva chega à portaria às 8:55
   - Realiza check-in no kiosk usando CPF ou QR Code
   - Sistema busca agendamento confirmado para 9:00
   - Validações automáticas:
     - □ Agendamento válido e confirmado
     - □ Motorista correto
     - □ Veículo correto
     - □ Documentação em dia
   - Sistema imprime guia de entrada
   - UCA (Unidade de Controle de Acesso) libera cancela
2. **Direcionamento à Baia**
   - Display Spider mostra: "BEM-VINDO JOÃO - BAIA 3"
   - Sistema selecionou automaticamente Baia 3 (disponível e otimizada)
   - Motorista dirige até a Baia 3
3. **Preparação da Operação**
   - Operador de baia recebe notificação no tablet
   - Verifica veículo fisicamente
   - Executa checklist de segurança no sistema:
     - □ Aterramento conectado
     - □ Extintor verificado
     - □ Válvulas em bom estado
     - □ Compartimentos vazios
     - □ Equipamentos de proteção
   - Confirma início da operação no sistema
4. **Início do Carregamento**

- Sistema backend envia comando MQTT para Edge Gateway
- **Tópico**: `{tenant}/gateways/{gatewayId}/startprocess`
- **Payload**: Programa de carregamento com 3 compartimentos

5. **Processamento no Edge Gateway**

- Gateway recebe comando MQTT
- `ProcessService` carrega script Lua correspondente
- Script executa sequência:

```
-- Compartimento 1: 15.000L Diesel
control:SetMessage("Carregando Comp 1", 1)
transaction1 = control:AuthorizeLoading(1, recipe_diesel, 15000)
control:WaitLoading(1)
control:FinalizeLoading(1, transaction1.Id)

-- Compartimento 2: 15.000L Diesel
control:SetMessage("Carregando Comp 2", 2)
transaction2 = control:AuthorizeLoading(2, recipe_diesel, 15000)
control:WaitLoading(2)
control:FinalizeLoading(2, transaction2.Id)

-- Compartimento 3: 10.000L Diesel
control:SetMessage("Carregando Comp 3", 3)
transaction3 = control:AuthorizeLoading(3, recipe_diesel, 10000)
control:WaitLoading(3)
control:FinalizeLoading(3, transaction3.Id)
```

6. **Interação com Equipamentos**

- Edge Gateway se comunica com Microload via driver
- Microload controla válvulas e bombas
- Balança Toledo pesa continuamente via Modbus
- Display Spider mostra progresso em tempo real:
  - "COMP 1: 15.000L □"
  - "COMP 2: 15.000L □"
  - "COMP 3: 8.456L → 10.000L"

7. **Telemetria em Tempo Real**

- Edge Gateway publica telemetria via MQTT
- **Frequência**: A cada 2 segundos
- **Dados**:
  - Volume carregado por compartimento
  - Vazão atual
  - Pressão
  - Temperatura
  - Status do equipamento
- Frontend recebe via SignalR
- Widgets no dashboard atualizam em tempo real
- Operador de torre monitora todas as baias simultaneamente

8. **Finalização Automática**

- Último compartimento atinge 10.000L
- Microload fecha válvulas automaticamente

- Gateway finaliza transação
- Sistema calcula totais:
  - Volume total: 40.000L □
  - Tempo decorrido: 23 minutos
  - Vazão média: 1.739 L/min
- Atualiza inventário automaticamente
- Gera ticket de carregamento

9. **Saída do Terminal**
   - Operador valida operação e libera veículo
   - Motorista recebe ticket impresso e digital (email)
   - Check-out automático no sistema
   - UCA libera cancela de saída
   - Display: "BOA VIAGEM JOÃO!"
10. **Pós-Operação**
    - Sistema atualiza status do agendamento: **CONCLUÍDO**
    - Gera registros de auditoria
    - Atualiza KPIs:
      - Taxa de ocupação
      - Tempo médio de operação
      - Conformidade (100%)
    - Notificação à transportadora: operação concluída
    - Dados disponíveis via API para integração com TMS

Resultado

- Operação totalmente automatizada
- Tempo total: 25 minutos (chegada à saída)
- Zero erros humanos
- Auditoria completa
- Experiência excelente para motorista
- Eficiência máxima para terminal

Benefícios Demonstrados

- □ Automação completa end-to-end
- □ Controle em tempo real
- □ Edge computing (resiliência)
- □ Integração com múltiplos protocolos
- □ Telemetria detalhada
- □ Rastreabilidade total
- □ Segurança e conformidade

5.3 Gestão Multi-Tenant

Cenário: Distribuidora com 5 Terminais

Contexto

- Distribuidora PetroBras Regional opera 5 terminais:
  - Terminal Santos (SP) - 300 ops/dia
  - Terminal Rio (RJ) - 250 ops/dia
  - Terminal BH (MG) - 150 ops/dia
  - Terminal Curitiba (PR) - 180 ops/dia
  - Terminal Porto Alegre (RS) - 200 ops/dia
- Necessidade de visão consolidada e gestão centralizada

Fluxo de Uso

1. **Configuração Inicial Multi-Tenant**
   - Super administrador cria tenant "PetroBras Regional"
   - Cria 5 instâncias (sub-tenants) para cada terminal
   - Configura hierarquia:

```
PetroBras Regional (Tenant Principal)
├── Santos
├── Rio
├── BH
├── Curitiba
└── Porto Alegre
```

   - Define administradores locais para cada terminal
   - Configura permissões cross-tenant para gestores regionais
2. **Operação Descentralizada**
   - Cada terminal opera de forma independente:
     - **Santos**: Configurou 12 baias, slots de 30 min
     - **Rio**: Configurou 10 baias, slots de 15 min
     - **BH**: Configurou 6 baias, slots de 60 min
   - Dados completamente isolados entre terminais
   - Cada terminal tem suas próprias regras e configurações
3. **Dashboard Consolidado Regional**
   - Gestor Regional acessa dashboard consolidado
   - Visualiza KPIs de todos os 5 terminais:

```
┌──────────────── VISÃO REGIONAL ────────────────┐
│                                                │
│  Total de Operações Hoje: 1.080                │
│  Taxa Média de Ocupação: 78%                   │
│  Tempo Médio de Operação: 28 min               │
│  SLA Cumprido: 94%                             │
│                                                │
│  ┌──────────────────────────────────────────┐  │
│  │ Terminal    | Ops  | Ocup | Tempo | SLA |  │
│  ├──────────────────────────────────────────┤  │
│  │ Santos      | 287  | 82%  | 26min | 96% |  │
│  │ Rio         | 241  | 79%  | 27min | 95% |  │
│  │ BH          | 148  | 71%  | 32min | 89% |  │
│  │ Curitiba    | 176  | 76%  | 29min | 93% |  │
│  │ P. Alegre   | 198  | 80%  | 27min | 97% |  │
│  └──────────────────────────────────────────┘  │
```

```
│                                          │
│  Alertas:                                │
│  • BH: Taxa de no-show acima de 5%       │
│  • Curitiba: Manutenção programada amanhã │
└──────────────────────────────────────────┘
```

4. **Análise Comparativa**
   - Gestor identifica que Terminal BH tem menor SLA
   - Investiga causas raiz:
     - Slots de 60 min (muito largos)
     - Menor taxa de ocupação
     - Processo de check-in mais lento
   - Acessa detalhes específicos do Terminal BH
   - Compara com melhores práticas de Santos
5. **Padronização e Melhoria**
   - Decisão: Padronizar slots de 30 min em todos os terminais
   - Implementa melhores práticas de Santos em BH:
     - Checklists otimizados
     - Treinamento de operadores
     - Processo de check-in acelerado
   - Monitora evolução nos próximos 30 dias
6. **Relatórios Consolidados**
   - Gera relatório mensal consolidado
   - Dados agregados de todos os terminais:
     - Volume total movimentado: 18.500.000 litros/mês
     - 32.400 operações/mês
     - Taxa média de ocupação: 78%
     - Economia estimada: R$ 2,3M/mês vs operação anterior
   - Exporta para apresentação à diretoria
7. **Gestão de Transportadoras Cross-Terminal**
   - Transportadora TransLog opera nos 5 terminais
   - Portal único para agendar em qualquer terminal
   - Dashboard consolidado da transportadora:
     - Frota de 50 caminhões
     - 487 operações/mês distribuídas
     - Performance por terminal
     - Faturamento integrado

Resultado

- Gestão centralizada com operação descentralizada
- Visibilidade total cross-terminal
- Economia de escala na gestão
- Padronização de melhores práticas
- Tomada de decisão baseada em dados

Benefícios Demonstrados

- □ Arquitetura multi-tenant nativa

- □ Isolamento de dados
- □ Dashboard consolidado
- □ Análise comparativa
- □ Economia de escala
- □ Gestão unificada

5.4 Resposta a Emergências

Cenário: Quebra de Equipamento e Reagendamento em Massa

Contexto

- Terminal Santos opera com 12 baias
- Às 10:30 da manhã, Baia 5 apresenta falha no preset loader
- 8 agendamentos confirmados para Baia 5 nas próximas 6 horas
- Necessidade de reagendar rapidamente sem impactar operação

Fluxo de Uso

1. **Detecção da Falha**
   - Edge Gateway na Baia 5 detecta falha no Microload
   - Envia alerta automático via MQTT
   - Dashboard do operador mostra □ **BAIA 5 - FALHA EQUIPAMENTO**
   - Sistema muda status da Baia 5 para "MANUTENÇÃO"
2. **Avaliação do Impacto**
   - Operador de torre acessa módulo de agendamento
   - Filtra agendamentos da Baia 5:

```
8 agendamentos afetados:
• 11:00 - TransLog (40.000L diesel)
• 11:30 - TransBrasil (30.000L gasolina)
• 12:00 - LogFácil (35.000L diesel)
• 13:00 - TransLog (40.000L diesel)
• 14:00 - RodoLog (25.000L etanol)
• 14:30 - TransBrasil (30.000L diesel)
• 15:30 - LogFácil (40.000L gasolina)
• 16:00 - RodoLog (35.000L diesel)
```

   - Sistema mostra motoristas já a caminho (2 com ETA < 1h)
3. **Realocação Inteligente**
   - Sistema sugere automaticamente realocações:
     - Analisa capacidade das outras 11 baias
     - Identifica slots disponíveis
     - Prioriza horários próximos aos originais
     - Considera compatibilidade de produtos

Sugestão do Sistema:

```
┌──────────────────────────────────────────────────────┐
│ REALOCAÇÃO AUTOMÁTICA SUGERIDA                       │
├──────────────────────────────────────────────────────┤
│ 11:00 TransLog    → Baia 3 (11:00) ( V ) Diesel      │
│ 11:30 TransBrasil → Baia 7 (11:30) ( V ) Gasolina    │
│ 12:00 LogFácil    → Baia 2 (12:15) ( ! ) +15min      │
│ 13:00 TransLog    → Baia 3 (13:00) ( V ) Diesel      │
│ 14:00 RodoLog     → Baia 9 (14:00) ( V ) Etanol      │
│ 14:30 TransBrasil → Baia 7 (14:45) ( ! ) +15min      │
│ 15:30 LogFácil    → Baia 2 (15:45) ( ! ) +15min      │
│ 16:00 RodoLog     → Baia 9 (16:00) ( V ) Diesel      │
│                                                      │
│ Impacto: 3 atrasos de 15min | 5 sem alteração        │
│                                                      │
│ [APROVAR REALOCAÇÃO] [AJUSTAR MANUALMENTE]           │
└──────────────────────────────────────────────────────┘
```

4. **Aprovação e Notificação Automática**
   - Operador revisa e aprova realocação
   - Sistema executa em lote:
     - Atualiza todos os 8 agendamentos
     - Gera novos guias de entrada
   - **Notificações Automáticas:**
     - **Email** para todas as transportadoras
     - **SMS** para motoristas (especialmente os em trânsito)

     ```
     SMS: Sr. João, agendamento 11:00 realocado.
     Nova BAIA: 3 (sem alteração de horário).
     Desculpe o transtorno. Terminal Santos.
     ```

   - Portal das transportadoras atualiza automaticamente

5. **Gestão de Motoristas em Trânsito**
   - Motorista João (11:00) recebe SMS às 10:35
   - Já está a 20km do terminal
   - Confirma recebimento via SMS: "OK"
   - Chega ao terminal às 10:55
   - Kiosk direciona automaticamente para **BAIA 3**
   - Operação ocorre normalmente às 11:00

6. **Acompanhamento da Situação**
   - Manutenção estima reparo em 4 horas
   - Operador mantém Baia 5 em manutenção no sistema
   - Sistema não oferece mais slots da Baia 5 automaticamente
   - Às 14:45, manutenção conclui reparo
   - Baia 5 testada e reativada no sistema
   - Sistema volta a distribuir carga nas 12 baias

7. **Análise Pós-Incidente**
   - Sistema gera relatório do incidente:
     - Duração: 4h15min

        - Agendamentos afetados: 8
        - Reagendamentos bem-sucedidos: 8 (100%)
        - No-shows: 0
        - Reclamações: 0
        - Tempo médio de atraso: 10 minutos
    - Satisfação dos clientes: Positiva (comunicação proativa apreciada)

Resultado

- Emergência gerenciada com eficiência
- Impacto minimizado através de realocação inteligente
- Comunicação proativa evitou no-shows
- Zero perda de operações
- Clientes informados e satisfeitos
- Terminal manteve 92% de capacidade durante falha

Benefícios Demonstrados

- □ Detecção automática de falhas
- □ Realocação inteligente
- □ Notificações automáticas multi-canal
- □ Reagendamento em lote
- □ Visibilidade em tempo real
- □ Resiliência operacional
- □ Comunicação proativa

5.5 Análise e Otimização Contínua

Cenário: Otimização Baseada em Dados Históricos

Contexto

- Terminal Rio opera há 6 meses com Autoload V4
- Gestor deseja otimizar operação baseado em dados acumulados
- 180.000 litros/dia em média, 250 operações/dia
- 10 baias operacionais

Fluxo de Uso

1. **Acesso ao Módulo de Analytics**
    - Gestor acessa dashboard de KPIs
    - Seleciona período de análise: Últimos 6 meses
    - Sistema processa 45.000 operações históricas
2. **Análise de Padrões de Demanda**
    - Gráfico de calor mostra distribuição de demanda:

```
MAPA DE CALOR - DEMANDA POR HORÁRIO E DIA

     | Seg  Ter  Qua  Qui  Sex  Sáb  Dom
▫▫▫▫▫┼▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫▫
```

```
06:00| ▒▒  ▓  ▓  ▓  ▒▒  ░  -
07:00| ▒▒  ▒▒  ▒▒  ▒▒  ▒▒  ▓  -
08:00| ██  ██  ██  ██  ██  ▒▒  ░
09:00| ██  ██  ██  ██  ██  ▒▒  ▓
10:00| ██  ██  ██  ██  ██  ▒▒  ▓
11:00| ▒▒  ▒▒  ▒▒  ▒▒  ▒▒  ▒▒  ▓
12:00| ▓  ▓  ▓  ▓  ▓  ▓  ░
13:00| ▒▒  ▒▒  ▒▒  ▒▒  ▒▒  ▓  -
14:00| ██  ██  ██  ██  ██  ▒▒  -
15:00| ▒▒  ▒▒  ▒▒  ▒▒  ▒▒  ▓  -
16:00| ▒▒  ▒▒  ▒▒  ▒▒  ▒▒  ░  -
17:00| ▓  ▓  ▓  ▓  ▓  -  -

Legenda: - Baixo  ░ Médio  ▓ Alto  ▒▒ Muito Alto  ██ Crítico
```

Insights Identificados:

- Picos: 8h-10h e 14h (críticos)
- Vale: 12h-13h (almoço)
- Sábado: 50% da demanda de dia útil
- Domingo: Fechado (sem demanda)

3. **Análise de Gargalos**
   - Sistema identifica:

```
▫ GARGALOS IDENTIFICADOS:

1. Horário 08:00-10:00 (Seg-Sex)
   • Taxa de ocupação: 95%
   • Tempo médio de espera: 45min
   • Taxa de no-show: 2%

2. Horário 14:00-15:00 (Seg-Sex)
   • Taxa de ocupação: 92%
   • Tempo médio de espera: 35min
   • Taxa de no-show: 1%

3. Produto: Gasolina
   • 60% das operações
   • Baias especializadas (4 de 10)
   • Tempo médio: 32min (vs 28min diesel)
```

4. **Simulação de Cenários**
   - Gestor usa ferramenta de simulação
   - Testa diferentes configurações:

Cenário 1: Aumentar slots no pico

```
Mudança: 08:00-10:00 → de 6 para 8 slots simultâneos
Resultado simulado:
  • Tempo de espera: 45min → 18min (-60%)
  • Taxa de ocupação: 95% → 87%
  • Capacidade extra: +40 ops/dia
```

Cenário 2: Diminuir slots no vale

```
Mudança: 12:00-13:00 → de 6 para 3 slots simultâneos
Resultado simulado:
  • Economia de recursos: 3 operadores
  • Impacto na demanda: Mínimo
  • Realocação para pico: Possível
```

5. **Implementação de Mudanças**
   - Gestor aprova ajustes baseados em simulação:
     - **Pico (8-10h, 14-15h)**: 6 → 8 slots simultâneos
     - **Vale (12-13h)**: 6 → 3 slots simultâneos
     - **Noite (17-19h)**: Mantém 4 slots
   - Atualiza regras de agendamento no sistema
   - Mudanças entram em vigor no dia seguinte
6. **Monitoramento de Resultados**
   - Após 30 dias, gestor compara KPIs:

```
┌──────────────────────────────────────────────┐
│ COMPARAÇÃO: ANTES vs DEPOIS                  │
├──────────────────────┬───────┬──────────┬────┤
│ Métrica              │ Antes │ Depois   │ Δ  │
├──────────────────────┼───────┼──────────┼────┤
│ Tempo médio espera   │ 42min │ 15min    │ -64% │
│ Taxa ocupação (pico) │ 95%   │ 85%      │ -10% │
│ Taxa ocupação (geral)│ 72%   │ 76%      │ +4%  │
│ Ops/dia              │ 250   │ 275      │ +10% │
│ SLA cumprido         │ 89%   │ 96%      │ +7%  │
│ NPS transportadoras  │ 45    │ 68       │ +23  │
│ Custo/operação       │ R$185 │ R$168    │ -9%  │
└──────────────────────┴───────┴──────────┴────┘
```

Resultados:

   - □ Tempo de espera reduzido drasticamente
   - □ Capacidade aumentada em 10%
   - □ Satisfação do cliente disparou
   - □ Economia de R$ 17 por operação
   - □ ROI da otimização: R$ 140.000/mês
7. **Otimização Contínua**
   - Gestor agenda revisões trimestrais
   - Configurações ajustadas sazonalmente:
     - Verão: Maior demanda de gasolina (+15%)
     - Inverno: Maior demanda de diesel (+10%)
   - Sistema aprende padrões e sugere ajustes automaticamente

Resultado

- Operação otimizada baseada em dados reais
- Melhoria significativa em todos os KPIs
- Aumento de capacidade sem investimento em hardware

- Satisfação do cliente alcançou níveis excelentes
- Economia mensal de R$ 140.000

Benefícios Demonstrados

- □ Analytics avançado
- □ Identificação de padrões
- □ Simulação de cenários
- □ Otimização baseada em dados
- □ Melhoria contínua mensurável
- □ ROI demonstrável

---

6. Diferenciais Competitivos

6.1 Vantagens Tecnológicas

Arquitetura Moderna e Escalável

- Stack tecnológico de ponta (.NET 8, Next.js 14, PostgreSQL)
- Cloud-native com suporte a containers
- Escalabilidade horizontal
- Performance otimizada

Edge Computing Industrial

- Operação resiliente mesmo offline
- Latência ultra-baixa para controle crítico
- Sincronização automática inteligente
- Única solução no mercado com edge verdadeiro

Multi-Tenancy Nativo

- Arquitetura planejada desde o início para múltiplos tenants
- Isolamento robusto de dados
- Gestão centralizada com operação descentralizada
- Economia de escala

6.2 Vantagens de Produto

Solução Completa End-to-End

- Cobre toda a jornada: agendamento → operação → auditoria
- Integração nativa entre todos os módulos
- Fonte única de verdade para dados
- Elimina necessidade de múltiplos sistemas

Flexibilidade e Configurabilidade

- Adaptável a diferentes tipos de terminal
- Parametrizável sem código
- Suporte a múltiplos protocolos industriais
- Workflows customizáveis

Experiência do Usuário Superior

- Interfaces modernas e intuitivas
- Portais self-service reduzem carga operacional
- Notificações proativas multi-canal
- Mobile-ready (responsive hoje, apps nativos no roadmap)

6.3 Vantagens de Negócio

Modelo de Implementação Faseado

- Início rápido com MVP
- Expansão gradual de funcionalidades
- Riscos mitigados
- ROI progressivo

Integrações Abertas

- REST API completa e documentada
- Webhooks para eventos
- Suporte a múltiplos protocolos (MQTT, SignalR)
- Ecossistema expansível

Suporte e Evolução

- Roadmap claro de evolução
- Releases frequentes
- Suporte técnico especializado
- Comunidade em formação

---

7. Conclusão

O **Autoload V4** representa uma transformação digital completa para o setor de terminais de granéis líquidos, combinando gestão inteligente, automação industrial e excelência em experiência do usuário.

Com uma arquitetura moderna, funcionalidades abrangentes e foco em resultados mensuráveis, o produto resolve problemas críticos de eficiência operacional, conformidade regulatória e satisfação do cliente.

Os casos de uso demonstram aplicações reais que geram impacto imediato e progressivo, com ROI típico em 12-18 meses e benefícios crescentes à medida que o sistema é adotado e otimizado.

O roadmap ambicioso e a visão de plataforma posicionam o Autoload V4 não apenas como uma ferramenta, mas como o **futuro da gestão de terminais logísticos**.

Documento atualizado em: 05 de março de 2026
Versão: 1.0
Autor: Milton Kiefer de Albuquerque Mello
Validado por: ---

II. Arquitetura e Software

Visão das camadas, componentes e integrações.

1. Visão Geral da Arquitetura

O **Autoload V4** é um sistema completo de gestão e automação para terminais de granéis líquidos, desenvolvido com arquitetura moderna, escalável e multi-tenant. O sistema é composto por **dois módulos principais** que trabalham de forma integrada:

- **Autoload Management**: Centraliza e consolida informações operacionais e de estoque, proporcionando uma visão integrada e estratégica para clientes e gestores
- **Autoload Operations**: Garante execução autônoma das operações dos terminais, com integração total aos sistemas de automação, permitindo completo controle na execução das operações multimodais

1.1. Arquitetura Distribuída

O sistema segue uma **arquitetura distribuída em camadas**:

- **Camada de Apresentação**: Interface web (Frontend) para interação com usuários
- **Camada de API**: OpenAPI REST que expõe funcionalidades para frontend e sistemas externos
- **Camada de Aplicação**: Backend (.NET 8) que gerencia toda lógica de negócio
- **Camada de Edge Computing**: Autoload Edgegateway para comunicação com dispositivos industriais
- **Camada de Dados**: Repositório de dados com suporte multi-tenant

1.2. Princípios Arquiteturais

- **Domain-Driven Design (DDD)**: Organização do código baseada em domínios de negócio
- **Multi-Tenancy**: Isolamento de dados por cliente/organização
- **API-First**: Backend expõe APIs REST para todos os módulos
- **Edge Computing**: Processamento descentralizado próximo aos dispositivos industriais
- **Real-Time Communication**: SignalR para comunicação bidirecional em tempo real
- **Microservices-Ready**: Arquitetura preparada para evolução para microserviços
- **Test-Driven Development (TDD)**: Metodologia de desenvolvimento de software onde os testes automatizados são escritos antes do código funcional.

1.3. Domínios de Negócio

O sistema é organizado em **dois módulos principais**:

Autoload Management

- **Cadastros:** Gerencia dados compartilhados entre módulos.
- **Contratos:** Administra contratos com clientes e armazenadores.
- **Pedidos:** Processa pedidos de movimentação conforme termos contratuais.
- **Agendamentos:** Aloca horários para carga/descarga.
- **Gerenciamento de Estoques:** Monitora níveis de estoque e conformidade.
- **Obrigações Legais:** Registra dados para conformidade regulatória.

Autoload Operations

- **Cadastros de Terminais:** Gerencia os equipamentos e estruturas internas dos terminais responsáveis por operar as movimentações.
- **Checkin/Checkout:** Registra entrada/saída de veículos e autoriza acesso.
- **Filas:** Otimiza e gerencia filas de veículos nos terminais.
- **Vistorias:** Administra processos de inspeção de veículos.
- **Acesso Físico:** Controla acesso a terminais para veículos/pessoas autorizadas.
- **Pesagem:** Gerencia pesagem de veículos antes/depois de carregamento/descarregamento.
- **Carregamento/Descarregamento:** Supervisiona operações de carregamento/descarregamento.
- **Telemetrias:** Habilita a criação de dashboards para monitoramento em tempo real das operações nos terminais.
- **Programações:** Orquestra movimentações agendadas.

Além disso, também há o Autoload Core, que é o Módulo base com entidades que estão presentes tanto no *Management* e *Operations*:

Autoload Core

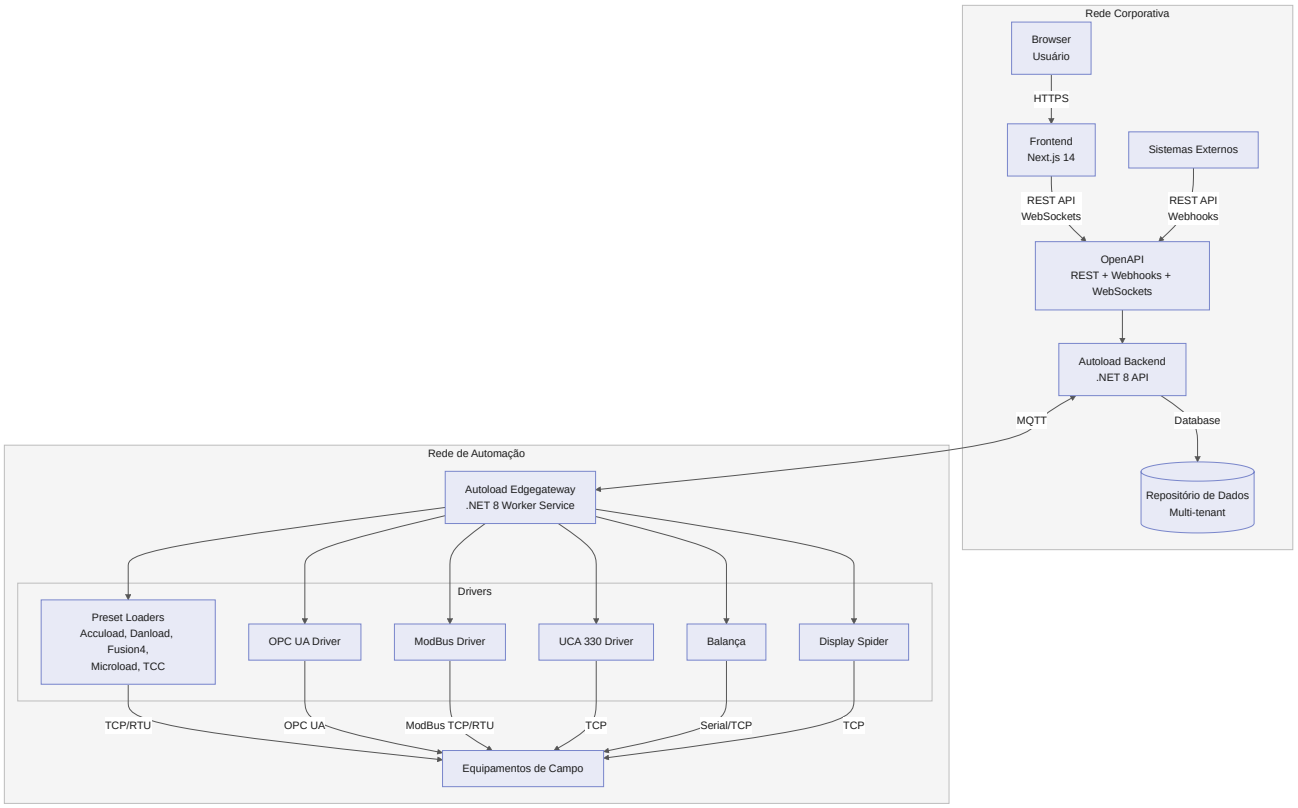
- **Multi-Cliente:** Suporta múltiplos assinantes com dados/configurações isolados.
- **Segurança:** Garante integridade de dados, autenticação e autorização.
- **Notificações:** Alerta usuários sobre eventos, operações e problemas.
- **Atributos:** Permite atributos personalizados para entidades.
- **Auditoria:** Registra todas as ações dos usuários para rastreabilidade.
- **Organizações:** Gerencia a estrutura de organizações utlizadas pelo assinantes.
- **Produtos:** Gerencia os produtos movimentados.

> **OBSERVAÇÃO**
>
> O cliente não é obrigado a utilizar todos os módulos disponíveis. Também há a possibilidade dele obter somente um deles (Management ou Operations)

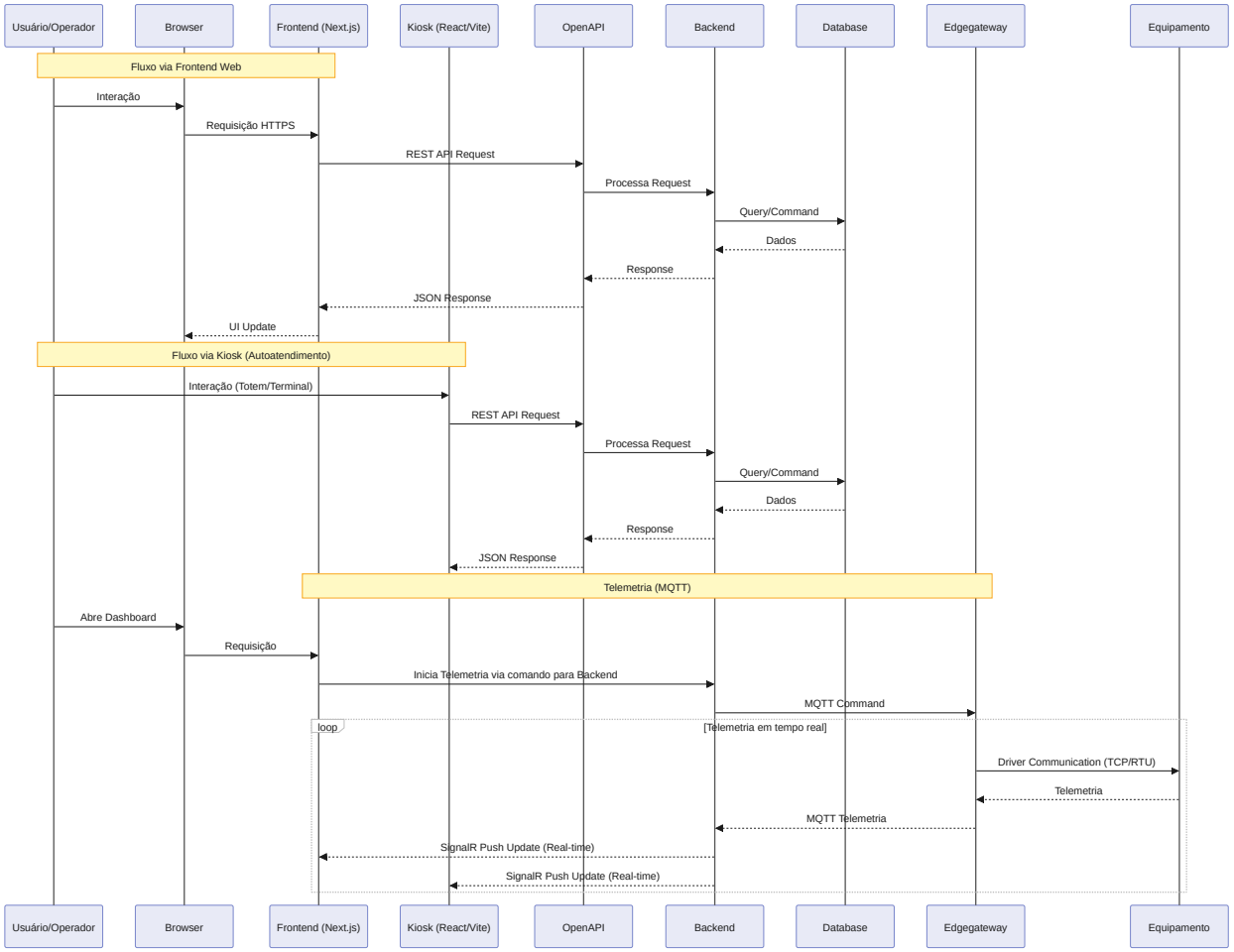
---

2. Diagrama de Arquitetura

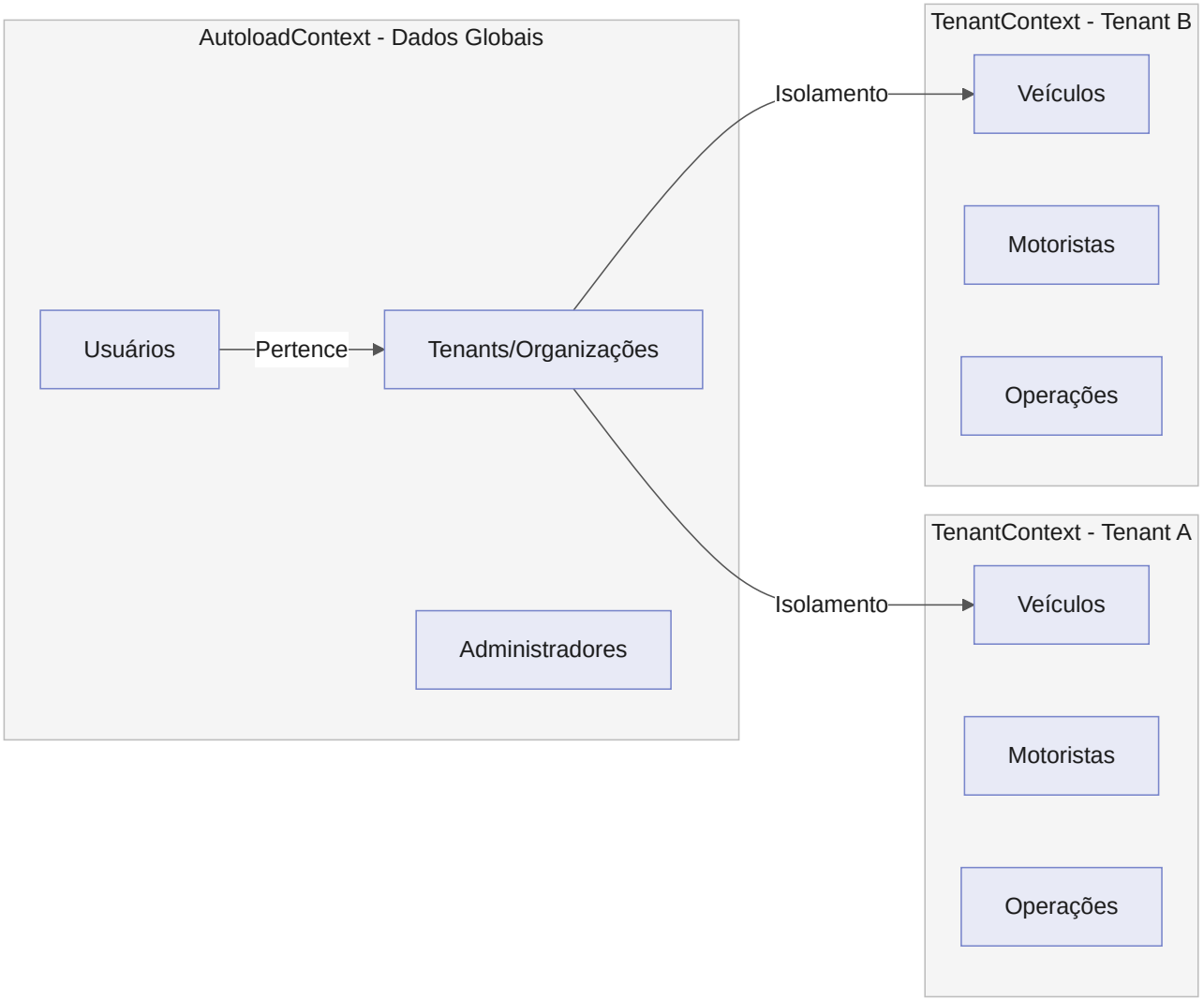
2.1. Visão Geral dos Componentes



2.2. Arquitetura de Comunicação



2.3. Arquitetura Multi-Tenant



3. Componentes Principais

3.1. Backend API (.NET 8)

Responsabilidades:

- Gerenciamento de autenticação e autorização
- Lógica de negócio dos módulos Management e Operations
- Persistência de dados via Entity Framework Core
- Exposição de OpenAPI REST versionadas
- Comunicação real-time via WebSockets (SignalR)
- Integração com Autoload Edgegateway via MQTT
- Webhooks para integração com sistemas externos

Características Técnicas:

- Framework: ASP.NET Core 8.0
- Autenticação: JWT Bearer + Windows Authentication (Negotiate)

- ORM: Entity Framework Core 8.0+
- API Documentation: OpenAPI/Swagger
- Versionamento de API: ASP.NET API Versioning
- Real-time: SignalR (WebSockets)
- Messaging: MQTT para comunicação com Edgegateway
- Monitoramento: Sentry.io integration
- Logs: Structured logging

Módulos:

Módulo	Responsabilidade
`AutoloadV4.Api`	Controllers, configuração de API, startup
`AutoloadV4.Domain.Commons`	Entidades base, serviços comuns, multi-tenancy
`AutoloadV4.Domain.Management`	Contratos, Pedidos, Agendamentos, Estoques, Programações
`AutoloadV4.Domain.Operations`	Ativos de Terminais, Cargas/Descargas, Filas, Checkin/Checkout, Vistorias
`AutoloadV4.Infrastructure`	DbContexts, Migrations, configuração de dados
`AutoloadV4.Lib.Pdf`	Geração de relatórios PDF

3.2. Frontend Web (Next.js 14)

Responsabilidades:

- Interface principal do sistema para gestão e operações
- Acesso via navegador padrão (Chrome, Firefox, Edge, Safari, etc)
- Dashboard de monitoramento em tempo real
- Gestão de contratos, pedidos e agendamentos (Management)
- Gestão de programações e operações (Operations)
- Controle de cargas/descargas, filas, checkin/checkout
- Gestão de vistorias e monitoramento de ativos
- Administração de sistema e multi-tenancy
- Integração com Autoload TMS (opcional)

Características Técnicas:

- Framework: Next.js 14 (App Router)
- Biblioteca UI: React 18
- Linguagem: TypeScript
- Estilização: Tailwind CSS + Radix UI
- Comunicação: REST API (Axios) + WebSockets (SignalR)
- Internacionalização: next-intl (PT-BR/EN)
- Estado: Zustand + React Hooks
- Editor de Texto: TipTap
- Monitoramento: Sentry.io

Principais Funcionalidades:

- **Autoload Management**: Contratos, Pedidos, Agendamentos, Estoques, Programações
- **Autoload Operations**: Dashboard Operacional, Configurações de Terminais, Dispositivos, Gateway, Programações Operacionais

- **Administração**: Usuários, Grupos, Organizações, Parâmetros, Auditoria

Estrutura de Rotas:

```
app/[locale]/
├── (auth)/                        # Rotas públicas de autenticação
│   ├── login/
│   ├── forgot-password/
│   └── change-password/
└── admin/                         # Área protegida
    ├── (common)/                  # Módulos administrativos (Core)
    │   ├── admins/
    │   ├── ai/
    │   ├── attributes/
    │   ├── audit/
    │   ├── groups/
    │   ├── instances/
    │   ├── jobs/
    │   ├── messages/
    │   ├── organizations/
    │   ├── parameters/
    │   ├── settings/
    │   └── users/
    │
    ├── (management)/              # Gestão
    │   ├── appointments/
    │   ├── drivers/
    │   ├── vehicle_compositions/
    │   └── vehicles/
    │
    └── (operations)/              # Operações
        ├── bays/
        ├── checklists/
        ├── circuits/
        ├── dashboard/
        ├── devices/
        ├── gateway/
        ├── operational-area/
        ├── pipelines/
        ├── presets/
        ├── products/
        ├── programs/
        ├── pumps/
        ├── storage-tanks/
        └── transactions/
```

3.3. Autoload Edgegateway (.NET 8)

Responsabilidades:

- Comunicação com equipamentos de campo via drivers especializados
- Suporte a múltiplos protocolos industriais (OPC UA, ModBus, Serial)
- Coleta de telemetria em tempo real
- Execução de processos de carregamento/descarregamento
- Execução de scripts Lua para automação

- Operação local com autonomia (cache offline)
- Sincronização com backend via MQTT
- Gerenciamento de dispositivos (Preset Loaders, Balanças, UCAs, Displays)

Características Técnicas:

- Framework: .NET 8 Worker Service
- Deployment: Windows Service / Linux Daemon
- Protocolo de Comunicação: MQTT (MQTTnet)
- Scripts: NLua (Lua)
- Protocolos Industriais: OPC UA, ModBus TCP/RTU, Serial (TCP/RTU)
- Sistemas Operacionais: Windows (Server, Server Core, Nano Server) e Linux (Ubuntu, Debian, CentOS, Alpine, etc.)

Drivers Disponíveis:

Driver	Protocolo	Dispositivos Suportados
Preset Loader	Serial/TCP	Accuload, Danload, Fusion4, Microload, TCC
OPC UA	OPC UA	Equipamentos industriais OPC UA
ModBus	ModBus TCP/RTU	PLCs, controladores ModBus
UCA 330	TCP	Unidade de Controle de Acesso Unidata 330
Balança	Serial/TCP	Balanças industriais (Toledo, etc)
Display	TCP	Display industrial Spider

Arquitetura de Serviços:

```
AutoloadV4.EdgeGateway.Services/
├── DeviceService           # Gerenciamento de drivers
├── GatewayService          # Estado do gateway
├── HeartbeatService        # Monitoramento de saúde
├── MQTTRPCService          # Chamadas RPC via MQTT
├── MQTTService             # Conexão MQTT com broker
├── ProcessService          # Execução de processos
├── ScriptService           # Execução de scripts Lua
├── StepService             # Definição de Passos, Timeouts e Logs
└── TelemetryService        # Coleta de telemetria
```

4. Tecnologias Utilizadas

4.1. Backend (.NET)

Categoria	Tecnologia	Versão/Detalhes	Uso
Runtime	.NET	8.0	Framework principal
Web Framework	ASP.NET Core	8.0	API REST e OpenAPI

Categoria	Tecnologia	Versão/Detalhes	Uso
ORM	Entity Framework Core	8.0+	Acesso a dados
Database	PostgreSQL	-	Banco de dados principal (suportado)
Database	SQL Server	2012+	Banco de dados suportado
Database	SQL Server LocalDB	-	Banco de dados suportado (desenvolvimento)
Database	In-Memory (EF Core)	-	Banco de dados para testes
Authentication	JWT Bearer	8.0	Autenticação de API
Authentication	Windows Auth (Negotiate)	8.0	Autenticação integrada
Real-time	SignalR	8.0	Comunicação WebSocket em tempo real
Messaging	MQTT (MQTTnet)	-	Comunicação com Edgegateway
Excel Generation	ClosedXML	0.105+	Geração de Excel
PDF Generation	Custom Lib	-	Geração de PDF
API Docs	Swashbuckle / OpenAPI	6.8+	Documentação Swagger
Monitoring	Sentry	6.0+	Monitoramento de erros
Testing	xUnit	-	Testes unitários e integração

Sistemas Operacionais Suportados:

- **Windows**: Nano Server, Server, Server Core (x64 e x86)
- **Linux**: Alpine, CentOS, Debian, Fedora, openSUSE, RedHat, SUSE Enterprise, Ubuntu (Arm32, Arm64, ppc64e, s390x e x64)

4.2. Frontend (Next.js)

Categoria	Tecnologia	Versão	Uso
Framework	Next.js	14.2+	Framework React com App Router
Runtime	Node.js	18.x/20.x	Runtime JavaScript
Biblioteca UI	React	18 (ReactJS)	Biblioteca de componentes
Linguagem	TypeScript	5.x	Linguagem principal
Estilização	Tailwind CSS	3.4.x (Tailwind)	Framework CSS utility-first
Componentes UI	Radix UI	1.x	Componentes acessíveis
Ícones	Lucide React	0.542+	Biblioteca de ícones
HTTP Client	Axios	1.x	Cliente HTTP para REST API
Real-time	SignalR	-	WebSocket/SSE para comunicação real-time

Categoria	Tecnologia	Versão	Uso
Formulários	React Hook Form	7.x	Gerenciamento de formulários
Validação	Zod	3.x	Validação de schemas
Datas	date-fns	4.x	Manipulação de datas
Editor	TipTap	2.10+	Editor de texto rico
i18n	next-intl	3.24+	Internacionalização (PT-BR/EN)
DnD	dnd-kit	6.x	Drag and Drop
Monitoring	Sentry	10.x+	Monitoramento de erros

Navegadores Suportados:

- Chrome, Firefox, Edge, Safari (versões atuais)

4.3. Autoload Edgegateway (.NET)

Categoria	Tecnologia	Versão/Detalhes	Uso
Runtime	.NET	8.0 (.Net 8.0)	Framework principal
Hosting	Worker Service	8.0 (.Net Worker)	Windows/Linux Service
MQTT	MQTTnet	- (MQTT)	Cliente MQTT para comunicação
Scripting	NLua	1.7.5+	Execução de scripts Lua
OPC	Custom Implementation	-	Comunicação OPC UA
ModBus	Custom Implementation	-	Comunicação ModBus TCP/RTU
Serial	System.IO.Ports	8.0 (TCP/RTU)	Comunicação serial

Sistemas Operacionais Suportados:

- **Windows**: Nano Server, Server, Server Core (x64 e x86)
- **Linux**: Alpine, CentOS, Debian, Fedora, openSUSE, RedHat, SUSE Enterprise, Ubuntu (Arm32, Arm64, ppc64e, s390x e x64)

Protocolos de Comunicação:

- **MQTT**: Comunicação com Backend
- **TCP/RTU**: Comunicação com equipamentos de campo
- **OPC UA**: Protocolo industrial
- **ModBus TCP/RTU**: Protocolo industrial
- **Serial**: Comunicação serial com dispositivos

4.4. Infraestrutura e DevOps

Categoria	Tecnologia	Uso
CI/CD	Azure Pipelines	Build e deployment automatizado
Controle de Versão	Git	Versionamento de código
Package Manager	NuGet	Pacotes .NET (privado Azure Artifacts)
Package Manager	npm	Pacotes JavaScript (Frontend)
Container	Docker	Containerização (opcional)
Monitoring	Sentry.io	Monitoramento de erros e performance
Message Broker	MQTT Broker	Comunicação Backend ↔ Edgegateway
API Gateway	Custom/Cloud	Gateway de API e resiliência

4.5. Padrões de Segurança

O Autoload V4 segue padrões internacionais de segurança e possui processos rigorosos de atualização:

Padrões Internacionais:

- **ISO/IEC 27001** - Sistema de Gestão de Segurança da Informação
- **ISO/IEC 27017** - Segurança em Cloud Computing
- **ISO/IEC 27018** - Proteção de Dados Pessoais na Nuvem

Padrões Específicos:

- **SOC 1** - Controles internos sobre relatórios financeiros
- **SOC 2** - Controles de segurança, disponibilidade e confidencialidade
- **SOC 3** - Relatórios públicos de controles

Processos de Atualização:

- Segregação de ambientes (Development, Staging, Production)
- Esteiras de desenvolvimento com workflows de aprovação
- Atualizações 100% automatizadas via CI/CD
- Patching contínuo e gestão de vulnerabilidades

---

5. Estrutura do Projeto

5.1. Estrutura Geral do Repositório

```
AUTOLOAD_V4/
├── AUTOLOAD_V4_Backend/              # Backend .NET 8
│   ├── AutoloadV4.sln                # Solution .NET
│   ├── azure-pipelines-*.yml         # Pipelines CI/CD
│   ├── nuget.config                  # Configuração NuGet
│   ├── CLAUDE.md                     # Documentação arquitetural
│   ├── GUIDELINES_*.md               # Guias de desenvolvimento
│   │
```

```
|   ├── AutoloadV4.Api/                # Projeto principal da API
|   |   ├── Controllers/               # Controllers REST
|   |   ├── DTOs/                      # Data Transfer Object
|   |   ├── Program.cs                 # Entry point
|   |   ├── appsettings.json           # Configurações
|   |   └── AutoloadV4.Api.csproj
|   |
|   ├── AutoloadV4.Domain.Commons/     # Domínio comum
|   |   ├── Models/                    # Entidades base
|   |   ├── Services/                  # Serviços comuns
|   |   ├── Enums/                     # Enumerações
|   |   ├── DTO/                       # Data Transfer Objects
|   |   └── Parameters/                # Parâmetros de configuração
|   |
|   ├── AutoloadV4.Domain.Management/  # Domínio de gestão
|   |   ├── Models/                    # Modelos (Vehicle, Driver, etc.)
|   |   ├── Services/                  # Serviços de negócio
|   |   ├── Enums/                     # Enumerações do domínio
|   |   └── DTO/                       # DTOs do domínio
|   |
|   ├── AutoloadV4.Domain.Operations/  # Domínio de operações
|   |   ├── Models/                    # Modelos (Program, Bay, etc.)
|   |   ├── Services/                  # Serviços operacionais
|   |   ├── Enums/                     # Enumerações
|   |   └── DTO/                       # DTOs
|   |
|   ├── AutoloadV4.Domain.Operations.Client/ # SDK cliente operações
|   |
|   ├── AutoloadV4.Infrastructure/     # Infraestrutura de dados
|   |   ├── Context/                   # DbContexts (Autoload, Tenant)
|   |   ├── Migrations/                # Migrations EF Core
|   |   |   ├── Autoload/              # Migrations AutoloadContext
|   |   |   └── Tenant/                # Migrations TenantContext
|   |   ├── DependencyInjection/       # Setup de DI
|   |   └── AutoloadV4.Infrastructure.csproj
|   |
|   ├── AutoloadV4.Lib.Pdf/            # Biblioteca de PDF
|   |
|   ├── AutoloadV4.Api.Test/           # Testes de integração da API
|   |
|   ├── AutoloadV4.Infrastructure.Test/ # Testes unitários de serviços
|   |
|   └── Documentation/                 # Documentação técnica
|
├── AUTOLOAD_V4_Frontend/              # Frontend Next.js 14
|   ├── azure-pipelines-*.yml          # Pipelines CI/CD
|   ├── package.json                   # Dependências npm
|   ├── next.config.mjs                # Configuração Next.js
|   ├── tailwind.config.ts             # Configuração Tailwind
|   ├── tsconfig.json                  # Configuração TypeScript
|   ├── .env.development               # Variáveis de ambiente (dev)
|   ├── .env.local                     # Variáveis de ambiente (local)
|   ├── CLAUDE.md                      # Documentação arquitetural
|   ├── guidelines/                    # Guias de desenvolvimento
|   |
|   ├── src/
|   |   ├── app/[locale]/              # App Router com i18n
```

```
│   │   │   ├── (auth)/                # Rotas de autenticação
│   │   │   │   ├── login/
│   │   │   │   ├── forgot-password/
│   │   │   │   └── change-password/
│   │   │   │
│   │   │   └── admin/                 # Área protegida
│   │   │       ├── (common)/          # Módulos comuns
│   │   │       │   ├── users/
│   │   │       │   ├── groups/
│   │   │       │   ├── instances/
│   │   │       │   ├── organizations/
│   │   │       │   ├── audit/
│   │   │       │   └── settings/
│   │   │       │
│   │   │       ├── (management)/      # Módulos de gestão
│   │   │       │   ├── vehicles/
│   │   │       │   ├── drivers/
│   │   │       │   ├── vehicle_compositions/
│   │   │       │   └── appointments/
│   │   │       │
│   │   │       └── (operations)/      # Módulos de operações
│   │   │           ├── dashboard/
│   │   │           ├── bays/
│   │   │           ├── circuits/
│   │   │           ├── programs/
│   │   │           ├── devices/
│   │   │           ├── gateway/
│   │   │           ├── pipelines/
│   │   │           └── checklists/
│   │   │
│   │   ├── components/                # Componentes React
│   │   │   ├── ui/                    # Componentes base (shadcn)
│   │   │   ├── forms/                 # Componentes de formulário
│   │   │   ├── layouts/               # Layouts
│   │   │   └── widgets/               # Widgets do dashboard
│   │   │
│   │   ├── services/                  # Serviços e API clients
│   │   │   ├── api/                   # Clientes de API
│   │   │   ├── signalr/               # Hubs SignalR
│   │   │   └── auth/                  # Autenticação
│   │   │
│   │   ├── hooks/                     # React Hooks customizados
│   │   │
│   │   ├── contexts/                  # React Contexts
│   │   │
│   │   ├── lib/                       # Utilitários
│   │   │
│   │   ├── types/                     # Definições TypeScript
│   │   │
│   │   └── i18n/                      # Internacionalização
│   │       ├── locales/
│   │       │   ├── pt-BR/
│   │       │   └── en/
│   │       └── config.ts
│   │
│   ├── public/                        # Arquivos estáticos
│   │
```

```
│   ├── features/                      # Features modulares
│   │
│   └── docs/                          # Documentação específica
│
├── AUTOLOAD_V4_EdgeGateway/           # Edge Gateway .NET 8
│   ├── AutoloadV4.EdgeGateway.sln     # Solution .NET
│   ├── azure-pipelines-*.yml          # Pipelines CI/CD
│   ├── CLAUDE.md                      # Documentação arquitetural
│   ├── GUIDELINES_*.md                # Guias de desenvolvimento
│   │
│   ├── AutoloadV4.EdgeGateway/        # Core library
│   │   ├── Interfaces/                # Interfaces de serviços
│   │   ├── Models/                    # Modelos de dados
│   │   ├── Services/                  # Serviços principais
│   │   └── Drivers/                   # Classe base de drivers
│   │
│   ├── AutoloadV4.EdgeGateway.Worker/  # Worker Service (entry point)
│   │   ├── Program.cs                 # Entry point
│   │   └── appsettings.json           # Configurações
│   │
│   ├── AutoloadV4.EdgeGateway.Drivers.Preset/    # Driver preset loaders
│   │   ├── Accuload/
│   │   ├── Danload/
│   │   ├── Fusion4/
│   │   └── Microload/
│   │
│   ├── AutoloadV4.EdgeGateway.Drivers.OPC/       # Driver OPC UA
│   │
│   ├── AutoloadV4.EdgeGateway.Drivers.ModBus/    # Driver ModBus
│   │
│   ├── AutoloadV4.EdgeGateway.Drivers.UCA/       # Driver UCA 330
│   │
│   ├── AutoloadV4.EdgeGateway.Drivers.TruckScale/# Driver balanças
│   │
│   ├── AutoloadV4.EdgeGateway.Drivers.Display/   # Driver displays
│   │
│   ├── AutoloadV4.EdgeGateway.Presets.Microload/ # Implementação Microload
│   │
│   ├── AutoloadV4.EdgeGateway.Test/              # Testes unitários
│   │
│   ├── AutoloadV4.EdgeGateway.Worker.Test/       # Testes worker
│   │
│   ├── Scripts/                       # Scripts Lua para processos
│   │
│   └── Tools/                         # Ferramentas auxiliares
│
└── AUTOLOAD_V4_KIOSK/                 # Kiosk React
    ├── azure-pipelines-*.yml          # Pipelines CI/CD
    ├── package.json                   # Dependências npm
    ├── vite.config.ts                 # Configuração Vite
    ├── tsconfig.json                  # Configuração TypeScript
    ├── index.html                     # HTML principal
    │
    ├── src/
    │   ├── main.tsx                   # Entry point React
    │   ├── App.tsx                    # Componente raiz
    │   │
```

```
    │   ├── pages/                    # Páginas
    │   │   ├── Login/
    │   │   ├── CheckIn/
    │   │   └── Appointments/
    │   │
    │   ├── components/               # Componentes React
    │   │   └── ui/                   # Componentes base
    │   │
    │   ├── services/                 # Serviços e API
    │   │
    │   ├── hooks/                    # React Hooks
    │   │
    │   ├── i18n/                     # Internacionalização
    │   │
    │   └── types/                    # Tipos TypeScript
    │
    └── public/                       # Arquivos estáticos
```

> **CONTEXTO:**
>
> **i18n** é a abreviação padrão para "internacionalização" (internationalization), referindo-se ao processo de projetar e desenvolver software, sites ou aplicações para que suportem múltiplos idiomas, culturas e regiões sem precisar alterar o código-base

5.2. Organização por Responsabilidades

Backend - Separação por Domínios

O backend segue uma arquitetura em camadas com separação clara de responsabilidades:

Camada de Apresentação (API)

- Controllers organizados por funcionalidade
- Versionamento de API
- Middleware de autenticação e autorização
- Configuração de CORS e segurança

Camada de Domínio

- **Commons**: Funcionalidades compartilhadas
  - Autenticação e autorização
  - Multi-tenancy
  - Auditoria
  - Parâmetros de sistema
- **Management**: Gestão de frota e agendamentos
  - Veículos e composições
  - Motoristas
  - Agendamentos (appointments)
  - Organizações
- **Operations**: Operações industriais
  - Programas de carregamento
  - Baias e circuitos

- Dispositivos e gateways
- Processos de carregamento
- Checklists operacionais

Camada de Infraestrutura

- Entity Framework Core contexts (AutoloadContext, TenantContext)
- Migrations separadas por contexto
- Configurações de entidades
- Repositórios (se necessário)

Frontend - Separação por Funcionalidade

O frontend utiliza o Next.js App Router com organização modular:

Route Groups

- `(auth)`: Rotas públicas de autenticação
- `(common)`: Administração comum do sistema
- `(management)`: Gestão de frota e agendamentos
- `(operations)`: Operações e monitoramento

Componentes

- `components/ui`: Componentes base reutilizáveis
- `components/forms`: Formulários e validações
- `components/layouts`: Layouts de página
- `components/widgets`: Widgets de dashboard

Serviços

- Clientes de API organizados por domínio
- Hubs SignalR para real-time
- Serviços de autenticação

Edge Gateway - Separação por Drivers

O Edge Gateway é modular, com cada driver em seu próprio projeto:

- Cada driver implementa a interface `IDriver`
- Drivers são carregados dinamicamente
- Comunicação via MQTT RPC
- Execução de scripts Lua para automação

---

6. Padrões Arquiteturais

6.1. Backend Patterns

Domain-Driven Design (DDD)

- **Entidades**: Objetos com identidade única (BaseEntity)
- **Value Objects**: Objetos sem identidade (ex: Address)
- **Aggregates**: Grupos de entidades (ex: Vehicle + VehicleComposition)

- **Services**: Lógica de negócio complexa
- **Repositories**: Abstração de acesso a dados (via EF Core DbSets)

Repository Pattern

Implementado através do Entity Framework Core:

```
public class UserService
{
    private readonly AutoloadContext _context;

    public async Task<User> GetByIdAsync(Guid id)
    {
        return await _context.Users.FindAsync(id);
    }
}
```

Dependency Injection

Todo o sistema utiliza DI nativo do .NET:

- Serviços registrados em `Program.cs`
- Scoped para serviços com DbContext
- Singleton para configurações
- Transient para serviços stateless

Multi-Tenancy Pattern

Dois DbContexts separados:

- **AutoloadContext**: Dados globais (usuários, tenants)
- **TenantContext**: Dados específicos do tenant (isolamento)

CQRS (Command Query Responsibility Segregation)

Separação entre operações de leitura e escrita:

- Queries: Retornam DTOs
- Commands: Modificam estado

6.2. Frontend Patterns

Component-Based Architecture

- Componentes funcionais React
- Hooks para estado e efeitos
- Composição sobre herança
- Props drilling evitado com Context API

Server Components + Client Components (Next.js 14)

- Server Components para dados estáticos

- Client Components para interatividade
- Hydration otimizada

Custom Hooks

Encapsulamento de lógica reutilizável:

```
export function useAuth() {
  const [user, setUser] = useState<User | null>(null);
  // lógica de autenticação
  return { user, login, logout };
}
```

Service Layer

Separação da lógica de API:

```
export class UserService {
  async getUsers(): Promise<User[]> {
    const response = await api.get('/api/users');
    return response.data;
  }
}
```

6.3. Edge Gateway Patterns

Driver Pattern

Todos os drivers implementam interface comum:

```
public abstract class Driver : IDriver
{
    public abstract Task ConnectAsync();
    public abstract Task<TelemetryData> ReadTelemetryAsync();
    public abstract Task SendCommandAsync(Command command);
}
```

Strategy Pattern

Processamento diferente por tipo de dispositivo:

```
public interface IDeviceStrategy
{
    Task ProcessDeviceData(byte[] rawData);
}
```

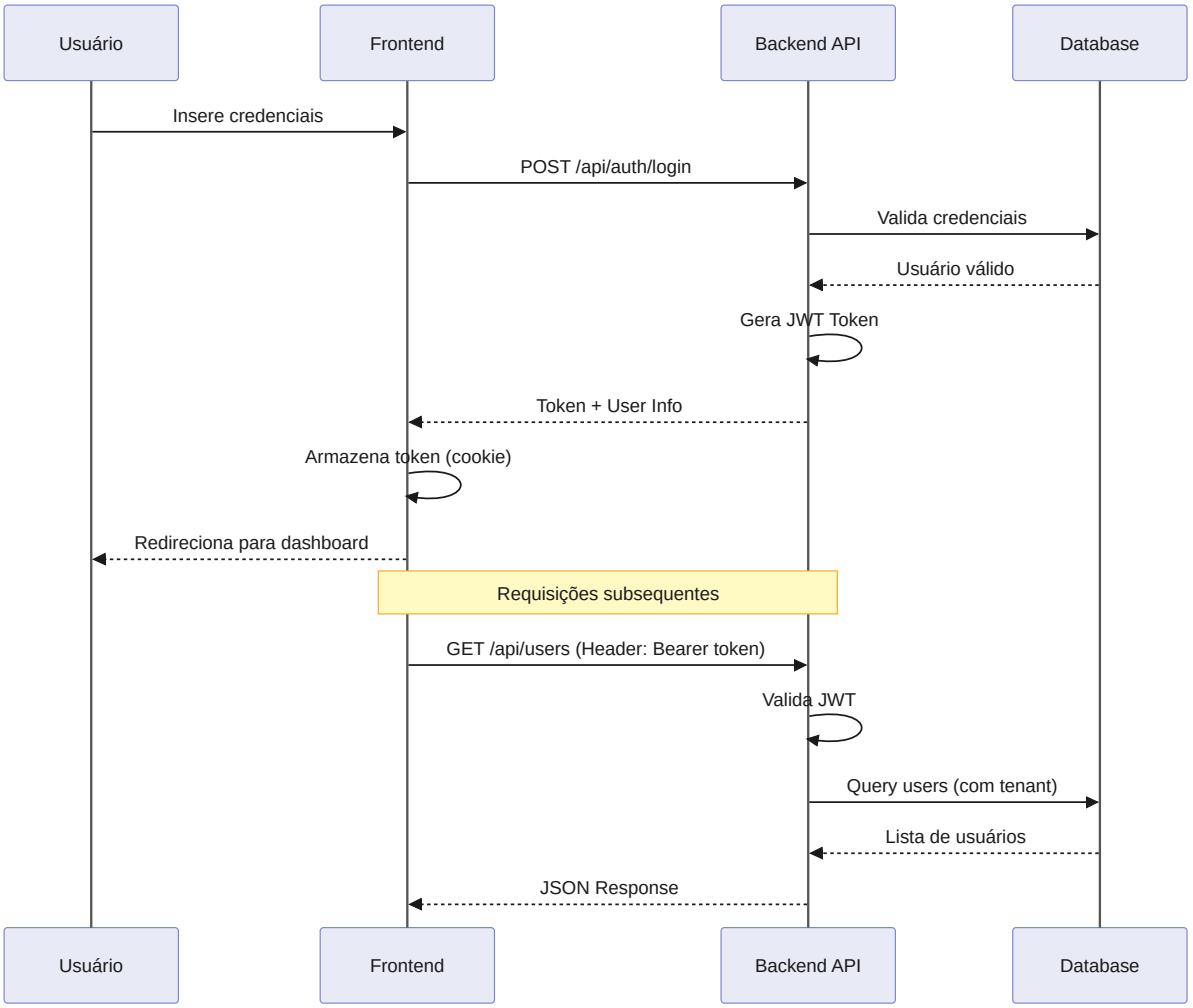
Observer Pattern

Telemetria publicada via eventos:

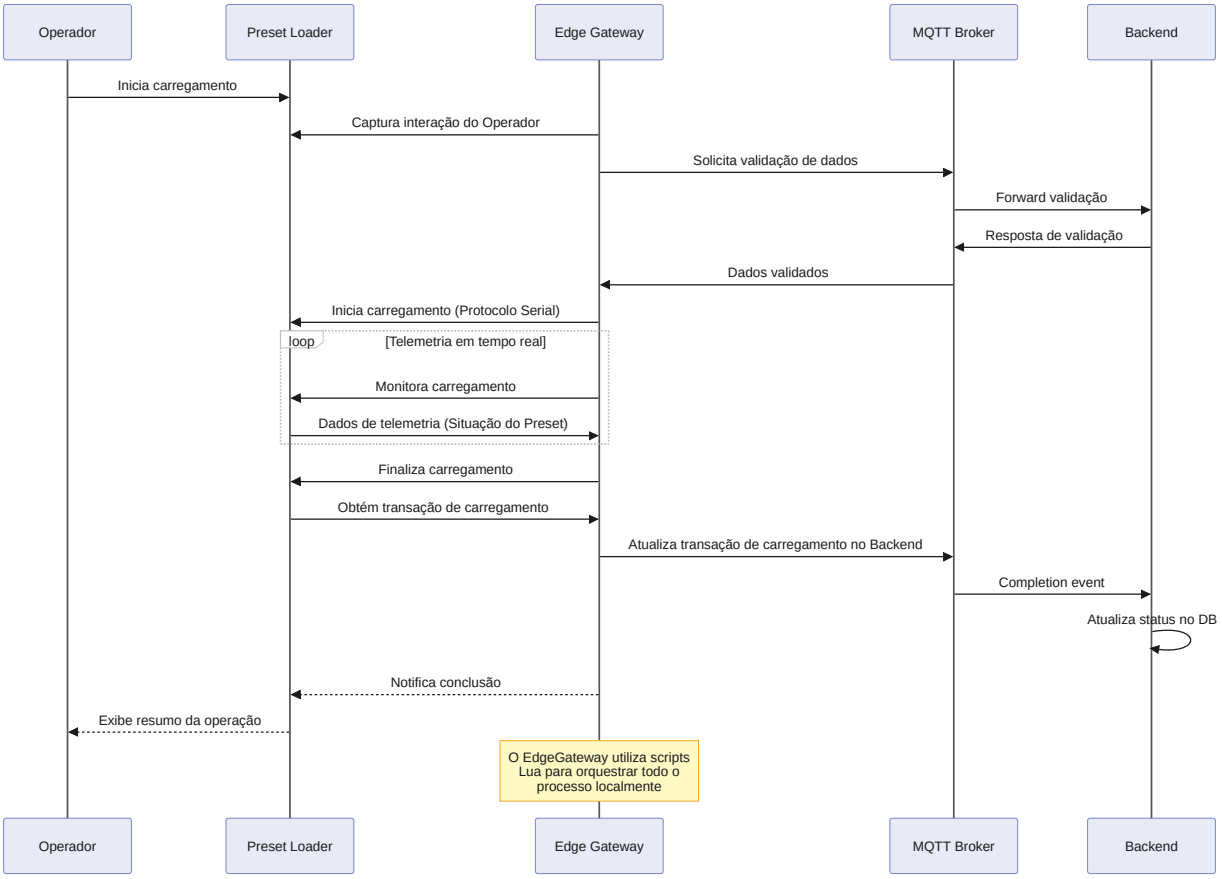
```
public event EventHandler<TelemetryEventArgs> TelemetryReceived;
```

7. Fluxo de Dados e Comunicação

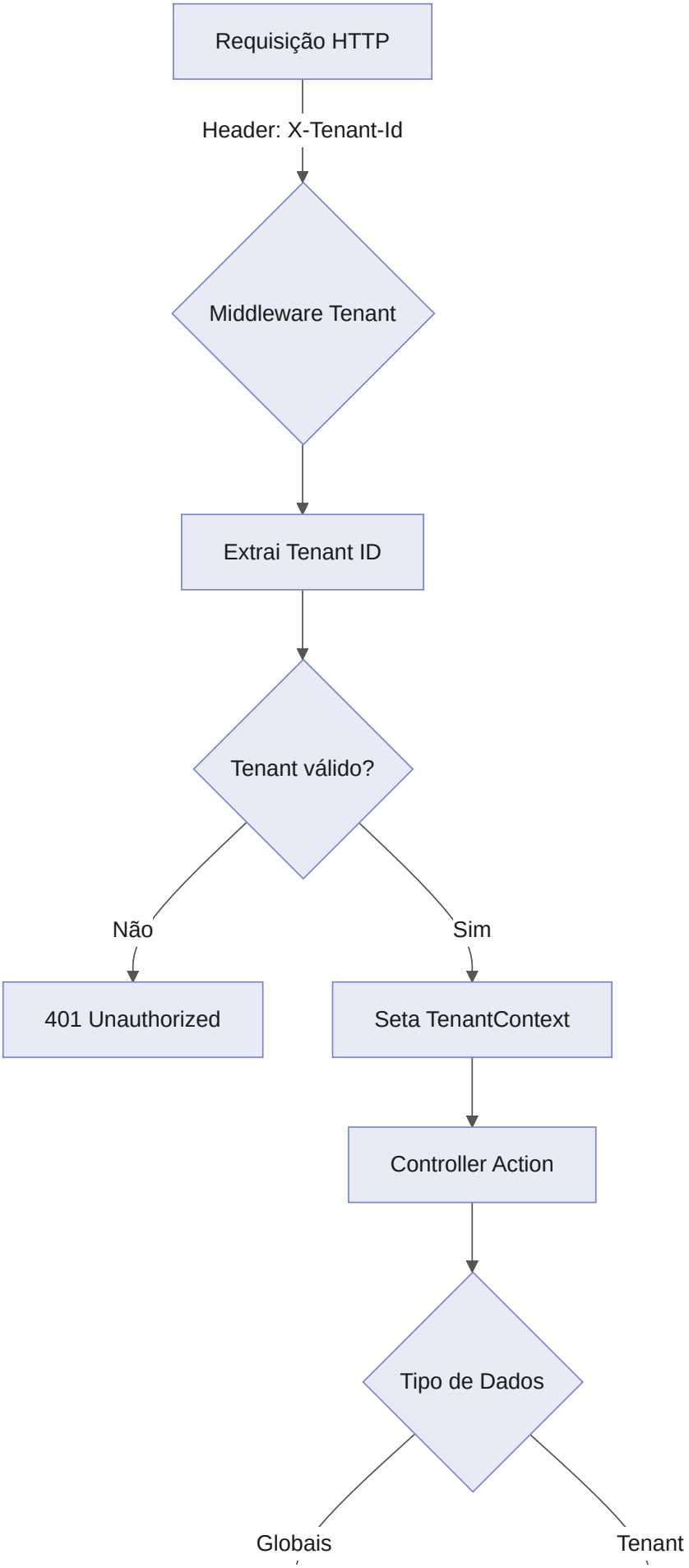
7.1. Fluxo de Autenticação

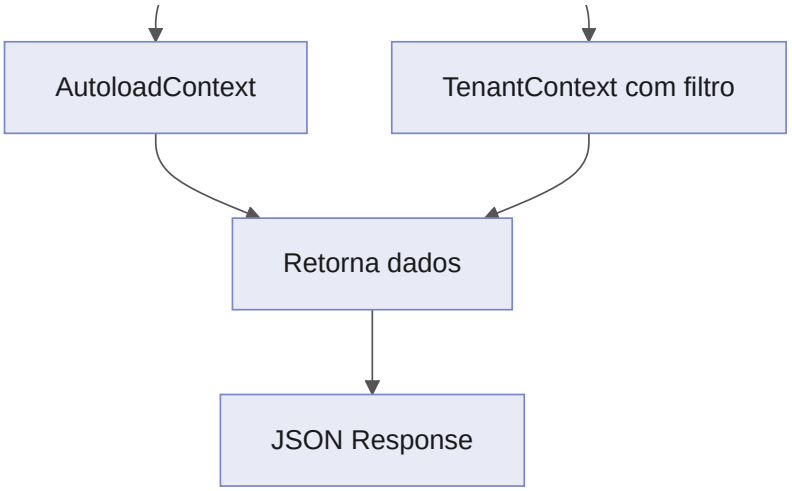


7.2. Fluxo de Operação de Carregamento



7.3. Fluxo Multi-Tenant


Requisição HTTP
Header: X-Tenant-Id
Middleware Tenant
Extrai Tenant ID
Tenant válido?
Não
Sim
401 Unauthorized
Seta TenantContext
Controller Action
Tipo de Dados
Globais
Tenant



7.4. Comunicação MQTT (Edge Gateway)

Tópicos MQTT:

```
autoload/gateway/{gatewayId}/telemetry       # Telemetria do gateway
autoload/gateway/{gatewayId}/command         # Comandos para gateway
autoload/gateway/{gatewayId}/rpc/request     # RPC Request
autoload/gateway/{gatewayId}/rpc/response    # RPC Response
autoload/device/{deviceId}/telemetry         # Telemetria de dispositivos
autoload/device/{deviceId}/command           # Comandos para dispositivos
```

Formato de mensagem:

```
{
  "messageId": "uuid",
  "timestamp": "2026-02-23T10:30:00Z",
  "type": "telemetry|command|rpc",
  "payload": {
    // dados específicos
  }
}
```

7.5. SignalR Real-Time Updates

Hubs Disponíveis:

- `TelemetryHub` (`/hubs/telemetry`): Telemetria de dispositivos e operações
- `MessageHub` (`/hubs/messages`): Notificações e mensagens em tempo real
- `HeartbeatHub` (`/hubs/heartbeats`): Monitoramento de conectividade de gateways e dispositivos

Fluxo de conexão:

1. Frontend conecta ao hub via WebSocket
2. Backend autentica conexão (JWT)
3. Cliente subscreve a grupos específicos (ex: `tenant-{tenantId}`)
4. Backend publica eventos para grupos
5. Clientes recebem atualizações em tempo real

8. Segurança

8.1. Autenticação e Autorização

- **JWT Tokens**: Para autenticação de API
- **Windows Authentication (Negotiate)**: Para integração com AD
- **Role-Based Access Control (RBAC)**: Permissões por função
- **Multi-Tenant Isolation**: Dados isolados por tenant
- **HTTPS Only**: Comunicação criptografada

8.2. Proteção de Dados

- **Entity Framework**: Proteção contra SQL Injection
- **Input Validation**: Validação de entrada com Zod/FluentValidation
- **CORS**: Configuração restritiva de CORS
- **Rate Limiting**: Proteção contra abuso de API
- **Audit Logging**: Registro de todas as operações sensíveis

8.3. Segurança de Comunicação

- **MQTT com TLS**: Comunicação criptografada com Edge Gateway
- **SignalR Authentication**: Autenticação de conexões WebSocket
- **API Versioning**: Compatibilidade e segurança de endpoints

---

9. Escalabilidade e Performance

9.1. Backend

- **Async/Await**: Todo código assíncrono
- **Database Indexing**: Índices em colunas frequentemente consultadas
- **Query Optimization**: LINQ otimizado, projeções específicas
- **Caching**: Cache em memória para dados frequentes
- **Connection Pooling**: Reutilização de conexões de DB

9.2. Frontend

- **Code Splitting**: Carregamento sob demanda
- **Server-Side Rendering (SSR)**: Renderização no servidor
- **Static Generation**: Páginas estáticas quando possível
- **Image Optimization**: Otimização automática de imagens
- **Lazy Loading**: Carregamento preguiçoso de componentes

9.3. Edge Gateway

- **Multithreading**: Drivers executam em threads separadas

- **Local Caching**: Cache local para operação offline
- **Batch Processing**: Processamento em lote de telemetria
- **Async I/O**: Operações de I/O não bloqueantes

---

10. Conclusão

O **Autoload V4** é uma solução moderna e escalável para gestão de terminais de granéis líquidos, construída com tecnologias de ponta e seguindo as melhores práticas de arquitetura de software. A separação em módulos (Backend, Frontend, Edge Gateway, Kiosk) permite desenvolvimento, teste e deploy independentes, facilitando a manutenção e evolução do sistema.

A arquitetura multi-tenant, comunicação em tempo real via SignalR e MQTT, e capacidade de edge computing garantem que o sistema possa operar de forma eficiente tanto online quanto offline, atendendo às necessidades críticas de operações industriais.

Documento atualizado em: 05 de março de 2026
Versão: 1.0
Autor: Milton Kiefer de Albuquerque Mello
Validado por: Gustavo Schmal
Última atualização tecnológica: .NET 8.0 / Next.js 14 / React 18

III. Aplicação

Fluxo operacional e principais módulos da aplicação.

1. Visão Geral da Aplicação

O **AUTOLOAD V4** é um sistema completo de gestão e automação para **terminais de graneis líquidos**. Ele abrange todo o ciclo operacional de carga e descarga, desde o agendamento de veículos ate a conclusão das operações de carregamento/descarregamento, passando por gestão de frota, controle de inventário de tanques, monitoramento em tempo real de equipamentos e rastreabilidade completa via trilha de auditoria.

Propósito Principal

Automatizar e gerenciar operações de movimentação de graneis líquidos em terminais, cobrindo:

- **Agendamento** de veículos e operações (scheduling)
- **Gestão de Frota** (motoristas, veículos, composições)
- **Controle Operacional** (programas de carga/descarga, operações, transações)
- **Monitoramento em Tempo Real** (telemetria de bais, tanques, bombas, presets)
- **Gestão de Ativos** (tanques de armazenamento, circuitos, dutos, bombas, dispositivos IoT)
- **Controle de Acesso e Auditoria** (permissões, trilha de auditoria, logs)

Stack Tecnológico

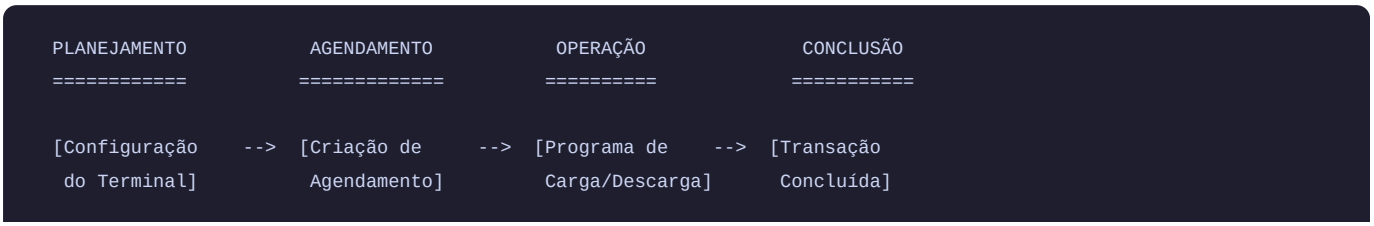
Camada	Tecnologia
Backend	.NET 8 (C#), PostgreSQL, SignalR, MQTT
Frontend	Next.js 14 (React 18), TypeScript, Tailwind CSS, Radix UI, Zustand
Comunicação Real-Time	SignalR (WebSocket) para telemetria, MQTT para gateways IoT
Autenticação	JWT Bearer Tokens + Windows AD (Negotiate)
Arquitetura	Multi-tenant, Domain-Driven Design (DDD)
Internacionalização	pt-BR, en (next-intl)

---

2. Fluxo de Funcionamento Operacional

O fluxo operacional típico do AUTOLOAD V4 segue uma sequência bem definida, desde o planejamento ate a conclusão da Operação:

2.1 Fluxo Completo de uma Operação Típica



```
- Bais              - Grupo de          - Criação do          - Registro de
- Circuitos           Agendamento         Programa              volumes
- Tanques           - Slots               - Operações          - Manifesto
- Produtos            Disponíveis       - Autorização          - Drainagem
- Presets           - Reserva do          no bay               - Pesagem
- Gateways            agendamento       - Carregamento/       - Trilha de
- Dispositivos      - Aprovação             Descarregamento       auditoria
                    - Confirmação         - Monitoramento
                    - Check-in              em tempo real
```

2.2 Detalhamento por Etapa

Etapa 1: Configuração do Terminal

Antes de qualquer Operação, o terminal precisa estar configurado:

1. **Áreas Operacionais** são cadastradas (zonas físicas do terminal)
2. **Bais de Carregamento** são configurados com capacidade, tipo de Operação (CARGA/DESCARGA/MISTA), produtos permitidos e vazão media
3. **Circuitos e Dutos** são mapeados conectando tanques de armazenamento aos bais
4. **Tanques de Armazenamento** são registrados com capacidade, produto armazenado e instrumentação
5. **Produtos** são catalogados com propriedades fisico-quimicas
6. **Presets** (equipamentos de medição) são configurados com componentes e receitas
7. **Gateways e Dispositivos IoT** são registrados para comunicação com equipamentos de campo
8. **Bombas** são cadastradas e vínculadas aos circuitos

Etapa 2: Planejamento e Agendamento

O ciclo de agendamento segue está sequência:

1. **Criação de Slots**: O planejador gera vagas (slots) para cada baia do terminal, definindo:
    - Período de datas (inclusivo)
    - Dias da semana permitidos
    - Janela de horário
    - Duração por slot
    - Restrições (tipo de produto, composição, cliente)
2. **Criação de Grupo de Agendamento**: Representa um pedido/contrato:
    - Terminal de destino
    - Tipo (ENTRADA/SAÍDA)
    - Volume planejado
    - Produtos e quantidades
    - Restrições de capacidade (volume diário máximo, agendamentos simultâneos)
3. **Reserva de Agendamento (RESERVED)**: O sistema encontra os primeiros slots Disponíveis e reserva:
    - Busca automática "o mais cedo possível" (As Soon As Possible)
    - Verifica compatibilidade de baia, produto, tipo de Operação
    - Verifica manutenções programadas
    - Respeita limites de operações simultâneas
4. **Aprovação (APPROVED)**: O planejador aprova a reserva:

- Define a composição veicular (cavalo + reboques)
- Atribui o motorista
- Valida compartimentos e volumes

5. **Confirmação (CONFIRMED)**: O veículo confirma presença:
   - Confirma chegada ao terminal
   - Prazo de Confirmação controlado pelo sistema
6. **Check-in (IN_PROGRESS)**: O Veículo entra no terminal:
   - Registro de entrada
   - Direcionamento para o bai designado

Etapa 3: Operação de Carga/Descarga

1. **Criação do Programa**: Um programa de Operação e criado vinculado ao agendamento:
   - Estado: DRAFT -> APPROVED -> CONFIRMED -> IN_PROGRESS
   - Modalidade: RODOVIARIO, FERROVIARIO, AQUAVIARIO ou DUTOVIARIO
   - Define operações, tanques envolvidos e volumes
2. **Autorização no Bai**: O operador autoriza o inicio do carregamento/descarregamento:
   - Seleciona o preset (equipamento de medição)
   - Configura circuito e duto
   - Habilita a bomba correspondente
3. **Carregamento/Descarregamento**: A Operação e executada:
   - **Transação** e criada no preset (equipamento de medição)
   - Status da Transação: IDLE -> AUTHORIZED -> LOADING -> LOADED
   - Telemetria em tempo real via MQTT (volume, vazão, temperatura, densidade)
   - Monitoramento via dashboard com widgets SignalR
4. **Monitoramento em Tempo Real**:
   - Dashboard com widgets configurados pelo Usuário
   - Dados de telemetria via SignalR (WebSocket)
   - Status de gateways, presets, tanques, bombas e bais
   - Heartbeat dos dispositivos IoT

Etapa 4: Conclusão

1. **Finalização da Transação**: O carregamento e concluido:
   - Volumes finais registrados
   - Medição Automática via instrumentação
2. **Manifesto**: Documento de movimentação gerado:
   - Estado: DRAFT -> APPROVED -> CONFIRMED -> IN_PROGRESS -> CLOSING -> COMPLETED
   - Registro de volumes, produtos, origem e destino
3. **Drainagem**: Procedimento de esvaziamento do circuito (quando aplicável)
4. **Pesagem**: Registro de peso bruto e tara (quando aplicavel)
5. **Checklist**: Verificações de segurança e conformidade
6. **Conclusão do Programa**: Estado final COMPLETED

- Volumes consolidados
- Trilha de auditoria completa
- Check-out do Veículo

7. **Conclusão do Agendamento (COMPLETED)**: Veículo liberado
   - Volume realizado atualizado no grupo de agendamento
   - Se todo volume planejado foi atingido, o grupo e concluído

---

3. Arquitetura Tecnica

3.1 Arquitetura Backend

O backend segue o padrão **Domain-Driven Design (DDD)** com separação clara em camadas:

```
AutoloadV4.Api/                  # Camada de Apresentação (Controllers, DTOs, Middleware)
AutoloadV4.Domain.Commons/       # Domínio Compartilhado (Users, Tenants, Parameters)
AutoloadV4.Domain.Management/    # Domínio de Gestão (Appointments, Vehicles, Drivers)
AutoloadV4.Domain.Operations/    # Domínio de Operações (Programs, Operations, Devices)
AutoloadV4.Infrastructure/       # Infraestrutura (EF Core, Repositories, Migrations)
AutoloadV4.Lib.Pdf/             # Utilitarios de Geração PDF
```

3.2 Multi-Tenancy

O sistema suporta múltiplos terminais (tenants) com isolamento de dados:

- **AutoloadContext**: Entidades globais (Users, Tenants, AuditTrails, ScheduledJobs)
- **TenantContext**: Dados específicos do terminal (Operations, Vehicles, Appointments)
- **TenantProvider**: Provedor que identifica o tenant ativo em runtime

3.3 Comunicação em Tempo Real

```
                  SignalR (WebSocket)
Frontend  <============================  Backend
(Dashboard)   Telemetria, Heartbeats,      (Hubs)
              Mensagens, Status

                    MQTT
Backend   <============================>  Edge Gateway
(Workers)   Comandos, Telemetria,           (Dispositivos
            RPC                              IoT)
```

SignalR Hubs:

- `MessageHub` (/hubs/messages) - Notificações em tempo real
- `TelemetryHub` (/hubs/telemetry) - Dados de telemetria de dispositivos
- `HeartbeatHub` (/hubs/heartbeats) - Monitoramento de gateways e dispositivos

3.4 Arquitetura Frontend

```
Next.js 14 (App Router)
├── [locale]/                  # Roteamento com internacionalização
│   ├── (auth)/                # Páginas publicas (login, recuperação de senha)
│   └── admin/                 # Area protegida
│       ├── (common)/          # Módulos de administração
│       ├── (management)/      # Módulos de Gestão de frota
│       └── (operations)/      # Módulos operacionais
├── services/                  # Camada de serviços API (35+ serviços)
├── hooks/                     # Hooks customizados (24 hooks)
├── components/                # Componentes reutilizáveis (50+)
├── types/                     # Definições TypeScript (35+ arquivos)
└── providers/                 # Provedores de contexto React
```

---

4. Principais Módulos da Aplicação

4.1 Módulo de Autenticação e Controle de Acesso

Funcionalidade: Gerencia autenticação, Autorização e sessões de usuário.

Componentes:

- Login com suporte a multi-tenant (Usuário seleciona o terminal)
- Autenticação de dois fatores (2FA) com timer de 60 segundos
- Suporte a Windows Active Directory (Negotiate)
- Chaves de acesso API para integração programática
- Recuperação e alteração de senha

Fluxo de Login:

1. Usuário informa o E-mail
2. Sistema retorna lista de tenants Disponíveis
3. Usuário seleciona o tenant e informa a senha
4. Se 2FA habilitado: insere o token
5. JWT e gerado e armazenado
6. Permissões carregadas no estado global (Zustand)

Permissões (55 tipos):

- Core, Terminal Setup, Gateway, Queues
- Access Control, Weighing, Loading/Unloading
- Checklist, Pump, Vehicle, Driver
- Custom Attributes, AI, Dashboard, Appointments

4.2 Módulo de Gestão de Frota (Management)

Funcionalidade: Gerencia motoristas, veículos, composições e agendamentos.

4.2.1 Motoristas (Drivers)

- Cadastro completo com dados pessoais e documentação

- Fluxo de Aprovação (PENDING -> APPROVED / REJECTED)
- Vinculação a agendamentos
- Atributos customizáveis

4.2.2 Veículos (Vehicles)

- Registro de frota com dados tecnicos
- Compartimentos com capacidades individuais
- Fluxo de Aprovação (PENDING -> APPROVED / REJECTED)
- Tipos: Cavalo-mecanico, Reboque, Semi-reboque

4.2.3 Composições Veiculares (Vehicle Compositions)

- Templates de composição (cavalo + reboques)
- Definição de compartimentos por Veículo
- Capacidade total calculada a partir dos compartimentos
- Vinculação a agendamentos

4.2.4 Agendamentos (Appointments)

- **Slots**: Vagas de tempo por baia do terminal
- **Grupos de Agendamento**: Representam pedidos/contratos com volume planejado
- **Agendamentos Individuais**: Operações agendadas com Veículo, motorista e produtos
- **Máquina de estados completa**: RESERVED -> APPROVED -> CONFIRMED -> IN_PROGRESS -> COMPLETED
- **Regras de expiração automática**: Timeout para Aprovação e Confirmação

---

4.3 Módulo de Operações (Operations)

Funcionalidade: Núcleo operacional que controla programas de carga/descarga, operações, transações e monitoramento.

4.3.1 Programas (Programs)

- Orquestrador principal de operações
- Maquina de estados: DRAFT -> APPROVED -> CONFIRMED -> IN_PROGRESS -> CLOSING -> COMPLETED
- Suporta 4 modalidades: RODOVIARIO, FERROVIARIO, AQUAVIARIO, DUTOVIARIO
- Contem múltiplas operações
- Steps (etapas) configurados sequencialmente
- Checklists de segurança Obrigatórios
- Time logs para rastreamento de tempo
- Anexos e documentos vinculados

4.3.2 Operações (Operations)

- Representam a ação fisica de carga/descarga
- Vínculadas a tanques de armazenamento (OperationTank)
- Medições de tanque (OperationTankMeasure) antes e depois da Operação
- Transações de volume medido por preset
- Manifestos de movimentação

4.3.3 Transações (Transactions)

- Registro de volume movimentado por preset
- Status: IDLE -> AUTHORIZED -> LOADING -> LOADED / ABORTED / PAUSED
- Componentes individuais (medidores, sensores)
- Dados de telemetria (volume, vazão, temperatura, densidade)

4.3.4 Manifestos (Manifests)

- Documentos de movimentação
- Maquina de estados própria
- Vinculados a operações e programas
- Assinatura digital

4.4 Módulo de Ativos do Terminal

Funcionalidade: Gerencia toda a infraestrutura fisica e logica do terminal.

4.4.1 Bais de Carregamento (Bays)

- Pontos físicos de carga/descarga
- Tipo de Operação: CARGA (OUTBOUND), DESCARGA (INBOUND), MISTA
- Lado do braco (Swing Arm): SIDE_A, SIDE_B
- Lado do slot: DIREITA, ESQUERDA
- Produtos permitidos (restrições)
- Vazão media por hora
- Periodos de manutenção

4.4.2 Tanques de Armazenamento (Storage Tanks)

- Registro de capacidade e produto armazenado
- Telemetria em tempo real (nível, temperatura, volume)
- Tabela de calibração para conversão volume/nível
- Histórico de medições

4.4.3 Circuitos (Circuits)

- Caminhos lógicos conectando tanques aos bais
- Configuração de dutos envolvidos
- Vinculação de bombas

4.4.4 Dutos (Pipelines)

- Interconexões físicas entre equipamentos
- Capacidade e tipo de produto

4.4.5 Bombas (Pumps)

- Equipamentos de transferência

- Status operacional em tempo real
- Vinculação a circuitos

4.4.6 Presets (Equipamentos de Medição)

- Configuração de hardware (medidores, sensores)
- Componentes individuais (meters)
- Receitas de produto (densidades, temperaturas de referência)
- Firmware e versão
- Modo de Operação
- Telemetria em tempo real

4.4.7 Produtos

- Catalogo de graneis líquidos
- Propriedades fisico-quimicas
- Grupos de produtos

4.5 Módulo de Gateways e Dispositivos IoT

Funcionalidade: Gerencia a comunicação com equipamentos de campo.

4.5.1 Gateways

- Concentradores de comunicação com dispositivos de campo
- Comunicação via MQTT
- Gerenciamento de processos (start/stop)
- Scripts customizados (Lua)
- Monitoramento de status (online/offline)
- Heartbeat para verificação de conectividade

4.5.2 Dispositivos (Devices)

- Equipamentos IoT individuais (sensores, atuadores, medidores)
- Drivers de comunicação (protocolos)
- Telemetria individual
- Heartbeat de conectividade

4.6 Módulo de Dashboard e Telemetria

Funcionalidade: Painel operacional em tempo real com widgets configuráveis.

Widgets Disponíveis:

- **Gateway Widget**: Status de conectividade dos gateways
- **Storage Tank Widget**: Nível de tanque em gauge visual
- **Bay Widget**: Status das baias de carregamento
- **Pump Widget**: Status das bombas
- **Preset Widget**: Status dos presets de medição

- **UCA Widget**: Status de controle de acesso
- **Kiosk Widget**: Display de quiosque

Funcionamento:

1. Usuário configura seus widgets preferidos (drag-and-drop)
2. Preferencias salvas no backend por Usuário
3. Dados em tempo real via SignalR
4. Atualização Automática sem refresh da página

4.7 Módulo de Administração

Funcionalidade: Configurações gerais do sistema, usuários e auditoria.

4.7.1 Usuários e Grupos

- CRUD de Usuários com atribuição de permissões
- Grupos de Usuários para permissões em lote
- Chaves de acesso API por Usuário

4.7.2 Organizações

- Cadastro de empresas/clientes
- Vinculação a agendamentos e operações

4.7.3 Parametros

- Configurações do sistema por tenant
- 11+ categorias de parâmetros
- Controle de login Simultâneo, timeouts, formatos

4.7.4 Atributos Customizados

- Campos dinâmicos adicionados a entidades existentes
- Sem necessidade de alteração de banco de dados
- Configuração por tenant

4.7.5 Trilha de Auditoria

- Registro completo de ações dos Usuários
- Historico de alterações em entidades
- Filtros por período, Usuário, entidade e ação

4.7.6 Jobs Agendados

- Monitoramento de tarefas em background
- Execução programada (workers)
- Status e histórico de execução

4.7.7 Mensagens

- Sistema de mensagens internas
- Notificações em tempo real via SignalR
- Contador de mensagens não lidas

4.7.8 Instâncias (Tenants)

- Gerenciamento de terminais/instâncias
- Configuração por tenant
- Isolamento de dados

4.8 Módulo de IA (AI Assistant)

Funcionalidade: Integração com assistente de inteligencia artificial para suporte operacional.

- Interface de chat com agente de IA
- URL do agente configurável por ambiente
- Consultas operacionais e suporte ao Usuário

---

5. Regras de Negócio

5.1 Regras de Agendamento

5.1.1 Geração de Slots

Regra	Descrição
Unicidade	Não podem existir slots duplicados para a mesma baia no mesmo horário
Dias da Semana	Slots so são gerados para os dias da semana permitidos
Manutenção	Baias em manutenção não recebem slots
Período Inclusivo	Data inicial e final são inclusivas na geração
Abrangência	Slots são gerados para TODAS as baias do terminal simultaneamente

Exemplo: 2 baias, período 02/02-04/02 (Seg-Qua), horário 08:00-10:00, duração 60min = 12 slots

5.1.2 Seleção de Slots Compatíveis

Ao buscar slots Disponíveis para um agendamento, o sistema aplica filtros em cascata:

1. **Compatibilidade de Tipo de Operação**: A baia deve suportar o tipo (CARGA/DESCARGA/MISTA)
2. **Compatibilidade de Produto**: Se a baia tem restrições de produto, o produto deve estar na lista
3. **Periodo de Manutenção**: A baia não pode estar desativada durante o slot
4. **Agendamentos Existentes**: Não pode haver sobreposição (estados CANCELED, EXPIRED, NO_SHOW são ignorados)
5. **Limite de Operações Simultâneas**: Respeita o máximo de agendamentos concorrentes do grupo

5.1.3 Estrategia de Alocação Automática ("As Soon As Possible")

```
1. Iniciar pelo primeiro dia da janela de agendamento
2. Verificar se o dia e permitido (dias úteis, feriados)
3. Iniciar pelo primeiro horário disponível do dia
4. Encontrar uma baia compativel com os requisitos
5. Criar o agendamento e avançar pela duração do agendamento
6. Repetir até que todo volume esteja agendado ou não haja mais slots
```

5.1.4 Bloqueio de Slots

- Slots podem ser bloqueados manualmente com motivo
- Slots ocupados não podem ser bloqueados ou deletados
- Slots bloqueados não aparecem como disponíveis

5.2 Regras de Grupos de Agendamento

Regra	Descrição
Campos Obrigatórios	Terminal, Tipo (INBOUND/OUTBOUND), Volume Planejado, pelo menos 1 produto
Modificação	Somente grupos ACTIVE podem ser modificados
Cancelamento em Cascata	Quando restrições mudam, agendamentos pendentes incompatíveis são cancelados
Conclusão Automática	Grupo é concluído quando todos agendamentos terminam e volume é atingido
Rastreamento	Volume planejado vs. realizado e atualizado conforme agendamentos completam

5.3 Regras de Aprovação de Agendamentos

Para aprovar um agendamento (RESERVED -> APPROVED), é obrigatório:

Validação	Descrição
Composição	Uma composição veicular deve ser atribuída
Motorista	Um motorista deve ser atribuído e estar APPROVED
Itens	Pelo menos um item (produto/volume) deve existir
Veículos	Veículos dos itens devem pertencer a composição
Compartimentos	Números de compartimento devem ser válidos para o Veículo
Duplicidade	Não pode haver compartimentos duplicados por Veículo
Capacidade	Volume planejado ≤ capacidade do compartimento
Prazo	Aprovação deve ocorrer antes do timeout configurado (padrão: 120min)

5.4 Regras de Confirmação

Regra	Descrição
Antecedência Minima	Confirmação requer antecedência mínima configurável (padrão: 30min)
Timeout	Não Confirmação dentro do prazo gera NO_SHOW (padrão: 60min após Aprovação)
Estado	Somente agendamentos APPROVED podem ser confirmados

5.5 Regras de Expiração e No-Show

Evento	Condição	Resultado
Expiração	Agendamento RESERVED não aprovado dentro do timeout	Estado -> EXPIRED
No-Show	Agendamento APPROVED/CONFIRMED não confirmado/presente	Estado -> NO_SHOW

5.6 Regras de Programas de Operação

Regra	Descrição
Modalidade	Cada programa tem uma modalidade: RODOVIARIO (1), FERROVIARIO (2), AQUAVIARIO (3), DUTOVIARIO (4)
Transição de Estados	Sequência obrigatória: DRAFT -> APPROVED -> CONFIRMED -> IN_PROGRESS -> CLOSING -> COMPLETED
Expiração	Programas podem expirar se não avançarem dentro do prazo
Cancelamento	Programas podem ser cancelados em qualquer estado ativo
Operações	Um programa contem múltiplas operações
Steps	Etapas sequenciais configuradas no programa
Checklists	Verificações obrigatórias antes de prosseguir

5.7 Regras de Transações

Regra	Descrição
Ciclo de Vida	IDLE -> AUTHORIZED -> LOADING -> LOADED
Abortamento	Transações podem ser abortadas (ABORTED)
Pausa	Transações podem ser pausadas (PAUSED)
Vinculação	Cada Transação esta vinculada a um preset e uma Operação
Componentes	Transações possuem componentes individuais (medidores)

5.8 Regras de Composição Veicular

Regra	Descrição
Sem Composição	Itens não podem ter VehicleId ou CompartmentNumber
Com Composição	Todos os itens devem ter VehicleId
Pertencimento	Veículos devem pertencer a composição
Compartimentos Validos	Números de compartimento devem ser válidos para o Veículo
Sem Duplicatas	Não pode haver compartimentos duplicados por Veículo
Limite de Volume	Volume planejado ≤ capacidade do compartimento

5.9 Regras de Aprovação de Motoristas e Veículos

Regra	Descrição
Motoristas	Devem ser aprovados (APPROVED) antes de serem atribuídos a agendamentos
Veículos	Devem ser aprovados (APPROVED) antes de serem usados em composições
Permissões	Requer permissões específicas: `vehicle_approval`, `driver_approval`

5.10 Regras de Controle de Acesso

Regra	Descrição
Autenticação	Todas as rotas admin requerem JWT valido
Sessão Única	Parâmetro `same_user_login` controla sessões simultâneas
Permissões Granulares	55 permissões por funcionalidade
Multi-Tenant	Dados isolados por terminal; Usuário pode pertencer a múltiplos tenants
Timeout de Sessão	Tokens JWT expiram conforme configuração

5.11 Regras de Parametrização

O sistema e altamente configurável por tenant:

Parametro	Descrição
Horário de Operação	Início e fim das operações do terminal
Dias de Operação	Dias da semana permitidos
Antecedência de Aprovação	Minutos minimos para Aprovação (padrão: 60)
Timeout de Aprovação	Minutos para expirar Aprovação (padrão: 120)
Antecedência de Confirmação	Minutos minimos para Confirmação (padrão: 30)

Parametro	Descrição
Timeout de Confirmação	Minutos para no-show (padrão: 60)
Login Simultâneo	Permite ou bloqueia múltiplas sessões

6. Maquinas de Estado

6.1 Agendamento (Appointment)

```
                Timeout
RESERVED ──────────────────────────> EXPIRED
    |
    | Aprovar (+ Composição
    |   + Motorista + Itens)
    ▼                 Timeout
APPROVED ──────────────────────────> NO_SHOW
    |
    | Confirmar
    | (Veículo Chegou)
    ▼                 Timeout
CONFIRMED ─────────────────────────> NO_SHOW
    |
    | Check-in
    ▼
IN_PROGRESS
    |
    | Check-out
    ▼
COMPLETED

* RESERVED, APPROVED e CONFIRMED podem ser CANCELADOS a qualquer momento
```

Validações de Transição:

- `CanBeApproved`: Estado == RESERVED
- `CanBeConfirmed`: Estado == APPROVED
- `CanBeCanceled`: Estado in [RESERVED, APPROVED, CONFIRMED]
- `CanBeStarted`: Estado == CONFIRMED
- `CanBeCompleted`: Estado == IN_PROGRESS
- `CanExpire`: Estado == RESERVED
- `CanBeMarkedNoShow`: Estado in [APPROVED, CONFIRMED]

6.2 Grupo de Agendamento (AppointmentGroup)

```
ACTIVE
    |
    ├─── Todos agendamentos concluídos ──────> COMPLETED
    |     + Volume atingido
```

```
    |
    └──── Cancelamento manual ──────────────────> CANCELED
```

6.3 Programa (Program)

```
DRAFT ──> APPROVED ──> CONFIRMED ──> IN_PROGRESS ──> CLOSING ──> COMPLETED
  |          |            |              |              |
  └──────────┴────────────┴──────────────┴──────────────┘
                          |
                  CANCELED / EXPIRED
```

6.4 Operação (Operation)

```
DRAFT ──> STARTED ──> COMPLETED
  |
  └─────> CANCELED
```

6.5 Manifesto (Manifest)

```
DRAFT ──> APPROVED ──> CONFIRMED ──> IN_PROGRESS ──> CLOSING ──> COMPLETED
  |          |            |              |              |
  └──────────┴────────────┴──────────────┴──────────────┘
                          |
                  CANCELED / EXPIRED
```

6.6 Transação (Transaction)

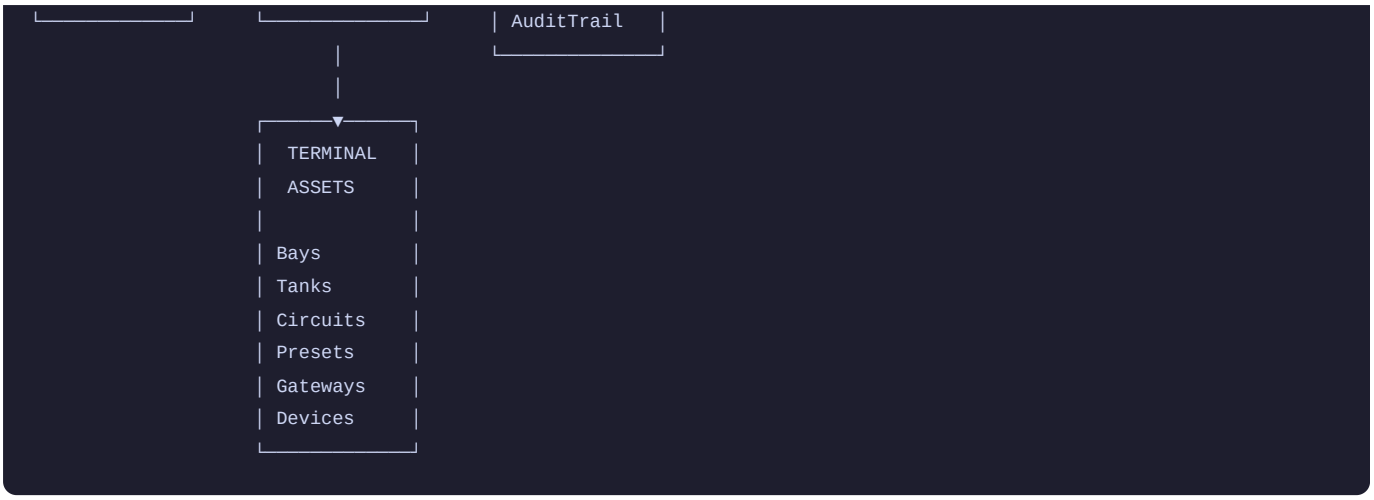
```
IDLE ──> AUTHORIZED ──> LOADING ──> LOADED
                          |
                          ├──> ABORTED
                          └──> PAUSED ──> LOADING (retomada)
```

---

7. Integração entre Módulos

7.1 Fluxo de Dados entre Módulos

```
┌──────────────┐     ┌──────────────┐     ┌──────────────┐
| MANAGEMENT   |────>|  OPERATIONS  |────>|   COMMONS    |
|              |     |              |     |              |
| Appointments|     | Programs     |     | Users        |
| Vehicles     |     | Operations   |     | Tenants      |
| Drivers      |     | Transactions |     | Parameters   |
| Compositions|     | Manifests    |     | Permissions  |
```



7.2 Pontos de Integração

Origem	Destino	Tipo	Descrição
Frontend	Backend	REST API	70+ endpoints para CRUD e ações
Frontend	Backend	SignalR (WS)	Telemetria e notificações em tempo real
Backend	Edge Gateway	MQTT	Comandos e telemetria de dispositivos IoT
Backend	Email (SMTP)	SMTP	Notificações por email
Backend	Sentry	HTTPS	Monitoramento de erros
Backend	OCI Storage	API	Armazenamento de arquivos (produção)
Management	Operations	ACL Service	ProductServiceClient (anti-corruption layer)

7.3 Workers em Background

Worker	Função
JobExecutionWorker	Executa jobs agendados (expiração, limpeza)
MQTTWorker	Gerencia comunicação com brokers MQTT
ProgramWorker	Monitora transições de estado de programas

8. Infraestrutura e Deployment

8.1 Ambientes

Ambiente	Swagger	TLS	Sentry	Logs
DEV	Habilitado	Não	Não	Verbose (Information)

Ambiente	Swagger	TLS	Sentry	Logs
HML	Restrito	Sim	Ativo	Warning
PRD	Desabilitado	Obrigatório	Ativo + Profiling	Warning

8.2 Deployment

- **CI/CD**: Azure DevOps Pipelines
- **Frontend**: Next.js standalone (Docker-ready)
- **Backend**: .NET 8 Web API
- **Database**: PostgreSQL com EF Core Migrations
- **Configuração**: Variáveis de ambiente via placeholders `#{VARIABLE}#`

8.3 Armazenamento de Arquivos

Provedor	Ambiente	Descrição
Local	DEV	Sistema de arquivos local
OCI (Oracle Cloud)	PRD/HML	Object Storage na nuvem
Database	Fallback	Armazenamento no banco de dados

8.4 Internacionalização

- **Idiomas suportados**: Português (pt-BR), Inglês (en)
- **Backend**: Messages.json por domínio (pt-BR, en)
- **Frontend**: 60+ arquivos de tradução organizados por Módulo
- **Erros de API**: Traduzidos via ApiErrorMessagesProvider

Documento atualizado em: 05 de março de 2026
Versão: 1.0
Autor: Milton Kiefer de Albuquerque Mello
Validado por: ---
Última atualização tecnológica: .NET 8.0 / Next.js 14 / React 18

IV. Integrações

Contratos, integrações externas e principais endpoints expostos.

1. Visão Geral das Integrações

1.1 Framework e Arquitetura

Item	Valor
Framework	ASP.NET Core 8.0
Estilo	REST (JSON)
Solution	`AutoloadV4.sln`
Entry point	`AutoloadV4.Api/Program.cs` → `ApiProgram.Main()`

A API segue uma arquitetura em camadas (Domain-Driven Design):

```
AutoloadV4.Api                    → Controllers, Middleware, Hubs, Workers
AutoloadV4.Domain.Commons         → Modelos compartilhados, serviços comuns
AutoloadV4.Domain.Management      → Domínio de gestão (veículos, motoristas, agendamentos)
AutoloadV4.Domain.Operations      → Domínio de operações (programas, carregamentos)
AutoloadV4.Domain.Operations.Client → ACL entre módulos (anti-corruption layer)
AutoloadV4.Infrastructure         → EF Core, migrações, injeção de dependência
AutoloadV4.Lib.Pdf                → Geração de PDFs
```

1.2 Multi-tenancy

O sistema opera com **dois DbContexts** distintos:

DbContext	Escopo	Dados
`AutoloadContext`	Global	Users, Tenants, Sessions, Audit Trails
`TenantContext`	Por tenant	Áreas, Bays, Circuits, Tanks, Products, Programs, Operations

O tenant é resolvido a partir do JWT token (claim `Tenant`) via `TenantProviderFromHttp`.

1.3 Localização (i18n)

Idiomas suportados:

- `pt-BR` (padrão)
- `en`

O idioma é determinado pelo header `Accept-Language` da requisição. A documentação Swagger também é gerada por idioma.

1.4 Comunicação em Tempo Real — SignalR

Três hubs SignalR para comunicação bidirecional em tempo real:

Hub	Endpoint	Finalidade
`MessageHub`	`/hubs/messages`	Notificações e mensagens do sistema
`TelemetryHub`	`/hubs/telemetry`	Dados de telemetria de dispositivos e operações
`HeartbeatHub`	`/hubs/heartbeats`	Heartbeat de gateways e dispositivos IoT

1.5 Serialização JSON

- Enums serializados como strings (`JsonStringEnumConverter`)
- Propriedades `null` são omitidas (`JsonIgnoreCondition.WhenWritingNull`)

1.6 Workers (Background Services)

Serviços em background habilitáveis por configuração:

Worker	Config Key	Função
`JobExecutionWorker`	`Workers:JobExecutionWorkerEnabled`	Execução de jobs assíncronos (email, SMS, WhatsApp)
`MQTTWorker`	`Workers:MQTTWorkerEnabled`	Comunicação MQTT com dispositivos IoT/Edge
`ProgramWorker`	`Workers:ProgramWorkerEnabled`	Controle do ciclo de vida de programas de carga

---

2. Autenticação e Segurança

2.1 JWT Bearer Token

Parâmetro	Valor
Algoritmo	HMAC-SHA256 (`SecurityAlgorithms.HmacSha256`)
Expiração	24 horas
Config	`Jwt:Key` (appsettings.json / variável de ambiente)
Validação	`ValidateLifetime = true`, `ValidateIssuerSigningKey = true`
Issuer/Audience	Não validados (`false`)

Claims incluídas no token:

Claim	Conteúdo
`NameIdentifier`	ID do usuário
`Email`	Email do usuário

Claim	Conteúdo
`Name`	Nome do usuário
`Jti`	ID da sessão (para controle de single-session)
`Tenant`	ID do tenant
`Locale`	Localidade (ex: `pt-BR`)
`Timezone`	Fuso horário (ex: `America/Sao_Paulo`)
`AuthenticationMethod`	Método(s) utilizado(s)
`Role`	Permissões do usuário (múltiplas claims)

2.2 Modos de Autenticação

O modo de autenticação é configurável por tenant via parâmetro `auth_mode`:

1. Basic (email + senha)

Autenticação direta por email e senha. Modo padrão.

```
POST /v1/auth/login
{
  "email": "user@example.com",
  "password": "senha123",
  "tenantId": "optional-tenant-id"
}
```

2. 2FA (Two-Factor Authentication)

Autenticação em duas etapas:

1. Login com email + senha → sistema envia código de 6 dígitos por email
2. Confirmação com o código recebido

```
POST /v1/auth/login       → retorna indicação de 2FA pendente
POST /v1/auth/login       → com o código 2FA no campo correspondente
```

3. Active Directory / LDAP

Autenticação delegada a um servidor LDAP/Active Directory.

Configuração (parâmetro `ldap` no banco):

Campo	Descrição
`domain`	Endereço do servidor LDAP
`port`	Porta (padrão: 389)

Campo	Descrição
`baseDN`	Base Distinguished Name
`query`	Filtro LDAP com placeholder `{username}`
`password`	Senha para binding (opcional)

Protocolo: `System.DirectoryServices.Protocols.LdapConnection` com `AuthType.Negotiate`.

4. OAuth (placeholder)

Integração OAuth prevista na arquitetura mas ainda não implementada.

2.3 API Keys (UserKey)

Para acesso programático (integrações M2M), o sistema suporta API Keys:

```
POST /v1/auth/login
{
  "key": "guid-da-api-key"
}
```

Modelo `UserKey`:

Campo	Tipo	Descrição
`Id`	string (GUID)	Identificador único
`UserId`	string	FK para User
`Name`	string	Nome descritivo
`Key`	string (GUID)	Chave de acesso (valor único)
`IsActive`	bool	Ativa/Inativa
`Created`	DateTimeOffset	Data de criação
`LastAccess`	DateTimeOffset	Último acesso
`Expiration`	DateTimeOffset	Expiração (opcional)

Operações de gerenciamento:

- `POST /v1/users/{id}/keys` — Criar chave
- `PUT /v1/users/{id}/keys/{keyId}/activate` — Ativar
- `PUT /v1/users/{id}/keys/{keyId}/deactivate` — Desativar
- `DELETE /v1/users/{id}/keys/{keyId}` — Remover

2.4 Sessões em Banco de Dados

As sessões JWT são rastreadas no banco de dados (`UserSession`):

Campo	Descrição
`UserId`	Identificador do usuário
`TokenId`	JTI do JWT
`DeviceInfo`	User-Agent + IP
`CreatedAt`	Criação da sessão
`ExpiresAt`	Expiração (30 minutos a partir da criação)

Controle de sessão (configurável por tenant via parâmetro `same_user_login`):

- **Single session**: novo login invalida sessões anteriores
- **Multiple session**: múltiplos logins simultâneos permitidos

A validação ocorre via `JwtSingleSessionExtensions` no middleware de autenticação.

2.5 Password Hashing

Parâmetro	Valor
Algoritmo	PBKDF2 (`Rfc2898DeriveBytes`)
Hash	SHA-256
Salt	16 bytes (128 bits), aleatório
Key	32 bytes (256 bits)
Iterações	10.000
Formato armazenado	`{ITERATIONS}.{BASE64_SALT}.{BASE64_KEY}`

Política de senha configurável:

- Comprimento mínimo/máximo
- Mínimo de caracteres maiúsculos, minúsculos, dígitos e especiais
- Histórico de senhas (impede reutilização)

2.6 RBAC (Role-Based Access Control)

Hierarquia de permissões:

```
Users → Groups → Permissions
```

- Cada usuário pertence a um ou mais **Groups**
- Cada grupo possui um conjunto de **Permissions**
- Permissions são incluídas como claims `Role` no JWT
- O tenant `autoload` (master) possui permissões especiais: `autoload`, `audit_management`, `job_management`, `user_management`, `tenant_management`

2.7 Segurança Adicional

Recurso	Implementação
CORS	Policy `CorsPolicy` com origins configuráveis (`Cors:Origins` , separadas por `;`)
HTTPS	Redirecionamento forçado via `UseHttpsRedirection()`
Middleware de exceções	`ExceptionMiddleware` — captura e padroniza erros
Sentry	Monitoramento de erros em produção (quando DSN configurado)
Negotiate	Suporte a Windows Authentication (esquema adicional)

---

3. Integrações com Serviços Externos

3.1 Email — SMTP via MailKit

Classe: `AutoloadV4.Domain.Commons.Services.EmailService`

Envio de emails HTML via protocolo SMTP utilizando a biblioteca **MailKit/MimeKit**.

> **SIMPLE MAIL TRANSFER PROTOCOL (SMTP)**
>
> é o protocolo padrão da internet para envio de e-mails

Configuração (`appsettings.json` seção `SMTP` ou parâmetro `SMTP` no banco):

Campo	Config Key	Descrição
Server	`SMTP:Server`	Host do servidor SMTP
Port	`SMTP:Port`	Porta (ex: 587)
Options	`SMTP:Options`	Opções de segurança: `None` , `Auto` , `SSL` , `TLS` , `TlsWhenAvailable`
SenderName	`SMTP:SenderName`	Nome do remetente
SenderEmail	`SMTP:SenderEmail`	Email do remetente
User	`SMTP:User`	Usuário de autenticação
Password	`SMTP:Password`	Senha de autenticação

Provedor sugerido: Brevo (ex-Sendinblue) como relay SMTP.

Uso: Envio de códigos 2FA, notificações e alertas do sistema.

3.2 SMS — Zenvia API

Classe: `AutoloadV4.Domain.Commons.Jobs.SendSmsJob`

Envio de SMS via **API REST da Zenvia**, executado como job assíncrono pelo `JobExecutionWorker`.

Configuração (parâmetro `zenvia` no banco, formato JSON):

```
{
  "sms": {
    "url": "https://api.zenvia.com/v2/channels/sms/messages",
    "key": "api-token",
    "sender": "Autoload",
    "reportId": "optional-report-id"
  }
}
```

Campo	Descrição
`sms.url`	Endpoint da API Zenvia para SMS
`sms.key`	Token de autenticação (header `X-API-TOKEN`)
`sms.sender`	Nome/número do remetente
`sms.reportId`	ID de relatório para tracking (opcional)

3.3 WhatsApp — Zenvia API

Classe: `AutoloadV4.Domain.Commons.Jobs.SendWhatsAppJob`

Envio de mensagens WhatsApp via **API REST da Zenvia**, executado como job assíncrono.

Configuração (parâmetro `zenvia` no banco, formato JSON):

```
{
  "whatsapp": {
    "url": "https://api.zenvia.com/v2/channels/whatsapp/messages",
    "key": "api-token",
    "sender": "sender-id"
  }
}
```

Campo	Descrição
`whatsapp.url`	Endpoint da API Zenvia para WhatsApp
`whatsapp.key`	Token de autenticação
`whatsapp.sender`	Identificador do remetente (número/account)

3.4 Armazenamento de Arquivos

O sistema suporta três provedores de armazenamento, configuráveis via `StorageProvider:Provider`:

OCI Object Storage

Classe: `FileStorageServiceOci`

Config Key	Descrição
`StorageProvider:Oci:TenancyOcid`	OCI Tenancy ID
`StorageProvider:Oci:UserOcid`	OCI User ID
`StorageProvider:Oci:Fingerprint`	Fingerprint da API key
`StorageProvider:Oci:PrivateKey`	Chave privada RSA
`StorageProvider:Oci:Region`	Região OCI
`StorageProvider:Oci:Namespace`	Namespace do Object Storage
`StorageProvider:Oci:BucketName`	Nome do bucket

Bibliotecas: `Oci.Common`, `Oci.ObjectstorageService`

Local (filesystem)

Classe: `FileStorageServiceLocal`

Config Key	Descrição
`StorageProvider:Local:BasePath`	Diretório base para armazenamento
`StorageProvider:Local:CreateDirectoryIfNotExists`	Criar diretório automaticamente (default: `true`)

Database

Classe: `FileStorageServiceDatabase`

Armazena o conteúdo dos arquivos diretamente no banco de dados na entidade `Attachment.Content`.

3.5 MQTT — MQTTnet

Classe: `AutoloadV4.Domain.Operations.Services.MQTTService`

Comunicação publish/subscribe com brokers MQTT para integração com dispositivos IoT e Edge Gateways.

Configuração (`appsettings.json` seção `MQTT`):

Config Key	Descrição	Default
`MQTT:Server`	Hostname do broker	—
`MQTT:Port`	Porta do broker	—
`MQTT:User`	Usuário (opcional)	—
`MQTT:Password`	Senha (opcional)	—
`MQTT:Protocol`	`mqtt` (TCP) ou `ws` (WebSocket)	`mqtt`

Config Key	Descrição	Default
`MQTT:UseTls`	Usar TLS/SSL	`false`
`MQTT:WebSocketPath`	Path para WebSocket (opcional)	—

Funcionalidades:

- Publish de mensagens JSON para tópicos
- Subscribe/Unsubscribe em tópicos
- Suporte a `ResponseTopic` e `CorrelationData` (MQTT 5.0)
- Gerenciamento de handlers de mensagens
- Reconexão automática

Biblioteca: `MQTTnet`

3.6 Sentry — Monitoramento de Erros

Integração condicional com **Sentry** para monitoramento de erros e profiling em produção.

Configuração (`appsettings.json` seção `Sentry`):

Config Key	Descrição
`Sentry:Dsn`	Data Source Name (URL do projeto Sentry)
`Sentry:Debug`	Modo debug (`true` / `false`)
`Sentry:Profiling`	Habilitar profiling de performance (`true` / `false`)

A integração só é ativada quando `Sentry:Dsn` contém um valor real (não o placeholder `#{SENTRY_DSN}#`).

3.7 Webhooks

Outbound — Webhooks de Programa

Classe: `AutoloadV4.Domain.Operations.Services.Programs.ProgramWebhookService`

Envia dados do programa para URLs externas quando eventos ocorrem durante o ciclo de vida de um programa de carga.

- Suporta autenticação via login (usuário/senha) ou Bearer token
- Payload: dados completos do programa em JSON
- Configurável por programa/tenant

Inbound — Webhooks de Status

Controller: `WebhooksController`

Recebe callbacks de serviços externos para atualização de status:

Endpoint	Origem	Finalidade
`POST /v1/webhooks/sms`	Zenvia	Status de entrega de SMS
`POST /v1/webhooks/whatsapp`	Zenvia	Status de entrega de WhatsApp

O `externalId` recebido é parseado para identificar a mensagem original e atualizar seu status.

3.8 Protocolos Industriais (Edge Gateway)

Os protocolos industriais são implementados no projeto **AutoloadV4.EdgeGateway** (processo separado):

OPC UA

Classe: `AutoloadV4.EdgeGateway.Drivers.OPC.DriverOPCClient`

- Implementa `IVariableDriver` para leitura/escrita de variáveis OPC UA
- Comunicação thread-safe via semáforo
- Simulador disponível: `PumpOPCSimulator` (servidor OPC UA virtual)

Portas Seriais / TCP-IP

Classe: `AutoloadV4.EdgeGateway.Drivers.Preset.DriverAccuload3`

- Suporte dual: comunicação serial (COM port) e TCP/IP
- Integração com preset loaders (Accuload3, Fusion4)
- Autorização de transações e monitoramento de status de carregamento

Classe: `AutoloadV4.EdgeGateway.Drivers.TruckScale.TruckScaleToledoDriver`

- Driver para balanças rodoviárias Toledo
- Suporte serial e TCP/IP
- Indicadores de estabilidade de peso configuráveis

3.9 LDAP — Active Directory

Autenticação de usuários contra servidores LDAP/Active Directory.

Implementado em `UserService.ValidateLdapUserExists()` utilizando `System.DirectoryServices.Protocols`.

3.10 Banco de Dados

Provedor	Suporte	Config
PostgreSQL	Primário	`Database:Provider = "postgresql"`
SQL Server	Fallback	`Database:Provider = "sqlserver"`

Configuração (`appsettings.json` seção `Database`):

Config Key	Descrição
`Database:Provider`	Provedor (`postgresql` ou `sqlserver`)
`Database:Server`	Host do servidor
`Database:Port`	Porta
`Database:User`	Usuário

Config Key	Descrição
`Database:Password`	Senha

ORM: **Entity Framework Core 8** com migrações code-first.

---

4. Versionamento da API

4.1 Estratégia

Item	Valor
Método	URL Segment Versioning
Formato	`/v{major}/...` (ex: `/v1/users`, `/v2/ping`)
Versão padrão	`v1.0`
Leitor de versão	`UrlSegmentApiVersionReader`
Convenção de namespace	`VersionByNamespaceConvention`
Formato do grupo	`'v'VVV` (ex: `v1`, `v2`)

4.2 Configuração

Definido em `AutoloadV4.Api.Extensions.ApiVersioningExtensions.RegisterApiVersioning()`:

```
options.DefaultApiVersion = new ApiVersion(1, 0);
options.AssumeDefaultVersionWhenUnspecified = true;
options.ReportApiVersions = true;  // Header api-supported-versions na resposta
options.ApiVersionReader = new UrlSegmentApiVersionReader();
options.Conventions.Add(new VersionByNamespaceConvention());
```

A versão é inferida automaticamente pelo namespace do controller:

- `Controllers.V1.*` → API v1
- `Controllers.V2.*` → API v2

4.3 Versões Disponíveis

Versão	Status	Controllers
v1	Estável, produção	52 controllers
v2	Em expansão	1 controller (`PingController`)

---

5. Documentação OpenAPI (Swagger/ReDoc)

5.1 Biblioteca

Swashbuckle.AspNetCore (integração com Swagger/OpenAPI 3.0).

5.2 Acesso

Recurso	URL
Swagger UI	`{base_url}/swagger`
Swagger JSON (v1, en-US)	`{base_url}/swagger/v1-en-US/swagger.json`
Swagger JSON (v1, pt-BR)	`{base_url}/swagger/v1-pt-BR/swagger.json`
Swagger JSON (v2, en-US)	`{base_url}/swagger/v2-en-US/swagger.json`
Swagger JSON (v2, pt-BR)	`{base_url}/swagger/v2-pt-BR/swagger.json`

5.3 Características

- **Documentação por versão e idioma**: cada combinação versão × idioma gera um documento separado
- **Autenticação no Swagger**: suporte a Bearer token via Security Definition
- **XML Comments**: documentação extraída dos comentários XML do código-fonte
- **Filtro de localização**: `SwaggerDocumentLocalizationFilter` traduz descrições por idioma
- **Filtro de enums**: `SwaggerEnumValueDescriptionSchemaFilter` adiciona descrições de valores de enums
- **Expansão padrão**: `DocExpansion.List` (endpoints visíveis, colapsados)

5.4 Autenticação no Swagger

Para testar endpoints autenticados no Swagger UI:

1. Execute `POST /v1/auth/login` com suas credenciais
2. Copie o token JWT retornado
3. Clique em **Authorize** (cadeado) no Swagger UI
4. Insira: `Bearer {seu-token}`
5. Os endpoints subsequentes incluirão o header `Authorization`

---

6. Principais Endpoints

6.1 AppointmentGroups

(`/v1/AppointmentGroups`)

Método	Endpoint	Descrição
POST	`/v1/AppointmentGroups`	AppointmentGroup_Create_Summary
GET	`/v1/AppointmentGroups`	AppointmentGroup_Get_Summary
PATCH	`/v1/AppointmentGroups/{id}`	AppointmentGroup_Update_Summary
DELETE	`/v1/AppointmentGroups/{id}`	AppointmentGroup_Delete_Summary
GET	`/v1/AppointmentGroups/{id}`	AppointmentGroup_GetOne_Summary
PATCH	`/v1/AppointmentGroups/{id}/planned-mass`	AppointmentGroup_UpdatePlannedVolume_Summary
POST	`/v1/AppointmentGroups/{id}/clients`	AppointmentGroup_AddClient_Summary
DELETE	`/v1/AppointmentGroups/{id}/clients/{clientId}`	AppointmentGroup_RemoveClient_Summary
POST	`/v1/AppointmentGroups/{id}/products`	AppointmentGroup_AddProduct_Summary
PATCH	`/v1/AppointmentGroups/{id}/products/{productId}`	AppointmentGroup_UpdateProductPlannedVolume_Summary
DELETE	`/v1/AppointmentGroups/{id}/products/{productId}`	AppointmentGroup_RemoveProduct_Summary
GET	`/v1/AppointmentGroups/{id}/appointments`	AppointmentGroup_GetAppointments_Summary

6.2 Appointments

(`/v1/Appointments`)

Método	Endpoint	Descrição
POST	`/v1/Appointments`	Creates a new appointment
GET	`/v1/Appointments`	Returns the list of appointments
POST	`/v1/Appointments/available-slots`	Appointment_GetAvailableSlots_Summary
POST	`/v1/Appointments/{id}/occupy`	Appointment_OccupySlot_Summary
POST	`/v1/Appointments/{id}/release`	Appointment_ReleaseSlot_Summary
PATCH	`/v1/Appointments/{id}`	Updates an existing appointment
DELETE	`/v1/Appointments/{id}`	Removes an appointment
GET	`/v1/Appointments/{id}`	Get appointment data
PATCH	`/v1/Appointments/{id}/items/{itemId}`	Updates the observed volume of an appointment item
POST	`/v1/Appointments/{id}/approve`	Approves an appointment by the requester (carrier)
POST	`/v1/Appointments/{id}/confirm`	Confirms a draft appointment

Método	Endpoint	Descrição
POST	`/v1/Appointments/{id}/start`	Starts the appointment operation (check-in)
POST	`/v1/Appointments/{id}/complete`	Completes the appointment operation (check-out)
POST	`/v1/Appointments/{id}/cancel`	Cancels an appointment
POST	`/v1/Appointments/{id}/expire`	Marks an appointment as expired
POST	`/v1/Appointments/{id}/mark-no-show`	Marks an appointment as no-show

6.3 AppointmentSlots

(`/v1/AppointmentSlots`)

Método	Endpoint	Descrição
POST	`/v1/AppointmentSlots/generate`	Generates slots for terminal bays
PATCH	`/v1/AppointmentSlots/{id}/block`	Blocks a specific slot
PATCH	`/v1/AppointmentSlots/{id}/unlock`	Unblocks a specific slot
DELETE	`/v1/AppointmentSlots/{id}`	Deletes a specific slot
GET	`/v1/AppointmentSlots/{id}`	Get appointment slot data
GET	`/v1/AppointmentSlots/available`	Get available appointment slots for a date
GET	`/v1/AppointmentSlots`	Returns the list of appointment slots

6.4 Areas

(`/v1/Areas`)

Método	Endpoint	Descrição
POST	`/v1/Areas`	Creates a new area
GET	`/v1/Areas`	Lists all areas with filtering and pagination support
PATCH	`/v1/Areas/{id}`	Updates an existing area
DELETE	`/v1/Areas/{id}`	Removes an existing area
GET	`/v1/Areas/{id}`	Gets a specific area by ID
PATCH	`/v1/Areas/{id}/activate`	Activates an existing area
PATCH	`/v1/Areas/{id}/deactivate`	Deactivates an existing area

6.5 AuditTrails

(`/v1/AuditTrails`)

Método	Endpoint	Descrição
GET	`/v1/AuditTrails`	Lists all audit trails with filtering and pagination support

6.6 Authentication

(`/v1/auth`)

Método	Endpoint	Descrição
POST	`/v1/auth/login`	Performs user login
POST	`/v1/auth/set_password`	Sets or resets a user's password
POST	`/v1/auth/change_password`	Changes the authenticated user's password
GET	`/v1/auth/current_user`	Gets authenticated user information
POST	`/v1/auth/logout`	Logs out the authenticated user

6.7 Bays

(`/v1/Bays`)

Método	Endpoint	Descrição
POST	`/v1/Bays`	Creates a new bay
GET	`/v1/Bays`	Lists all bays with filtering and pagination support
PATCH	`/v1/Bays/{id}`	Updates an existing bay
DELETE	`/v1/Bays/{id}`	Removes an existing bay
GET	`/v1/Bays/{id}`	Gets a specific bay by ID
POST	`/v1/Bays/{id}/circuits`	Adds a circuit to an existing bay
PATCH	`/v1/Bays/{id}/circuits/{circuitId}`	Updates a circuit of an existing bay
DELETE	`/v1/Bays/{id}/circuits/{circuitId}`	Removes a circuit from an existing bay
PATCH	`/v1/Bays/{id}/activate`	Activates an existing bay

Método	Endpoint	Descrição
PATCH	`/v1/Bays/{id}/deactivate`	Deactivates an existing bay
GET	`/v1/Bays/{id}/telemetry`	Gets the last recorded telemetry of a bay

6.8 Checklists

(`/v1/Checklists`)

Método	Endpoint	Descrição
GET	`/v1/Checklists`	Lists programs that have pending or rejected checklist questions
POST	`/v1/Checklists/{programId}/questions/{questionId}/reset`	Restores (resets) a checklist question's answer to the initial state

6.9 Circuits

(`/v1/Circuits`)

Método	Endpoint	Descrição
POST	`/v1/Circuits`	Creates a new circuit
GET	`/v1/Circuits`	Lists all circuits with filtering and pagination support
PATCH	`/v1/Circuits/{id}`	Updates an existing circuit
DELETE	`/v1/Circuits/{id}`	Removes an existing circuit
GET	`/v1/Circuits/{id}`	Gets a specific circuit by ID
PATCH	`/v1/Circuits/{id}/activate`	Activates an existing circuit
PATCH	`/v1/Circuits/{id}/deactivate`	Deactivates an existing circuit

6.10 CustomAttributes

(`/v1/CustomAttributes`)

Método	Endpoint	Descrição
POST	`/v1/CustomAttributes`	Creates a new custom attribute
GET	`/v1/CustomAttributes`	Lists all custom attributes with filtering and pagination support

Método	Endpoint	Descrição
PATCH	`/v1/CustomAttributes/{id}`	Updates an existing custom attribute
DELETE	`/v1/CustomAttributes/{id}`	Removes an existing custom attribute
GET	`/v1/CustomAttributes/{id}`	Gets a specific custom attribute by ID

6.11 Devices

(`/v1/Devices`)

Método	Endpoint	Descrição
POST	`/v1/Devices`	Creates a new device
GET	`/v1/Devices`	Lists all devices with filtering and pagination support
PATCH	`/v1/Devices/{id}`	Updates an existing device
DELETE	`/v1/Devices/{id}`	Removes an existing device
GET	`/v1/Devices/{id}`	Gets a specific device by ID
POST	`/v1/Devices/{id}/execute`	Executes a specific method on a device
POST	`/v1/Devices/{id}/{attributeId}/attachments`	Adds an attachment to a device's custom attribute
DELETE	`/v1/Devices/{id}/{attributeId}/attachments/{attachmentId}`	Removes an attachment from a device's custom attribute
GET	`/v1/Devices/drivers`	Gets the list of available device drivers
GET	`/v1/Devices/{id}/telemetry`	Gets telemetry of a specific device
GET	`/v1/Devices/{id}/{attributeId}/attachments/{attachmentId}`	Gets the content of an attachment from a device's custom attribute

6.12 Drainages

(`/v1/programs/

Método	Endpoint	Descrição
POST	`/v1/programs/{pid}/operations/{oid}/drainages`	Creates a new drainage for a program operation
GET	`/v1/programs/{pid}/operations/{oid}/drainages`	Lists all drainages for a program operation with filtering and pagination support

Método	Endpoint	Descrição
DELETE	`/v1/programs/{pid}/operations/{oid}/drainages/{id}`	Deletes an existing drainage from a program operation
GET	`/v1/programs/{pid}/operations/{oid}/drainages/{id}`	Gets a specific drainage from a program operation by ID

6.13 Drivers

(`/v1/Drivers`)

Método	Endpoint	Descrição
POST	`/v1/Drivers`	Creates a new driver
GET	`/v1/Drivers`	Lists all drivers with filter and pagination support
PATCH	`/v1/Drivers/{id}`	Updates an existing driver
DELETE	`/v1/Drivers/{id}`	Deletes an existing driver
GET	`/v1/Drivers/{id}`	Gets a specific driver by ID
PATCH	`/v1/Drivers/{id}/activate`	Activates an existing driver
PATCH	`/v1/Drivers/{id}/deactivate`	Deactivates an existing driver
PATCH	`/v1/Drivers/{id}/approve`	Approves a pending driver
PATCH	`/v1/Drivers/{id}/reject`	Rejects a pending driver
POST	`/v1/Drivers/{id}/{attributeId}/attachments`	Driver_AddAttachment_Summary
DELETE	`/v1/Drivers/{id}/{attributeId}/attachments/{attachmentId}`	Driver_DeleteAttachment_Summary
GET	`/v1/Drivers/{id}/{attributeId}/attachments/{attachmentId}`	Driver_GetAttachment_Summary

6.14 Enums

(`/v1/Enums`)

Método	Endpoint	Descrição
GET	`/v1/Enums/{enumName}`	Returns a dictionary with the values and descriptions of the specified enum

6.15 Exports

(`/v1/exports/...`)

Método	Endpoint	Descrição
POST	`/v1/exports/programs`	Exports the list of programs in the requested format (CSV/XLSX)
POST	`/v1/exports/programs/{pid}/events`	Exports the events of a program in the requested format
POST	`/v1/exports/programs/{pid}/weighings`	Exports the weighings of a program in the requested format
POST	`/v1/exports/programs/{pid}/steps`	Exports the steps of a program in the requested format
POST	`/v1/exports/programs/{pid}/timelog`	Exports the timelogs of a program
POST	`/v1/exports/programs/{pid}/operations`	Exports the list of operations of a program
POST	`/v1/exports/programs/{pid}/operations/{oid}/operators`	Exports the operators of a program operation
POST	`/v1/exports/programs/{pid}/operations/{oid}/manifests`	Exports the manifests of a program operation
POST	`/v1/exports/programs/{pid}/operations/{oid}/drainages`	Exports the drainages of an operation
POST	`/v1/exports/programs/{pid}/operations/{oid}/tanks`	Exports the tanks associated with an operation
POST	`/v1/exports/programs/{pid}/operations/{oid}/tanks/{tid}/measures`	Exports measurements of a tank associated with an operation
POST	`/v1/exports/programs/{pid}/operations/{oid}/tanks/{tid}/programs`	Exports the list of programs linked to a tank
POST	`/v1/exports/programs/{programId}/attachments`	Exports the list of program attachments
POST	`/v1/exports/areas`	Exports the list of areas in the requested format
POST	`/v1/exports/auditTrails`	Exports the audit trail list in the requested format
POST	`/v1/exports/bays`	Exports the list of bays in the requested format
POST	`/v1/exports/circuits`	Exports the list of circuits in the requested format
POST	`/v1/exports/customAttributes`	Exports the list of custom attributes in the requested format
POST	`/v1/exports/devices`	Exports the list of devices in the requested format
POST	`/v1/exports/gateways`	Exports the list of gateways in the requested format

Método	Endpoint	Descrição
POST	`/v1/exports/gateways/{id}/processes`	Exports the list of processes of a gateway
POST	`/v1/exports/scripts`	Exports the list of gateway scripts
POST	`/v1/exports/groups`	Exports the list of groups in the requested format
POST	`/v1/exports/jobexecutions`	Exports the list of job executions in the requested format
POST	`/v1/exports/messages`	Exports the list of user messages
POST	`/v1/exports/messages/all`	Exports all messages (with filters)
POST	`/v1/exports/organizations`	Exports the list of organizations in the requested format
POST	`/v1/exports/parameters`	Exports the list of parameters in the requested format
POST	`/v1/exports/pipelines`	Exports the list of pipelines in the requested format
POST	`/v1/exports/presets`	Exports the list of presets in the requested format
POST	`/v1/exports/products`	Exports the list of products in the requested format
POST	`/v1/exports/scheduledJobs`	Exports the list of scheduled jobs in the requested format
POST	`/v1/exports/storageTanks`	Exports the list of storage tanks in the requested format
POST	`/v1/exports/tenants`	Exports the list of tenants in the requested format
POST	`/v1/exports/transactions`	Exports the list of transactions in the requested format
POST	`/v1/exports/users`	Exports the list of users in the requested format

6.16 Gateways

(`/v1/Gateways`)

Método	Endpoint	Descrição
POST	`/v1/Gateways`	Creates a new gateway

Método	Endpoint	Descrição
GET	`/v1/Gateways`	Lists all gateways with support for filters and pagination
PATCH	`/v1/Gateways/{id}`	Updates an existing gateway
DELETE	`/v1/Gateways/{id}`	Removes an existing gateway
GET	`/v1/Gateways/{id}`	Gets a specific gateway by ID
PATCH	`/v1/Gateways/{id}/turn_on`	Activates an existing gateway
PATCH	`/v1/Gateways/{id}/turn_off`	Deactivates an existing gateway
POST	`/v1/Gateways/{id}/processes`	Creates a new process for an existing gateway
GET	`/v1/Gateways/{id}/processes`	Lists all processes of a gateway
DELETE	`/v1/Gateways/{id}/processes/{processId}`	Removes a process from an existing gateway
GET	`/v1/Gateways/{id}/processes/{processId}`	Gets a specific process of a gateway
PATCH	`/v1/Gateways/{id}/processes/{processId}/start`	Starts a gateway process
PATCH	`/v1/Gateways/{id}/processes/{processId}/stop`	Stops a gateway process

6.17 GatewayScripts

(`/v1/scripts`)

Método	Endpoint	Descrição
POST	`/v1/scripts`	Creates a new gateway script
GET	`/v1/scripts`	Lists all gateway scripts with support for filters and pagination
PATCH	`/v1/scripts/{id}`	Updates an existing gateway script
DELETE	`/v1/scripts/{id}`	Removes an existing gateway script
GET	`/v1/scripts/{id}`	Gets a specific gateway script by ID

6.18 Groups

(`/v1/Groups`)

Método	Endpoint	Descrição
POST	`/v1/Groups`	Creates a new user group
GET	`/v1/Groups`	Lists all groups with support for filters and pagination

Método	Endpoint	Descrição
PATCH	`/v1/Groups/{groupId}`	Updates an existing group
DELETE	`/v1/Groups/{groupId}`	Removes an existing group
GET	`/v1/Groups/{groupId}`	Gets a specific group by ID
PATCH	`/v1/Groups/{groupId}/activate`	Activates an existing group
PATCH	`/v1/Groups/{groupId}/deactivate`	Deactivates an existing group
POST	`/v1/Groups/{groupId}/user/{userId}`	Adds a user to an existing group
DELETE	`/v1/Groups/{groupId}/user/{userId}`	Removes a user from an existing group

6.19 Heartbeats

(`/v1/Heartbeats`)

Método	Endpoint	Descrição
GET	`/v1/Heartbeats`	Retrieves all heartbeats for the current tenant

6.20 JobExecution

(`/v1/JobExecution`)

Método	Endpoint	Descrição
POST	`/v1/JobExecution/submit`	Submits a new job for execution
GET	`/v1/JobExecution/{id}`	Gets a specific job by ID
GET	`/v1/JobExecution`	Lists all jobs with support for filters and pagination
PATCH	`/v1/JobExecution/{id}/reexecute`	Re-executes an existing job

6.21 Kiosks

(`/v1/kiosks`)

Método	Endpoint	Descrição
GET	`/v1/kiosks/{kioskName}/checkin`	Gets a program for check-in based on the provided values
PATCH	`/v1/kiosks/{kioskName}/checkin`	Confirms a program check-in

Método	Endpoint	Descrição
GET	`/v1/kiosks/{kioskName}/checkin/{pid}/ticket`	Generates a check-in ticket for a program
GET	`/v1/kiosks/{kioskName}/checkout`	Gets a program for check-out based on the provided values
PATCH	`/v1/kiosks/{kioskName}/checkout`	Confirms a program check-out
GET	`/v1/kiosks/{kioskName}/checkout/{pid}/ticket`	Generates a check-out ticket for a program

6.22 Manifests

(`/v1/programs/

Método	Endpoint	Descrição
POST	`/v1/programs/{pid}/operations/{oid}/manifests`	Creates a new manifest for a program operation
GET	`/v1/programs/{pid}/operations/{oid}/manifests`	Lists all manifests for a program operation
PATCH	`/v1/programs/{pid}/operations/{oid}/manifests/{id}/state`	Updates the state of an existing manifest
PATCH	`/v1/programs/{pid}/operations/{oid}/manifests/{id}`	Updates an existing manifest
DELETE	`/v1/programs/{pid}/operations/{oid}/manifests/{id}`	Deletes an existing manifest
GET	`/v1/programs/{pid}/operations/{oid}/manifests/{id}`	Gets a specific manifest by ID

6.23 Messages

(`/v1/Messages`)

Método	Endpoint	Descrição
POST	`/v1/Messages/send_email_test`	Sends a test email
POST	`/v1/Messages/send_sms_test`	Sends a test SMS
POST	`/v1/Messages/send_whatsapp_test`	Sends a test WhatsApp message
POST	`/v1/Messages/subscriptions`	Subscribes the current user to a message topic
GET	`/v1/Messages/subscriptions`	Gets the list of subscriptions of the current user
DELETE	`/v1/Messages/subscriptions/{topic}`	Unsubscribes from a message topic
POST	`/v1/Messages`	Creates a new message and delivers it to subscribed users
GET	`/v1/Messages`	Gets unread messages of the current user
POST	`/v1/Messages/deliver`	Delivers a message to a specific user

Método	Endpoint	Descrição
PATCH	`/v1/Messages/{id}/acknowledge`	Marks a message as read
PATCH	`/v1/Messages/{id}/close`	Closes a message
DELETE	`/v1/Messages/{id}`	Removes a message from the system
GET	`/v1/Messages/{id}`	Gets a specific message by ID
GET	`/v1/Messages/topics`	Gets available topics
GET	`/v1/Messages/all`	Gets all system messages

6.24 Operations

(`/v1/programs/

Método	Endpoint	Descrição
POST	`/v1/programs/{pid}/operations`	Creates a new operation for a program
GET	`/v1/programs/{pid}/operations`	Lists all operations for a program
PATCH	`/v1/programs/{pid}/operations/{id}/start`	Starts the operation
PATCH	`/v1/programs/{pid}/operations/{id}/complete`	Completes the operation
PATCH	`/v1/programs/{pid}/operations/{id}/cancel`	Cancels the operation
PATCH	`/v1/programs/{pid}/operations/{id}`	Updates an existing operation
DELETE	`/v1/programs/{pid}/operations/{id}`	Deletes an existing operation
GET	`/v1/programs/{pid}/operations/{id}`	Gets a specific operation by ID
POST	`/v1/programs/{pid}/operations/swap`	Swaps the labels of two operations
GET	`/v1/programs/{pid}/operations/{id}/report`	Generates a PDF report
GET	`/v1/programs/{pid}/operations/{id}/json_report`	Gets a JSON report

6.25 OperationTanks

(`/v1/programs/

Método	Endpoint	Descrição
POST	`/v1/programs/{pid}/operations/{oid}/tanks`	Associates a tank with a program operation
GET	`/v1/programs/{pid}/operations/{oid}/tanks`	Lists all tanks associated with an operation

Método	Endpoint	Descrição
DELETE	`/v1/programs/{pid}/operations/{oid}/tanks/{id}`	Removes a tank association
GET	`/v1/programs/{pid}/operations/{oid}/tanks/{id}`	Gets a specific tank association

6.26 OperationTanksMeasure

(`/v1/programs/

Método	Endpoint	Descrição
POST	`/v1/programs/{pid}/operations/{oid}/tanks/{tid}/measures`	Creates a new measure
GET	`/v1/programs/{pid}/operations/{oid}/tanks/{tid}/measures`	Lists all measures
PATCH	`/v1/programs/{pid}/operations/{oid}/tanks/{tid}/measures/{id}`	Updates an existing measure
DELETE	`/v1/programs/{pid}/operations/{oid}/tanks/{tid}/measures/{id}`	Deletes an existing measure
GET	`/v1/programs/{pid}/operations/{oid}/tanks/{tid}/measures/{id}`	Gets a specific measure

6.27 OperationTanksProgram

(`/v1/programs/

Método	Endpoint	Descrição
POST	`/v1/programs/{pid}/operations/{oid}/tanks/{tid}/programs/{linkedProgramId}`	Links a program to an operation tank
DELETE	`/v1/programs/{pid}/operations/{oid}/tanks/{tid}/programs/{id}`	Unlinks a program
GET	`/v1/programs/{pid}/operations/{oid}/tanks/{tid}/programs/{id}`	Gets a specific link
GET	`/v1/programs/{pid}/operations/{oid}/tanks/{tid}/programs`	Lists linked programs

6.28 OperationTransaction

(`/v1/programs/

Método	Endpoint	Descrição
POST	`/v1/programs/{pid}/operations/{oid}/transactions`	Creates a transaction
GET	`/v1/programs/{pid}/operations/{oid}/transactions`	Lists transactions

Método	Endpoint	Descrição
POST	`/v1/programs/{pid}/operations/{oid}/transactions/{id}`	Associates a transaction
GET	`/v1/programs/{pid}/operations/{oid}/transactions/{id}`	Gets a transaction by ID
DELETE	`/v1/programs/{pid}/operations/{oid}/transactions/{tid}`	Removes a transaction

6.29 Operators

(`/v1/programs/

Método	Endpoint	Descrição
POST	`/v1/programs/{pid}/operations/{oid}/operators`	Adds an operator
GET	`/v1/programs/{pid}/operations/{oid}/operators`	Lists operators
PATCH	`/v1/programs/{pid}/operations/{oid}/operators/{id}`	Updates an operator
DELETE	`/v1/programs/{pid}/operations/{oid}/operators/{id}`	Removes an operator
GET	`/v1/programs/{pid}/operations/{oid}/operators/{id}`	Gets an operator by ID

6.30 Organizations

(`/v1/Organizations`)

Método	Endpoint	Descrição
POST	`/v1/Organizations`	Creates a new organization
GET	`/v1/Organizations`	Lists organizations
PATCH	`/v1/Organizations/{id}`	Updates an organization
DELETE	`/v1/Organizations/{id}`	Deletes an organization
GET	`/v1/Organizations/{id}`	Gets an organization by ID
PATCH	`/v1/Organizations/{id}/activate`	Activates an organization
PATCH	`/v1/Organizations/{id}/deactivate`	Deactivates an organization
POST	`/v1/Organizations/{id}/add_comment`	Adds a comment
PATCH	`/v1/Organizations/{id}/approve`	Approves an organization
PATCH	`/v1/Organizations/{id}/reject`	Rejects an organization
PATCH	`/v1/Organizations/{id}/parent/{parentId}`	Sets parent organization

Método	Endpoint	Descrição
DELETE	`/v1/Organizations/{id}/parent`	Removes parent organization
GET	`/v1/Organizations/{id}/parent`	Gets parent organization
POST	`/v1/Organizations/{id}/{attributeId}/attachments`	Adds an attachment
DELETE	`/v1/Organizations/{id}/{attributeId}/attachments/{attachmentId}`	Removes attachment
GET	`/v1/Organizations/{id}/children`	Gets child organizations
GET	`/v1/Organizations/hierarchy`	Gets hierarchy
GET	`/v1/Organizations/parents`	Gets possible parents

6.31 Parameters

(`/v1/Parameters`)

Método	Endpoint	Descrição
PATCH	`/v1/Parameters/{id}`	Updates the value of a system parameter
GET	`/v1/Parameters/{id}`	Gets a specific parameter by ID
PATCH	`/v1/Parameters/{id}/default`	Restores the default value of a system parameter
GET	`/v1/Parameters/modules`	Gets the list of modules available in the system
GET	`/v1/Parameters`	Lists system parameters

6.32 Permissions

(`/v1/Permissions`)

Método	Endpoint	Descrição
GET	`/v1/Permissions`	Retrieves all permissions available for the current tenant

6.33 Pipelines

(`/v1/Pipelines`)

Método	Endpoint	Descrição
POST	`/v1/Pipelines`	Creates a new operation pipeline

Método	Endpoint	Descrição
GET	`/v1/Pipelines`	Lists all pipelines
PATCH	`/v1/Pipelines/{id}`	Updates an existing pipeline
DELETE	`/v1/Pipelines/{id}`	Removes an existing pipeline
GET	`/v1/Pipelines/{id}`	Gets a specific pipeline by ID
PATCH	`/v1/Pipelines/{id}/activate`	Activates an existing pipeline
PATCH	`/v1/Pipelines/{id}/deactivate`	Deactivates an existing pipeline

6.34 Presets

(`/v1/Presets`)

Método	Endpoint	Descrição
POST	`/v1/Presets`	Creates a new preset
GET	`/v1/Presets`	Gets a paginated list of presets
PATCH	`/v1/Presets/{id}`	Updates an existing preset
DELETE	`/v1/Presets/{id}`	Deletes a preset
GET	`/v1/Presets/{id}`	Gets a single preset by ID
POST	`/v1/Presets/{id}/components`	Adds a meter to an existing preset
PATCH	`/v1/Presets/{id}/components/{meterId}`	Updates a meter in a preset
DELETE	`/v1/Presets/{id}/components/{meterId}`	Removes a meter from a preset
POST	`/v1/Presets/{id}/recipes`	Adds a recipe to an existing preset
GET	`/v1/Presets/{id}/recipes`	Gets preset recipes
PATCH	`/v1/Presets/{id}/recipes/{recipeId}`	Updates a recipe
DELETE	`/v1/Presets/{id}/recipes/{recipeId}`	Removes a recipe
GET	`/v1/Presets/{id}/telemetry`	Gets telemetry data for a preset
POST	`/v1/Presets/{id}/firmware`	Updates preset firmware
PATCH	`/v1/Presets/{id}/set_mode/{mode}`	Sets preset mode
PATCH	`/v1/Presets/{id}/{state}`	Locks or unlocks preset

6.35 Products

(`/v1/Products`)

Método	Endpoint	Descrição
POST	`/v1/Products`	Creates a new product
GET	`/v1/Products`	Lists all products
PATCH	`/v1/Products/{id}`	Updates an existing product
DELETE	`/v1/Products/{id}`	Removes a product
GET	`/v1/Products/{id}`	Gets a product by ID
PATCH	`/v1/Products/{id}/enable`	Enables a product
PATCH	`/v1/Products/{id}/disable`	Disables a product
POST	`/v1/Products/{id}/{attributeId}/attachments`	Adds attachment
DELETE	`/v1/Products/{id}/{attributeId}/attachments/{attachmentId}`	Removes attachment
GET	`/v1/Products/groups`	Gets product groups
GET	`/v1/Products/{id}/{attributeId}/attachments/{attachmentId}`	Gets attachment content

6.36 ProgramAttachments

(`/v1/programs/

Método	Endpoint	Descrição
POST	`/v1/programs/{programId}/attachments`	Uploads attachment
GET	`/v1/programs/{programId}/attachments`	Lists attachments
DELETE	`/v1/programs/{programId}/attachments/{attachmentId}`	Deletes attachment
GET	`/v1/programs/{programId}/attachments/{attachmentId}`	Gets attachment metadata
GET	`/v1/programs/{programId}/attachments/{attachmentId}/content`	Downloads attachment content

6.37 ProgramChecklists

(`/v1/programs/

Método	Endpoint	Descrição
GET	`/v1/programs/{programId}/checklists`	Gets checklists for program
GET	`/v1/programs/{programId}/checklists/{stepId}`	Gets checklist questions
POST	`/v1/programs/{programId}/checklists/{stepId}`	Creates checklist questions
PATCH	`/v1/programs/{programId}/checklists/{stepId}`	Updates checklist answers
GET	`/v1/programs/{programId}/checklists/{stepId}/status`	Gets consolidated checklist status

6.38 Programs

(`/v1/programs`)

Método	Endpoint	Descrição
POST	`/v1/programs`	Creates a new program
GET	`/v1/programs`	Lists all programs
PATCH	`/v1/programs/{id}`	Updates program
DELETE	`/v1/programs/{id}`	Deletes program
GET	`/v1/programs/{id}`	Gets program by ID
PATCH	`/v1/programs/{id}/state`	Updates program state
PATCH	`/v1/programs/{id}/to_next_step`	Moves to next step
PATCH	`/v1/programs/{id}/to_next_step/{nextStepId}`	Moves to specific step
GET	`/v1/programs/labels/{modal}`	Gets parameter labels
GET	`/v1/programs/modals`	Gets modals
POST	`/v1/programs/{pid}/events`	Creates event
GET	`/v1/programs/{pid}/events`	Lists events
DELETE	`/v1/programs/{pid}/events/{eid}`	Deletes event
GET	`/v1/programs/{pid}/events/{eid}`	Gets event
POST	`/v1/programs/{pid}/weighings`	Creates weighing
GET	`/v1/programs/{pid}/weighings`	Lists weighings
PATCH	`/v1/programs/{pid}/weighings/{wid}`	Updates weighing
DELETE	`/v1/programs/{pid}/weighings/{wid}`	Deletes weighing

Método	Endpoint	Descrição
GET	`/v1/programs/{pid}/weighings/{wid}`	Gets weighing

6.39 Pumps

(`/v1/Pumps`)

Método	Endpoint	Descrição
POST	`/v1/Pumps`	Creates pump
GET	`/v1/Pumps`	Lists pumps
PATCH	`/v1/Pumps/{id}`	Updates pump
DELETE	`/v1/Pumps/{id}`	Removes pump
GET	`/v1/Pumps/{id}`	Gets pump
PATCH	`/v1/Pumps/{id}/state`	Changes pump state
PATCH	`/v1/Pumps/{id}/mode/{mode}`	Sets pump mode

6.40 Queues

(`/v1/Queues`)

Método	Endpoint	Descrição
POST	`/v1/Queues/process`	Runs queue prioritization
GET	`/v1/Queues/{terminalId}/monitor`	Returns scheduling data

6.41 ScheduledJobs

(`/v1/ScheduledJobs`)

Método	Endpoint	Descrição
POST	`/v1/ScheduledJobs`	Schedules new job
GET	`/v1/ScheduledJobs`	Lists scheduled jobs
PATCH	`/v1/ScheduledJobs/{id}/activate`	Activates job
PATCH	`/v1/ScheduledJobs/{id}/deactivate`	Deactivates job

Método	Endpoint	Descrição
DELETE	`/v1/ScheduledJobs/{id}`	Removes job
GET	`/v1/ScheduledJobs/{id}`	Gets job by ID

6.42 Steps

(`/v1/programs/

Método	Endpoint	Descrição
POST	`/v1/programs/{pid}/steps`	Creates step
GET	`/v1/programs/{pid}/steps`	Lists steps
PATCH	`/v1/programs/{pid}/steps/{id}`	Updates step
DELETE	`/v1/programs/{pid}/steps/{id}`	Deletes step
GET	`/v1/programs/{pid}/steps/{id}`	Gets step

6.43 StorageTanks

(`/v1/StorageTanks`)

Método	Endpoint	Descrição
POST	`/v1/StorageTanks`	Creates tank
GET	`/v1/StorageTanks`	Lists tanks
PATCH	`/v1/StorageTanks/{id}`	Updates tank
DELETE	`/v1/StorageTanks/{id}`	Deletes tank
GET	`/v1/StorageTanks/{id}`	Gets tank
POST	`/v1/StorageTanks/{id}/circuits/{cid}`	Adds circuit
DELETE	`/v1/StorageTanks/{id}/circuits/{cid}`	Removes circuit
PATCH	`/v1/StorageTanks/{id}/activate`	Activates tank
PATCH	`/v1/StorageTanks/{id}/deactivate`	Deactivates tank
GET	`/v1/StorageTanks/{id}/operations`	Lists operations
POST	`/v1/StorageTanks/{id}/calculateVariables`	Calculates measures
GET	`/v1/StorageTanks/{id}/telemetry`	Gets telemetry

Método	Endpoint	Descrição
POST	`/v1/StorageTanks/{id}/calibrationTable`	Adds calibration table
PATCH	`/v1/StorageTanks/{id}/calibrationTable`	Updates calibration table
DELETE	`/v1/StorageTanks/{id}/calibrationTable`	Removes calibration table

6.44 Telemetry

(`/v1/Telemetry`)

Método	Endpoint	Descrição
POST	`/v1/Telemetry/{entityType}/{entityId}`	Starts telemetry collection
DELETE	`/v1/Telemetry/{entityType}/{entityId}`	Stops telemetry collection
GET	`/v1/Telemetry/{entityType}/{entityId}`	Gets telemetry data
GET	`/v1/Telemetry`	Gets telemetry for tenant

6.45 Tenants

(`/v1/Tenants`)

Método	Endpoint	Descrição
POST	`/v1/Tenants`	Creates tenant
GET	`/v1/Tenants`	Lists tenants
PATCH	`/v1/Tenants/{id}`	Updates tenant
DELETE	`/v1/Tenants/{id}`	Deletes tenant
GET	`/v1/Tenants/{id}`	Gets tenant
PATCH	`/v1/Tenants/{id}/name/{name}`	Updates tenant name
PATCH	`/v1/Tenants/{id}/activate`	Activates tenant
PATCH	`/v1/Tenants/{id}/deactivate`	Deactivates tenant
GET	`/v1/Tenants/modules`	Lists modules
GET	`/v1/Tenants/modes`	Lists transportation modes

6.46 Ticket

(`/v1/programs/

Método	Endpoint	Descrição
GET	`/v1/programs/{programId}/ticket`	Generates program ticket

6.47 TimeLog

(`/v1/programs/

Método	Endpoint	Descrição
POST	`/v1/programs/{pid}/timelog`	Creates timelog
GET	`/v1/programs/{pid}/timelog`	Lists timelogs
GET	`/v1/programs/{pid}/timelog/{id}`	Gets timelog

6.48 Transactions

(`/v1/Transactions`)

Método	Endpoint	Descrição
POST	`/v1/Transactions`	Creates transaction
GET	`/v1/Transactions`	Lists transactions
PATCH	`/v1/Transactions/{id}`	Updates transaction
GET	`/v1/Transactions/{id}`	Gets transaction
PATCH	`/v1/Transactions/{id}/complete`	Completes transaction

6.49 Users

(`/v1/Users`)

Método	Endpoint	Descrição
POST	`/v1/Users`	Creates user
GET	`/v1/Users`	Lists users
PATCH	`/v1/Users/{userId}/enable_login`	Enables login
PATCH	`/v1/Users/{userId}/disable_login`	Disables login

Método	Endpoint	Descrição
PATCH	`/v1/Users/{userId}/enable_key`	Enables key
PATCH	`/v1/Users/{userId}/disable_key`	Disables key
PATCH	`/v1/Users/{userId}/activate`	Activates user
PATCH	`/v1/Users/{userId}/deactivate`	Deactivates user
PATCH	`/v1/Users/{userId}/accept_terms_of_use`	Accepts terms
PATCH	`/v1/Users/reset_terms_of_use`	Resets terms
DELETE	`/v1/Users/{userId}`	Deletes user
GET	`/v1/Users/{id}/groups`	Lists user groups
GET	`/v1/Users/{id}`	Gets user with groups
POST	`/v1/Users/{userId}/keys`	Creates access key
GET	`/v1/Users/{userId}/keys`	Lists access keys
PATCH	`/v1/Users/{userId}/keys/{userKeyId}/activate`	Activates key
PATCH	`/v1/Users/{userId}/keys/{userKeyId}/deactivate`	Deactivates key
DELETE	`/v1/Users/{userId}/keys/{userKeyId}`	Deletes key
PATCH	`/v1/Users/{userId}/name/{username}`	Updates name
PATCH	`/v1/Users/{userId}/email/{userEmail}`	Updates email
PATCH	`/v1/Users/{userId}/phone`	Updates phone
POST	`/v1/Users/{userId}/preferences`	Updates preferences
GET	`/v1/Users/{userId}/preferences`	Gets preferences
DELETE	`/v1/Users/{userId}/preferences/{userPreferenceId}`	Deletes preference
GET	`/v1/Users/{userId}/preferences/{userPreferenceId}`	Gets preference by ID
POST	`/v1/Users/{userId}/tenant/{tenantId}`	Adds user to tenant
DELETE	`/v1/Users/{userId}/tenant/{tenantId}`	Removes user from tenant

6.50 VehicleCompositions

(`/v1/VehicleCompositions`)

Método	Endpoint	Descrição
POST	`/v1/VehicleCompositions`	Creates vehicle composition
GET	`/v1/VehicleCompositions`	Lists vehicle compositions
PATCH	`/v1/VehicleCompositions/{id}`	Updates vehicle composition
DELETE	`/v1/VehicleCompositions/{id}`	Deletes vehicle composition
GET	`/v1/VehicleCompositions/{id}`	Gets vehicle composition
GET	`/v1/VehicleCompositions/types`	Gets vehicle composition types
PUT	`/v1/VehicleCompositions/types`	Updates vehicle composition types

6.51 Vehicles

(`/v1/Vehicles`)

Método	Endpoint	Descrição
POST	`/v1/Vehicles`	Creates vehicle
GET	`/v1/Vehicles`	Lists vehicles
PATCH	`/v1/Vehicles/{id}`	Updates vehicle
DELETE	`/v1/Vehicles/{id}`	Removes vehicle
GET	`/v1/Vehicles/{id}`	Gets vehicle
PATCH	`/v1/Vehicles/{id}/activate`	Activates vehicle
PATCH	`/v1/Vehicles/{id}/deactivate`	Deactivates vehicle
PATCH	`/v1/Vehicles/{id}/approve`	Approves vehicle
PATCH	`/v1/Vehicles/{id}/reject`	Rejects vehicle
POST	`/v1/Vehicles/{id}/{attributeId}/attachments`	Adds attachment
DELETE	`/v1/Vehicles/{id}/{attributeId}/attachments/{attachmentId}`	Removes attachment
GET	`/v1/Vehicles/{id}/{attributeId}/attachments/{attachmentId}`	Gets attachment

6.52 Webhooks

(`/v1/webhooks`)

Método	Endpoint	Descrição
POST	`/v1/webhooks/events/sms`	Receives SMS status events
POST	`/v1/webhooks/events/whatsapp`	Receives WhatsApp status events

7. Resumo de Contadores

Categoria	Quantidade
Controllers V1	52
Controllers V2	1
SignalR Hubs	3
Integrações externas	10
Idiomas suportados	2
Modos de autenticação	4 (+1 placeholder)
Provedores de storage	3
Background workers	3

Documento atualizado em: 05 de março de 2026
Versão: 1.0 **Autor**: Milton Kiefer de Albuquerque Mello
Validado por: ---
Última atualização tecnológica: .NET 8.0 / Next.js 14 / React 18

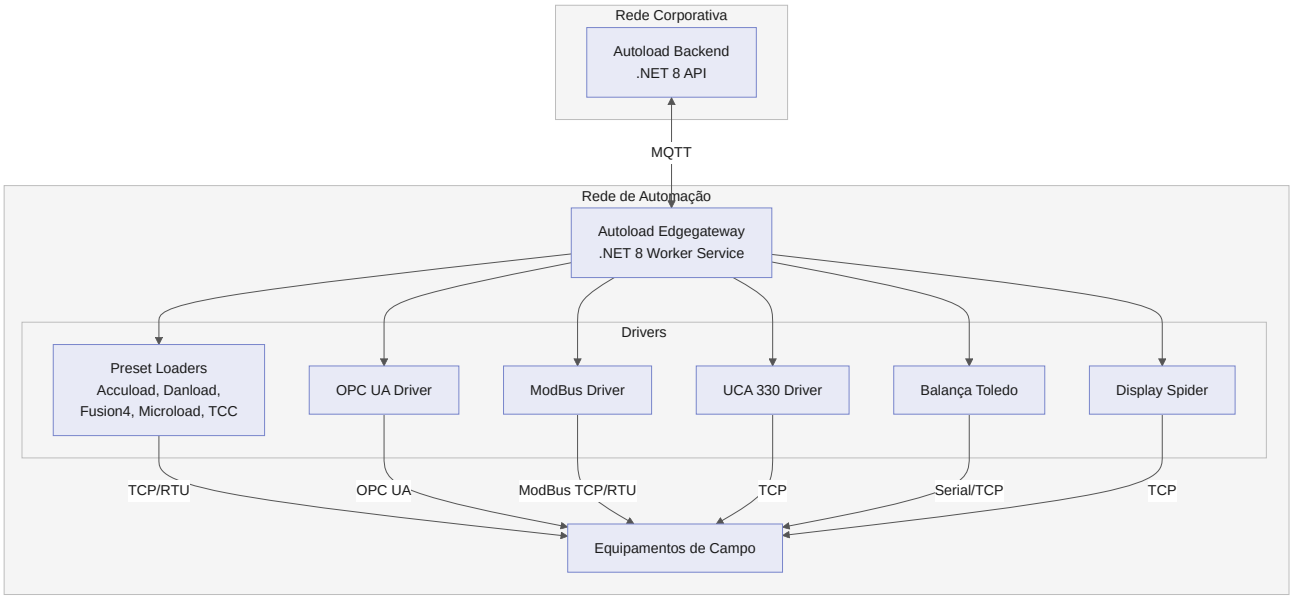
V. Drivers

Integrações e adaptadores para sistemas externos.

1. Visão Geral de Drivers

O **Autoload Edgegateway** é o componente de edge computing do Autoload V4, atuando como ponte entre o Backend e os dispositivos industriais de campo. Os **drivers** são os módulos responsáveis por estabelecer e gerenciar a comunicação direta com esses equipamentos, traduzindo comandos do sistema em operações de protocolo industrial e coletando telemetria em tempo real.

1.1. Arquitetura dos Drivers no Sistema



1.2. Fundamentos da Arquitetura de Drivers

Todos os drivers do Autoload Edgegateway seguem uma arquitetura unificada:

- **Classe abstrata** `Driver`: Base que implementa a interface `IDriver`, fornecendo propriedades comuns (`Status`, `StopRequested`, `Device`, `Logger`) e implementações padrão dos hooks de lifecycle.
- **Interface** `IDriver`: Contrato base que todo driver deve cumprir — conexão, desconexão, telemetria e hooks de ciclo de vida.
- **Interfaces especializadas**: Cada categoria de driver implementa interfaces adicionais que definem operações específicas do domínio.
- **Carregamento dinâmico**: Drivers são instanciados em runtime via reflection, utilizando as informações de `DriverType` e `DriverTypeAssembly` registradas no arquivo `DeviceDrivers.json`.

1.3. Princípios Fundamentais

Princípio	Descrição
Thread Safety	Drivers podem ser acessados por múltiplas threads simultaneamente (telemetria, processos, RPC)
Cancelamento Cooperativo	Drivers verificam `StopRequested` e encerram operações graciosamente

Princípio	Descrição
Resiliência	Tratamento de erros de comunicação sem travar o sistema, com reconexão automática
Observabilidade	Logging estruturado via `ILogger<T>` para diagnóstico

2. Tipos de Drivers

O Autoload Edgegateway possui **16 implementações de drivers** organizadas em **6 categorias**, registradas centralizadamente no arquivo `DeviceDrivers.json`.

2.1. Tabela Completa de Drivers

Categoria	Driver	Classe	Protocolo	Assembly
Preset	Accuload 3	`DriverAccuload3`	TCP/Serial (RTU)	`AutoloadV4.EdgeGateway.Drivers.Preset`
Preset	Accuload 4	`DriverAccuload4`	TCP	`AutoloadV4.EdgeGateway.Drivers.Preset`
Preset	Danload 6000	`DriverDanload6000`	TCP/RTU	`AutoloadV4.EdgeGateway.Drivers.Preset`
Preset	Danload 8000	`DriverDanload8000`	TCP/RTU	`AutoloadV4.EdgeGateway.Drivers.Preset`
Preset	Fusion4	`DriverFusion4`	TCP/RTU	`AutoloadV4.EdgeGateway.Drivers.Preset`
Preset	Microload	`DriverMicroload`	TCP/RTU	`AutoloadV4.EdgeGateway.Drivers.Preset`
Preset	TCC	`DriverTCC`	TCP/RTU	`AutoloadV4.EdgeGateway.Drivers.Preset`
Preset	Mock	`DriverMock`	Mock (testes)	`AutoloadV4.EdgeGateway.Drivers.Preset`
OPC	OPC Client	`DriverOPCClient`	OPC UA	`AutoloadV4.EdgeGateway.Drivers.OPC`
OPC	OPC Client Mock	`DriverOPCClientMock`	Mock (testes)	`AutoloadV4.EdgeGateway.Drivers.OPC`
Modbus	Modbus Client	`DriverModBus`	ModBus TCP	`AutoloadV4.EdgeGateway.Drivers.ModBus`
Modbus	Modbus Client Mock	`DriverModBusMock`	Mock (testes)	`AutoloadV4.EdgeGateway.Drivers.ModBus`
Display	Display Spider	`DriverSpider`	TCP	`AutoloadV4.EdgeGateway.Drivers.Display`
TruckScale	Toledo	`TruckScaleToledoDriver`	Serial/TCP	`AutoloadV4.EdgeGateway.Drivers.TruckScale`
UCA	Unidata 330	`DriverUCA330`	TCP	`AutoloadV4.EdgeGateway.Drivers.UCA`
UCA	UCA Mock	`DriverUCAMock`	Mock (testes)	`AutoloadV4.EdgeGateway.Drivers.UCA`

> **GLOSSÁRIO**

- OPC (Open Platform Communications): padrão universal de interoperabilidade para troca segura e confiável de dados na automação industrial e sistemas corporativos;
- Modbus: protocolo de comunicação industrial usado para conectar dispositivos eletrônicos como CLPs, sensores e inversores
- UCA (Unidade de Controle de Acesso): dispositivo para controle de acesso físico em terminais

2.2. Drivers Preset

Os drivers Preset são responsáveis pela comunicação com **preset loaders** — equipamentos que controlam o carregamento/descarregamento de granéis líquidos em terminais.

Interfaces implementadas: `IDriverPreset` (operações de baixo nível) + `IPresetDriverV4` (operações de alto nível)

Operações principais:

- Autorização de carregamento (`AuthorizeLoading`)
- Monitoramento do processo (`WaitLoading` , `GetProcessStatus`)
- Finalização de transação (`FinalizeLoading`)
- Consulta de telemetria por braço/medidor (`GetTelemetryData`)
- Controle de modo operacional (`SetMode`)
- Gerenciamento de receitas e alarmes
- Exibição de mensagens no display do preset (`SetMessage` , `WaitForData`)
- Controle do swing arm (`GetSwingArmSide`)

Driver	Descrição
Accuload 4	Comunicação TCP/IP pura. Protocolo proprietário Accuload com frames STX/ETX/LRC.
Accuload 3	Suporte dual Serial (RS-232) e TCP. Mesmo protocolo Accuload, com seleção via configuração `protocol` .
Danload 6000/8000	Equipamentos Daniel. Comunicação TCP/RTU.
Fusion4	Loaders Fusion4 MSCL. Utiliza Modbus sobre TCP.
Microload	Preset loaders Microload com implementação dedicada.
TCC	Controladores TCC com protocolo de mensagens proprietário.
Mock	Simulador para testes sem hardware físico.

2.3. Drivers OPC

Os drivers OPC fazem leitura e escrita de variáveis industriais via protocolo OPC UA.

Interface implementada: `IVariableDriver`

Operações principais:

- Leitura de variável individual (`GetVariable`)
- Leitura de múltiplas variáveis (`GetVariables`)
- Escrita de variável individual (`SetVariable`)
- Escrita de múltiplas variáveis (`SetVariables`)
- Coleta de telemetria de todas as variáveis configuradas (`GetTelemetryData`)

Driver	Descrição
OPC Client	Cliente OPC UA real. Suporta URLs `opc.tcp://` (UA).
OPC Client Mock	Simulador para testes sem servidor OPC.

2.4. Drivers Modbus

Os drivers Modbus fazem leitura e escrita de registradores e coils em dispositivos ModBus TCP.

Interface implementada: `IVariableDriver`

Operações principais:

- Leitura/escrita de variáveis com suporte a múltiplos tipos de dados (`ushort`, `short`, `uint`, `int`, `bool`, `float-little`, `float-big`, `string`)
- Leitura de holding registers e coils
- Coleta de telemetria (`GetTelemetryData`)

Driver	Descrição
Modbus Client	Cliente ModBus TCP (implementação customizada).
Modbus Client Mock	Simulador para testes sem dispositivos ModBus.

2.5. Driver Display

O driver Display controla displays industriais Spider para exibição de mensagens nos terminais.

Interface implementada: `IDisplayDriver`

Operações principais:

- Exibição de mensagem estática (`SetMessage`)
- Exibição de mensagem com título (`SetMessageWithTitle`)
- Exibição de mensagem rolante (`SetScrollingMessage`)
- Limpar display (`ClearDisplay`)

Driver	Descrição
Display Spider	Comunicação TCP direta com displays Spider. Comandos enviados em formato binário.

2.6. Driver TruckScale (Balança)

O driver TruckScale faz a leitura de peso de balanças industriais Toledo.

Interface implementada: `ITruckScaleDriver`

Operações principais:

- Leitura de peso síncrona (`GetWeight`)
- Leitura de peso assíncrona (`GetWeightAsync`)
- Coleta de telemetria com peso atual (`GetTelemetryData`)

Driver	Descrição
Toledo	Suporte dual Serial (RS-232) e TCP. Compatível com modelos IND560 e 8540.

2.7. Drivers UCA (Unidade de Controle de Acesso)

Os drivers UCA controlam unidades de controle de acesso Unidata 330, usadas para autenticação de motoristas e controle de portões em terminais.

Interface implementada: `IDriverUCA`

Operações principais:

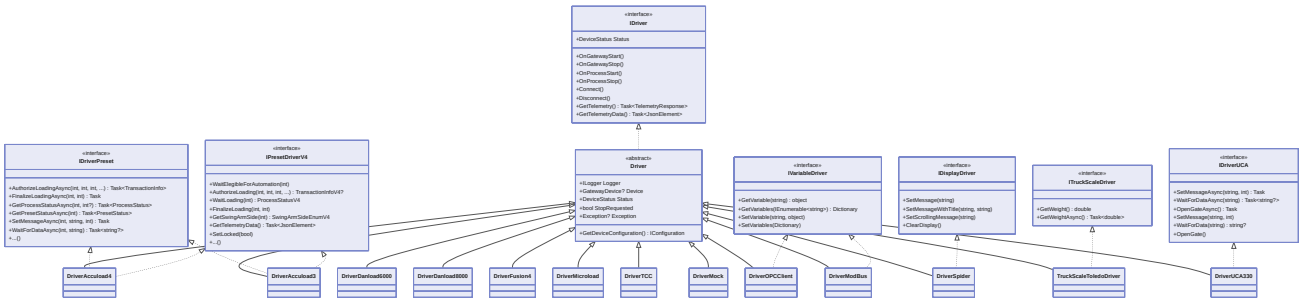
- Exibição de mensagens no display da UCA (`SetMessageAsync`)
- Aguardar entrada de dados do operador (`WaitForDataAsync`)
- Abertura de portão (`OpenGateAsync`)
- Consulta de status do portão (`GetTelemetryData`)

Driver	Descrição
Unidata 330	Comunicação TCP com protocolo de comandos (display, teclado, portão).
UCA Mock	Simulador para testes sem hardware UCA.

---

3. Hierarquia de Classes e Interfaces

3.1. Diagrama de Hierarquia



> **PARA MELHOR QUALIDADE, CONFIRA O DIAGRAMA EM UMA NOVA ABA:**
>
> Abrir diagrama

3.2. Descrição das Interfaces

`IDriver` — Interface Base

Contrato que todo driver deve implementar. Define o ciclo de vida básico e coleta de telemetria.

```
public interface IDriver
{
    DeviceStatus Status { get; set; }
    void OnGatewayStart();
    void OnGatewayStop();
    void OnProcessStart();
    void OnProcessStop();
    void Connect();
    void Disconnect();
    Task<TelemetryResponse> GetTelemetry();
    Task<JsonElement> GetTelemetryData();
}
```

`IDriverPreset` — Operações de Baixo Nível para Preset Loaders

Interface detalhada com operações granulares para controle de preset loaders. Inclui métodos assíncronos para autorização, finalização, consulta de status, gerenciamento de receitas, alarmes, transações e controle de modo operacional.

Métodos principais: `AuthorizeLoadingAsync`, `FinalizeLoadingAsync`, `GetProcessStatusAsync`, `GetPresetStatusAsync`, `GetMetersStatusAsync`, `SetMessageAsync`, `WaitForDataAsync`, `SetRecipeAsync`, `GetTransactionHistoryAsync`, `OnStopBatchAlarmAsync`, `GetPresetDateAsync`, `MarkAsCompletedAsync`

`IPresetDriverV4` — Operações de Alto Nível para Preset Loaders

Interface simplificada para uso em scripts Lua. Fornece métodos síncronos e de alto nível para o fluxo de carregamento.

Métodos principais: `WaitElegibleForAutomation`, `AuthorizeLoading`, `WaitLoading`, `FinalizeLoading`, `GetSwingArmSide`, `GetProcessStatus`, `SetLocked`, `SetRecipe`

```
public interface IPresetDriverV4
    {
        public void WaitElegibleForAutomation(int arm);
        public string? WaitForArm(string message);
        public void SetMessage(int arm, string message, int timeout);
        public string? WaitForData(int arm, string message);
        public void ClearDisplay(int arm);
        public TransactionInfoV4? AuthorizeLoading(int arm, int recipe, int volume, AdditiveMask? am = null, string? rec
        public ProcessStatusV4 WaitLoading(int arm);
        public void FinalizeLoading(int arm, int transactionId);
        public SwingArmSideEnumV4 GetSwingArmSide(int arm);
        public ProcessStatusV4 GetProcessStatus(int arm);
        public TransactionV4? GetTransaction(int arm, int transactionId);
        public Task<JsonElement> GetTelemetryData();
        public void ResetPreset(int arm);
        public int GetNextTransactionId(int arm);
        void SetMode(int arm, PresetMode mode);
        public void SetLocked(bool locked);
        public void SetRecipe(int arm, int recipeId, PresetRecipesV4 presetRecipe);
    }
```

`IVariableDriver` — Leitura/Escrita de Variáveis (OPC, Modbus)

Interface para drivers que trabalham com variáveis industriais nomeadas.

```
public interface IVariableDriver
{
    object GetVariable(string variableName);
    Dictionary<string, object> GetVariables(IEnumerable<string> variableNames);
    void SetVariable(string variableName, object value);
    void SetVariables(Dictionary<string, object> variables);
}
```

`IDisplayDriver` — Controle de Displays

Interface para controle de displays industriais com suporte a mensagens estáticas, com título e rolantes.

```
public interface IDisplayDriver
{
    void SetScrollingMessage(string message);
    void SetMessage(string message);
    void SetMessageWithTitle(string title, string message);
    void ClearDisplay();
}
```

`ITruckScaleDriver` — Leitura de Peso

Interface para balanças industriais com leitura síncrona e assíncrona.

```
public interface ITruckScaleDriver
{
    Task<double> GetWeightAsync();
    double GetWeight();
}
```

`IDriverUCA` — Controle de Acesso

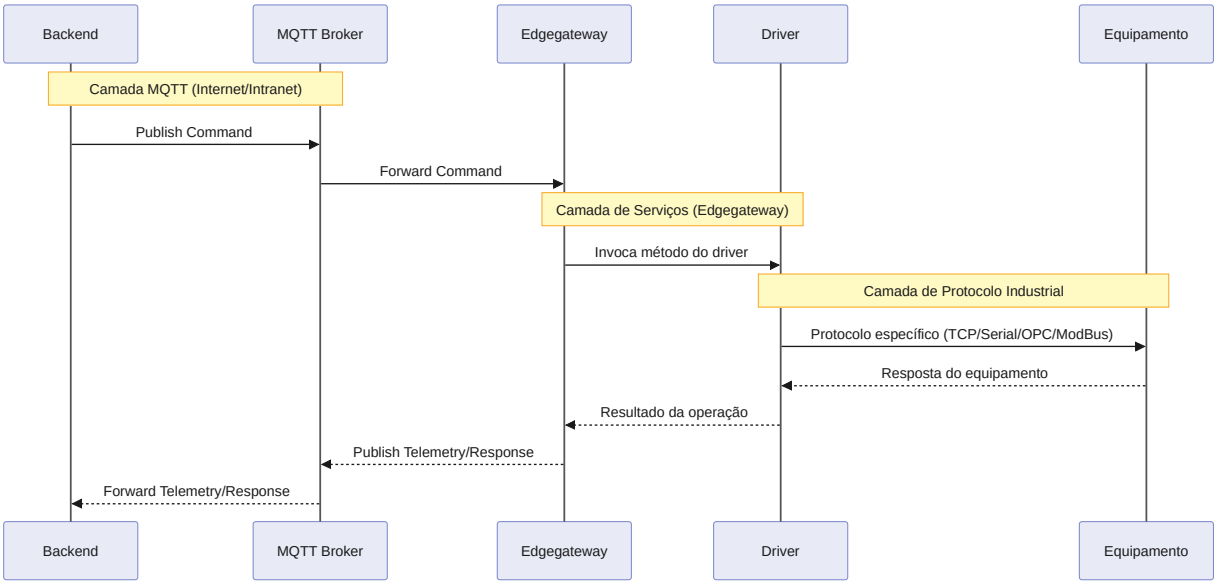
Interface para unidades de controle de acesso, com versões síncronas e assíncronas de cada operação.

```
public interface IDriverUCA
{
    Task SetMessageAsync(string message, int timeout = 0);
    Task<string?> WaitForDataAsync(string message);
    Task OpenGateAsync();
    void SetMessage(string message, int timeout = 0);
    string? WaitForData(string message);
    void OpenGate();
}
```

---

4. Comunicação entre Sistema e Drivers

4.1. Visão Geral dos Protocolos



4.2. MQTT (Backend ↔ Edgegateway)

O MQTT é o protocolo de comunicação entre o Backend e o Edgegateway, conforme descrito na seção 7.4 do documento de arquitetura.

Tópicos MQTT:

```
autoload/gateway/{gatewayId}/telemetry       # Telemetria do gateway
autoload/gateway/{gatewayId}/command          # Comandos para gateway
autoload/gateway/{gatewayId}/rpc/request      # RPC Request
autoload/gateway/{gatewayId}/rpc/response     # RPC Response
autoload/device/{deviceId}/telemetry          # Telemetria de dispositivos
autoload/device/{deviceId}/command            # Comandos para dispositivos
```

Formato de mensagem:

```
{
  "messageId": "uuid",
  "timestamp": "2026-03-03T10:30:00Z",
  "type": "telemetry|command|rpc",
  "payload": {
    // dados específicos da operação
  }
}
```

Operações MQTT do Gateway:

Tópico	Operação	Descrição
`{TenantId}/gateways/{GatewayId}/start`	Iniciar gateway	Conecta todos os drivers
`{TenantId}/gateways/{GatewayId}/stop`	Parar gateway	Desconecta todos os drivers
`{TenantId}/gateways/{GatewayId}/startprocess`	Iniciar processo	Inicia script Lua para carregamento

Tópico	Operação	Descrição
`{TenantId}/gateways/{GatewayId}/stopprocess`	Parar processo	Para processo em execução
`{TenantId}/gateways/{GatewayId}/starttelemetry`	Iniciar telemetria	Inicia coleta periódica de telemetria
`{TenantId}/gateways/{GatewayId}/request`	RPC Request	Chamada remota de métodos do driver
`{TenantId}/gateways/{GatewayId}/init`	Inicializar	Recebe configuração de dispositivos

Reconexão: O Edgegateway implementa reconexão automática ao broker MQTT com intervalo configurável via `GATEWAY_RECONNECT_INTERVAL`.

4.3. TCP/RTU — Preset Loaders

Os preset loaders utilizam protocolos proprietários sobre TCP ou Serial (RTU).

Protocolo Accuload (Accuload 3 / Accuload 4)

O protocolo Accuload utiliza frames com a seguinte estrutura:

```
┌──────┬──────────┬──────────┬──────┬──────┐
│ STX  │ Address  │ Content  │ ETX  │ LRC  │
│ 0x02 │ 2 bytes  │ N bytes  │ 0x03 │ 1byte│
└──────┴──────────┴──────────┴──────┴──────┘
```

Campo	Tamanho	Descrição
STX	1 byte (0x02)	Start of Text — início do frame
Address	2 bytes ASCII	Endereço interno do preset (ex: "01")
Content	N bytes ASCII	Comando ou resposta (ex: "EQ", "RK", "WD")
ETX	1 byte (0x03)	End of Text — fim do conteúdo
LRC	1 byte	Longitudinal Redundancy Check — XOR de todos os bytes entre STX e ETX (exclusive)

Resposta: A resposta segue o mesmo formato, com um byte adicional `0x7F` (DEL) no final do frame como terminador.

Comandos comuns:

Comando	Descrição
`EQ`	Consulta de status (Equipment Query)
`RK`	Leitura do teclado (Read Keyboard)
`WD`	Escrita no display (Write Display)

Exemplo de comunicação:

```
Request:  [0x02] "01" "EQ" [0x03] [LRC]
Response: [0x00] [0x02] "01" "<status_data>" [0x03] [LRC] [0x7F]
```

Fusion4 (Modbus sobre TCP)

O Fusion4 utiliza protocolo Modbus sobre conexão TCP para comunicação com os loaders MSCL.

TCC (Protocolo de Mensagens)

O TCC utiliza um protocolo de mensagens proprietário com frames estruturados.

4.4. OPC UA — Variáveis Industriais

O driver OPC suporta dois modos de conexão:

Modo	Formato da URL	Exemplo
OPC UA	`opc.tcp://<host>:<port>`	`opc.tcp://localhost:4840`

Fluxo de comunicação:

1. Conexão ao servidor OPC via URL configurada
2. Leitura de variáveis utilizando `NodeId` (ex: `ns=2;s=VariableNodeId`)
3. Escrita de variáveis via `SetVariable` / `SetVariables`
4. Coleta periódica de telemetria de todas as variáveis configuradas

4.5. ModBus TCP — Variáveis Industriais

O driver ModBus utiliza uma implementação customizada para comunicação ModBus TCP.

Tipos de dados suportados:

Tipo	Registros	Descrição
`ushort`	1 holding register	Inteiro sem sinal 16-bit
`short`	1 holding register	Inteiro com sinal 16-bit
`uint`	2 holding registers	Inteiro sem sinal 32-bit
`int`	2 holding registers	Inteiro com sinal 32-bit
`bool`	1 coil	Booleano (com posição de bit)
`float-little`	2 holding registers	Float 32-bit (little-endian)
`float-big`	2 holding registers	Float 32-bit (big-endian)
`string`	N holding registers	String ASCII (tamanho configurável)

> **VALIDAÇÃO**
>
> Os tipos de dados suportados foram definidos no `AutoloadV4.EdgeGateway.Drivers.Modbus` dentro da função `GetVariable()`

4.6. Serial RS-232 — Accuload 3, Balanças

Dispositivos que suportam comunicação serial utilizam os seguintes parâmetros configuráveis:

Parâmetro	Tipo	Descrição
`port`	string	Porta serial (ex: `COM3`, `/dev/ttyUSB0`)
`baudRate`	int	Taxa de transmissão (ex: 9600, 19200)
`parity`	Parity enum	Paridade (None, Odd, Even)
`dataBits`	int	Bits de dados (7 ou 8)
`stopBits`	StopBits enum	Bits de parada (One, Two)

4.7. TCP Direto — Display Spider, UCA 330

Os drivers Display Spider e UCA 330 utilizam conexão TCP direta:

- **Display Spider**: Comandos enviados em formato binário (byte arrays) com mensagens formatadas pelo `SpiderMessageHelper`
- **UCA 330**: Protocolo baseado em comandos de texto (ex: `D` para limpar display, `G` para posicionar texto, `R` para desenhar retângulo)

> **GLOSSÁRIO:**
>
> - Display Spider: Painéis eletrônicos de LED para sinalização, horários e mensagens
> - UCA 330: Unidade de Controle de Acesso inteligente desenvolvida para identificar veículos e usuários

4.8. Thread Safety

Todos os drivers utilizam `SemaphoreSlim` para garantir acesso serializado ao stream de comunicação:

```
private readonly SemaphoreSlim _semaphore = new(1, 1);

protected virtual async Task<string> PresetRequest(string request, int arm)
{
    await _semaphore.WaitAsync();
    try
    {
        EnsureConnected();
        // Operação de I/O protegida
        await _stream.WriteAsync(buffer);
        return await ReadResponseAsync();
    }
    finally
    {
        _semaphore.Release();  // SEMPRE liberar no finally
    }
}
```

4.9. Reconexão Automática

Todos os drivers implementam o padrão `EnsureConnected()` para reconexão transparente:

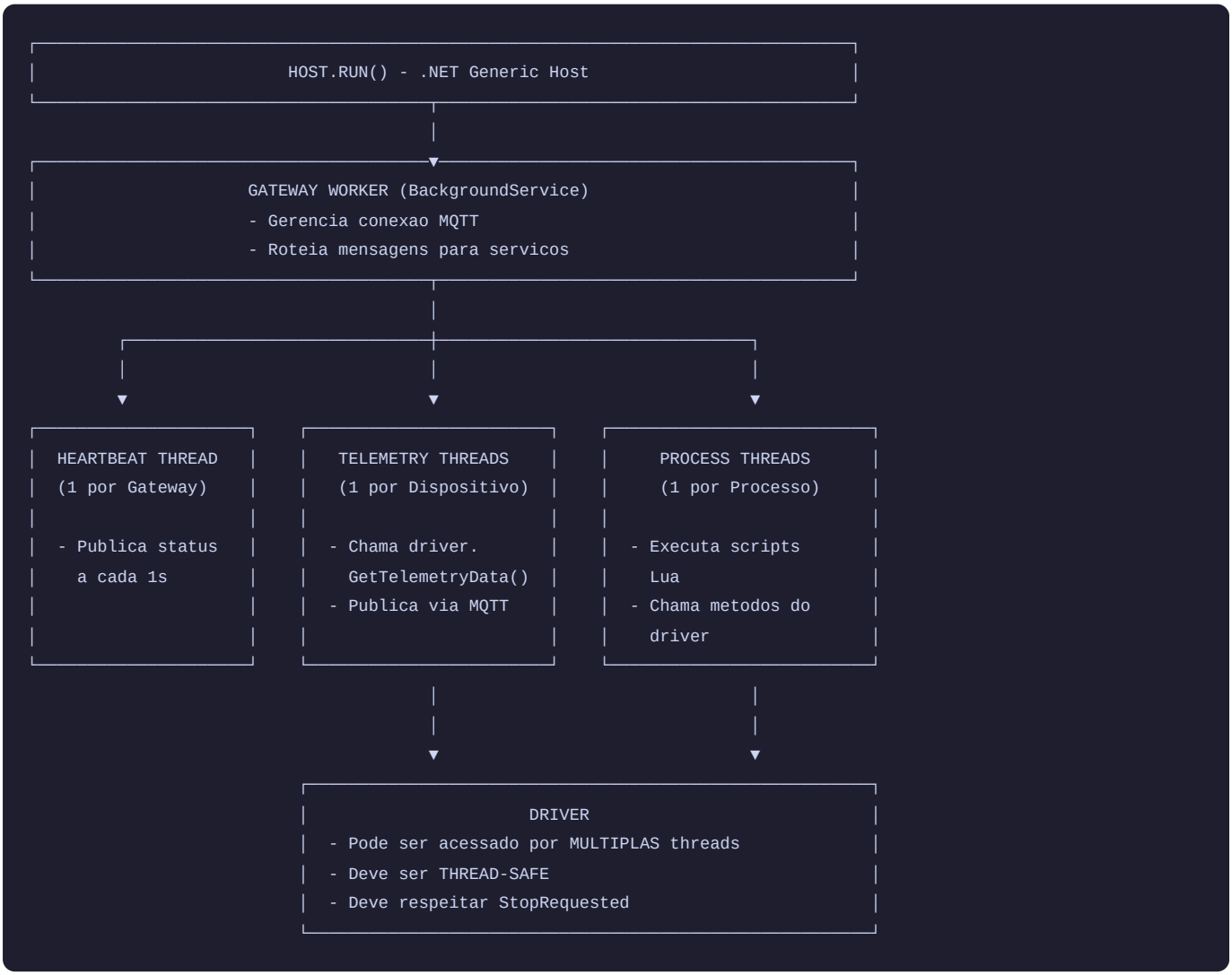
```
private void EnsureConnected()
{
    if (_client is null || !_client.Connected)
    {
        Logger.LogInformation("Device is not connected. Reconnecting...");
        Disconnect();
        Connect();
    }
}
```

Este método é chamado antes de cada operação de I/O, garantindo que a conexão seja restabelecida automaticamente em caso de queda.

---

5. Arquitetura de Threads do EdgeGateway

5.1 Hieraquia de Threads



5.2 Contextos de Chamada ao Driver

Um driver pode ser chamado em diferentes contextos:

Contexto	Thread	Metodos Tipicos
Telemetria	TelemetryThread (1 por device)	`GetTelemetryData()`
Processo	ProcessThread (1 por processo)	`WaitLoading()`, `AuthorizeLoading()`, `FinalizeLoading()`
Lifecycle	GatewayWorker	`OnGatewayStart()`, `OnGatewayStop()`, `OnProcessStart()`, `OnProcessStop()`
RPC	Task.Run (por requisicao)	Qualquer metodo publico via reflexao

---

6. Configurações dos Drivers

6.1. DeviceDrivers.json — Registro Centralizado

O arquivo `DeviceDrivers.json` localizado em `AutoloadV4.Domain.Operations/Resources/` registra todos os drivers disponíveis no sistema. Cada entrada contém:

```
{
  "PresetAccuload4": {
    "Name": "Preset Accuload 4",
    "Type": "Preset",
    "Permission": "devices_loading_control_management",
    "DriverType": "AutoloadV4.EdgeGateway.Drivers.Preset.DriverAccuload4",
    "DriverTypeAssembly": "AutoloadV4.EdgeGateway.Drivers.Preset"
  },
  "OPCClient": {
    "Name": "OPC Client",
    "Type": "OPC",
    "Permission": "device_opc_management",
    "DriverType": "AutoloadV4.EdgeGateway.Drivers.OPC.DriverOPCClient",
    "DriverTypeAssembly": "AutoloadV4.EdgeGateway.Drivers.OPC"
  }
}
```

Campo	Descrição
Name	Nome de exibição do driver
Type	Categoria do driver (`Preset` , `OPC` , `Modbus` , `Display` , `TruckScale` , `UCA`)
Permission	Permissão necessária para gerenciar dispositivos deste tipo
DriverType	Fully qualified name da classe (usado via reflection para instanciação)
DriverTypeAssembly	Nome do assembly que contém a classe do driver

Estrutura completa do `DeviceDrivers.json`:

```
{
  "DisplaySpider":     { "Name": "Display Spider",      "Type": "Display",     "Permission": "device_display_management"
  "ModbusClient":      { "Name": "Modbus Client",       "Type": "Modbus",      "Permission": "device_modbus_management",
```

```
  "ModbusClientMock":  { "Name": "Modbus Client Mock",  "Type": "Modbus",      "Permission": "device_modbus_management",
  "OPCClient":         { "Name": "OPC Client",          "Type": "OPC",         "Permission": "device_opc_management",
  "OPCClientMock":     { "Name": "OPC Client Mock",     "Type": "OPC",         "Permission": "device_opc_management",
  "PresetAccuload3":   { "Name": "Preset Accuload 3",   "Type": "Preset",      "Permission": "devices_loading_control_ma
  "PresetAccuload4":   { "Name": "Preset Accuload 4",   "Type": "Preset",      "Permission": "devices_loading_control_ma
  "PresetDanload6000": { "Name": "Preset Danload 6000", "Type": "Preset",      "Permission": "devices_loading_control_ma
  "PresetDanload8000": { "Name": "Preset Danload 8000", "Type": "Preset",      "Permission": "devices_loading_control_ma
  "PresetFusion4":     { "Name": "Preset Fusion4",      "Type": "Preset",      "Permission": "devices_loading_control_ma
  "PresetMicroload":   { "Name": "Preset Microload",    "Type": "Preset",      "Permission": "devices_loading_control_ma
  "PresetTCC":         { "Name": "Preset TCC",          "Type": "Preset",      "Permission": "devices_loading_control_ma
  "PresetMock":        { "Name": "Preset Mock",         "Type": "Preset",      "Permission": "devices_loading_control_ma
  "TruckScaleToledo":  { "Name": "Truck Scale Toledo",  "Type": "TruckScale",  "Permission": "devices_truckscale_managem
  "UCAMock":           { "Name": "UCA Mock",            "Type": "UCA",         "Permission": "devices_uca_management",
  "UCAUnidata330":     { "Name": "UCA Unidata 330",     "Type": "UCA",         "Permission": "devices_uca_management",
}
```

6.2. Configuração por Driver

Cada dispositivo possui uma configuração JSON armazenada no banco de dados e injetada no driver via `GatewayDevice.Configuration`. Os parâmetros variam por tipo de driver.

Preset Accuload 4 (TCP)

```
{
  "name": "Accuload4-Bay01",
  "host": "192.168.1.100",
  "port": 10001,
  "timeout": 5000
}
```

Parâmetro	Tipo	Obrigatório	Descrição
`name`	string	Sim	Nome de identificação do preset
`host`	string	Sim	Endereço IP do equipamento
`port`	int	Sim	Porta TCP
`timeout`	int	Sim	Timeout de comunicação em milissegundos

Preset Accuload 3 (Serial/TCP)

```
{
  "name": "Accuload3-Bay02",
  "protocol": "serial",
  "port": "COM3",
  "timeout": 5000,
  "baudRate": 9600,
  "parity": "None",
  "dataBits": 8,
```

```
  "stopBits": "One"
}
```

Ou em modo TCP:

```
{
  "name": "Accuload3-Bay02",
  "protocol": "network",
  "host": "192.168.1.101",
  "port": 10001,
  "timeout": 5000
}
```

Parâmetro	Tipo	Obrigatório	Descrição
`name`	string	Sim	Nome de identificação do preset
`protocol`	string	Sim	Modo de conexão: `"serial"` ou `"network"`
`host`	string	Se network	Endereço IP do equipamento
`port`	string/int	Sim	Porta serial (ex: `"COM3"`) ou porta TCP
`timeout`	int	Sim	Timeout de comunicação em milissegundos
`baudRate`	int	Se serial	Taxa de transmissão (ex: 9600)
`parity`	string	Se serial	Paridade: `"None"`, `"Odd"`, `"Even"`
`dataBits`	int	Se serial	Bits de dados (7 ou 8)
`stopBits`	string	Se serial	Bits de parada: `"One"`, `"Two"`

ModBus Client

```
{
  "host": "192.168.1.200",
  "port": 502,
  "timeout": 60000,
  "variables": {
    "Temperature": {
      "register": "100",
      "type": "float-big",
      "position": 0,
      "length": 2
    },
    "PumpStatus": {
      "register": "200",
      "type": "bool",
      "position": 0,
      "length": 1
    },
    "TankLevel": {
      "register": "300",
```

```
      "type": "uint",
      "position": 0,
      "length": 2
    }
  }
}
```

Parâmetro	Tipo	Obrigatório	Descrição
`host`	string	Sim	Endereço IP do dispositivo ModBus
`port`	int	Sim	Porta TCP (padrão ModBus: 502)
`timeout`	int	Não	Timeout em ms (padrão: 60000)
`variables`	object	Sim	Mapa de variáveis com configuração individual
`variables.*.register`	string	Sim	Endereço do registrador ModBus
`variables.*.type`	string	Sim	Tipo de dado: `ushort`, `short`, `uint`, `int`, `bool`, `float-little`, `float-big`, `string`
`variables.*.position`	int	Sim	Posição do bit (para tipo `bool`)
`variables.*.length`	int	Sim	Número de registradores (para `string` e `float`)

OPC Client

```
{
  "url": "opc.tcp://192.168.1.50:4840",
  "variables": [
    {
      "Name": "Temperature",
      "NodeId": "ns=2;s=Channel1.Device1.Temperature"
    },
    {
      "Name": "Pressure",
      "NodeId": "ns=2;s=Channel1.Device1.Pressure"
    }
  ]
}
```

Parâmetro	Tipo	Obrigatório	Descrição
`url`	string	Sim	URL do servidor OPC UA (`opc.tcp://`)
`variables`	array	Sim	Lista de variáveis a monitorar
`variables[].Name`	string	Sim	Nome da variável (identificador no sistema)
`variables[].NodeId`	string	Sim	Node ID no servidor OPC (ex: `ns=2;s=...`)

Display Spider

```
{
  "host": "192.168.1.150",
  "port": 3000
}
```

Parâmetro	Tipo	Obrigatório	Descrição
`host`	string	Sim	Endereço IP do display Spider
`port`	int	Sim	Porta TCP do display

TruckScale Toledo

```
{
  "name": "Balanca-01",
  "host": "192.168.1.180",
  "port": "5001",
  "connectionType": "TCP",
  "concatWeightStabilized": ")0",
  "concatWeightNotStabilized": ")8",
  "truckScaleModel": "IND560",
  "timeout": 5000
}
```

Ou em modo Serial:

```
{
  "name": "Balanca-01",
  "host": "localhost",
  "port": "COM4",
  "connectionType": "Serial",
  "concatWeightStabilized": "i0",
  "concatWeightNotStabilized": "i8",
  "truckScaleModel": "8540",
  "timeout": 5000,
  "baudRate": 9600,
  "parity": "None",
  "dataBits": 8,
  "stopBits": "One"
}
```

Parâmetro	Tipo	Obrigatório	Descrição
`name`	string	Sim	Nome de identificação da balança
`host`	string	Sim	Endereço IP (TCP) ou hostname (Serial)
`port`	string	Sim	Porta TCP ou porta serial (ex: `"COM4"`)
`connectionType`	string	Sim	Tipo de conexão: `"TCP"` ou `"Serial"`

Parâmetro	Tipo	Obrigatório	Descrição
`concatWeightStabilized`	string	Sim	Prefixo do peso estabilizado (modelo-dependente: `")0"` para IND560, `"i0"` para 8540)
`concatWeightNotStabilized`	string	Sim	Prefixo do peso não estabilizado (modelo-dependente: `")8"` para IND560, `"i8"` para 8540)
`truckScaleModel`	string	Sim	Modelo da balança (`"IND560"`, `"8540"`)
`timeout`	int	Sim	Timeout de comunicação em milissegundos
`baudRate`	int	Se Serial	Taxa de transmissão
`parity`	string	Se Serial	Paridade
`dataBits`	int	Se Serial	Bits de dados
`stopBits`	string	Se Serial	Bits de parada

UCA Unidata 330

```
{
  "name": "UCA-Portaria01",
  "host": "192.168.1.170",
  "port": 4001,
  "charactersPerLine": 20,
  "linesPerMessage": 4,
  "especialMessageLine": 1,
  "firstMessageLine": 4,
  "topLeftBerderColumn": 0,
  "topLeftBorderLine": 0,
  "bottomRightBorderColumn": 39,
  "bottomRightBorderLine": 15,
  "passwordfieldLength": 6,
  "passwordFieldFlag": "S",
  "finishWithSizeFlag": "S",
  "alphanumericFlag": "N",
  "bigSourceFlag": "S",
  "simultaneousReadingFlag": "N"
}
```

Parâmetro	Tipo	Padrão	Descrição
`name`	string	—	Nome de identificação da UCA
`host`	string	—	Endereço IP da UCA
`port`	int	—	Porta TCP da UCA
`charactersPerLine`	int	20	Caracteres por linha do display
`linesPerMessage`	int	4	Linhas por mensagem

Parâmetro	Tipo	Padrão	Descrição
`especialMessageLine`	int	1	Linha para mensagem especial (cabeçalho)
`firstMessageLine`	int	4	Primeira linha do conteúdo
`topLeftBerderColumn`	int	0	Coluna superior esquerda da borda
`topLeftBorderLine`	int	0	Linha superior esquerda da borda
`bottomRightBorderColumn`	int	39	Coluna inferior direita da borda
`bottomRightBorderLine`	int	15	Linha inferior direita da borda
`passwordfieldLength`	int	6	Tamanho do campo de senha
`passwordFieldFlag`	string	"S"	Campo de senha: `"S"` (sim) ou `"N"` (não)
`finishWithSizeFlag`	string	"S"	Finalizar com tamanho: `"S"` ou `"N"`
`alphanumericFlag`	string	"N"	Alfanumérico: `"S"` (sim) ou `"N"` (numérico)
`bigSourceFlag`	string	"S"	Fonte grande: `"S"` ou `"N"` (pequena)
`simultaneousReadingFlag`	string	"N"	Leitura simultânea: `"S"` ou `"N"`

6.3. Variáveis de Ambiente do EdgeGateway

O Edgegateway utiliza variáveis de ambiente para configuração global, definidas no `appsettings.json`:

```
{
  "EdgeGateway": {
    "Id": "#{GATEWAY_ID}#",
    "TenantId": "#{TENANT_ID}#",
    "RpcPrefix": "#{RPC_PREFIX}#",
    "ReconnectInterval": "#{GATEWAY_RECONNECT_INTERVAL}#",
    "PoolingInterval": "#{GATEWAY_POOLING_INTERVAL}#"
  },
  "MQTT": {
    "Server": "#{MQTT_SERVER}#",
    "Port": "#{MQTT_PORT}#",
    "User": "#{MQTT_USER}#",
    "Password": "#{MQTT_PASSWORD}#",
    "Protocol": "#{MQTT_PROTOCOL}#",
    "UseTls": "#{MQTT_USE_TLS}#",
    "WebSocketPath": "#{MQTT_WEB_SOCKET_PATH}#",
    "ReconnectInterval": "#{MQTT_RECONNECT_INTERVAL}#"
  }
}
```

Variável	Descrição
`GATEWAY_ID`	Identificador único do gateway

Variável	Descrição
`TENANT_ID`	ID do tenant no sistema
`MQTT_SERVER`	Endereço do broker MQTT
`MQTT_PORT`	Porta do broker (padrão: 8883 para TLS)
`MQTT_USER`	Usuário de autenticação MQTT
`MQTT_PASSWORD`	Senha de autenticação MQTT
`GATEWAY_RECONNECT_INTERVAL`	Intervalo de reconexão em milissegundos
`GATEWAY_POLLING_INTERVAL`	Intervalo de polling de telemetria em milissegundos

6.4. Permissões por Tipo de Driver

Cada tipo de driver exige uma permissão específica no sistema para gerenciamento de dispositivos:

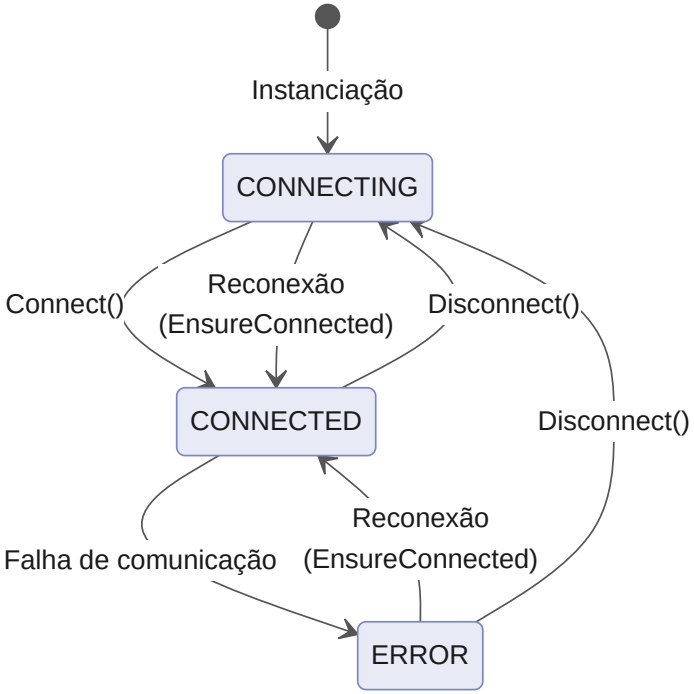
Tipo	Permissão	Drivers
Preset	`devices_loading_control_management`	Accuload 3/4, Danload 6000/8000, Fusion4, Microload, TCC, Mock
OPC	`device_opc_management`	OPC Client, OPC Client Mock
Modbus	`device_modbus_management`	Modbus Client, Modbus Client Mock
Display	`device_display_management`	Display Spider
TruckScale	`devices_truckscale_management`	Truck Scale Toledo
UCA	`devices_uca_management`	UCA Unidata 330, UCA Mock

---

7. Ciclo de Vida dos Drivers

7.1. Status do Driver

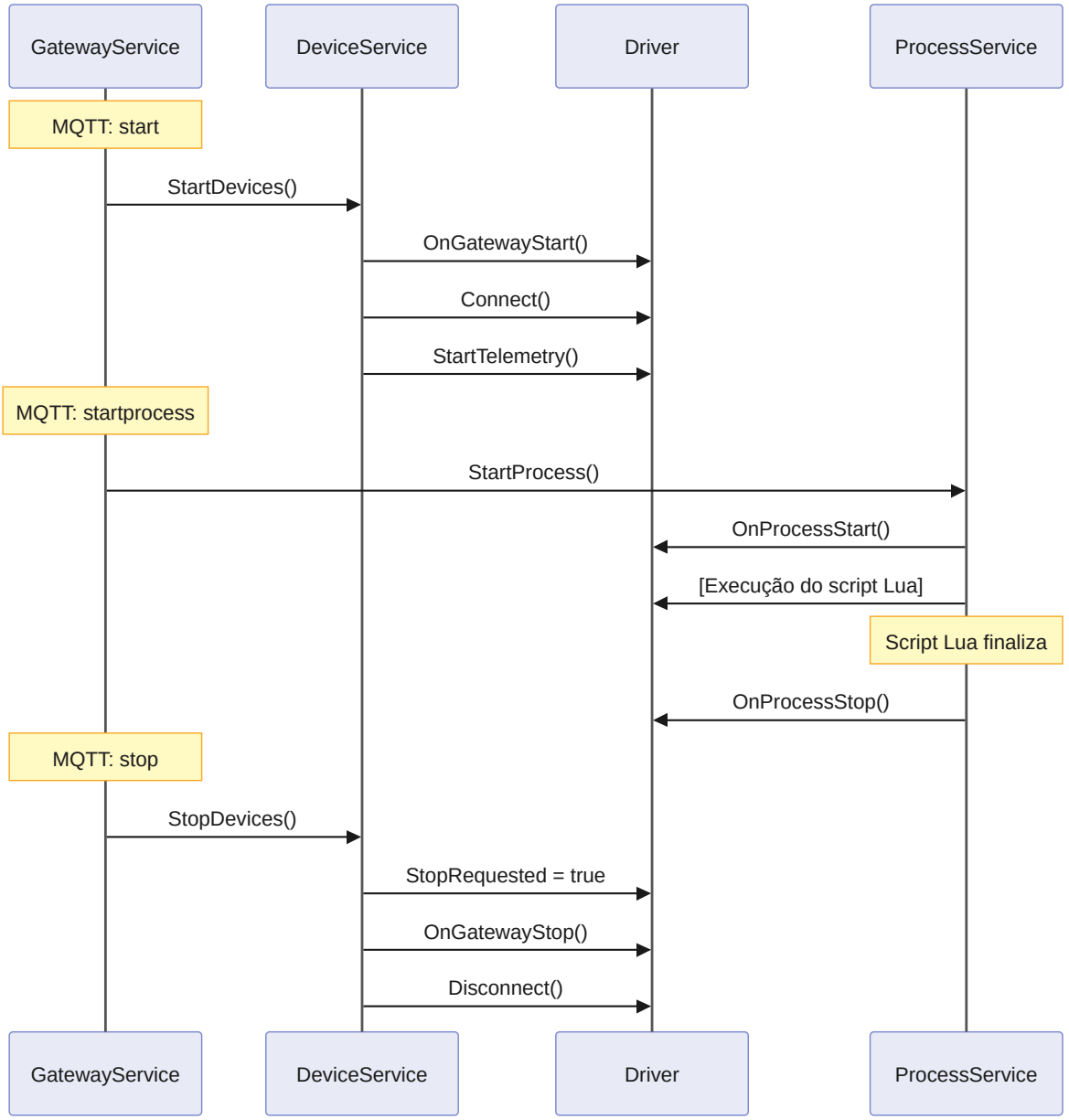
Os drivers possuem três estados possíveis, definidos pelo enum `DeviceStatus`:



Status	Valor	Descrição
`CONNECTING`	0	Estado inicial. Driver instanciado mas não conectado ao equipamento.
`CONNECTED`	1	Conexão ativa com o equipamento. Operações disponíveis.
`ERROR`	4	Falha de comunicação. Reconexão será tentada automaticamente.

7.2. Hooks de Lifecycle

Os drivers possuem hooks que são invocados em momentos específicos do ciclo de vida:



Hook	Quando é chamado	Uso típico
`OnGatewayStart()`	Quando o gateway inicia	Carregar configuração, inicializar estado
`OnGatewayStop()`	Quando o gateway para	Fechar conexões, liberar recursos
`OnProcessStart()`	Quando um processo que usa o driver inicia	Resetar flags, preparar para nova operação
`OnProcessStop()`	Quando um processo que usa o driver finaliza	Limpar estado do processo, exibir mensagem offline

7.3. Padrão EnsureConnected

O padrão `EnsureConnected()` é implementado em todos os drivers para garantir reconexão automática transparente antes de cada operação:

```
private void EnsureConnected()
{
    if (_client is null || !_client.Connected)
    {
        Logger.LogInformation("{Driver}: Connection lost, reconnecting...", GetType().Name);
        Disconnect();
        Connect();
    }
}
```

7.4. Cancelamento Cooperativo (StopRequested)

O EdgeGateway usa **cancelamento cooperativo**. Quando uma parada é solicitada:

1. `Gateway.StopRequested = true` - Flag global
2. `driver.StopRequested = true` - Flag por driver
3. `process.StopRequested = true` - Flag por processo

> **ATENÇÃO**
>
> O Driver **DEVE** verificar `StopRequested` e encerrar operações graciosamente.

```
MQTT: stop ──► GatewayService.StopGateway()
                    │
                    ├──► Gateway.StopRequested = true
                    │
                    ├──► Aguarda TelemetrySessions terminarem
                    │         └──► Dependem de GetTelemetryData() retornar
                    │
                    ├──► driver.StopRequested = true (para cada driver)
                    │
                    ├──► process.StopRequested = true (para cada processo)
                    │
                    ├──► Aguarda todas as Tasks de processo terminarem
                    │         └──► Dependem de métodos do driver retornarem
                    │
                    └──► driver.OnGatewayStop() (para cada driver)
```

Todo loop e operação de espera no driver deve verificar `StopRequested`:

```
while (condicao && !StopRequested)
{
    await Task.Delay(1000);
    // operação
}
```

7.5. Telemetria

A telemetria é coletada periodicamente pelo `TelemetryService` em threads dedicadas (uma por dispositivo).

`GetTelemetry()` — Método da classe base `Driver` que monta a resposta de telemetria:

```
public virtual async Task<TelemetryResponse> GetTelemetry()
{
    var telemetry = new TelemetryResponse()
    {
        DeviceId = Device?.Id ?? "Unknown",
        DeviceName = Device?.Name ?? "Unknown",
        DeviceStatus = Status,
    };
    try
    {
        telemetry.Data = await GetTelemetryData();
    }
    catch (Exception exception)
    {
        telemetry.Error = exception.Message;
    }
    return telemetry;
}
```

`GetTelemetryData()` — Método virtual sobrescrito por cada driver para coletar dados específicos do equipamento.

> **OBS:**
>
> Deve ser rápido e não bloquear o sistema.

---

8. Drivers Mock para Testes

O Autoload V4 inclui drivers mock e simuladores para permitir desenvolvimento e testes sem hardware físico.

8.1. Drivers Mock Disponíveis

Driver Mock	Categoria	Descrição
`DriverMock`	Preset	Simula operações de preset loader (autorização, carregamento, finalização)
`DriverModBusMock`	Modbus	Simula leitura/escrita de variáveis ModBus
`DriverOPCClientMock`	OPC	Simula leitura/escrita de variáveis OPC
`DriverUCAMock`	UCA	Simula operações da unidade de controle de acesso

8.2. Simuladores Standalone

Para testes mais realistas, o projeto inclui simuladores que executam como aplicações standalone:

Simulador	Porta Padrão	Comando
OPC UA Server	4840	`dotnet run --project AutoloadV4.OPCUAServerSimulator.Console`
TruckScale (Balança)	—	`dotnet run --project AutoloadV4.TruckScaleSimulator`

Simulador	Porta Padrão	Comando
UCA 330	—	`dotnet run --project AutoloadV4.UCASimulator.Console`

8.3. Uso em Desenvolvimento

Para utilizar drivers mock, basta configurar o dispositivo no sistema apontando para o driver mock correspondente (ex: `PresetMock` em vez de `PresetAccuload4`). A configuração do dispositivo pode ser mínima ou vazia, já que drivers mock não requerem conexão real.

---

9. Conclusão

Os Drivers do **Autoload V4** constituem a camada fundamental de integração entre o sistema e os equipamentos industriais de campo. Através de uma arquitetura modular e extensível — baseada em classes abstratas, interfaces bem definidas e exposição via Lua scripting — o EdgeGateway consegue suportar uma ampla variedade de protocolos e dispositivos, desde preset loaders (Accuload, Danload, Fusion4, Microload, TCC) até balanças Toledo, displays Spider, unidades UCA 330 e servidores OPC UA e ModBus.

O ciclo de vida padronizado com hooks, reconexão automática (`EnsureConnected`) e cancelamento cooperativo (`StopRequested`) garante robustez e resiliência na operação contínua. Complementarmente, a disponibilidade de drivers mock e simuladores standalone viabiliza o desenvolvimento e testes independentes de hardware físico, acelerando o ciclo de entrega e reduzindo custos de validação.

Documento atualizado em: 05 de março de 2026
Versão: 1.0
Autor: Milton Kiefer de Albuquerque Mello
Validado por: ---
Última atualização tecnológica: .NET 8.0 / Next.js 14 / React 18

VI. Banco de Dados

Modelo de dados, schemas e estratégia de armazenamento.

1. Visão Geral do Banco de Dados

1.1. Arquitetura Dual-Context

O Autoload V4 utiliza dois `DbContext` do Entity Framework Core separados para implementar multi-tenancy:

Contexto	Banco de Dados	Escopo	Tabelas
`AutoloadContext`	`autoload`	Global (compartilhado)	9
`TenantContext`	`autoload_[TENANT_ID]`	Por tenant (isolado)	83
Total			**92**

1.2. Convenção de Nomes de Tabelas

As tabelas do `TenantContext` utilizam prefixos para organização por domínio:

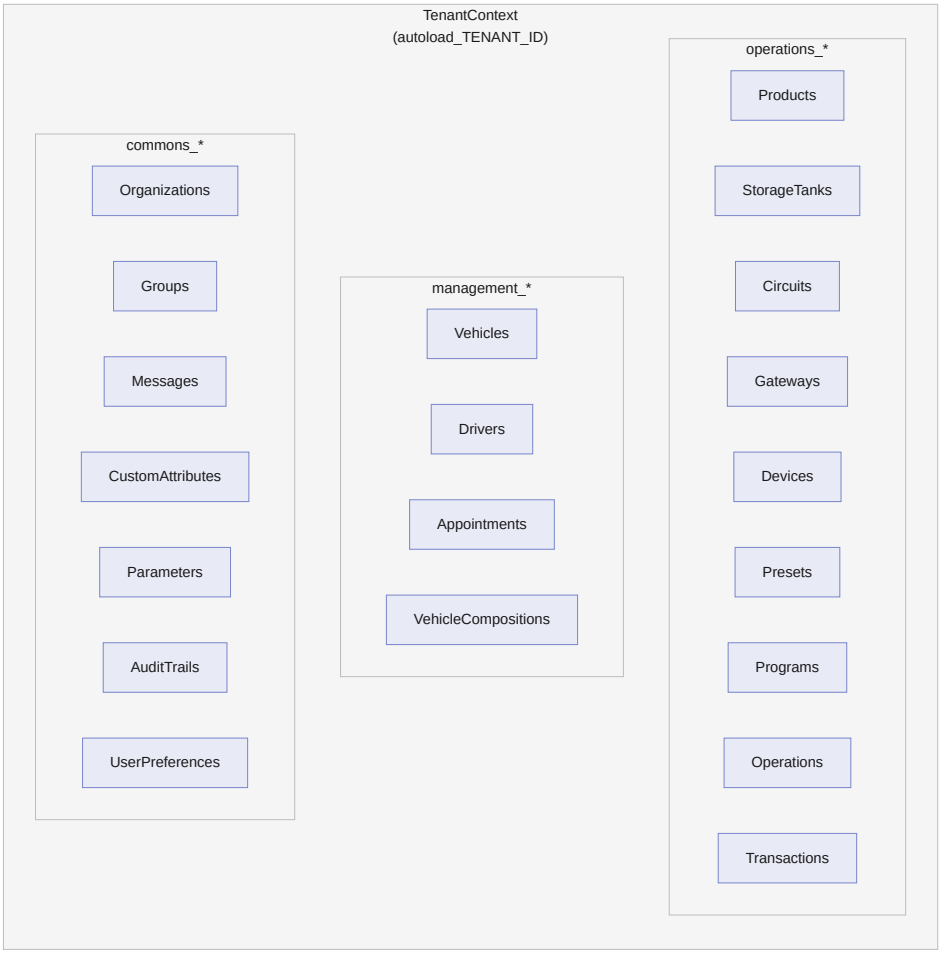
Domínio	Prefixo	Tabelas	Descrição
Commons	`commons_`	13	Entidades compartilhadas entre módulos
Management	`management_`	18	Gestão de frota e agendamentos
Operations	`operations_`	52	Operações de carregamento/descarregamento

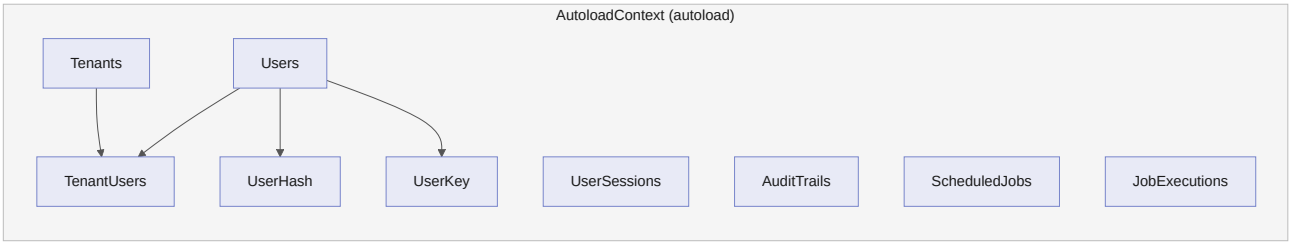
1.3. Providers Suportados

O sistema já foi *homologado* com diferentes providers de banco de dados, configurados via `Database:Provider`:

Provider	Valor Config	Connection String
PostgreSQL	`postgresql`	`Server={server};Port={port};User Id={user};Password={password};Database={tenantId};`
SQL Server	`sqlserver`	`Server={server};Database={tenantId};User Id={user};Password={password};...`
SQL Server LocalDB	`localdb`	`Data Source=(localdb)\MSSQLLocalDB;Initial Catalog={tenantId}`
In-Memory (testes)	`inmemory`	N/A

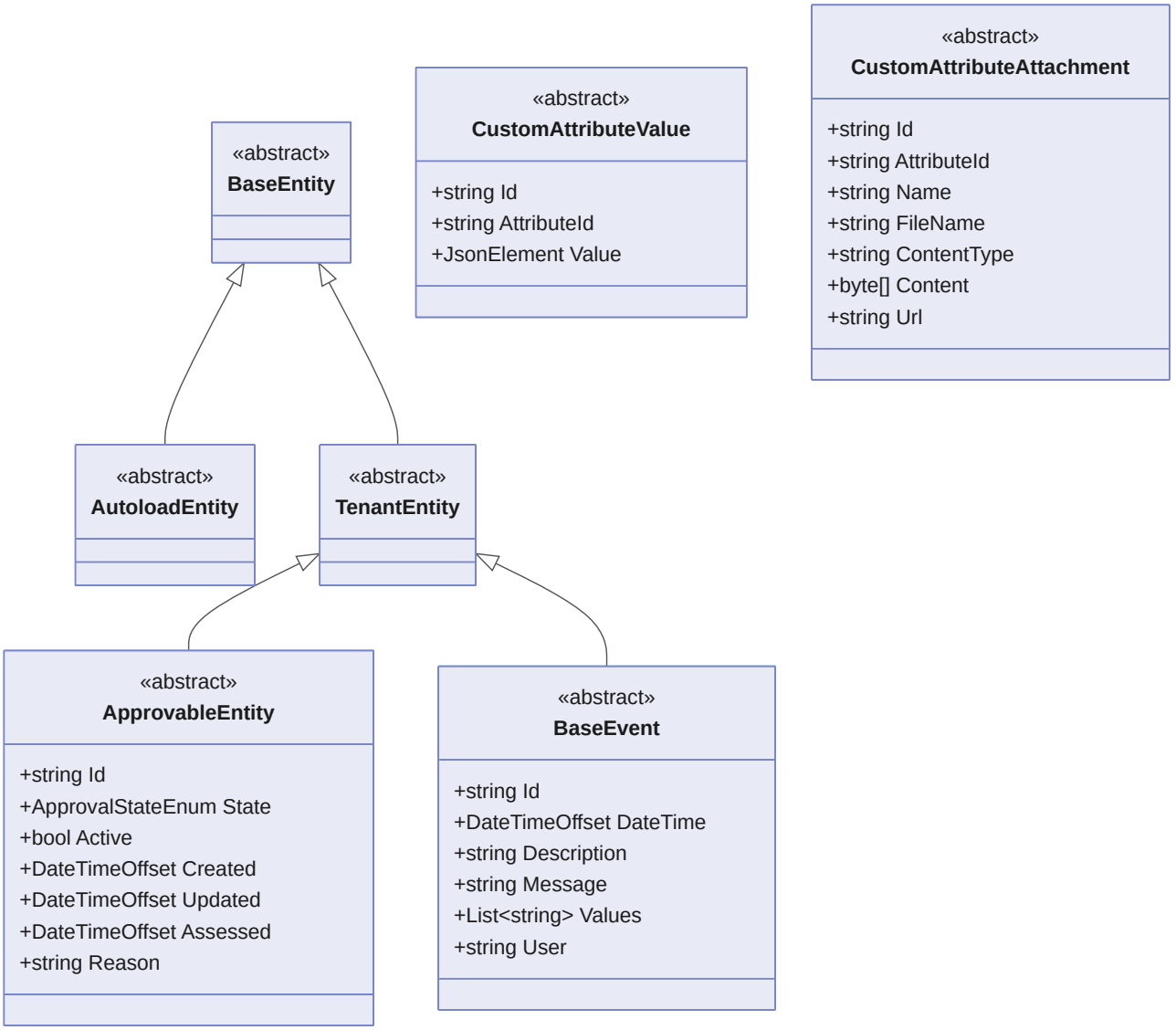
1.4. Diagrama de Alto Nível





2. Hierarquia de Entidades Base

2.1. Diagrama de Classes



2.2. Descrição das Classes Base

Classe	Herda de	Uso
`BaseEntity`	—	Raiz abstrata de todas as entidades persistidas
`AutoloadEntity`	`BaseEntity`	Entidades do `AutoloadContext` (banco global)
`TenantEntity`	`BaseEntity`	Entidades do `TenantContext` (banco por tenant)
`ApprovableEntity`	`TenantEntity`	Entidades com workflow de aprovação (Vehicle, Driver)
`BaseEvent`	`TenantEntity`	Eventos de auditoria de entidades (VehicleEvent, DriverEvent, etc.)
`CustomAttributeValue`	—	Valores de atributos customizáveis (abstract, sem herança de BaseEntity)
`CustomAttributeAttachment`	—	Anexos de atributos customizáveis (abstract, sem herança de BaseEntity)

2.3. Padrão Custom Attributes

O sistema suporta atributos dinâmicos em 5 tipos de entidades via o padrão `CustomAttributeValue` / `CustomAttributeAttachment`:

Entidade	Value Table	Attachment Table
Organization	`OrganizationAttributeValue`	`OrganizationAttributeAttachment`
Vehicle	`VehicleAttributeValue`	`VehicleAttributeAttachment`
Driver	`DriverAttributeValue`	`DriverAttributeAttachment`
Product	`ProductAttributeValue`	`ProductAttributeAttachment`
Device	`DeviceAttributeValue`	`DeviceAttributeAttachment`

---

3. Padrões Arquiteturais do Banco de Dados

3.1. Multi-Tenancy

- **AutoloadContext** (`autoload`): dados globais compartilhados entre todos os tenants (usuários, tenants, sessões, jobs)
- **TenantContext** (`autoload_[TENANT_ID]`): banco isolado por tenant com dados operacionais, de gestão e comuns

3.2. Chaves Compostas

O sistema utiliza extensivamente chaves compostas para entidades filhas:

Padrão	PK	Exemplo
Entidade filha	`(ParentId, Id)`	`ProgramEvent(ProgramId, Id)`
State tracking	`(ParentId, State, Start)`	`ProgramState(ProgramId, State, Start)`
Junction table	`(EntityAId, EntityBId)`	`TenantUser(TenantId, UserId)`
Componente	`(ParentId, Number)`	`VehicleCompartment(VehicleId, Number)`

3.3. JSON Storage

Campos complexos são armazenados como JSON (JSONB no PostgreSQL) via `HasConversion` :

Entidade	Campo	Tipo .NET	Descrição
AutoloadAuditTrail	PreviousValues / NewValues	`JsonElement`	Valores antes/depois da alteração
Parameter	Value	`JsonElement`	Valor do parâmetro (string, number, bool, json)
UserPreferences	Values	`JsonElement`	Preferências do usuário
Device	Configuration	`JsonElement`	Configuração do dispositivo

Entidade	Campo	Tipo .NET	Descrição
CustomAttributeValue	Value	`JsonElement`	Valor do atributo customizável
ProgramVariable	Value	`JsonElement`	Variável da programação
ChecklistQuestion	AnswerValue	`JsonElement`	Resposta do checklist
VehicleCompartment	Markers	`List<double>`	Marcadores volumétricos
AppointmentSlot	ClientIds, ProductIds, etc.	`List<string>`	Filtros do slot

3.4. State Machine

Quatro entidades utilizam tabelas de histórico de estado com chave composta `(EntityId, State, Start)`:

Entidade	Tabela de Estado	Enum
Program	`ProgramState`	`ProgramStateEnum`
Operation	`OperationState`	`ProgramOperationStateEnum`
Appointment	`AppointmentState`	`AppointmentStateEnum`
Manifest	`ManifestState`	`ProgramOperationManifestStateEnum`

3.5. Cascade Deletes

Estratégia	Uso	Exemplo
`CASCADE`	Filhas diretas obrigatórias	Program → ProgramEvent, ProgramState, ProgramStep
`RESTRICT`	Referências que não podem ser removidas	Circuit → Terminal (Organization)
`ClientCascade`	Cascade controlado pela aplicação	Operation → OperationTank
`SetNull`	FK anulada ao remover pai	AppointmentSlot → Appointment

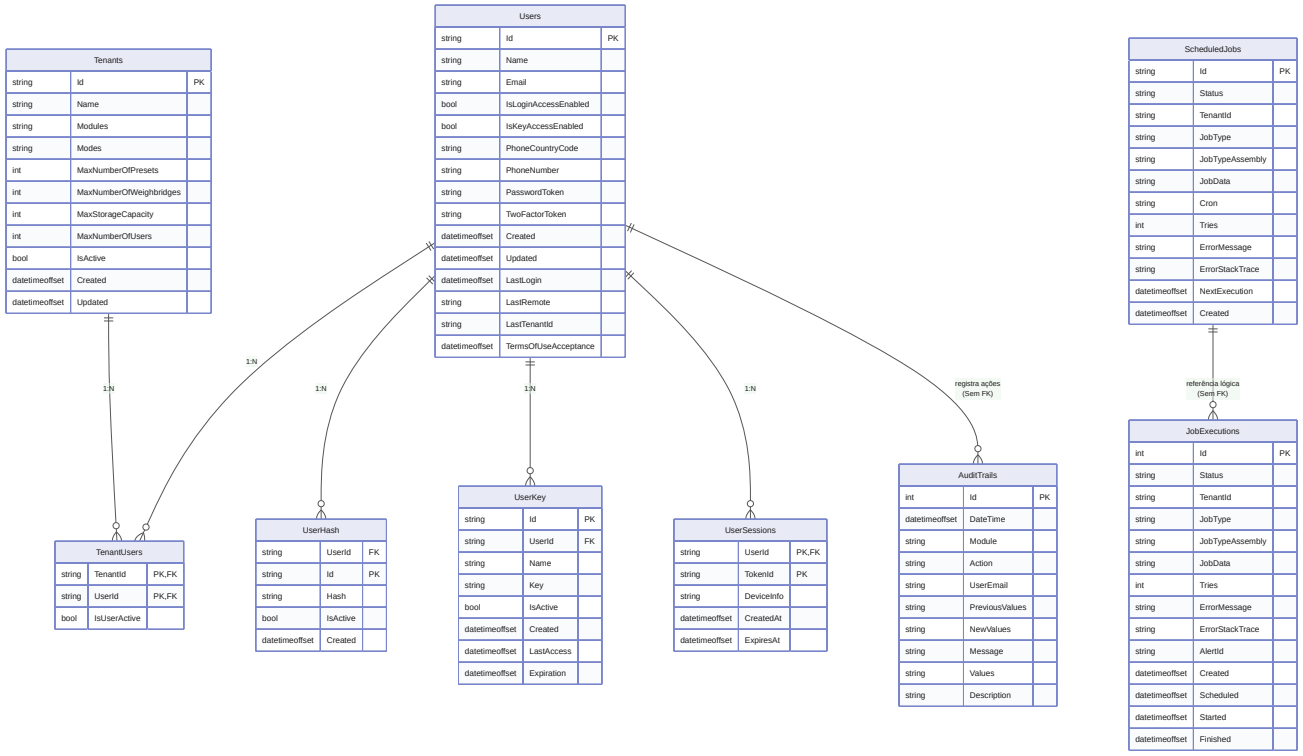
> **OBSERVAÇÃO:**
>
> **Cascade Deletes:** restrição de banco de dados SQL que remove automaticamente linhas filhas dependentes quando o registro pai correspondente é excluído

---

4. AutoloadContext — Banco de Dados Global

O `AutoloadContext` gerencia 9 tabelas no banco `autoload`, responsáveis por autenticação, autorização, auditoria e jobs agendados.

4.1. Diagrama ER



Confira o diagrama completo em numa nova aba: Abrir diagrama

> **OBSERVAÇÃO DO DIAGRAMA**
>
> **Relacionamento adicionado:** A coluna `UserEmail` em `AuditTrails` referencia logicamente um usuário do sistema, embora não exista FK física no banco de dados.
>
> **Relacionamento sem FK física:** A coluna `UserId` em `UserSessions` é parte da PK composta `(UserId, TokenId)`, mas não possui foreign key física para `Users.Id` no banco de dados. A integridade referencial é garantida apenas pela aplicação.

4.2. Tenants e TenantUsers

Tenants

Representa os tenants (terminais) do sistema. Cada tenant possui um banco de dados isolado.

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	N	**PK** — Identificador do tenant (ex: `autoload_terminal_01`)
`Name`	`string`	N	Nome de exibição do tenant
`Modules`	`List<string>`	N	Módulos habilitados para o tenant
`Modes`	`List<string>`	N	Modos de operação habilitados
`MaxNumberOfPresets`	`int?`	S	Limite de presets (licença)
`MaxNumberOfWeighbridges`	`int?`	S	Limite de balanças (licença)

Coluna	Tipo	Nulo	Descrição
`MaxStorageCapacity`	`int?`	S	Capacidade máxima de armazenamento (licença)
`MaxNumberOfUsers`	`int?`	S	Limite de usuários (licença)
`IsActive`	`bool`	N	Tenant ativo/inativo
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

TenantUsers

Tabela de junção N:N entre Tenants e Users.

Coluna	Tipo	Nulo	Descrição
`TenantId`	`string`	N	**PK/FK** → `Tenants.Id` (CASCADE)
`UserId`	`string`	N	**PK/FK** → `Users.Id` (CASCADE)
`IsUserActive`	`bool`	N	Usuário ativo neste tenant

4.3. Users, UserHash, UserKey

Users

Usuários globais do sistema, compartilhados entre tenants.

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK** — Identificador do usuário
`Name`	`string`	N	Nome completo
`Email`	`string`	N	Email (login)
`IsLoginAccessEnabled`	`bool`	N	Acesso via login habilitado
`IsKeyAccessEnabled`	`bool`	N	Acesso via API key habilitado
`PhoneCountryCode`	`string?`	S	Código do país do telefone
`PhoneNumber`	`string?`	S	Número de telefone
`PasswordToken`	`string?`	S	Token de reset de senha
`TwoFactorToken`	`string?`	S	Token de autenticação 2FA
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

Coluna	Tipo	Nulo	Descrição
`LastLogin`	`DateTimeOffset?`	S	Último login realizado
`LastRemote`	`string?`	S	Último IP de acesso
`LastTenantId`	`string?`	S	Último tenant acessado
`TermsOfUseAcceptance`	`DateTimeOffset?`	S	Data de aceite dos termos de uso

UserHash

Hashes de senha do usuário, permitindo histórico de senhas.

Coluna	Tipo	Nulo	Descrição
`UserId`	`string`	N	**PK/FK** → `Users.Id` (CASCADE)
`Id`	`string`	N	**PK** — Identificador do hash
`Hash`	`string`	N	Hash da senha (PBKDF2-SHA256)
`IsActive`	`bool`	N	Hash ativo (senha atual)
`Created`	`DateTimeOffset`	N	Data de criação

UserKey

Chaves de API para acesso programático.

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	N	**PK** — Identificador da chave
`UserId`	`string`	N	**FK** → `Users.Id`
`Name`	`string`	N	Nome descritivo da chave
`Key`	`string`	N	Valor da chave (hash)
`IsActive`	`bool`	N	Chave ativa/inativa
`Created`	`DateTimeOffset`	N	Data de criação
`LastAccess`	`DateTimeOffset?`	S	Último uso da chave
`Expiration`	`DateTimeOffset?`	S	Data de expiração

4.4. UserSessions

Gerencia sessões ativas dos usuários (JWT tokens).

Coluna	Tipo	Nulo	Descrição
`UserId`	`string`	N	**PK** — Identificador do usuário
`TokenId`	`string`	N	**PK** — Identificador do token JWT
`DeviceInfo`	`string`	N	Informações do dispositivo
`CreatedAt`	`DateTimeOffset`	N	Data de criação da sessão
`ExpiresAt`	`DateTimeOffset`	N	Data de expiração da sessão

4.5. AuditTrails (Global)

Trilha de auditoria do contexto global.

Coluna	Tipo	Nulo	Descrição
`Id`	`int`	N	**PK** — Auto-incremento
`DateTime`	`DateTimeOffset`	N	Data/hora do evento
`Module`	`string`	N	Módulo afetado
`Action`	`string`	N	Ação realizada (CREATE, UPDATE, DELETE)
`UserEmail`	`string?`	N	Email do usuário responsável
`PreviousValues`	`JsonElement`	N	Valores antes da alteração (JSON)
`NewValues`	`JsonElement`	N	Valores após a alteração (JSON)
`Message`	`string?`	S	Mensagem descritiva
`Values`	`List<string>?`	S	Valores adicionais
`Description`	`string?`	S	Descrição detalhada

4.6. ScheduledJobs e JobExecutions

ScheduledJobs

Jobs agendados com expressão cron para execução periódica.

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	N	**PK** — Identificador do job
`Status`	`ScheduledJobStatusEnum`	N	Status do job
`TenantId`	`string`	N	Tenant associado
`JobType`	`string`	N	Tipo (classe) do job

Coluna	Tipo	Nulo	Descrição
`JobTypeAssembly`	`string`	N	Assembly que contém o tipo
`JobData`	`string`	N	Dados serializados do job (JSON)
`Cron`	`string`	N	Expressão cron de agendamento
`Tries`	`int`	N	Número de tentativas (default: 1)
`ErrorMessage`	`string?`	S	Mensagem de erro da última execução
`ErrorStackTrace`	`string?`	S	Stack trace do erro
`NextExecution`	`DateTimeOffset`	N	Próxima execução agendada
`Created`	`DateTimeOffset`	N	Data de criação

JobExecutions

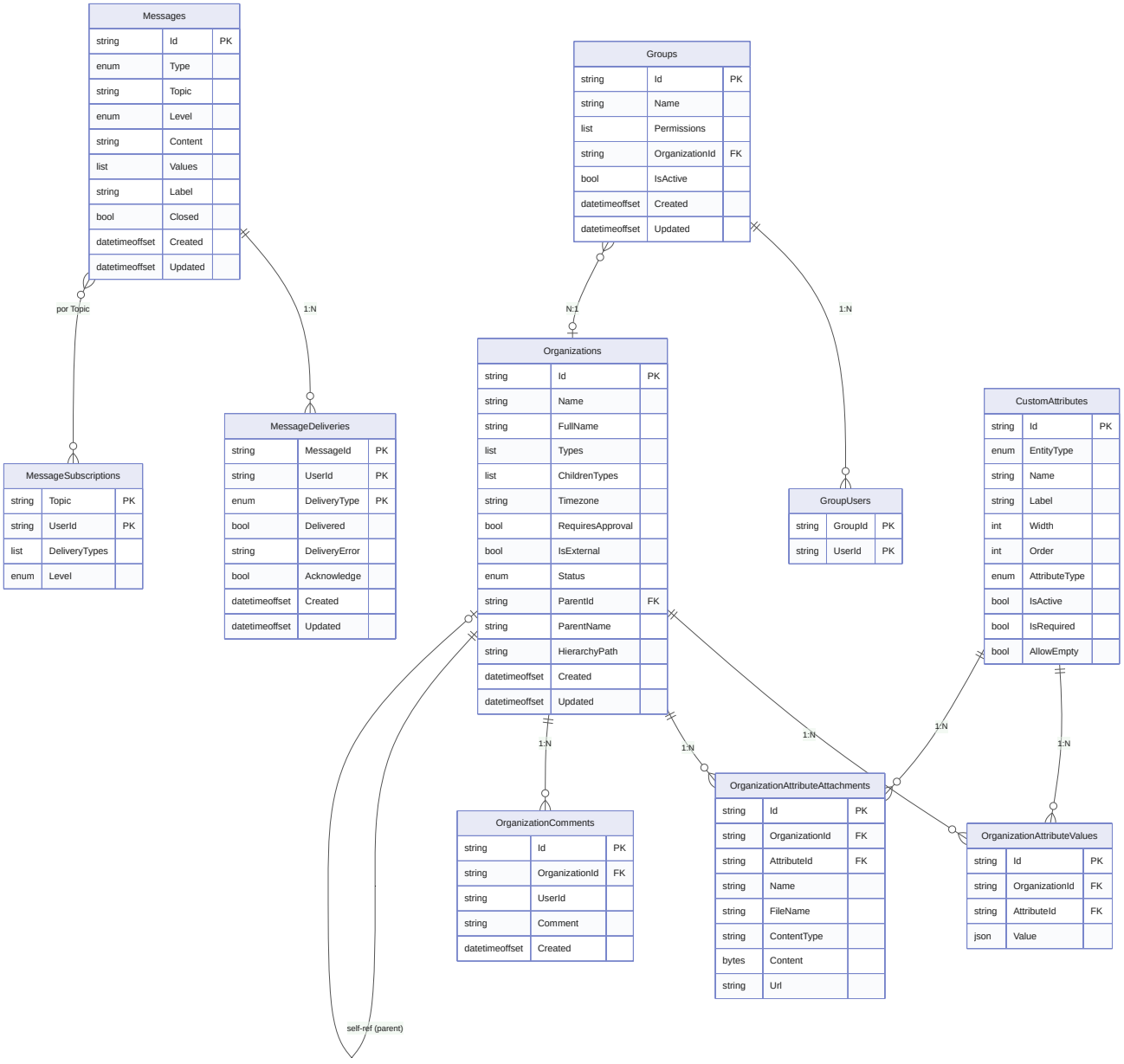
Execuções individuais de jobs (enfileirados ou disparados por schedule).

Coluna	Tipo	Nulo	Descrição
`Id`	`int`	N	**PK** — Auto-incremento
`Status`	`JobExecutionStatusEnum`	N	Status da execução
`TenantId`	`string`	N	Tenant associado
`JobType`	`string`	N	Tipo (classe) do job
`JobTypeAssembly`	`string`	N	Assembly que contém o tipo
`JobData`	`string`	N	Dados serializados do job (JSON)
`Tries`	`int`	N	Tentativas realizadas
`ErrorMessage`	`string?`	S	Mensagem de erro
`ErrorStackTrace`	`string?`	S	Stack trace do erro
`AlertId`	`string?`	S	ID do alerta gerado
`Created`	`DateTimeOffset`	N	Data de criação
`Scheduled`	`DateTimeOffset`	N	Data agendada para execução
`Started`	`DateTimeOffset?`	S	Data de início da execução
`Finished`	`DateTimeOffset?`	S	Data de término da execução

5. TenantContext — Domínio Commons (13 tabelas)

O domínio Commons contém entidades compartilhadas entre os módulos do sistema, como organizações, grupos, mensagens e parâmetros.

5.1. Diagrama ER



OBSERVAÇÃO DO DIAGRAMA

Relacionamento adicionado: Vínculo lógico entre `Messages.Topic` e `MessageSubscriptions.Topic`. Não existe FK física, mas o relacionamento reflete o comportamento da aplicação.

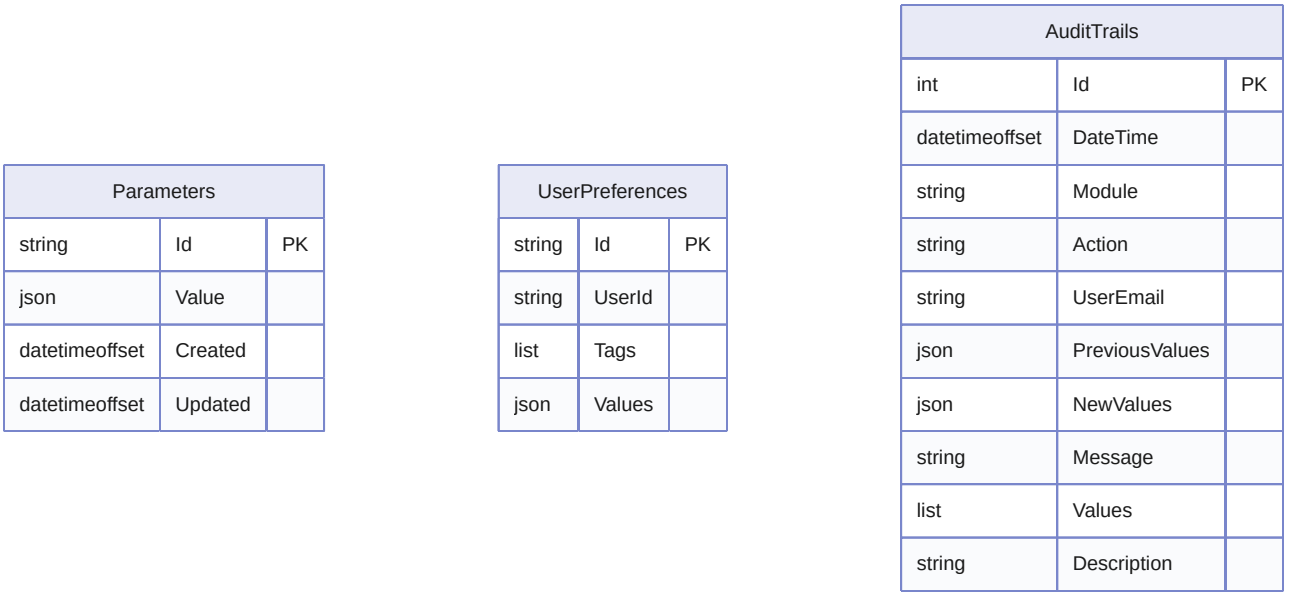
INFO:

A tabela de organização tem uma auto-referência, pois algumas organizações podem ser "pais" de outras

Tabelas Independentes (sem relacionamentos no diagrama)

As tabelas abaixo não possuem foreign keys e são independentes de outras entidades no domínio Commons. Não fazem parte do diagrama ER visual, mas continuam documentadas nas seções 5.6, 5.7 e 5.8.

Tabela	Descrição	Motivo do isolamento
`Parameters`	Tabela key-value de configuração do tenant	Sem FK — configuração standalone
`UserPreferences`	Preferências de usuário por tenant	`UserId` referencia `Users` do AutoloadContext (outro banco) — sem FK local
`AuditTrails`	Trilha de auditoria do tenant	Sem FK — registro standalone de auditoria



5.2. Organizations

Organizações representam terminais, transportadoras, clientes e outros participantes do sistema, organizadas hierarquicamente.

commons_Organizations

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK** — Identificador da organização
`Name`	`string`	N	Nome curto
`FullName`	`string`	N	Nome completo / razão social
`Types`	`List<OrganizationTypeEnum>`	N	Tipos (TERMINAL, CARRIER, CLIENT, etc.)
`ChildrenTypes`	`List<OrganizationTypeEnum>`	N	Tipos permitidos para filhos

Coluna	Tipo	Nulo	Descrição
`Timezone`	`string?`	S	Fuso horário (ex: `America/Sao_Paulo`)
`RequiresApproval`	`bool`	N	Exige aprovação para operações
`IsExternal`	`bool`	N	Organização externa
`Status`	`OrganizationStatusEnum`	N	Status da organização
`ParentId`	`string?`	S	**FK** → `Organizations.Id` (RESTRICT) — Organização pai
`ParentName`	`string?`	S	Nome da organização pai (cache)
`HierarchyPath`	`string`	N	Caminho hierárquico (default: `/`)
`Created`	`DateTimeOffset?`	S	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

Índices: `ParentId`

commons_OrganizationAttributeValues

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK** — Identificador do valor
`OrganizationId`	`string?`	S	**FK** → `Organizations.Id` (CASCADE)
`AttributeId`	`string`	N	**FK** → `CustomAttributes.Id` (RESTRICT)
`Value`	`JsonElement`	N	Valor do atributo (JSON)

commons_OrganizationAttributeAttachments

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`OrganizationId`	`string?`	S	**FK** → `Organizations.Id` (CASCADE)
`AttributeId`	`string`	N	**FK** → `CustomAttributes.Id` (RESTRICT)
`Name`	`string`	N	Nome do anexo
`FileName`	`string`	N	Nome do arquivo
`ContentType`	`string`	N	MIME type
`Content`	`byte[]?`	S	Conteúdo binário
`Url`	`string?`	S	URL de acesso

commons_OrganizationComments

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	N	**PK**
`OrganizationId`	`string`	N	**FK** → `Organizations.Id` (CASCADE)
`UserId`	`string`	N	ID do usuário que comentou
`Comment`	`string`	N	Texto do comentário
`Created`	`DateTimeOffset`	N	Data de criação

5.3. Groups e Permissões

commons_Groups

Grupos de permissões atribuídos a usuários dentro do tenant.

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	N	**PK**
`Name`	`string`	N	Nome do grupo
`Permissions`	`List<string>`	N	Lista de permissões
`OrganizationId`	`string?`	S	**FK** → `Organizations.Id` — Escopo organizacional
`IsActive`	`bool`	N	Grupo ativo/inativo
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

commons_GroupUsers

Coluna	Tipo	Nulo	Descrição
`GroupId`	`string`	N	**PK/FK** → `Groups.Id` (CASCADE)
`UserId`	`string`	N	**PK** — ID do usuário

Índices: `UserId`

5.4. Mensagens

commons_Messages

Sistema de notificações internas.

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	N	**PK**
`Type`	`MessageTypeEnum`	N	Tipo da mensagem
`Topic`	`string`	N	Tópico (ex: `program`, `appointment`)
`Level`	`MessageLevelEnum`	N	Nível (INFO, WARNING, ERROR, etc.)
`Content`	`string`	N	Conteúdo da mensagem (template)
`Values`	`List<string>`	N	Valores para interpolação no template
`Label`	`string?`	S	Label de agrupamento
`Closed`	`bool`	N	Mensagem encerrada (default: false)
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

Índices: `Created` ; `(Label, Closed)`

commons_MessageDeliveries

Coluna	Tipo	Nulo	Descrição
`MessageId`	`string`	N	**PK/FK** → `Messages.Id`
`UserId`	`string`	N	**PK** — ID do destinatário
`DeliveryType`	`MessageDeliveryTypeEnum`	N	**PK** — Tipo de entrega (APP, EMAIL, SMS)
`Delivered`	`bool`	N	Entrega realizada
`DeliveryError`	`string?`	S	Mensagem de erro de entrega
`Acknowledge`	`bool`	N	Leitura confirmada
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

commons_MessageSubscriptions

Coluna	Tipo	Nulo	Descrição
`Topic`	`string`	N	**PK** — Tópico de inscrição
`UserId`	`string`	N	**PK** — ID do usuário
`DeliveryTypes`	`List<MessageDeliveryTypeEnum>`	N	Tipos de entrega desejados
`Level`	`MessageLevelEnum`	N	Nível mínimo de notificação

5.5. Atributos Customizáveis

commons_CustomAttributes

Definições de atributos dinâmicos para entidades extensíveis.

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`EntityType`	`CustomAttributeEntityEnum`	N	Tipo de entidade (ORGANIZATION, VEHICLE, DRIVER, PRODUCT, DEVICE)
`Name`	`string`	N	Nome interno do atributo
`Label`	`string`	N	Label de exibição
`Width`	`int`	N	Largura no formulário (default: 3)
`Order`	`int`	N	Ordem de exibição
`AttributeType`	`CustomAttributeTypeEnum`	N	Tipo (CHARACTERS, NUMBER, LIST, DATETIME, ATTACHMENT)
`IsActive`	`bool`	N	Atributo ativo
`IsRequired`	`bool`	N	Preenchimento obrigatório
`AllowEmpty`	`bool`	N	Permitir valor vazio (default: true)
`CharactersMinLength`	`int?`	S	Comprimento mínimo (type: CHARACTERS)
`CharactersMaxLength`	`int?`	S	Comprimento máximo (type: CHARACTERS)
`CharactersRegex`	`string?`	S	Regex de validação (type: CHARACTERS)
`NumberMin`	`int?`	S	Valor mínimo (type: NUMBER)
`NumberMax`	`int?`	S	Valor máximo (type: NUMBER)
`NumberDecimals`	`int?`	S	Casas decimais (type: NUMBER)
`ListValues`	`List<string>`	N	Opções disponíveis (type: LIST)
`ListAllowMultiple`	`bool`	N	Permitir múltipla seleção (type: LIST)
`DateTimeAllowPast`	`bool`	N	Permitir datas passadas (type: DATETIME)
`DateTimeAllowFuture`	`bool`	N	Permitir datas futuras (type: DATETIME)
`AttachmentExtensions`	`List<string>`	N	Extensões permitidas (type: ATTACHMENT)
`AttachmentMaxSize`	`int?`	S	Tamanho máximo em bytes (type: ATTACHMENT)
`AttachmentMaxFiles`	`int?`	S	Número máximo de arquivos (type: ATTACHMENT)

5.6. Parâmetros

commons_Parameters

Parâmetros de configuração do tenant (modo de autenticação, SMTP, políticas de senha, etc.).

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK** — Identificador do parâmetro (ex: `auth_mode`, `smtp`)
`Value`	`JsonElement?`	S	Valor do parâmetro (JSON)
`Created`	`DateTimeOffset?`	S	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

5.7. Preferências de Usuário

commons_UserPreferences

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	N	**PK**
`UserId`	`string`	N	ID do usuário
`Tags`	`List<string>`	N	Tags de categorização
`Values`	`JsonElement`	N	Valores das preferências (JSON)

5.8. Auditoria do Tenant

commons_AuditTrails

Trilha de auditoria no contexto do tenant. Estrutura idêntica ao `AuditTrails` global.

Coluna	Tipo	Nulo	Descrição
`Id`	`int`	N	**PK** — Auto-incremento
`DateTime`	`DateTimeOffset`	N	Data/hora do evento
`Module`	`string`	N	Módulo afetado
`Action`	`string`	N	Ação realizada
`UserEmail`	`string?`	N	Email do usuário responsável
`PreviousValues`	`JsonElement`	N	Valores antes da alteração (JSON)
`NewValues`	`JsonElement`	N	Valores após a alteração (JSON)
`Message`	`string?`	S	Mensagem descritiva

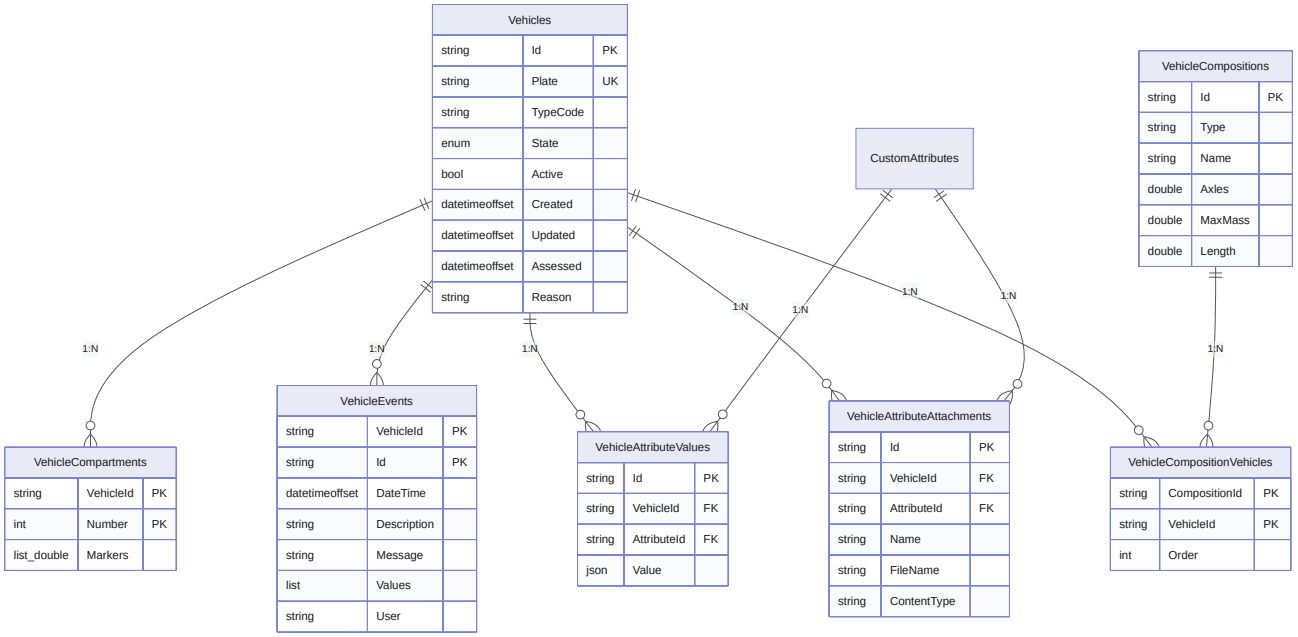
Coluna	Tipo	Nulo	Descrição
`Values`	`List<string>?`	S	Valores adicionais
`Description`	`string?`	S	Descrição detalhada

---

6. TenantContext — Domínio Management (18 tabelas)

O domínio Management gerencia a frota (veículos e motoristas) e o agendamento de operações.

6.1. Diagrama ER — Veículos



6.2. Veículos

management_Vehicles

Veículos cadastrados no sistema com workflow de aprovação.

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK** — Herdado de `ApprovableEntity`
`Plate`	`string`	N	Placa do veículo (**índice único**)
`TypeCode`	`string`	N	Código do tipo de veículo
`State`	`ApprovalStateEnum`	N	Estado de aprovação (PENDING, APPROVED, REJECTED)
`Active`	`bool`	N	Veículo ativo (default: true)

Coluna	Tipo	Nulo	Descrição
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização
`Assessed`	`DateTimeOffset?`	S	Data da última avaliação
`Reason`	`string?`	S	Motivo da última avaliação

Índices: `Plate` (unique); `State`; `TypeCode`

management_VehicleCompartments

Coluna	Tipo	Nulo	Descrição
`VehicleId`	`string`	N	**PK/FK** → `Vehicles.Id` (CASCADE)
`Number`	`int`	N	**PK** — Número do compartimento
`Markers`	`List<double>`	N	Marcadores volumétricos (JSON)

management_VehicleEvents

Coluna	Tipo	Nulo	Descrição
`VehicleId`	`string`	N	**PK/FK** → `Vehicles.Id` (CASCADE)
`Id`	`string`	N	**PK** — Herdado de `BaseEvent`
`DateTime`	`DateTimeOffset`	N	Data/hora do evento
`Description`	`string`	N	Descrição do evento
`Message`	`string`	N	Mensagem do evento
`Values`	`List<string>`	N	Valores adicionais
`User`	`string`	N	Usuário responsável

management_VehicleAttributeValues / management_VehicleAttributeAttachments

Mesma estrutura de `OrganizationAttributeValues` / `OrganizationAttributeAttachments` com FK para `Vehicles.Id` ao invés de `Organizations.Id`. FK para `CustomAttributes.Id` com delete RESTRICT.

management_VehicleCompositions

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	N	**PK**
`Type`	`string`	N	Tipo de composição

Coluna	Tipo	Nulo	Descrição
`Name`	`string`	N	Nome da composição
`Axles`	`double`	N	Número de eixos
`MaxMass`	`double`	N	Massa máxima permitida
`Length`	`double`	N	Comprimento total

Índices: `Type`

management_VehicleCompositionVehicles

Coluna	Tipo	Nulo	Descrição
`CompositionId`	`string`	N	**PK/FK** → `VehicleCompositions.Id` (CASCADE)
`VehicleId`	`string`	N	**PK/FK** → `Vehicles.Id` (RESTRICT)
`Order`	`int`	N	Ordem na composição

6.3. Motoristas

management_Drivers

Motoristas com workflow de aprovação (herda de `ApprovableEntity`).

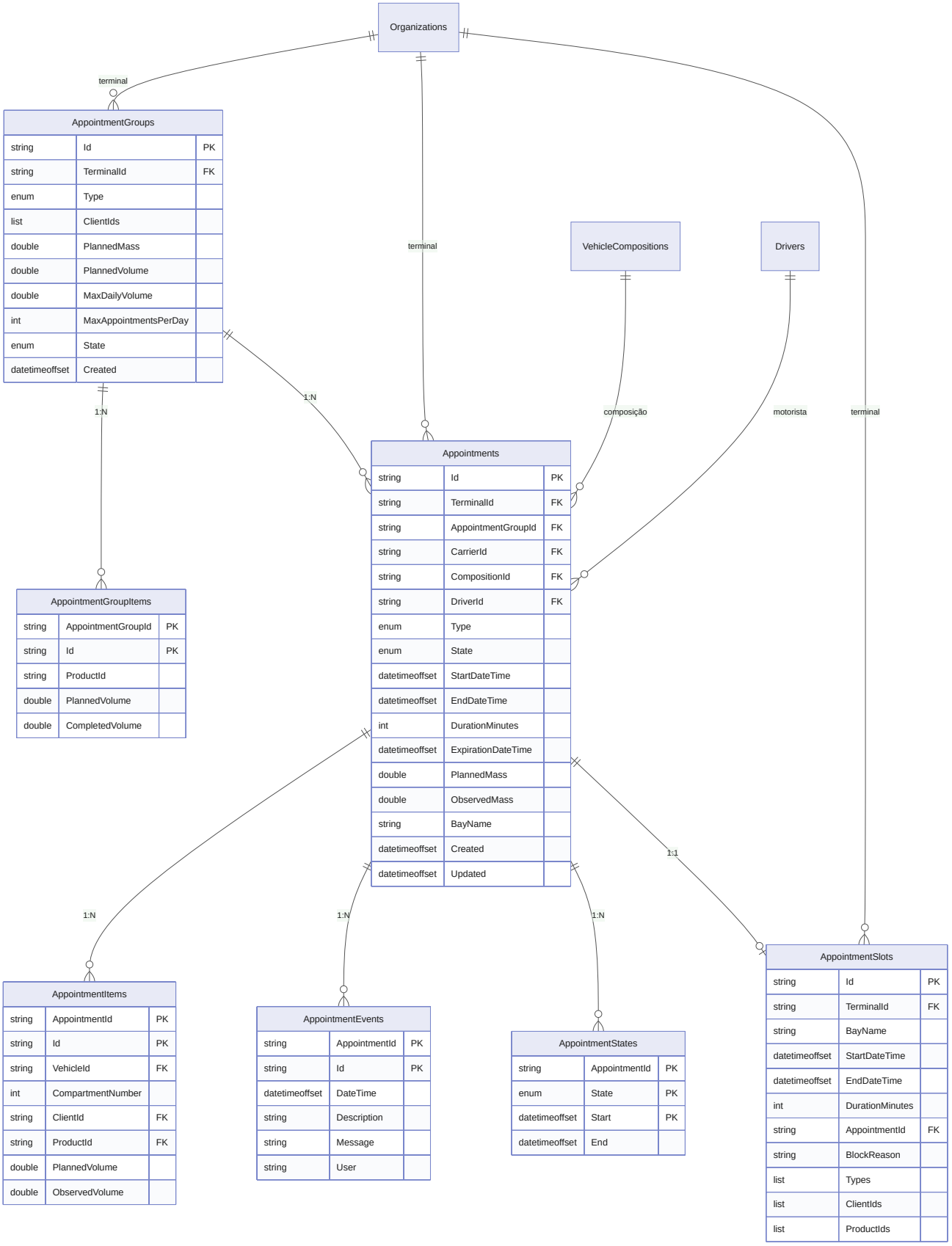
Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Code`	`string`	N	Código do motorista (**índice único**)
`Name`	`string`	N	Nome completo
`Email`	`string?`	S	Email do motorista
`State`	`ApprovalStateEnum`	N	Estado de aprovação
`Active`	`bool`	N	Motorista ativo
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização
`Assessed`	`DateTimeOffset?`	S	Data da última avaliação
`Reason`	`string?`	S	Motivo da avaliação

Índices: `Code` (unique); `Email`; `State`; `Active`

management_DriverEvents / management_DriverAttributeValues / management_DriverAttributeAttachments

Mesma estrutura de VehicleEvents / VehicleAttributeValues / VehicleAttributeAttachments com FK para `Drivers.Id`.

6.4. Diagrama ER — Agendamentos



6.5. Agendamentos

management_Appointments

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`TerminalId`	`string`	N	**FK** → `Organizations.Id` — Terminal
`AppointmentGroupId`	`string?`	S	**FK** → `AppointmentGroups.Id`
`CarrierId`	`string?`	S	**FK** → `Organizations.Id` — Transportadora
`CompositionId`	`string?`	S	**FK** → `VehicleCompositions.Id`
`DriverId`	`string?`	S	**FK** → `Drivers.Id`
`Type`	`AppointmentTypeEnum`	N	Tipo de agendamento
`State`	`AppointmentStateEnum`	N	Estado atual (default: RESERVED)
`StartDateTime`	`DateTimeOffset`	N	Início agendado
`EndDateTime`	`DateTimeOffset`	N	Fim agendado
`DurationMinutes`	`int`	N	Duração em minutos
`ExpirationDateTime`	`DateTimeOffset`	N	Expiração do agendamento
`NoShowDateTime`	`DateTimeOffset`	N	Limite para no-show
`CancellationReason`	`string?`	S	Motivo do cancelamento
`BayName`	`string?`	S	Nome da baia alocada
`PlannedMass`	`double`	N	Massa planejada
`ObservedMass`	`double`	N	Massa observada
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

Índices: `StartDateTime` ; `State` ; `TerminalId` ; `(TerminalId, StartDateTime)` ; `AppointmentGroupId`

management_AppointmentItems

Coluna	Tipo	Nulo	Descrição
`AppointmentId`	`string`	N	**PK/FK** → `Appointments.Id`
`Id`	`string`	N	**PK**
`VehicleId`	`string?`	S	**FK** → `Vehicles.Id`
`CompartmentNumber`	`int`	N	Número do compartimento

Coluna	Tipo	Nulo	Descrição
`ClientId`	`string?`	S	**FK** → `Organizations.Id`
`ProductId`	`string`	N	**FK** → `Products.Id`
`PlannedVolume`	`double`	N	Volume planejado
`ObservedVolume`	`double`	N	Volume observado

Índices: `AppointmentId`

management_AppointmentEvents

Coluna	Tipo	Nulo	Descrição
`AppointmentId`	`string`	N	**PK/FK** → `Appointments.Id`
`Id`	`string`	N	**PK** — Herdado de `BaseEvent`
`DateTime`	`DateTimeOffset`	N	Data/hora
`Description`	`string`	N	Descrição
`Message`	`string`	N	Mensagem
`Values`	`List<string>`	N	Valores (JSON)
`User`	`string`	N	Usuário

management_AppointmentStates

Coluna	Tipo	Nulo	Descrição
`AppointmentId`	`string`	N	**PK/FK** → `Appointments.Id` (CASCADE)
`State`	`AppointmentStateEnum`	N	**PK** — Estado
`Start`	`DateTimeOffset`	N	**PK** — Início do estado
`End`	`DateTimeOffset?`	S	Fim do estado (transição)

6.6. Grupos de Agendamento

management_AppointmentGroups

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`TerminalId`	`string`	N	**FK** → `Organizations.Id`
`Type`	`AppointmentTypeEnum`	N	Tipo de agendamento

Coluna	Tipo	Nulo	Descrição
`ClientIds`	`List<string>`	N	IDs de clientes associados
`ClientNames`	`List<string>`	N	Nomes (cache)
`PlannedMass`	`double`	N	Massa planejada
`PlannedVolume`	`double`	N	Volume planejado
`CompletedVolume`	`double`	N	Volume completado
`MaxDailyVolume`	`double`	N	Volume máximo diário
`MaxAppointmentsPerDay`	`int`	N	Máximo de agendamentos por dia
`MaxSimultaneousAppointments`	`int`	N	Máximo simultâneo
`Weekdays`	`List<DayOfWeek>`	N	Dias da semana permitidos
`StartTimeLocal`	`TimeSpan`	N	Horário de início
`EndTimeLocal`	`TimeSpan`	N	Horário de término
`ScheduleFromDate`	`DateOnly`	N	Data início do período
`ScheduleToDate`	`DateOnly`	N	Data fim do período
`State`	`AppointmentGroupStateEnum`	N	Estado (default: ACTIVE)
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

Índices: `TerminalId` ; `State` ; `(TerminalId, State)`

management_AppointmentGroupItems

Coluna	Tipo	Nulo	Descrição
`AppointmentGroupId`	`string`	N	**PK/FK** → `AppointmentGroups.Id`
`Id`	`string`	N	**PK**
`ProductId`	`string`	N	ID do produto
`ProductName`	`string?`	S	Nome do produto (cache)
`PlannedVolume`	`double`	N	Volume planejado
`CompletedVolume`	`double`	N	Volume completado

6.7. Slots de Agendamento

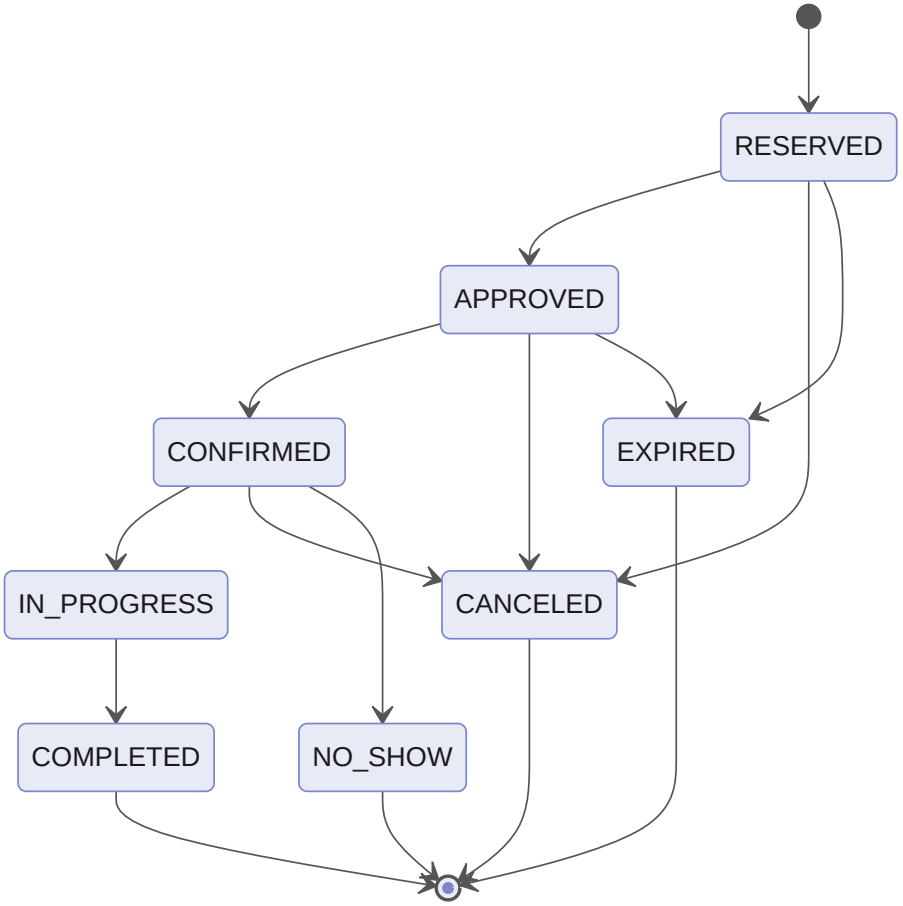
management_AppointmentSlots

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`TerminalId`	`string`	N	**FK** → `Organizations.Id`
`BayName`	`string`	N	Nome da baia
`StartDateTime`	`DateTimeOffset`	N	Início do slot
`EndDateTime`	`DateTimeOffset`	N	Fim do slot
`DurationMinutes`	`int`	N	Duração em minutos
`AppointmentId`	`string?`	S	**FK** → `Appointments.Id` (SET NULL)
`BlockReason`	`string?`	S	Motivo do bloqueio (null = disponível)
`Types`	`List<AppointmentTypeEnum>?`	S	Tipos permitidos (JSON, null = sem restrição)
`ClientIds`	`List<string>?`	S	Clientes permitidos (JSON, null = sem restrição)
`ClientNames`	`List<string>?`	S	Nomes (cache, JSON)
`ProductIds`	`List<string>?`	S	Produtos permitidos (JSON)
`ProductNames`	`List<string>?`	S	Nomes (cache, JSON)
`ProductGroupIds`	`List<string>?`	S	Grupos de produto (JSON)
`CompositionTypeIds`	`List<string>?`	S	Tipos de composição (JSON)
`CompositionTypeNames`	`List<string>?`	S	Nomes (cache, JSON)

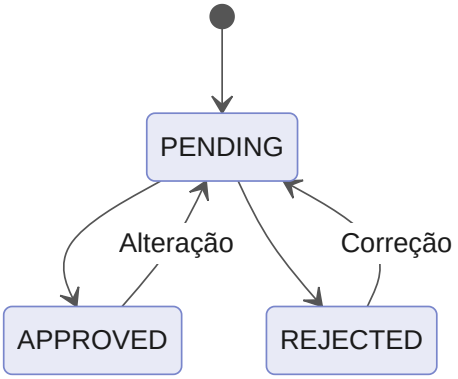
Índices: `TerminalId`; `(TerminalId, BayName)`; `(TerminalId, BayName, StartDateTime)` (unique); `AppointmentId`

6.8. Diagramas de Estado

Ciclo de Vida do Appointment



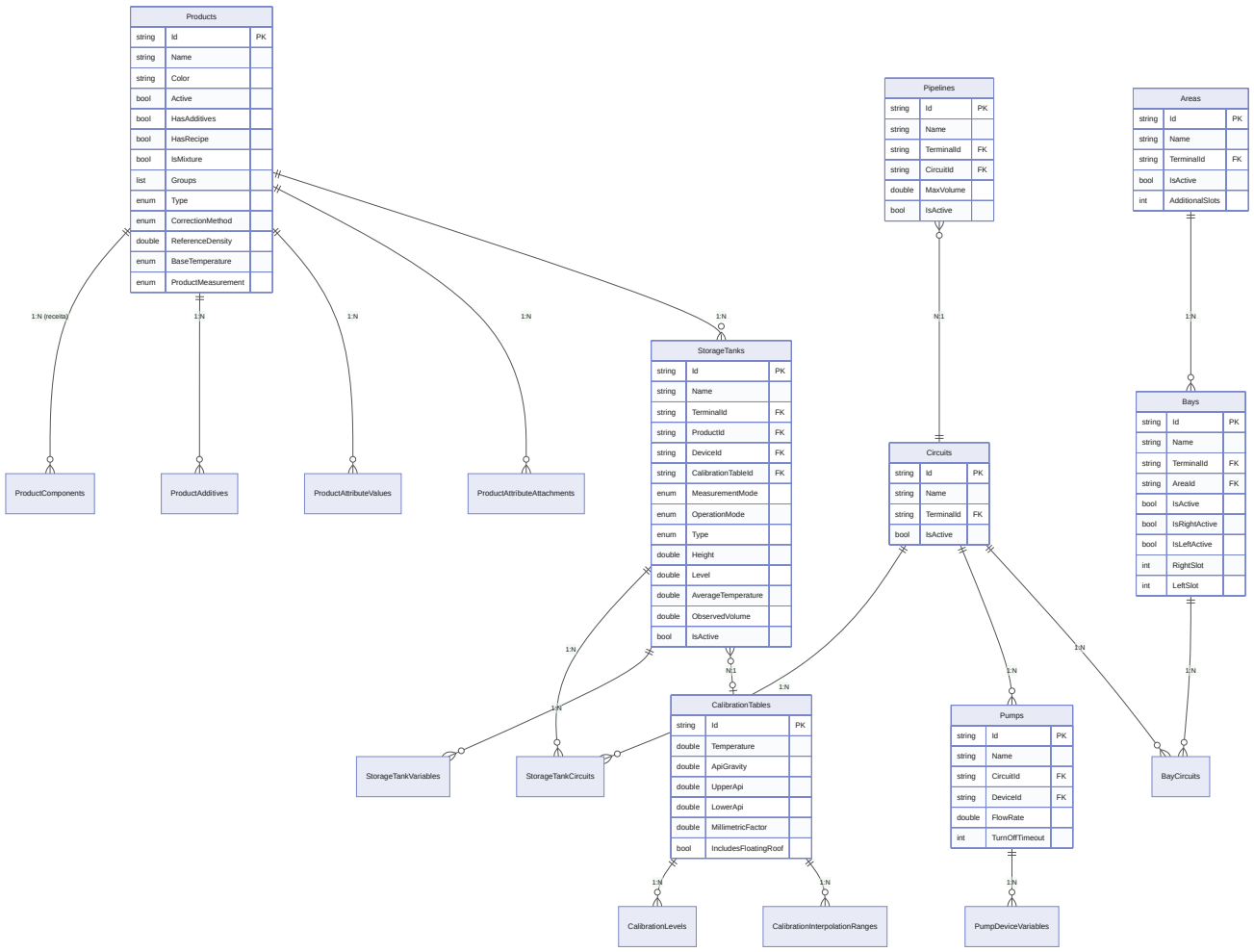
Workflow de Aprovação (Vehicle / Driver)



7. TenantContext — Domínio Operations: Infraestrutura (18 tabelas)

Tabelas de infraestrutura que definem os ativos físicos do terminal: produtos, tanques, circuitos, áreas, baias, pipelines, bombas e calibração.

7.1. Diagrama ER



Confira o diagrama completo em numa nova aba: Abrir diagrama

7.2. Produtos

operations_Products

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Name`	`string`	N	Nome do produto
`Color`	`string`	N	Cor de exibição (default: `#6668d9`)
`Active`	`bool`	N	Produto ativo (default: true)
`HasAdditives`	`bool`	N	Possui aditivos
`HasConcentration`	`bool`	N	Possui concentração
`HasRecipe`	`bool`	N	Possui receita (mistura)
`IsMixture`	`bool`	N	É uma mistura

Coluna	Tipo	Nulo	Descrição
`Groups`	`List<string>`	N	Grupos do produto
`Type`	`ProductTypeEnum`	N	Tipo (default: REFINED_PRODUCT)
`CorrectionMethod`	`ProductCorrectionMethodEnum`	N	Método de correção volumétrica
`CoefficientThermalExpansion`	`double?`	S	Coeficiente de expansão térmica
`ReferenceDensity`	`double?`	S	Densidade de referência
`MinReferenceDensity`	`double?`	S	Densidade mínima de referência
`MaxReferenceDensity`	`double?`	S	Densidade máxima de referência
`BaseTemperature`	`ProductBaseTemperatureEnum`	N	Temperatura base de correção
`ProductMeasurement`	`UnitTypeEnum`	N	Unidade de medição (default: VOLUME)
`Created`	`DateTimeOffset?`	S	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

operations_ProductComponents

Componentes da receita de um produto (mistura).

Coluna	Tipo	Nulo	Descrição
`ProductId`	`string`	N	**PK/FK** → `Products.Id`
`ComponentId`	`string`	N	**PK/FK** → `Products.Id` (RESTRICT) — Produto componente
`Percentual`	`double?`	S	Percentual na mistura
`ComponentVolume`	`double?`	S	Volume do componente
`ProductVolume`	`double?`	S	Volume do produto final

operations_ProductAdditives

Coluna	Tipo	Nulo	Descrição
`ProductId`	`string`	N	**PK/FK** → `Products.Id`
`AdditiveId`	`string`	N	**PK/FK** → `Products.Id` (RESTRICT) — Produto aditivo
`RatioType`	`ProductAdditiveRatioTypeEnum`	N	Tipo de proporção (default: PPM)
`Numerator`	`double`	N	Numerador da proporção
`Denominator`	`double`	N	Denominador da proporção

operations_ProductAttributeValues / operations_ProductAttributeAttachments

Mesma estrutura de atributos customizáveis com FK para `Products.Id`.

7.3. Tanques de Armazenamento

operations_StorageTanks

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Name`	`string`	N	Nome do tanque
`IsActive`	`bool`	N	Tanque ativo
`MeasurementMode`	`StorageTankMeasurementModeEnum`	N	Modo de medição (default: AUTO)
`OperationMode`	`StorageTankModeEnum`	N	Modo de operação (default: OPENED)
`CustomOperationMode`	`string?`	S	Modo customizado
`Type`	`StorageTankTypeEnum`	N	Tipo (default: CYLINDRICAL)
`Height`	`double?`	S	Altura total
`Level`	`double?`	S	Nível atual
`AverageTemperature`	`double?`	S	Temperatura média
`BaseDensity`	`double?`	S	Densidade base
`AmbientDensity`	`double?`	S	Densidade ambiente
`ObservedVolume`	`double?`	S	Volume observado
`Concentration`	`double?`	S	Concentração
`ExternalTemperature`	`double?`	S	Temperatura externa
`WaterLevel`	`double?`	S	Nível de água
`Material`	`StorageTankMaterialEnum?`	S	Material do tanque
`CalculateCTSh`	`bool?`	S	Calcular CTSh
`TerminalId`	`string`	N	**FK** → `Organizations.Id`
`ProductId`	`string?`	S	**FK** → `Products.Id`
`DeviceId`	`string?`	S	**FK** → `Devices.Id`
`MaxOperationalLevel`	`double?`	S	Nível operacional máximo
`MinOperationalLevel`	`double?`	S	Nível operacional mínimo
`HighHighThreshold`	`double?`	S	Alarme muito alto
`HighThreshold`	`double?`	S	Alarme alto

Coluna	Tipo	Nulo	Descrição
`LowThreshold`	`double?`	S	Alarme baixo
`LowLowThreshold`	`double?`	S	Alarme muito baixo
`CalibrationTableId`	`string?`	S	**FK** → `CalibrationTables.Id`
`HasFloatingRoof`	`bool?`	S	Teto flutuante
`RoofWeight`	`double?`	S	Peso do teto
`CriticalZoneHeight`	`double?`	S	Altura da zona crítica
`Created`	`DateTimeOffset?`	S	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

Índices: `(TerminalId, ProductId)`

operations_StorageTankCircuits

Coluna	Tipo	Nulo	Descrição
`StorageTankId`	`string`	N	**PK/FK** → `StorageTanks.Id`
`CircuitId`	`string`	N	**PK/FK** → `Circuits.Id`

Índices: `CircuitId`

operations_StorageTankVariables

Coluna	Tipo	Nulo	Descrição
`StorageTankId`	`string`	N	**PK/FK** → `StorageTanks.Id`
`Variable`	`StorageTankDeviceVariableEnum`	N	**PK** — Variável do dispositivo
`Name`	`string?`	S	Nome da variável no dispositivo
`Value`	`double?`	S	Valor atual

7.4. Circuitos

operations_Circuits

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Name`	`string`	N	Nome do circuito
`IsActive`	`bool`	N	Circuito ativo

Coluna	Tipo	Nulo	Descrição
`TerminalId`	`string?`	S	**FK** → `Organizations.Id` (RESTRICT)
`Created`	`DateTimeOffset?`	S	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

7.5. Áreas e Baias

operations_Areas

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Name`	`string`	N	Nome da área
`IsActive`	`bool`	N	Área ativa
`TerminalId`	`string`	N	**FK** → `Organizations.Id`
`AdditionalSlots`	`int`	N	Slots adicionais
`Created`	`DateTimeOffset?`	S	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

operations_Bays

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Name`	`string`	N	Nome da baia
`TerminalId`	`string?`	S	**FK** → `Organizations.Id`
`AreaId`	`string`	N	**FK** → `Areas.Id`
`IsActive`	`bool`	N	Baia ativa
`IsRightActive`	`bool`	N	Lado direito ativo (default: true)
`NameRight`	`string?`	S	Nome do lado direito
`RightSlot`	`int`	N	Slot do lado direito (default: 1)
`RightSupportedVehicleTypes`	`List<string>`	N	Tipos de veículo suportados (direita)
`IsLeftActive`	`bool`	N	Lado esquerdo ativo (default: true)
`NameLeft`	`string?`	S	Nome do lado esquerdo

Coluna	Tipo	Nulo	Descrição
`LeftSlot`	`int`	N	Slot do lado esquerdo (default: 1)
`LeftSupportedVehicleTypes`	`List<string>`	N	Tipos de veículo suportados (esquerda)
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

operations_BayCircuits

Coluna	Tipo	Nulo	Descrição
`BayId`	`string`	N	**PK/FK** → `Bays.Id`
`CircuitId`	`string`	N	**PK/FK** → `Circuits.Id`
`OutboundTopLeft`	`bool`	N	Saída topo esquerda (default: true)
`OutboundTopRight`	`bool`	N	Saída topo direita (default: true)
`OutboundBottomLeft`	`bool`	N	Saída base esquerda (default: true)
`OutboundBottomRight`	`bool`	N	Saída base direita (default: true)
`InboundBottomLeft`	`bool`	N	Entrada base esquerda (default: true)
`InboundBottomRight`	`bool`	N	Entrada base direita (default: true)
`IsLeftActive`	`bool`	N	Lado esquerdo ativo (default: true)
`IsRightActive`	`bool`	N	Lado direito ativo (default: true)
`IsOutboundActive`	`bool`	N	Saída ativa (default: true)
`IsInboundActive`	`bool`	N	Entrada ativa (default: true)

7.6. Pipelines

operations_Pipelines

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Name`	`string`	N	Nome do pipeline
`IsActive`	`bool`	N	Pipeline ativo
`TerminalId`	`string?`	S	**FK** → `Organizations.Id` (RESTRICT)
`MaxVolume`	`double`	N	Volume máximo

Coluna	Tipo	Nulo	Descrição
`CircuitId`	`string`	N	**FK** → `Circuits.Id`
`Created`	`DateTimeOffset?`	S	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

7.7. Bombas

operations_Pumps

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Name`	`string`	N	Nome da bomba
`CircuitId`	`string?`	S	**FK** → `Circuits.Id`
`DeviceId`	`string?`	S	**FK** → `Devices.Id`
`FlowRate`	`double`	N	Vazão
`TurnOffTimeout`	`int`	N	Timeout de desligamento (segundos)
`Created`	`DateTimeOffset?`	S	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

operations_PumpDeviceVariables

Coluna	Tipo	Nulo	Descrição
`PumpId`	`string`	N	**PK/FK** → `Pumps.Id`
`Variable`	`PumpDeviceVariableEnum`	N	**PK** — Variável do dispositivo
`Name`	`string?`	S	Nome da variável no dispositivo

7.8. Calibração

operations_CalibrationTables

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Temperature`	`double?`	S	Temperatura de calibração
`ApiGravity`	`double?`	S	Gravidade API

Coluna	Tipo	Nulo	Descrição
`UpperApi`	`double?`	S	API superior
`LowerApi`	`double?`	S	API inferior
`MillimetricFactor`	`double?`	S	Fator milimétrico
`IncludesFloatingRoof`	`bool?`	S	Inclui teto flutuante
`Created`	`DateTimeOffset?`	S	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

operations_CalibrationLevels

Coluna	Tipo	Nulo	Descrição
`CalibrationTableId`	`string`	N	**PK/FK** → `CalibrationTables.Id`
`Level`	`double`	N	**PK** — Nível (altura)
`Volume`	`double?`	S	Volume correspondente

operations_CalibrationInterpolationRanges

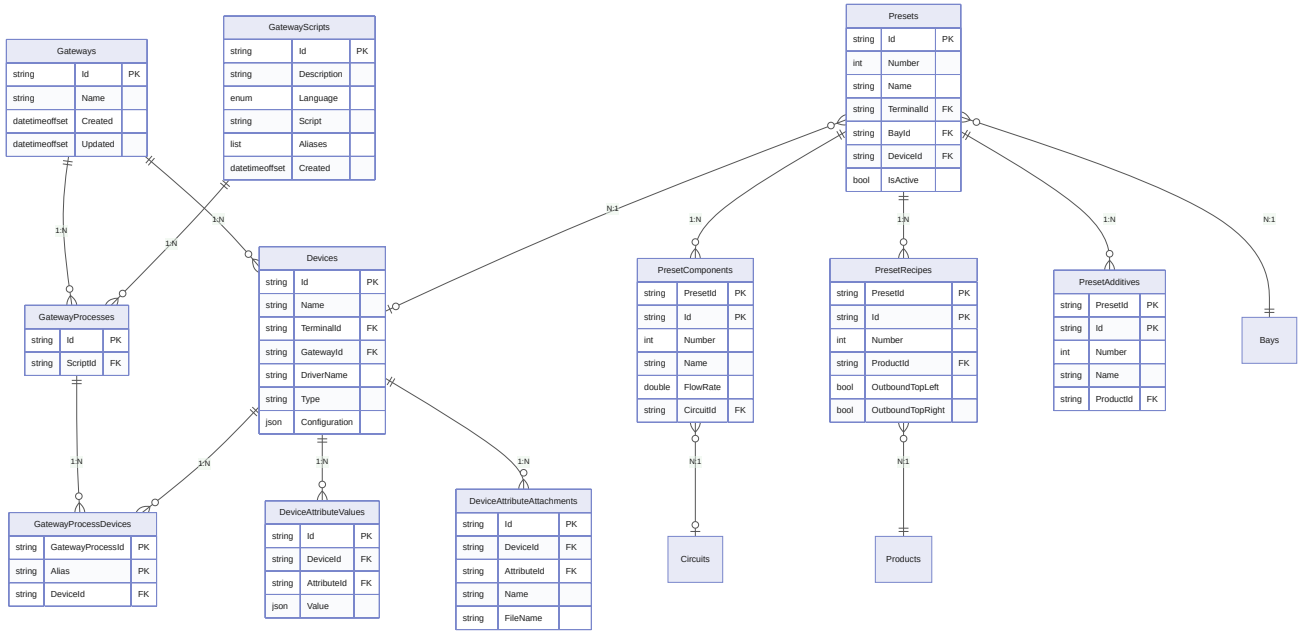
Coluna	Tipo	Nulo	Descrição
`CalibrationTableId`	`string`	N	**PK/FK** → `CalibrationTables.Id`
`StartLevel`	`int`	N	**PK** — Nível inicial do range
`EndLevel`	`int`	N	Nível final do range
`Ring`	`int?`	S	Anel do tanque
`MillimetricFactor`	`double`	N	Fator milimétrico
`Value1` a `Value9`	`double`	N	Valores de interpolação (9 colunas)

---

8. TenantContext — Domínio Operations: Dispositivos e Gateways (11 tabelas)

Tabelas que gerenciam a comunicação com equipamentos de campo: gateways, dispositivos (PLCs, controladores de baia) e presets de carregamento.

8.1. Diagrama ER



Confira o diagrama completo em numa nova aba: Abrir diagrama

8.2. Gateways

operations_Gateways

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	N	**PK**
`Name`	`string`	N	Nome do gateway
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

operations_GatewayScripts

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	N	**PK**
`Description`	`string`	N	Descrição do script
`Language`	`GatewayScriptLanguageEnum`	N	Linguagem (default: UNDEFINED)
`Script`	`string?`	S	Código do script
`Aliases`	`List<string>`	N	Aliases para referência
`Created`	`DateTimeOffset`	N	Data de criação

Coluna	Tipo	Nulo	Descrição
`Updated`	`DateTimeOffset?`	S	Data da última atualização

operations_GatewayProcesses

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`ScriptId`	`string`	N	**FK** → `GatewayScripts.Id` (RESTRICT)
`GatewayId`	`string?`	S	**FK** → `Gateways.Id` (NoAction)

operations_GatewayProcessDevices

Coluna	Tipo	Nulo	Descrição
`GatewayProcessId`	`string`	N	**PK/FK** → `GatewayProcesses.Id` (CASCADE)
`Alias`	`string`	N	**PK** — Alias do dispositivo no processo
`DeviceId`	`string`	N	**FK** → `Devices.Id` (RESTRICT)

8.3. Dispositivos

operations_Devices

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Name`	`string`	N	Nome do dispositivo
`TerminalId`	`string`	N	**FK** → `Organizations.Id`
`GatewayId`	`string`	N	**FK** → `Gateways.Id`
`DriverName`	`string`	N	Nome do driver de comunicação
`Type`	`string?`	S	Tipo do dispositivo
`Configuration`	`JsonElement`	N	Configuração do dispositivo (JSON)
`Created`	`DateTimeOffset?`	S	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

operations_DeviceAttributeValues / operations_DeviceAttributeAttachments

Mesma estrutura de atributos customizáveis com FK para `Devices.Id` (CASCADE) e `CustomAttributes.Id` (RESTRICT).

8.4. Presets

operations_Presets

Presets representam pontos de carregamento configurados em uma baia.

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Number`	`int`	N	Número do preset
`Name`	`string`	N	Nome do preset
`IsActive`	`bool`	N	Preset ativo
`TerminalId`	`string?`	S	**FK** → `Organizations.Id` (RESTRICT)
`BayId`	`string?`	S	**FK** → `Bays.Id`
`DeviceId`	`string?`	S	**FK** → `Devices.Id`
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset`	N	Data da última atualização

operations_PresetComponents

Coluna	Tipo	Nulo	Descrição
`PresetId`	`string`	N	**PK/FK** → `Presets.Id`
`Id`	`string`	S	**PK**
`Number`	`int`	N	Número do componente
`Name`	`string`	N	Nome do componente
`FlowRate`	`double`	N	Vazão
`CircuitId`	`string?`	S	**FK** → `Circuits.Id`

operations_PresetRecipes

Coluna	Tipo	Nulo	Descrição
`PresetId`	`string`	N	**PK/FK** → `Presets.Id`
`Id`	`string`	S	**PK**
`Number`	`int`	N	Número da receita
`ProductId`	`string`	N	**FK** → `Products.Id`

Coluna	Tipo	Nulo	Descrição
`OutboundTopLeft` / `OutboundTopRight` / `OutboundBottomLeft` / `OutboundBottomRight`	`bool`	N	Braços de saída habilitados
`InboundBottomLeft` / `InboundBottomRight`	`bool`	N	Braços de entrada habilitados
`IsLeftActive` / `IsRightActive`	`bool`	N	Lados ativos
`IsOutboundActive` / `IsInboundActive`	`bool`	N	Direções ativas

Índices: `ProductId`

operations_PresetAdditives

Coluna	Tipo	Nulo	Descrição
`PresetId`	`string`	N	**PK/FK** → `Presets.Id` (CASCADE)
`Id`	`string`	S	**PK**
`Number`	`int`	N	Número do aditivo
`Name`	`string`	N	Nome do aditivo
`ProductId`	`string`	N	**FK** → `Products.Id` (RESTRICT)

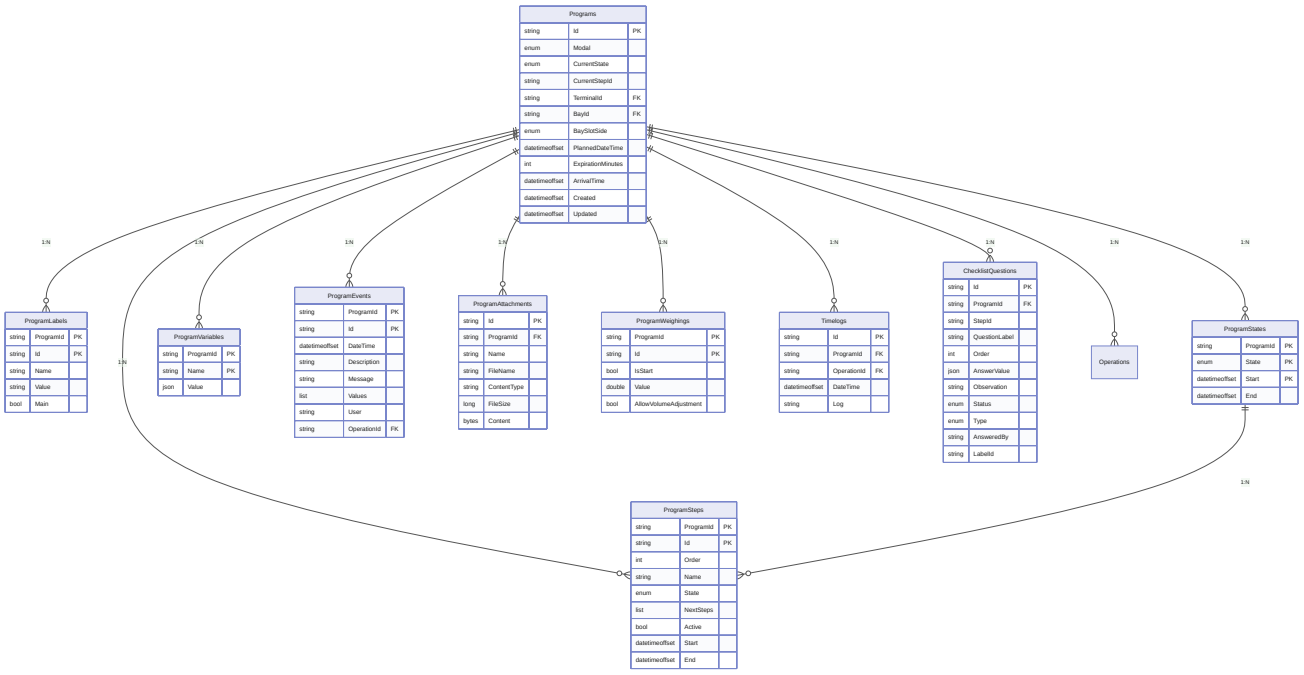
Índices: `ProductId`

---

9. TenantContext — Domínio Operations: Programações (10 tabelas)

Programações representam o fluxo completo de um carregamento/descarregamento, desde o rascunho até a conclusão.

9.1. Diagrama ER



LIGAR PROGRAMSTEPS COM PROGRAMSTATES

Confira o diagrama completo em numa nova aba: Abrir diagrama

9.2. Programas

operations_Programs

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Modal`	`ProgramModalEnum`	N	Modal de transporte
`CurrentState`	`ProgramStateEnum?`	S	Estado atual
`CurrentStepId`	`string?`	S	Step atual
`TerminalId`	`string`	N	**FK** → `Organizations.Id` (RESTRICT)
`BayId`	`string?`	S	**FK** → `Bays.Id` (RESTRICT)
`BaySlotSide`	`BaySlotSideEnum?`	S	Lado do slot na baia
`PlannedDateTime`	`DateTimeOffset?`	S	Data/hora planejada
`ExpirationMinutes`	`int?`	S	Minutos para expiração
`ArrivalTime`	`DateTimeOffset?`	S	Hora de chegada
`Created`	`DateTimeOffset`	N	Data de criação

Coluna	Tipo	Nulo	Descrição
`Updated`	`DateTimeOffset?`	S	Data da última atualização

operations_ProgramLabels

Labels identificam a programação (Ticket, Placa, NF, etc.).

Coluna	Tipo	Nulo	Descrição
`ProgramId`	`string`	N	**PK/FK** → `Programs.Id` (CASCADE)
`Id`	`string`	N	**PK** — Tipo do label (ex: `ticket`, `plate`)
`Name`	`string?`	S	Nome de exibição
`Value`	`string?`	S	Valor do label
`Main`	`bool`	N	Label principal

Índices: `Value`

operations_ProgramStates

Coluna	Tipo	Nulo	Descrição
`ProgramId`	`string`	N	**PK/FK** → `Programs.Id` (CASCADE)
`State`	`ProgramStateEnum`	N	**PK** — Estado
`Start`	`DateTimeOffset`	N	**PK** — Início do estado
`End`	`DateTimeOffset?`	S	Fim do estado

operations_ProgramSteps

Coluna	Tipo	Nulo	Descrição
`ProgramId`	`string`	N	**PK/FK** → `Programs.Id` (CASCADE)
`Id`	`string`	N	**PK** — Identificador do step
`Order`	`int`	N	Ordem de execução
`Name`	`string`	N	Nome do step
`State`	`ProgramStateEnum`	N	Estado associado
`NextSteps`	`List<string>`	N	IDs dos próximos steps
`Active`	`bool`	N	Step ativo (default: true)
`Start`	`DateTimeOffset?`	S	Data de início

Coluna	Tipo	Nulo	Descrição
`End`	`DateTimeOffset?`	S	Data de conclusão

operations_ProgramVariables

Coluna	Tipo	Nulo	Descrição
`ProgramId`	`string`	N	**PK/FK** → `Programs.Id`
`Name`	`string`	N	**PK** — Nome da variável
`Value`	`JsonElement`	N	Valor da variável (JSON)

9.3. Eventos e Timelogs

operations_ProgramEvents

Coluna	Tipo	Nulo	Descrição
`ProgramId`	`string`	N	**PK/FK** → `Programs.Id` (CASCADE)
`Id`	`string`	S	**PK**
`DateTime`	`DateTimeOffset?`	S	Data/hora do evento
`Description`	`string?`	S	Descrição
`Message`	`string?`	S	Mensagem
`Values`	`List<string>`	N	Valores
`User`	`string?`	S	Usuário
`OperationId`	`string?`	S	**FK** → `Operations.Id` (RESTRICT)

operations_Timelogs

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`ProgramId`	`string?`	S	**FK** → `Programs.Id` (CASCADE)
`OperationId`	`string?`	S	**FK** → `Operations.Id` (RESTRICT)
`DateTime`	`DateTimeOffset?`	S	Data/hora
`Log`	`string`	N	Texto do log

Índices: `ProgramId`

9.4. Anexos e Pesagens

operations_ProgramAttachments

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`ProgramId`	`string?`	S	**FK** → `Programs.Id` (CASCADE)
`Name`	`string`	N	Nome do anexo
`FileName`	`string`	N	Nome do arquivo
`ContentType`	`string`	N	MIME type
`FileSize`	`long`	N	Tamanho do arquivo
`Content`	`byte[]`	N	Conteúdo binário
`Created`	`DateTime`	N	Data de criação

operations_ProgramWeighings

Coluna	Tipo	Nulo	Descrição
`ProgramId`	`string`	N	**PK/FK** → `Programs.Id` (CASCADE)
`Id`	`string`	S	**PK**
`IsStart`	`bool`	N	Pesagem de entrada (true) ou saída (false)
`Value`	`double`	N	Peso medido
`AllowVolumeAdjustment`	`bool`	N	Permitir ajuste de volume (default: true)
`Created`	`DateTimeOffset?`	S	Data de criação

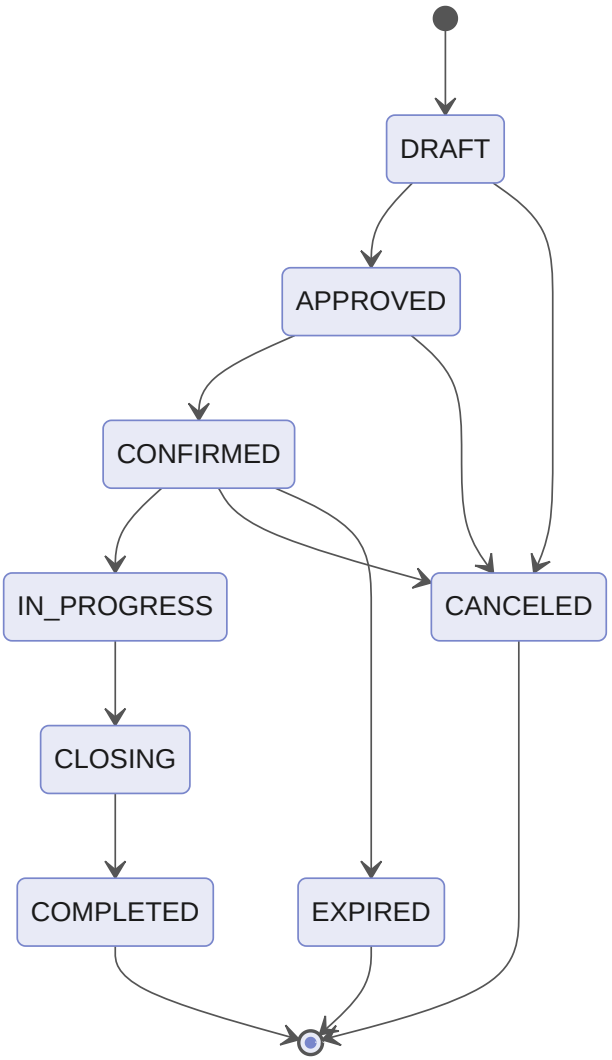
9.5. Checklist

operations_ChecklistQuestions

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`ProgramId`	`string`	N	**FK** → `Programs.Id` (CASCADE)
`StepId`	`string`	N	ID do step associado
`QuestionLabel`	`string`	N	Texto da pergunta
`Order`	`int`	N	Ordem de exibição

Coluna	Tipo	Nulo	Descrição
`AnswerValue`	`JsonElement?`	S	Resposta (JSON)
`Observation`	`string?`	S	Observação
`Status`	`ChecklistStatusEnum`	N	Status (default: PENDING)
`Type`	`ChecklistQuestionTypeEnum`	N	Tipo de resposta (default: BOOLEAN)
`AnsweredBy`	`string?`	S	Usuário que respondeu
`LabelId`	`string?`	S	Label associado
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

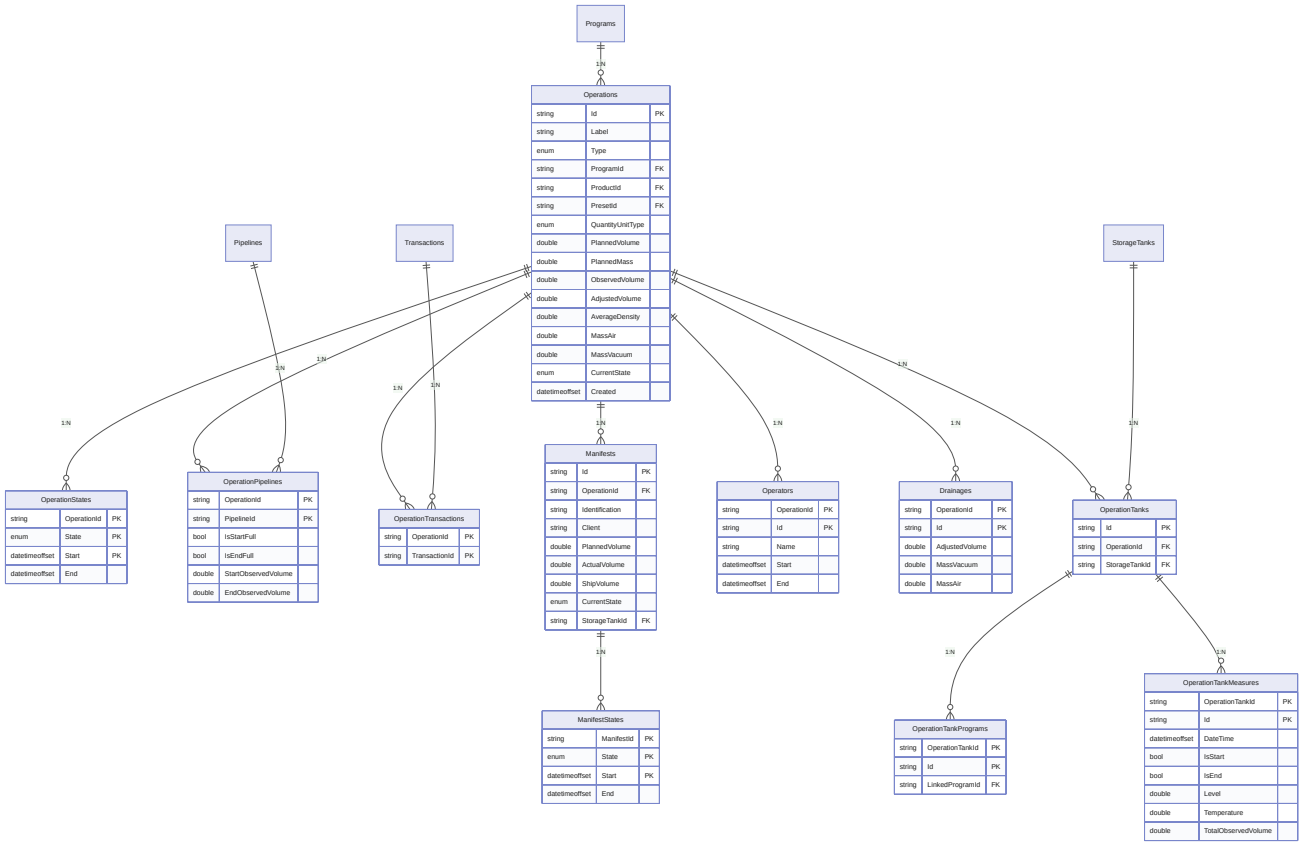
9.6. Diagrama de Estados — Ciclo de Vida do Program



10. TenantContext — Domínio Operations: Operações (11 tabelas)

Operações representam o carregamento/descarregamento efetivo de um produto dentro de uma programação.

10.1. Diagrama ER



Confira o diagrama completo em numa nova aba: Abrir diagrama

10.2. Operações

operations_Operations

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Label`	`string`	N	Rótulo da operação
`Type`	`TransactionTypeEnum`	N	Tipo (OUTBOUND, INBOUND, etc.)
`ProgramId`	`string?`	S	**FK** → `Programs.Id` (CASCADE)
`ProductId`	`string`	N	**FK** → `Products.Id` (RESTRICT)
`PresetId`	`string?`	S	**FK** → `Presets.Id` (RESTRICT)

Coluna	Tipo	Nulo	Descrição
`QuantityUnitType`	`UnitTypeEnum?`	S	Unidade de quantidade
`PlannedVolume`	`double?`	S	Volume planejado
`PlannedMass`	`double?`	S	Massa planejada
`ObservedVolume`	`double?`	S	Volume observado
`AdjustedVolume`	`double?`	S	Volume ajustado (corrigido)
`AverageDensity`	`double?`	S	Densidade média
`MassAir`	`double?`	S	Massa ao ar
`MassVacuum`	`double?`	S	Massa no vácuo
`CurrentState`	`ProgramOperationStateEnum?`	S	Estado atual
`Created`	`DateTimeOffset`	N	Data de criação
`Updated`	`DateTimeOffset?`	S	Data da última atualização

operations_OperationStates

Coluna	Tipo	Nulo	Descrição
`OperationId`	`string`	N	**PK/FK** → `Operations.Id` (CASCADE)
`State`	`ProgramOperationStateEnum`	N	**PK** — Estado
`Start`	`DateTimeOffset`	N	**PK** — Início
`End`	`DateTimeOffset?`	S	Fim do estado

operations_OperationPipelines

Pipelines utilizados durante a operação com medições de início e fim.

Coluna	Tipo	Nulo	Descrição
`OperationId`	`string`	N	**PK/FK** → `Operations.Id` (CASCADE)
`PipelineId`	`string`	N	**PK/FK** → `Pipelines.Id` (RESTRICT)
`IsStartFull`	`bool`	N	Pipeline cheio no início
`IsEndFull`	`bool`	N	Pipeline cheio no fim
`StartDateTime`	`DateTimeOffset`	N	Data/hora medição inicial
`StartAverageTemperature`	`double?`	S	Temperatura média inicial
`StartObservedVolume`	`double?`	S	Volume observado inicial

Coluna	Tipo	Nulo	Descrição
`StartDensityFactor`	`double?`	S	Fator de densidade inicial
`StartAdjustedVolume`	`double?`	S	Volume ajustado inicial
`StartAverageDensity`	`double?`	S	Densidade média inicial
`StartBaseDensity`	`double?`	S	Densidade base inicial
`StartMassVacuum`	`double?`	S	Massa no vácuo inicial
`StartMassAirFactor`	`double?`	S	Fator de massa ao ar inicial
`StartMassAir`	`double?`	S	Massa ao ar inicial
`EndDateTime`	`DateTimeOffset`	N	Data/hora medição final
`End...` (mesmos campos)		S	Medições finais (mesma estrutura)

operations_OperationTransactions

Coluna	Tipo	Nulo	Descrição
`OperationId`	`string`	N	**PK/FK** → `Operations.Id` (CASCADE)
`TransactionId`	`string`	N	**PK/FK** → `Transactions.Id` (RESTRICT)

10.3. Tanques de Operação

operations_OperationTanks

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`OperationId`	`string?`	S	**FK** → `Operations.Id` (ClientCascade)
`StorageTankId`	`string`	N	**FK** → `StorageTanks.Id` (RESTRICT)

operations_OperationTankMeasures

Coluna	Tipo	Nulo	Descrição
`OperationTankId`	`string`	N	**PK/FK** → `OperationTanks.Id` (CASCADE)
`Id`	`string`	S	**PK**
`DateTime`	`DateTimeOffset`	N	Data/hora da medição
`IsStart`	`bool`	N	Medição de início
`IsEnd`	`bool`	N	Medição de fim

Coluna	Tipo	Nulo	Descrição
`Level`	`double?`	S	Nível
`Temperature`	`double?`	S	Temperatura
`TotalObservedVolume`	`double?`	S	Volume total observado
`FloatRoofAdjustment`	`double?`	S	Ajuste do teto flutuante
`GrossObservedVolume`	`double?`	S	Volume bruto observado
`DensityFactor`	`double?`	S	Fator de densidade
`GrossStandardVolume`	`double?`	S	Volume bruto padrão
`AverageDensity`	`double?`	S	Densidade média
`BaseDensity`	`double?`	S	Densidade base
`MassAir`	`double?`	S	Massa ao ar
`MassAirFactor`	`double?`	S	Fator de massa ao ar
`MassVacuum`	`double?`	S	Massa no vácuo

operations_OperationTankPrograms

Coluna	Tipo	Nulo	Descrição
`OperationTankId`	`string`	N	**PK/FK** → `OperationTanks.Id` (CASCADE)
`Id`	`string`	S	**PK**
`LinkedProgramId`	`string?`	S	**FK** → `Programs.Id` (RESTRICT)

10.4. Operadores

operations_Operators

Coluna	Tipo	Nulo	Descrição
`OperationId`	`string`	N	**PK/FK** → `Operations.Id` (CASCADE)
`Id`	`string`	S	**PK**
`Name`	`string`	N	Nome do operador
`Start`	`DateTimeOffset`	N	Início do turno
`End`	`DateTimeOffset`	N	Fim do turno

10.5. Drenagens

operations_Drainages

Coluna	Tipo	Nulo	Descrição
`OperationId`	`string`	N	**PK/FK** → `Operations.Id` (CASCADE)
`Id`	`string`	S	**PK**
`AdjustedVolume`	`double?`	S	Volume ajustado da drenagem
`MassVacuum`	`double?`	S	Massa no vácuo
`MassAir`	`double?`	S	Massa ao ar

10.6. Manifestos

operations_Manifests

Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`OperationId`	`string?`	S	**FK** → `Operations.Id` (CASCADE)
`Identification`	`string?`	S	Identificação do manifesto
`Client`	`string?`	S	Cliente
`PlannedVolume`	`double?`	S	Volume planejado
`PlannedMassAir`	`double?`	S	Massa ao ar planejada
`PlannedMassVacuum`	`double?`	S	Massa no vácuo planejada
`ActualVolume`	`double?`	S	Volume real
`ActualMassAir`	`double?`	S	Massa ao ar real
`ActualMassVacuum`	`double?`	S	Massa no vácuo real
`ShipVolume`	`double?`	S	Volume do navio
`ShipMassAir`	`double?`	S	Massa ao ar do navio
`ShipMassVacuum`	`double?`	S	Massa no vácuo do navio
`CurrentState`	`ProgramOperationManifestStateEnum?`	S	Estado atual
`StorageTankId`	`string?`	S	**FK** → `StorageTanks.Id` (RESTRICT)

operations_ManifestStates

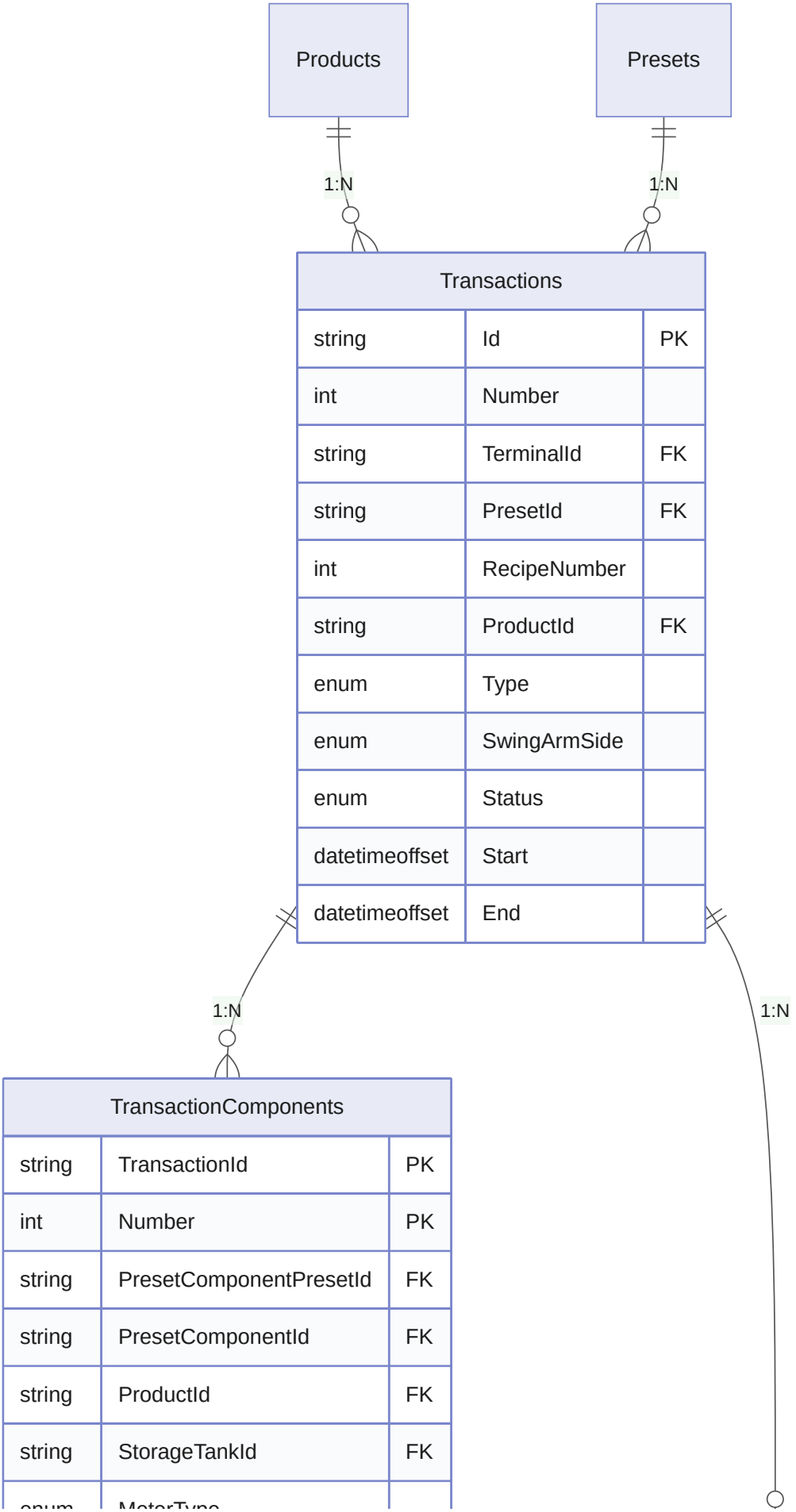
Coluna	Tipo	Nulo	Descrição
`ManifestId`	`string`	N	**PK/FK** → `Manifests.Id` (CASCADE)
`State`	`ProgramOperationManifestStateEnum`	N	**PK** — Estado
`Start`	`DateTimeOffset`	N	**PK** — Início
`End`	`DateTimeOffset?`	S	Fim do estado

---

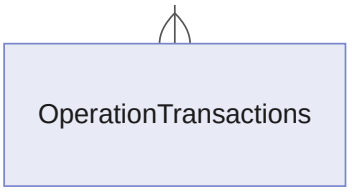
11. TenantContext — Domínio Operations: Transações (2 tabelas)

Transações representam os registros de carregamento/descarregamento gerados pelos equipamentos de campo.

11.1. Diagrama ER


Products
Presets
1:N
1:N
Transactions
string Id PK
int Number
string TerminalId FK
string PresetId FK
int RecipeNumber
string ProductId FK
enum Type
enum SwingArmSide
enum Status
datetimeoffset Start
datetimeoffset End
1:N
1:N
TransactionComponents
string TransactionId PK
int Number PK
string PresetComponentPresetId FK
string PresetComponentId FK
string ProductId FK
string StorageTankId FK

enum	MeterType	
double	AverageTemperature	
double	ObservedVolume	
double	AdjustedVolume	
double	AverageDensity	
double	BaseDensity	
double	Concentration	
double	MassAir	
double	MassVacuum	
double	StartTotalizer	
double	EndTotalizer	



11.2. Transações

operations_Transactions

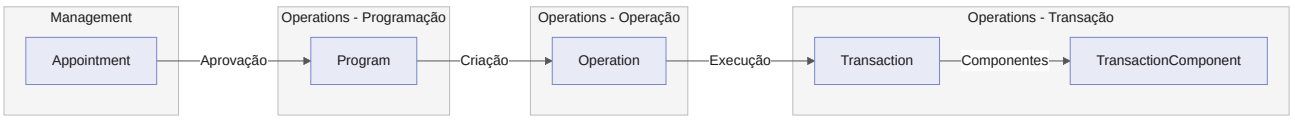
Coluna	Tipo	Nulo	Descrição
`Id`	`string`	S	**PK**
`Number`	`int?`	S	Número da transação
`TerminalId`	`string?`	S	**FK** → `Organizations.Id` (RESTRICT)
`PresetId`	`string?`	S	**FK** → `Presets.Id` (RESTRICT)
`RecipeNumber`	`int?`	S	Número da receita utilizada
`ProductId`	`string?`	S	**FK** → `Products.Id` (RESTRICT)
`Type`	`TransactionTypeEnum?`	S	Tipo (OUTBOUND, INBOUND, etc.)
`SwingArmSide`	`SwingArmSideEnum?`	S	Lado do braço de carregamento
`Status`	`TransactionStatusEnum?`	S	Status (default: PENDING)
`Start`	`DateTimeOffset?`	S	Início da transação
`End`	`DateTimeOffset?`	S	Fim da transação

operations_TransactionComponents

Coluna	Tipo	Nulo	Descrição
`TransactionId`	`string`	N	**PK/FK** → `Transactions.Id` (CASCADE)
`Number`	`int`	N	**PK** — Número do componente
`PresetComponentPresetId`	`string?`	S	FK → `PresetComponents` (parte 1, SET NULL)
`PresetComponentId`	`string?`	S	FK → `PresetComponents` (parte 2, SET NULL)
`ProductId`	`string?`	S	**FK** → `Products.Id` (RESTRICT)
`StorageTankId`	`string?`	S	**FK** → `StorageTanks.Id` (RESTRICT)
`MeterType`	`MeterTypeEnum?`	S	Tipo do medidor
`AverageTemperature`	`double?`	S	Temperatura média
`ObservedVolume`	`double?`	S	Volume observado
`AdjustedVolume`	`double?`	S	Volume ajustado
`AverageDensity`	`double?`	S	Densidade média
`BaseDensity`	`double?`	S	Densidade base
`Concentration`	`double?`	S	Concentração
`MassAir`	`double?`	S	Massa ao ar
`MassVacuum`	`double?`	S	Massa no vácuo
`StartTotalizer`	`double?`	S	Totalizador inicial
`EndTotalizer`	`double?`	S	Totalizador final

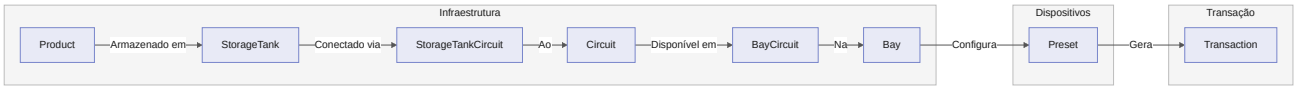
12. Fluxos de Dados Integrados

12.1. Agendamento até Operação



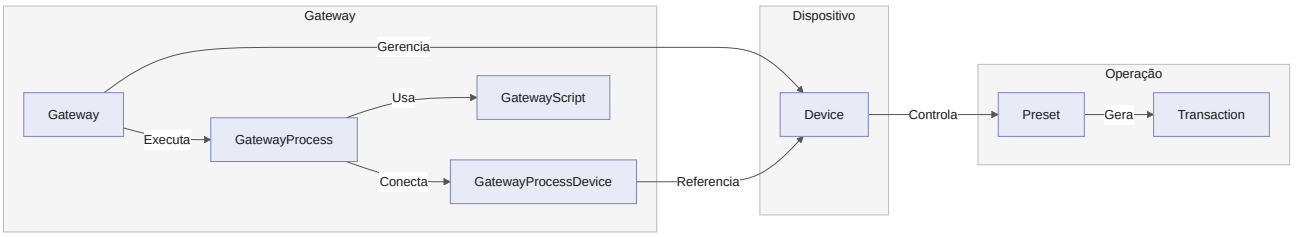
Fluxo: Um `Appointment` (agendamento) gera um `Program` (programação). O Program contém uma ou mais `Operations` (operações de carregamento/descarregamento). Cada Operation é associada a `Transactions` (transações de medição enviadas pelo equipamento de campo). Cada Transaction possui `TransactionComponents` com os dados volumétricos por componente.

12.2. Produto até Carregamento



Fluxo: Um `Product` é armazenado em `StorageTanks`, conectados a `Circuits` via `StorageTankCircuits`. Os Circuits são disponibilizados em `Bays` via `BayCircuits`. Cada Bay possui `Presets` configurados que definem como o carregamento é realizado pelo equipamento. O Preset gera `Transactions` com os dados de medição.

12.3. Gateway até Transação



Fluxo: Um `Gateway` é o intermediário entre o Autoload e os equipamentos de campo. Ele gerencia `Devices` (PLCs, controladores) e executa `GatewayProcesses` (scripts de comunicação). Cada processo usa um `GatewayScript` e conecta aliases a `Devices` via `GatewayProcessDevices`. Os Devices controlam `Presets`, que geram `Transactions`.

---

13. Conclusão

O banco de dados do Autoload V4 é estruturado sobre uma **arquitetura dual-context** que separa dados globais (9 tabelas no `AutoloadContext`) de dados operacionais isolados por cliente (83 tabelas no `TenantContext`), totalizando **92 tabelas**. A estratégia **database-per-tenant** garante isolamento completo entre organizações, enquanto o banco global centraliza autenticação, permissões e auditoria administrativa.

O modelo de dados organiza as 83 tabelas do tenant em **cinco domínios** bem definidos: Commons (13 tabelas de configuração e auditoria), Management (18 tabelas de frota e agendamento), Infraestrutura (18 tabelas de tanques, circuitos e baías), Dispositivos e Gateways (11 tabelas de comunicação com equipamentos de campo), Programações (10 tabelas de planejamento de carga), Operações (11 tabelas de execução) e Transações (2 tabelas de medição volumétrica).

Padrões arquiteturais consistentes permeiam toda a base: **entidades base** (`BaseEntity`, `TenantBaseEntity`) garantem campos padronizados de auditoria e rastreabilidade; **soft delete** preserva histórico sem perda de dados; **índices compostos** e **chaves únicas** asseguram integridade referencial; e **colunas JSON** (`JsonElement`) oferecem flexibilidade para dados dinâmicos sem sacrificar a estrutura relacional. Os **fluxos de dados integrados** — do agendamento à transação, do produto ao carregamento, e do gateway ao equipamento — demonstram como os domínios se conectam para suportar o ciclo completo de operações de carregamento e descarregamento de combustíveis.

Documento atualizado em: 11 de março de 2026
Versão: 1.0
Autor: Milton Kiefer de Albuquerque Mello
Validado por: Daniel Ribeiro
Total de tabelas documentadas: 92 (9 globais + 83 tenant)

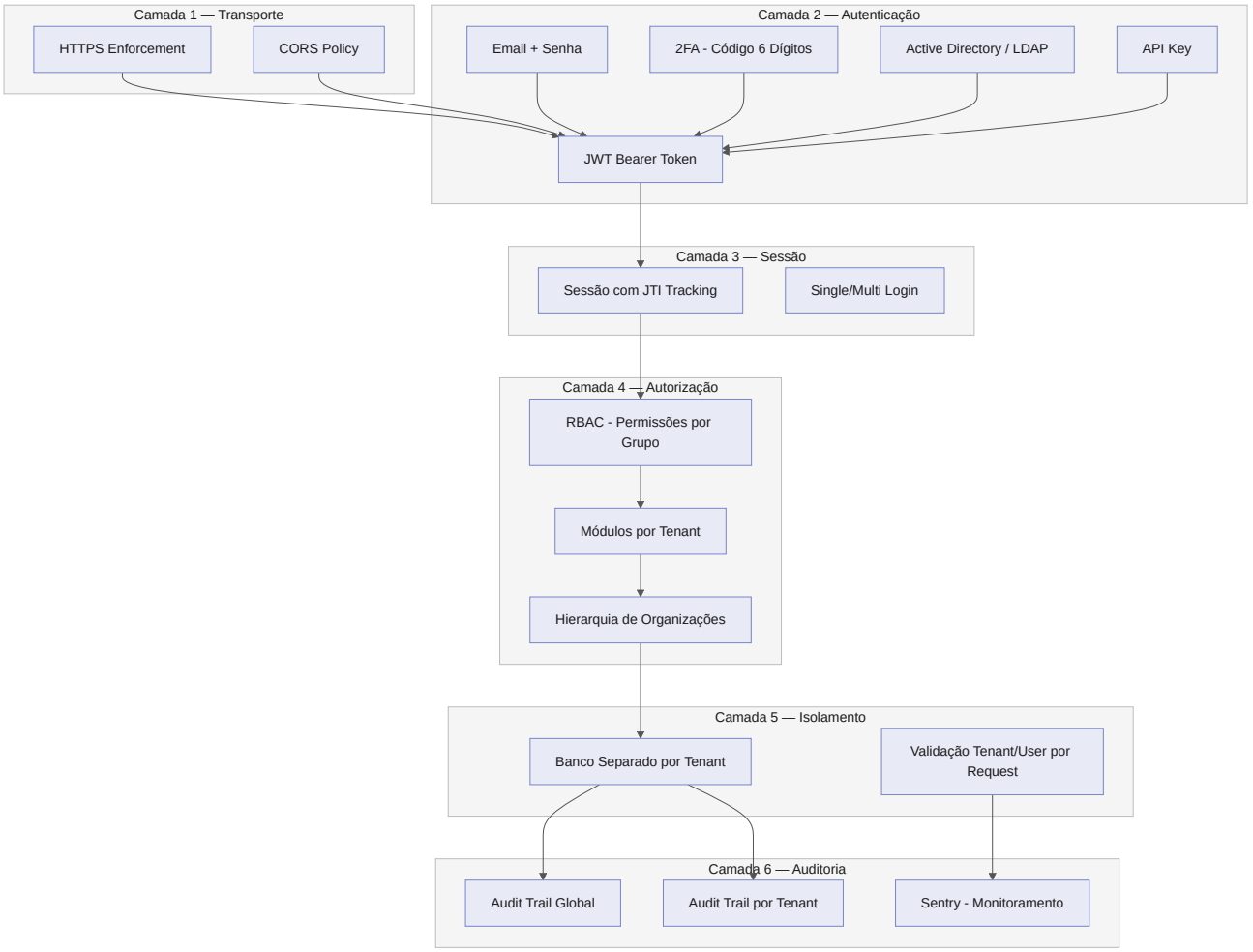
VII. Segurança

Autenticação, autorização e práticas de proteção de dados.

1. Visão Geral da Segurança

O Autoload V4 implementa um modelo de segurança em camadas que combina **autenticação multi-modo**, **autorização baseada em papéis (RBAC)** e **isolamento multi-tenant** com banco de dados separado por tenant. A arquitetura segue o princípio de defesa em profundidade, onde cada camada adiciona uma barreira de proteção independente.

1.1. Camadas de Segurança



> **OBSERVAÇÃO**
>
> O Produto é flexível e as camadas de segurança podem ser customizáveis por cliente durante a instalação

1.2. Tecnologias e Padrões

Componente	Tecnologia	Descrição
Autenticação	JWT Bearer + HMAC-SHA256	Tokens com claims de usuário, tenant e permissões
Hashing de Senhas	PBKDF2-SHA256	Salt 128-bit, key 256-bit, 10.000 iterações
Autorização	RBAC (Role-Based Access Control)	55 permissões vinculadas a módulos e grupos
Diretório	Active Directory / LDAP	Integração opcional via protocolo LDAP
Multi-Tenant	Database-per-Tenant	Banco de dados isolado por tenant com `TenantContext` separado
Auditoria	Dual Audit Trail	Registros globais (`AutoloadAuditTrail`) e por tenant (`TenantAuditTrail`)
Monitoramento	Sentry	Captura de exceções com contexto de tenant e usuário
Validação	Data Annotations Customizadas	Atributos `NoWhiteSpace` , `ValidEnumValue` , `GreaterThanZero`

---

2. Autenticação

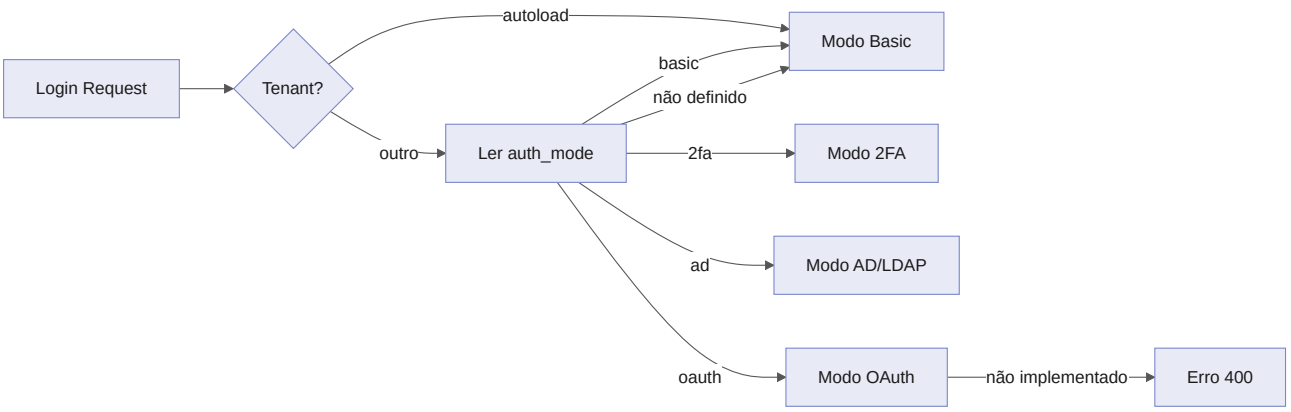
O Autoload V4 suporta quatro modos de autenticação, configuráveis por tenant via parâmetro `auth_mode` . O tenant administrativo `"autoload"` sempre utiliza o modo `basic` .

2.1. Modos de Autenticação

Modo	Chave	Descrição	Status
Basic	`basic`	Email + senha com hash PBKDF2	Implementado
2FA	`2fa`	Email + senha + código 6 dígitos por email	Implementado
Active Directory	`ad`	Autenticação via LDAP/AD corporativo	Implementado
OAuth	`oauth`	Autenticação via provedor OAuth externo	Não implementado

Resolução do modo de autenticação:

```
Tenant = "autoload" → sempre "basic"
Tenant ≠ "autoload" → parâmetro "auth_mode" do ParameterService (default: "basic")
```



2.2. JWT Tokens

Todos os modos de autenticação, ao validar com sucesso, geram um **JWT Bearer Token** assinado com HMAC-SHA256. O token é o artefato central de identidade do sistema.

Configuração do JWT:

Parâmetro	Valor	Origem
Algoritmo de assinatura	HMAC-SHA256	Hardcoded
Chave de assinatura	`Jwt:Key`	`appsettings.json` / variável de ambiente
Expiração	1 dia	Hardcoded (`DateTime.Now.AddDays(1)`)
Validar Issuer	`false`	`ServiceCollectionExtensions`
Validar Audience	`false`	`ServiceCollectionExtensions`
Validar Lifetime	`true`	`ServiceCollectionExtensions`
Validar Signing Key	`true`	`ServiceCollectionExtensions`

Estrutura de Claims do JWT:

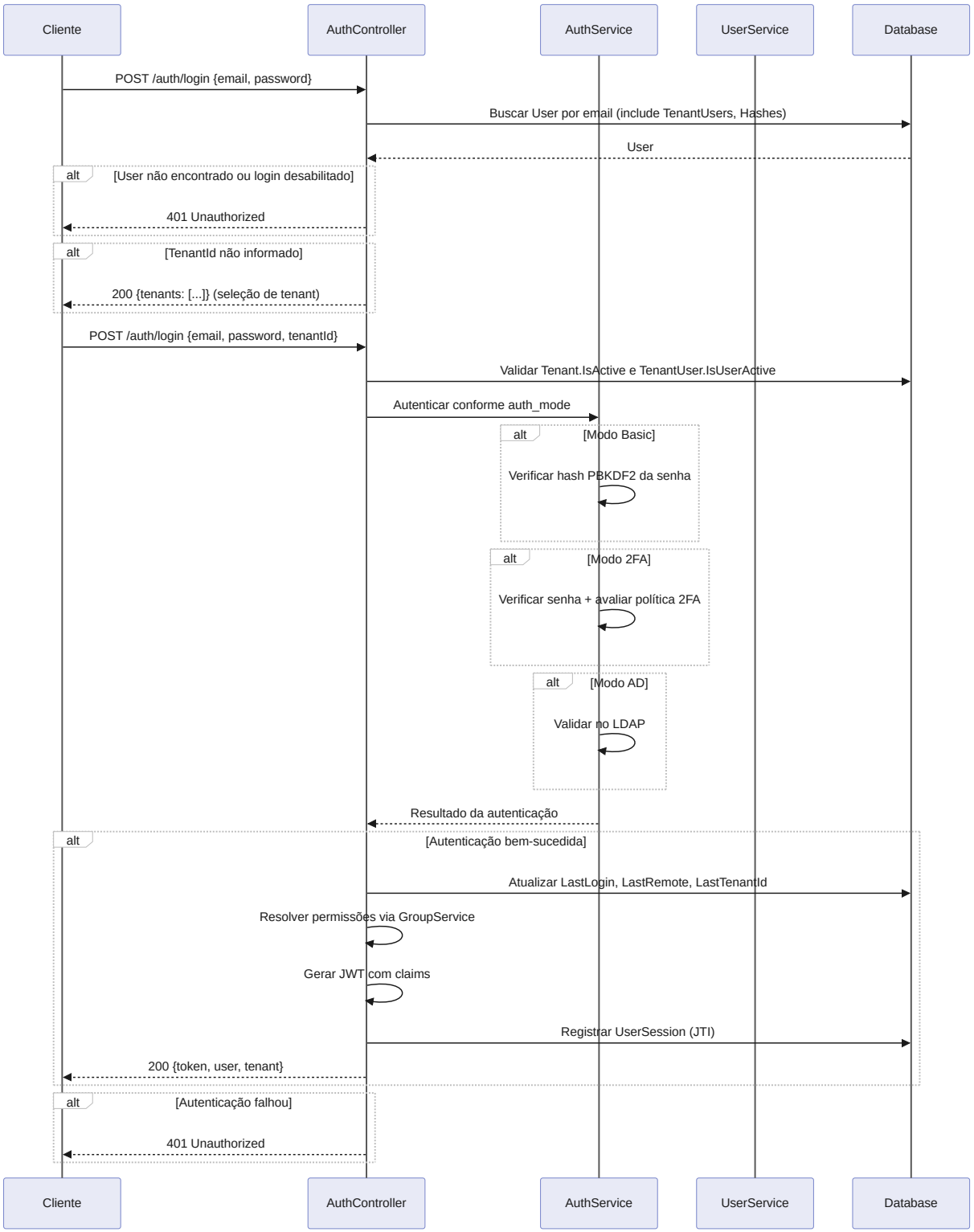
Claim	Tipo	Exemplo
`NameIdentifier`	`string`	ID do usuário (GUID hex 32 chars)
`Email`	`string`	`usuario@empresa.com`
`Name`	`string`	`Nome do Usuário`
`Jti`	`string`	ID único da sessão (GUID hex 32 chars)
`Tenant`	`string` (custom)	`empresa_abc`
`Locale`	`string` (custom)	`pt-BR` (default)
`Timezone`	`string` (custom)	`America/Sao_Paulo` (default)

Claim	Tipo	Exemplo
`AuthenticationMethod`	`string[]`	`["email", "basic"]`, `["email", "2fa"]`, `["key"]`
`Role`	`string[]`	Lista de permissões: `["user_management", "group_management", ...]`

Esquema de autenticação dual: O pipeline aceita tanto **JWT Bearer** quanto **Negotiate (Windows/Kerberos)**, com política padrão de autorização exigindo pelo menos um dos dois.

2.3. Login via Email/Senha

O fluxo principal de login segue um processo em duas etapas quando o usuário pertence a múltiplos tenants.



Validações executadas na ordem:

1. Usuário existe no banco global
2. `IsLoginAccessEnabled` está ativo
3. Usuário possui pelo menos um tenant associado
4. `TenantId` informado — tenant está ativo (`Tenant.IsActive`)
5. Usuário está ativo no tenant (`TenantUser.IsUserActive`)

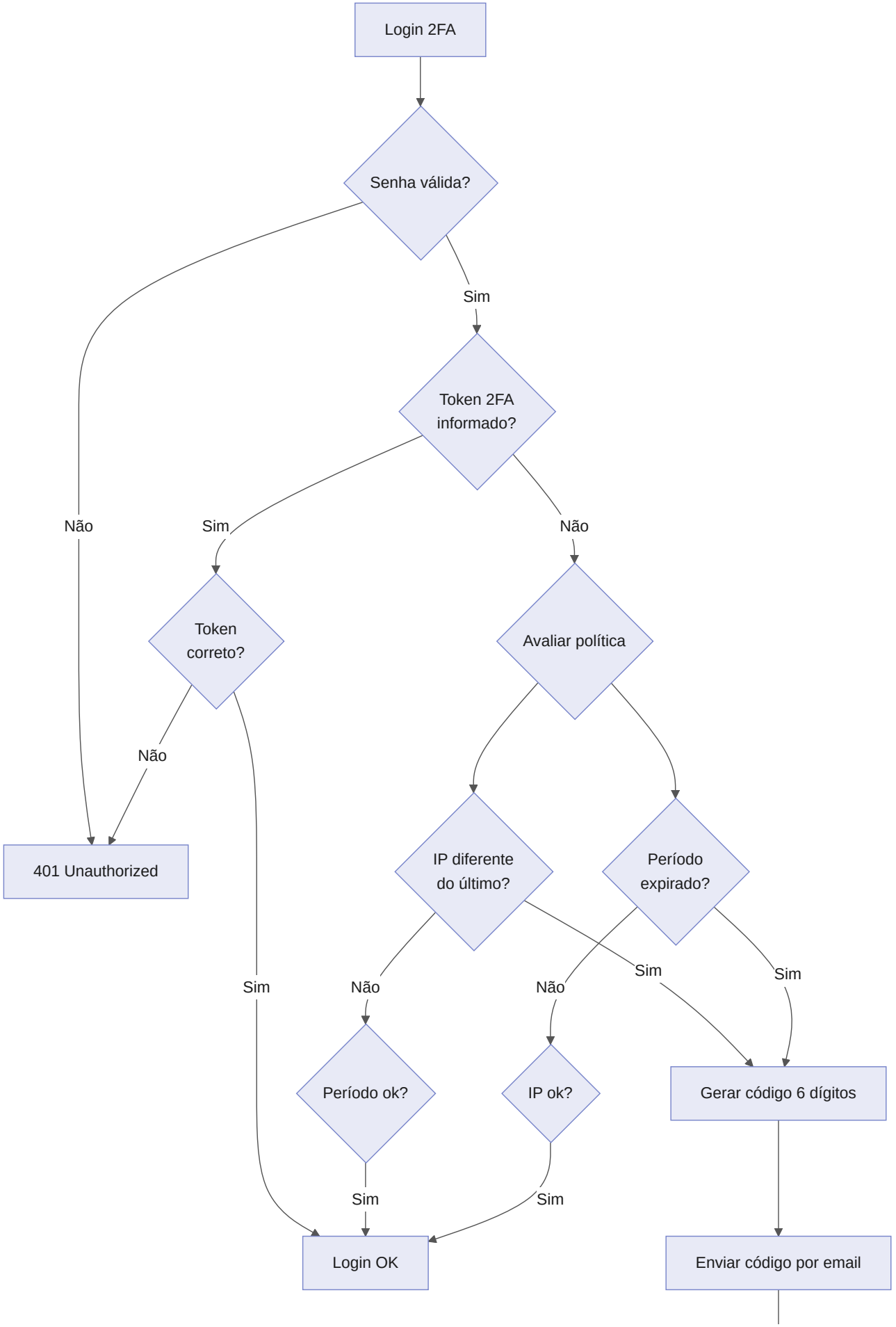
6. Política de sessão concorrente (`same_user_login`)
7. Autenticação conforme modo configurado
8. Geração do token JWT

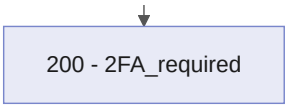
2.4. Autenticação 2FA

O modo 2FA estende o modo basic com uma camada adicional de verificação por código de 6 dígitos enviado por email.

Política 2FA (configurável via parâmetro `2fa_policy` por tenant):

Parâmetro	Tipo	Default	Descrição
`required_from_other_remote`	`bool`	`true`	Exigir 2FA quando IP é diferente do último login
`required_period`	`int` (dias)	—	Exigir 2FA após N dias desde o último login


Login 2FA
Senha válida?
Não
Sim
Token 2FA informado?
Não
Sim
Não
Token correto?
Avaliar política
Não
401 Unauthorized
IP diferente do último?
Período expirado?
Sim
Não
Não
Sim
Sim
Período ok?
IP ok?
Gerar código 6 dígitos
Sim
Sim
Login OK
Enviar código por email



Geração do código 2FA:

- Método: `ServicesHelper.Create2FAToken()`
- Formato: 6 dígitos zero-padded (`000000` a `999999`)
- Armazenamento: campo `User.TwoFactorToken` no banco
- Entrega: enviado ao email do usuário
- Limpeza: `User.Clean2FA()` após validação bem-sucedida

2.5. Login via API Key

O sistema suporta autenticação por API Key para integrações de máquina-a-máquina (M2M), sem necessidade de senha.

Modelo UserKey:

Campo	Tipo	Descrição
`Key`	`string`	Chave de API única
`Name`	`string`	Nome descritivo da chave
`IsActive`	`bool`	Se a chave está ativa
`Expiration`	`DateTimeOffset`	Data de expiração (`MinValue` = sem expiração)
`LastAccess`	`DateTimeOffset?`	Último uso da chave

Validações na autenticação por API Key:

1. Usuário existe
2. Chave existe na coleção `User.Keys`
3. `User.IsKeyAccessEnabled = true`
4. `UserKey.IsActive = true`
5. `UserKey.Expiration` não expirou (comparação com `DateTimeOffset.UtcNow`)
6. Usuário possui pelo menos um tenant

Claims do JWT gerado: O método de autenticação é registrado como `["key"]` (ao invés de `["email", "basic"]`).

2.6. Integração Active Directory (LDAP)

O modo AD permite autenticação contra um servidor LDAP/Active Directory corporativo.

Configuração LDAP (parâmetro `ldap` do `ParameterService`):

Propriedade	Tipo	Default	Descrição
`domain`	`string`	**Obrigatório**	Domínio LDAP
`base_dn`	`string`	**Obrigatório**	Distinguished Name base para busca
`query`	`string`	**Obrigatório**	Filtro LDAP (template com `{username}`)

Propriedade	Tipo	Default	Descrição
`port`	`int`	`389`	Porta do servidor LDAP
`password`	`string`	—	Se `"true"`, senha enviada vazia (service account)

Fluxo de autenticação LDAP:

1. Extrair username do email (`email.Split("@")[0]`)
2. Conectar ao servidor LDAP com domain, baseDN, query e port configurados
3. Validar existência do usuário via `UserService.ValidateLdapUserExists()`
4. Se válido, gerar JWT normalmente

Restrição: Reset de senha é **bloqueado** para tenants com modo `ad` — o gerenciamento de senhas é responsabilidade do Active Directory.

2.7. Hashing de Senhas

O sistema utiliza **PBKDF2-SHA256** para armazenamento seguro de senhas.

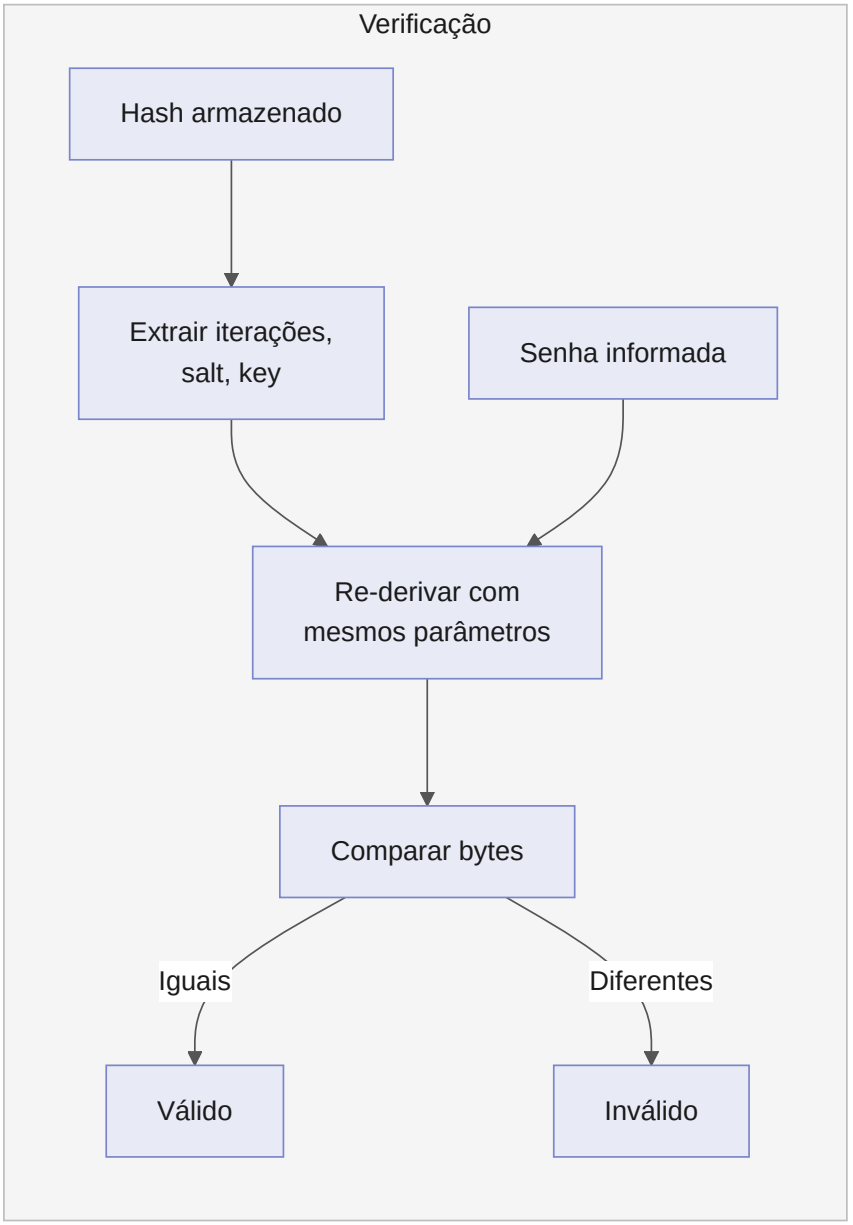
Parâmetros do algoritmo:

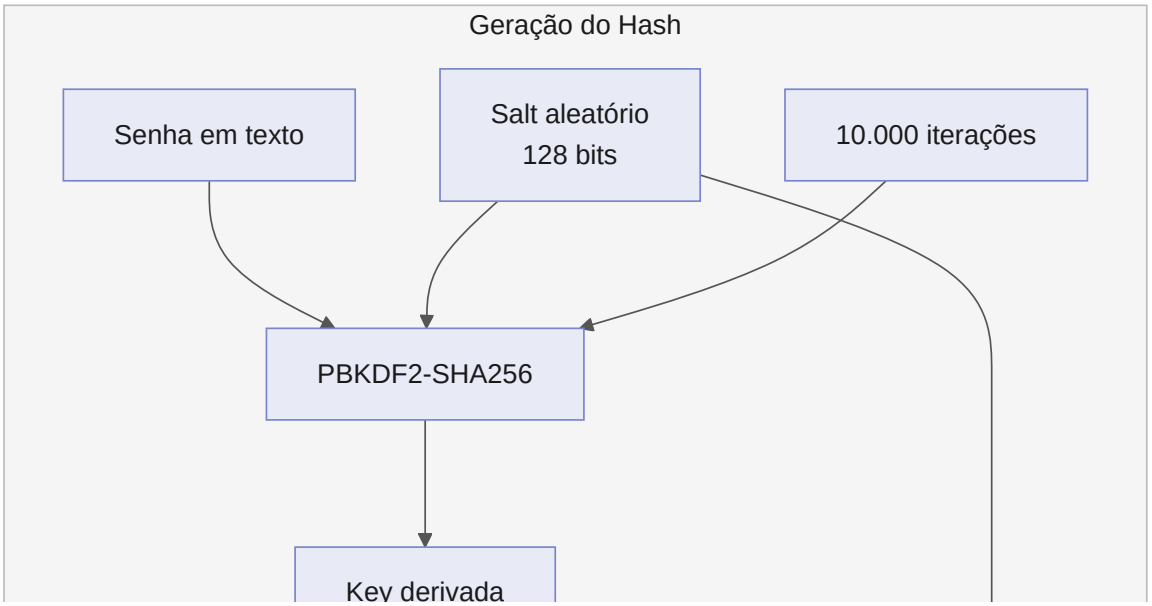
Parâmetro	Valor
Algoritmo	`Rfc2898DeriveBytes` (PBKDF2)
Hash interno	SHA-256
Tamanho do salt	16 bytes (128 bits)
Tamanho da key	32 bytes (256 bits)
Iterações	10.000

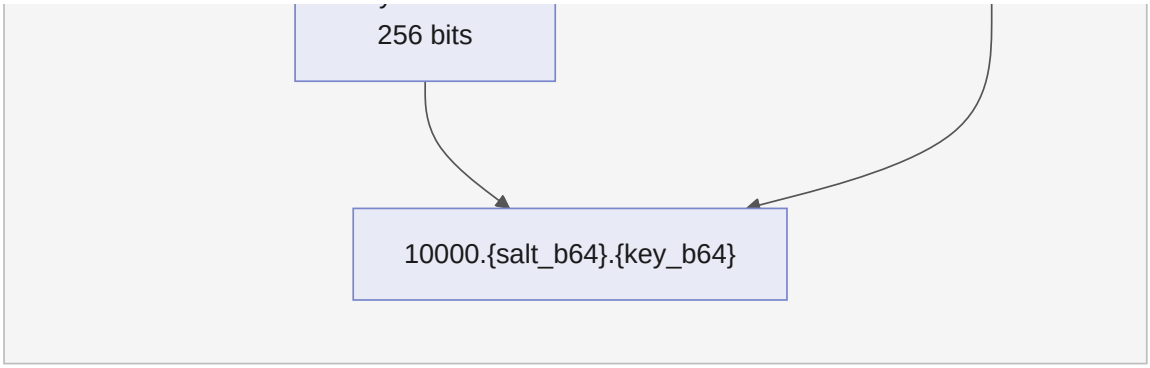
Formato de armazenamento:

```
{iterações}.{salt_base64}.{key_base64}
```

Exemplo: `10000.abc123salt==.xyz789key==`


Verificação
Hash armazenado
Extrair iterações, salt, key
Senha informada
Re-derivar com mesmos parâmetros
Comparar bytes
Iguais
Diferentes
Válido
Inválido


Geração do Hash
Senha em texto
Salt aleatório 128 bits
10.000 iterações
PBKDF2-SHA256
Key derivada



2.8. Políticas de Senha

As políticas de senha são configuráveis por tenant via parâmetro `password_policy` do `ParameterService`.

Regras configuráveis:

Regra	Chave JSON	Tipo	Descrição
Comprimento máximo	`maxLength`	`int`	Número máximo de caracteres
Comprimento mínimo	`minLength`	`int`	Número mínimo de caracteres
Letras maiúsculas	`minUpper`	`int`	Mínimo de caracteres maiúsculos
Letras minúsculas	`minLower`	`int`	Mínimo de caracteres minúsculos
Dígitos	`minNumber`	`int`	Mínimo de caracteres numéricos
Caracteres especiais	`minSpecial`	`int`	Mínimo de caracteres especiais
Caracteres especiais permitidos	`specialChars`	`string`	Conjunto de caracteres considerados especiais
Histórico de senhas	`lastUsedPasswords`	`int`	Quantidade de senhas anteriores que não podem ser reutilizadas

Comportamento:

- Todas as violações são coletadas e retornadas juntas como `ServiceExceptionCollection`
- O tenant `"autoload"` (administrativo) **não aplica** política de senha — permite qualquer formato
- O histórico de senhas é mantido via entidades `UserHash`

2.9. Gerenciamento de Sessões

O sistema utiliza `UserSession` para rastrear sessões ativas via JTI (JWT ID) tracking.

Modelo UserSession:

Campo	Tipo	Descrição
`UserId`	`string`	ID do usuário
`TokenId`	`string`	JTI do token JWT

Campo	Tipo	Descrição
`DeviceInfo`	`string`	User-Agent + IP (`{userAgent}|{ipAddress}`)
`ExpiresAt`	`DateTime`	Expiração da sessão (UTC + 30 minutos)

Modos de sessão (configurável via parâmetro `same_user_login`):

Parâmetro	Comportamento
`same_user_login = true`	Permite múltiplas sessões simultâneas (invalidando anteriores ao registrar nova)
`same_user_login = false`	Login bloqueado se já existe sessão ativa

Validação de sessão (middleware JWT):

O middleware `JwtSingleSessionExtensions` intercepta todo token JWT validado e, para tenants com `same_user_login = false`, verifica se o JTI do token corresponde a uma sessão ativa via `UserSessionService.ValidateTokenAsync()`. Se a sessão foi invalidada por um login mais recente, retorna **401 Unauthorized** com mensagem `"This session has been invalidated by a newer login"`.

O tenant `"autoload"` é isento desta validação (sempre permite sessões concorrentes).

Logout: O endpoint `POST /auth/logout` invalida **todas** as sessões do usuário chamando `InvalidateUserTokens(userId)`.

---

3. Autorização

O Autoload V4 implementa um modelo de **RBAC (Role-Based Access Control)** com permissões vinculadas a módulos, atribuídas via grupos e opcionalmente escopadas por organização.

3.1. Modelo de Permissões

O sistema define **55 permissões** no arquivo `Permissions.json`, cada uma com nomes localizados (en, pt-BR, es-ES, fr-FR). As permissões são organizadas por domínio funcional.

Permissões por categoria:

Categoria	Permissões
Core / Administração	`audit_management`, `custom_attribute_management`, `group_management`, `job_management`, `message_management`, `parameter_management`, `tenant_management`, `user_management`
Organizações	`organization_approval`, `organization_management`, `organization_view`
Terminais	`area_management`, `bay_management`, `circuit_management`, `pipeline_management`, `product_management`, `storagetank_management`, `transaction_management`
Programas	`program_management`, `program_pipeline_management`, `program_railway_management`, `program_roadway_management`, `program_waterway_management`
Dispositivos	`device_management`, `device_display_management`, `device_modbus_management`, `device_opc_management`, `gateway_management`
Controle de Carga	`preset_management`, `devices_loading_control_management`, `pump_management`

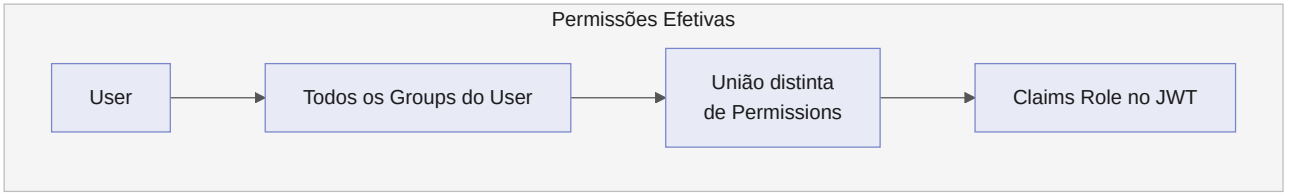
Categoria	Permissões
Controle de Acesso	`devices_uca_management`, `devices_truckscale_management`
Dashboard	`dashboard_management`, `dashboard_bay_widget`, `dashboard_bay_widget_management`, `dashboard_kiosk_widget`, `dashboard_kiosk_widget_management`, `dashboard_preset_widget`, `dashboard_preset_widget_management`, `dashboard_pump_widget`, `dashboard_pump_widget_management`, `dashboard_tank_widget`, `dashboard_tank_widget_management`, `dashboard_uca_widget`, `dashboard_uca_widget_management`
Veículos e Motoristas	`vehicle_management`, `vehicle_approval`, `driver_management`, `driver_approval`
Agendamentos	`appointment_approval`, `appointment_management`, `appointment_slot_management`
Filas	`queue_management`
Checklists	`checklist_management`
IA	`autointelligence`

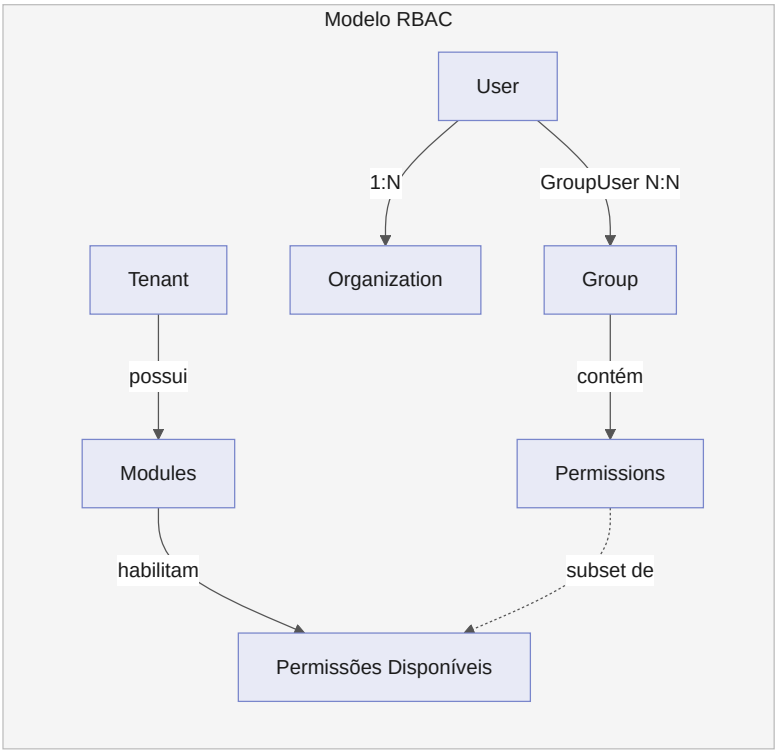
3.2. Grupos e Permissões

Grupos são a unidade de atribuição de permissões a usuários. Cada grupo pertence ao banco de dados de um tenant específico.

Modelo Group:

Campo	Tipo	Descrição
`Id`	`string`	Identificador único
`Name`	`string`	Nome do grupo (único por tenant)
`Permissions`	`List<string>`	Lista de IDs de permissões
`OrganizationId`	`string?`	Escopo por organização (null = global)
`IsActive`	`bool`	Se o grupo está ativo
`Users`	`List<GroupUser>`	Relação N:N com usuários





Regras de segurança do GroupService:

- Não é possível remover `group_management` do último grupo que a possui
- Não é possível remover `user_management` do último grupo que a possui
- Não é possível remover um usuário de um grupo se isso deixaria zero usuários com `user_management`
- Nomes de grupo são únicos dentro do tenant
- Grupos devem ser desativados (`IsActive = false`) antes de serem excluídos

Permissões do tenant administrativo `"autoload"`: Hardcoded como `["autoload", "audit_management", "job_management", "user_management", "tenant_management"]`.

3.3. Módulos e Feature Flags

Módulos controlam quais permissões estão disponíveis para cada tenant. O sistema define **13 módulos** com dependências entre si.

Módulos e suas dependências:

Módulo	Dependências	Permissões Concedidas
`core`	—	`audit_management`, `custom_attribute_management`, `group_management`, `job_management`, `message_management`, `organization_approval`, `organization_management`,

Módulo	Dependências	Permissões Concedidas
		`organization_view`, `parameter_management`, `tenant_management`, `user_management`
`terminals`	`core`	`area_management`, `bay_management`, `circuit_management`, `dashboard_*_tank/bay`, `pipeline_management`, `product_management`, `program_*`, `storagetank_management`, `transaction_management`
`gateways`	`core`	`device_management`, `device_display_management`, `device_modbus_management`, `device_opc_management`, `gateway_management`
`kiosks`	`terminals`, `gateways`	`dashboard_kiosk_widget`, `dashboard_kiosk_widget_management`
`queues`	`terminals`	`queue_management`
`access_control`	`terminals`, `gateways`	`devices_uca_management`, `dashboard_uca_widget*`
`weighing_control`	`terminals`, `gateways`	`devices_uca_management`, `devices_truckscale_management`
`load_unload_control`	`terminals`, `gateways`	`preset_management`, `devices_loading_control_management`, `dashboard_preset_widget*`
`pump_control`	`terminals`, `gateways`, `load_unload_control`	`pump_management`, `dashboard_pump_widget*`
`checklists`	`core`	`checklist_management`
`vehicles_drivers`	`core`	`vehicle_management`, `vehicle_approval`, `driver_management`, `driver_approval`
`appointments`	`vehicles_drivers`	`appointment_approval`, `appointment_management`, `appointment_slot_management`
`autointelligence`	`core`	`autointelligence`

Resolução de dependências: O `ModuleService.ValidateModules()` resolve dependências recursivamente e sempre inclui `core` como módulo base. Módulos desconhecidos geram `ServiceException`.

Validação Tenant → Módulo → Permissão: O `PermissionService.Validate()` garante que uma permissão só pode ser atribuída a um grupo se o tenant possui o módulo correspondente. O fluxo:

1. Ler `Tenant.Modules` (ex: `["terminals", "gateways"]`)
2. Para cada módulo, coletar as permissões que ele concede
3. Unir todas as permissões em `availablePermissions`
4. Validar que cada permissão solicitada está em `availablePermissions`

3.4. Autorização por Organização

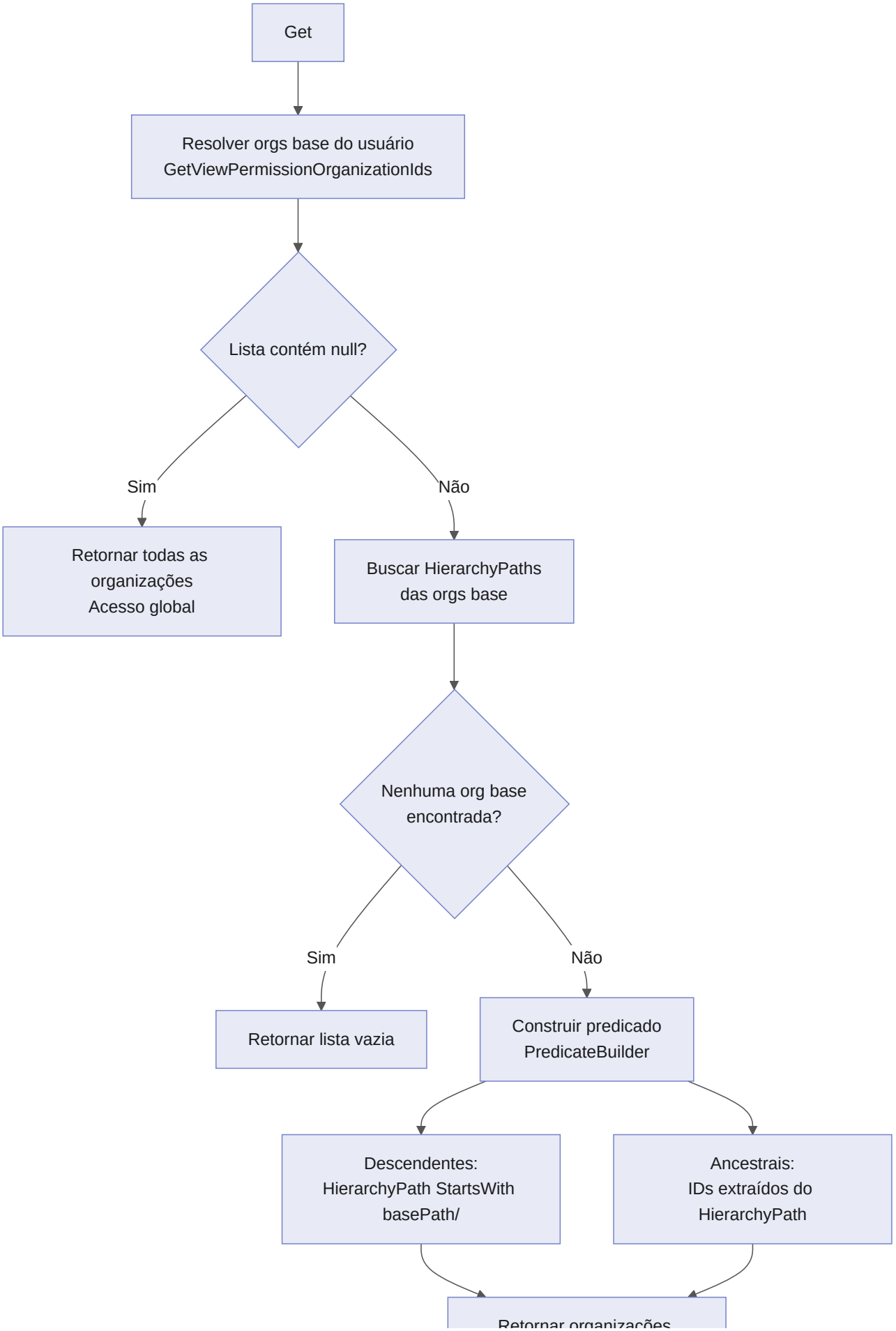
A autorização por organização determina quais organizações cada usuário pode visualizar ou gerenciar. As organizações acessíveis são resolvidas a partir dos grupos do usuário via `GetViewPermissionOrganizationIds(userId)`, que agrega todas as organizações associadas aos grupos do usuário com permissões `organization_view`, `organization_management` ou `organization_approval`.

Hierarquia de Organizações:

O campo `HierarchyPath` (materialized path) permite consultas eficientes de ancestrais e descendentes:

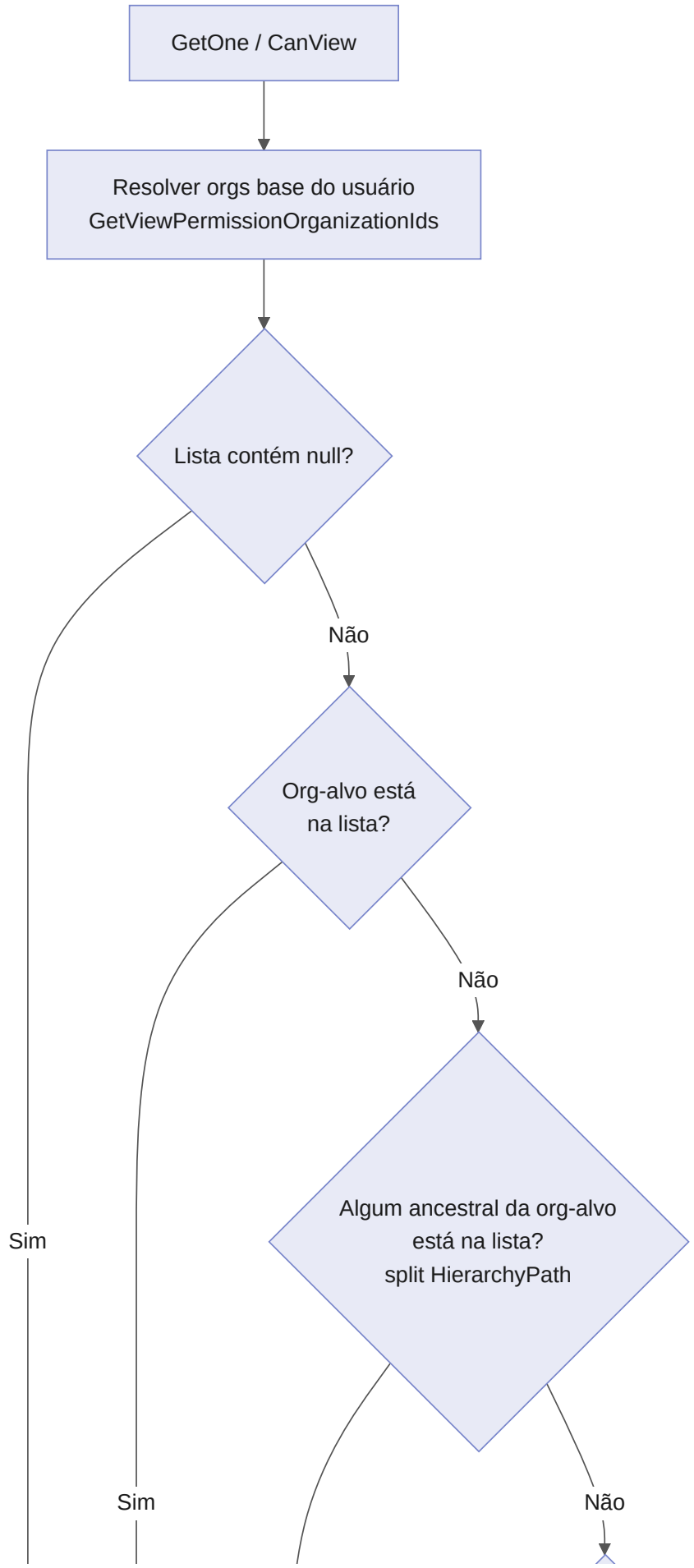
```
/org_raiz
/org_raiz/org_filial_sp
/org_raiz/org_filial_sp/org_terminal_01
/org_raiz/org_filial_rj
```

Diagrama A — `Get()` (listagem):


Get
Resolver orgs base do usuário
GetViewPermissionOrganizationIds
Lista contém null?
Sim
Não
Retornar todas as
organizações
Acesso global
Buscar HierarchyPaths
das orgs base
Nenhuma org base
encontrada?
Sim
Não
Retornar lista vazia
Construir predicado
PredicateBuilder
Descendentes:
HierarchyPath StartsWith
basePath/
Ancestrais:
IDs extraídos do
HierarchyPath
Retornar organizações

Retornar organizações filtradas

Diagrama B — `GetOne()` / `CanView()` (acesso individual):


GetOne / CanView
Resolver orgs base do usuário
GetViewPermissionOrganizationIds
Lista contém null?
Sim
Não
Org-alvo está
na lista?
Sim
Não
Algum ancestral da org-alvo
está na lista?
split HierarchyPath
Sim
Não

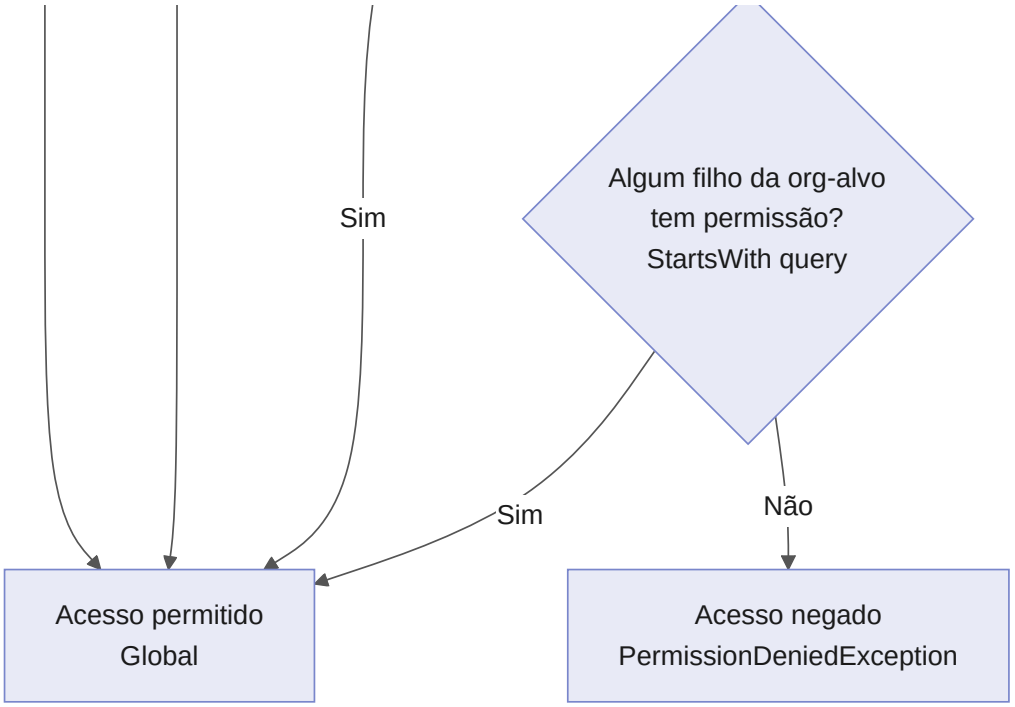
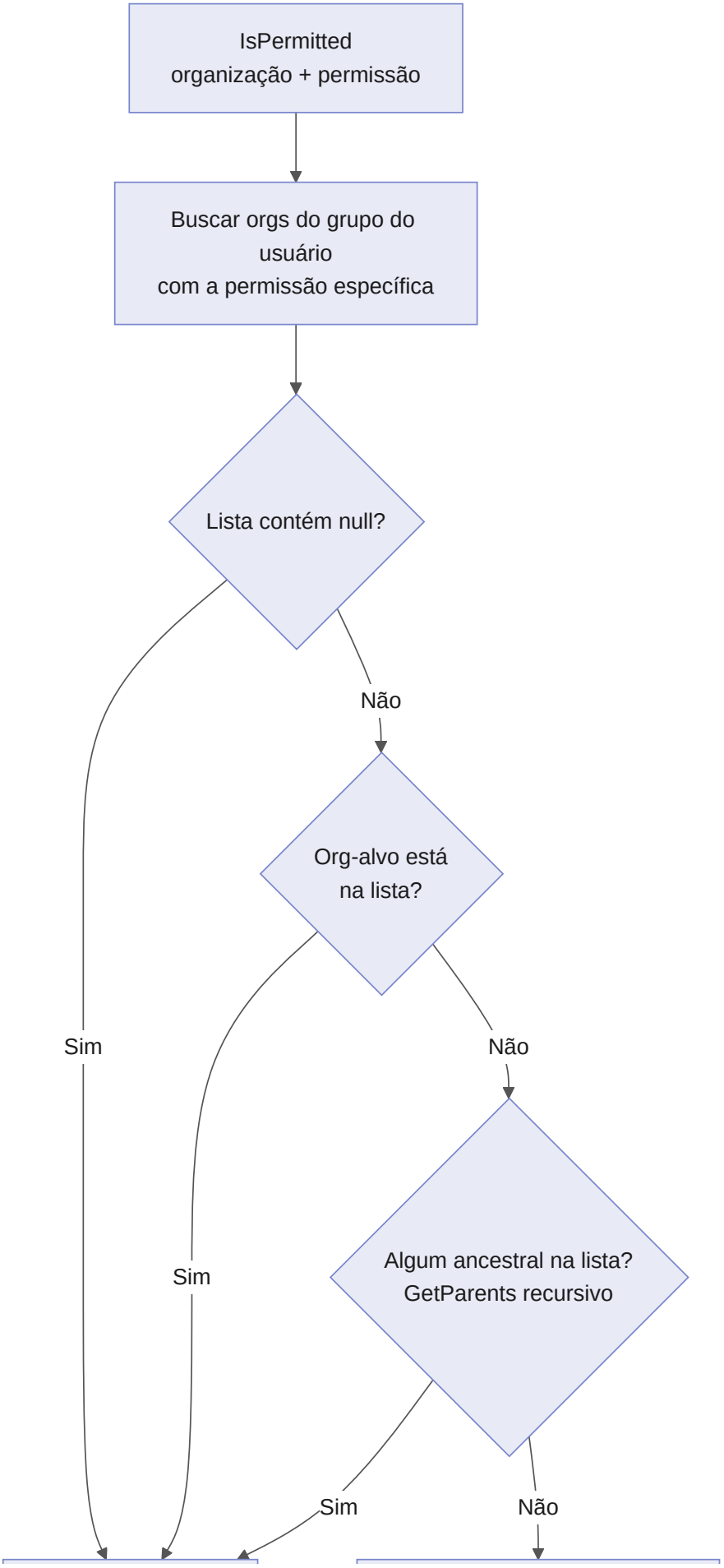
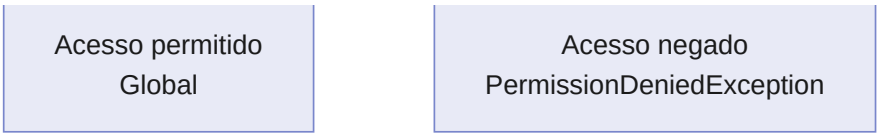


Diagrama C — `IsPermitted()` (escrita):


IsPermitted
organização + permissão
Buscar orgs do grupo do usuário
com a permissão específica
Lista contém null?
Sim
Não
Org-alvo está na lista?
Sim
Não
Algum ancestral na lista?
GetParents recursivo
Sim
Não



Regras de visibilidade — Leitura (`Get()` e `GetOne()`):

- **Permissões consideradas**: `organization_view`, `organization_management`, `organization_approval`
- **Global**: lista contém `null` → acesso a todas as organizações
- **Descendentes**: orgs cujo `HierarchyPath` começa com o path da org base (`StartsWith basePath/`)
- **Ancestrais**: IDs extraídos do `HierarchyPath` da org base (organizações pai na árvore)
- **Visibilidade ascendente**: em `CanView()`, se algum **filho** da org-alvo está na lista de orgs do usuário, o acesso à org-alvo é concedido

Regras de visibilidade — Escrita (`IsPermitted()`):

- **Permissão verificada**: a permissão específica da operação (ex: `organization_management`)
- **Global**: lista contém `null` → acesso permitido
- **Direto**: org-alvo na lista → acesso permitido
- **Ancestral**: algum ancestral da org-alvo na lista via `GetParents` recursivo → acesso permitido
- **Sem visibilidade ascendente**: filhos da org-alvo não concedem acesso
- **Verificação na org pai**: operações de escrita chamam `EnsurePermission(GetParent(organization), ...)` — a permissão é verificada na organização **pai**, não na própria org

Permissões por operação:

Operação	Permissão Requerida	Org Verificada
Criar / Atualizar	`organization_management`	Org **pai**
Ativar / Desativar / Excluir	`organization_management`	Org **pai**
Alterar pai	`organization_management`	Org **pai**
Aprovar / Rejeitar	`organization_approval`	Org **pai**
Comentar	`organization_management` OU `organization_approval`	Org **pai**
Visualizar (lista)	`organization_view` OU `organization_management` OU `organization_approval`	Via `Get()`
Visualizar (individual)	`organization_view` OU `organization_management` OU `organization_approval`	Via `CanView()`

3.5. Proteção de Endpoints

Todos os controllers utilizam o atributo `[Authorize(Roles = "...")]` para proteger endpoints. As permissões são verificadas contra as claims `Role` do JWT.

BaseController — classe base que extrai informações de segurança dos claims:

Propriedade	Claim	Default
`CurrentTenantId`	`Tenant`	`null`

Propriedade	Claim	Default
`CurrentUserId`	`NameIdentifier`	`null`
`CurrentAuthenticationMethod`	`AuthenticationMethod`	`[]`
`CurrentRoles`	`Role`	`[]`
`CurrentOrganizationId`	`Organization`	`null`
`CurrentLocale`	`Locale`	`pt-BR`
`CurrentTimezone`	`Timezone`	`America/Sao_Paulo`

Padrão de proteção nos controllers:

```
[Authorize(Roles = "user_management")]       → Requer permissão user_management
[Authorize(Roles = "organization_management,organization_approval")] → Requer qualquer uma
[Authorize]                                  → Requer apenas autenticação (qualquer permissão)
```

Endpoints públicos (sem `[Authorize]`):

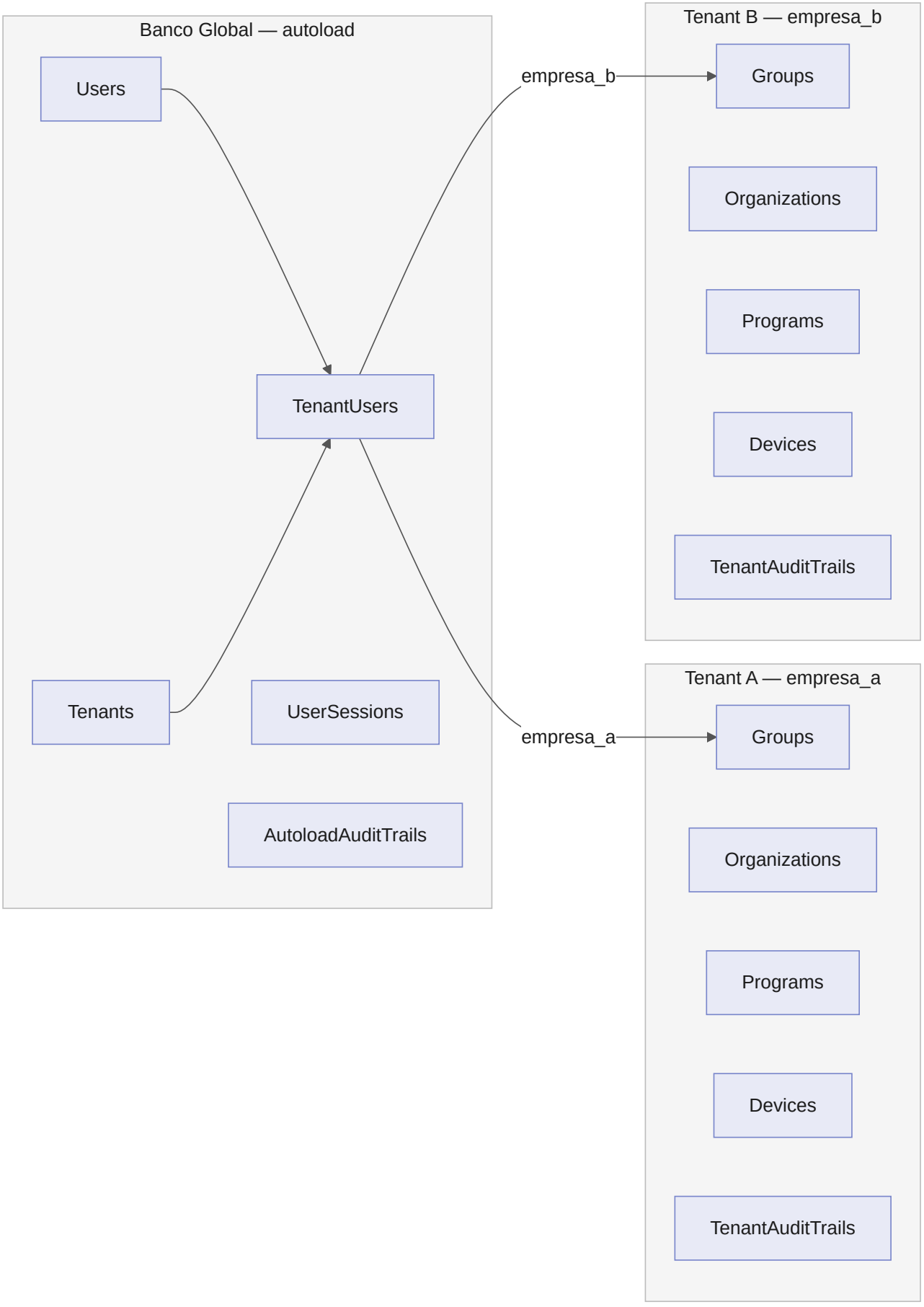
- `POST /auth/login` — login
- `POST /auth/set_password` — solicitação e conclusão de reset de senha

3.6. Isolamento Multi-Tenant

O Autoload V4 utiliza a estratégia **database-per-tenant** para isolamento completo de dados entre tenants.

Dois contextos de banco de dados:

Contexto	Banco	Entidades	Classe Base
`AutoloadContext`	`autoload` (fixo)	`Tenant`, `TenantUser`, `User`, `UserSession`, `JobExecution`, `ScheduledJob`, `AutoloadAuditTrail`	`AutoloadEntity`
`TenantContext`	Nome do tenant (ex: `empresa_abc`)	`Group`, `GroupUser`, `Organization`, e todas as entidades de domínio	`TenantEntity`


Banco Global — autoload
Users
TenantUsers
Tenants
UserSessions
AutoloadAuditTrails
empresa_b
empresa_a
Tenant B — empresa_b
Groups
Organizations
Programs
Devices
TenantAuditTrails
Tenant A — empresa_a
Groups
Organizations
Programs
Devices
TenantAuditTrails

Mecanismos de isolamento:

1. **Resolução de banco por request:** `ITenantProvider.GetTenantId()` determina qual banco conectar a cada request. O `TenantId` é usado diretamente como nome do banco na connection string.
2. **Validação de tenant ativo:** Cada login valida `Tenant.IsActive` e `TenantUser.IsUserActive`.
3. **Provedores HTTP:** `TenantProviderFromHttp`, `UserProviderFromHttp` e `IntlProviderFromHttp` são registrados como serviços scoped, extraindo contexto do request HTTP.
4. **Separação de repositórios:** `IAutoloadRepository<T> where T : AutoloadEntity` vs `IBaseTenantRepository<T> where T : TenantEntity` — o sistema de tipos impede acesso cruzado acidental.
5. **Associação User-Tenant:** A tabela `TenantUser` (chave composta `TenantId + UserId`) controla quais usuários têm acesso a quais tenants, com flag `IsUserActive` independente por tenant.

---

4. Boas Práticas de Segurança

4.1. Criptografia de Dados em Trânsito

HTTPS Enforcement:

O middleware `UseHttpsRedirection()` redireciona todas as requisições HTTP para HTTPS, garantindo criptografia TLS em trânsito.

> **OBSERVAÇÃO:**
>
> O **HTTPS** utiliza protocolos criptográficos *(SSL/TLS)* para proteger a troca de dados entre o navegador e o servidor, impedindo interceptações. O **HTTP** transmite dados em texto simples, sendo inseguro para informações sensíveis, enquanto o **HTTPS** é o padrão atual, melhorando a confiança do usuário e o SEO.

JWT Signing:

Tokens JWT são assinados com HMAC-SHA256, garantindo integridade e autenticidade. A chave simétrica é configurada via `Jwt:Key` e validada na inicialização da aplicação — a aplicação não inicia sem chave configurada.

SignalR/WebSocket:

Três hubs SignalR estão expostos em `/hubs/messages`, `/hubs/telemetry` e `/hubs/heartbeats`. Estes herdam a proteção HTTPS do servidor. Autenticação JWT via query string para hubs está planejada mas ainda não implementada.

4.2. Criptografia de Dados em Repouso

Senhas: Armazenadas como hash PBKDF2-SHA256 (nunca em texto plano). Veja seção 2.7 Hashing de Senhas.

Campos sensíveis: Propriedades como `Users` em `GroupUser` e navigation properties utilizam `[JsonIgnore]` para evitar exposição acidental em respostas da API.

Serialização: A configuração global `JsonIgnoreCondition.WhenWritingNull` omite campos nulos das respostas JSON, reduzindo a superfície de dados expostos.

4.3. Validação de Entrada

O sistema utiliza **atributos de validação customizados** além da validação automática do `[ApiController]`.

Atributos de validação:

Atributo	Validação	Mensagem de Erro
`NoWhiteSpace`	String não pode ser null, vazia ou apenas espaços	`"Validation error. {0} cannot be null, empty or only spaces"`
`ValidEnumValue`	Valor deve ser membro válido do enum especificado	`"Validation error. {0} has an invalid value"`
`ValidEnumList`	Todos os itens da lista devem ser membros válidos do enum	`"One or more values in {0} are not valid {1} values"`
`GreaterThanZero`	Valor inteiro deve ser maior que zero	`"Validation error. {0} must be greater than zero"`

Padrão de validação: Todos os atributos lançam `ServiceException` ao invés de `ValidationResult`, garantindo formato de erro consistente via `ExceptionMiddleware`.

Model Validation: O atributo `[ApiController]` habilita validação automática de model binding, retornando 400 Bad Request para payloads inválidos antes de atingir o código do controller.

4.4. Prevenção de SQL Injection

O sistema utiliza **Entity Framework Core** como ORM exclusivo, o que garante:

- **Queries parametrizadas:** Todas as consultas LINQ-to-Entities são traduzidas para SQL parametrizado pelo EF Core
- **Nenhum raw SQL:** Não há uso de `FromSqlRaw`, `ExecuteSqlRaw` ou concatenação de strings SQL no código-fonte
- **PredicateBuilder:** Consultas dinâmicas (como filtro de organizações por HierarchyPath) utilizam `PredicateBuilder` que gera expressões LINQ seguras

4.5. CORS

Configuração (via `appsettings.json`):

```
{
  "Cors": {
    "Origins": "https://app.empresa.com;https://admin.empresa.com"
  }
}
```

Configuração	Valor
Origins	Configurável via `Cors:Origins` (separadas por `;`)
Headers	Todos permitidos (`AllowAnyHeader`)
Methods	Todos permitidos (`AllowAnyMethod`)
Credentials	Habilitado (`AllowCredentials`)

Comportamento de segurança:

- Se `Cors:Origins` não estiver configurado, **nenhuma política CORS é registrada**, o que bloqueia requisições cross-origin por padrão
- `AllowCredentials` é necessário para envio de tokens JWT em requisições cross-origin
- A política é nomeada `"CorsPolicy"` e aplicada globalmente no pipeline

4.6. Auditoria

O sistema implementa **auditoria dual** com registros separados para operações globais e operações por tenant.

AutoloadAuditTrail (global):

Campo	Descrição
`DateTime`	Timestamp UTC da operação
`UserEmail`	Email do usuário que executou a ação
`Module`	Módulo afetado (ex: `"Users"`, `"Tenants"`)
`Action`	Tipo de operação (ex: `"Create"`, `"Update"`, `"Delete"`)
`PreviousValues`	JSON dos valores anteriores (filtrado por campos alterados)
`NewValues`	JSON dos novos valores
`Description`	Descrição localizada (traduzida via `MessageHelper`)

TenantAuditTrail (por tenant):

Mesmos campos que o `AutoloadAuditTrail`, armazenado no banco de dados do tenant. Suporta **padrão deferred** — captura estado anterior antes da operação e estado novo após, persistindo em `SaveRecord()`.

Filtragem diferencial: Quando `PreviousValues` e `NewValues` são objetos JSON, o sistema filtra automaticamente o `PreviousValues` para incluir apenas as propriedades que existem em `NewValues`, mostrando somente os campos que foram alterados.

4.7. Tratamento de Erros

O `ExceptionMiddleware` captura todas as exceções não tratadas e retorna respostas JSON padronizadas.

Tipo de Exceção	HTTP Status	Descrição
`ServiceExceptionCollection`	400 Bad Request	Múltiplas violações de regra de negócio (agregadas)
`ServiceException`	400 Bad Request	Violação única de regra de negócio
`PermissionDeniedException`	403 Forbidden	Permissões insuficientes
`Exception` (genérica)	500 Internal Server Error	Erros inesperados

Funcionalidades de segurança:

- **Localização de mensagens:** Erros são traduzidos usando `LocaleHelper.GetLocale()` e `MessageHelper.TranslateAndReplace()` com placeholders parametrizados
- **Integração Sentry:** Todas as exceções são enviadas ao Sentry com tags de contexto (`tenantId`, `userId`, `requestPath`)
- **Posição no pipeline:** O middleware é posicionado **antes** de `UseAuthentication()`, capturando inclusive erros de autenticação

4.8. Segurança de Upload de Arquivos

O `ProgramAttachmentService` implementa validação rigorosa de uploads.

Configuração (via `appsettings.json`):

Parâmetro	Chave	Exemplo
Tamanho máximo	`Attachments:MaxSize`	`"5MB"` (default)
Content-types permitidos	`Attachments:AllowedContentTypes`	Lista de MIME types

Validações (em ordem):

1. **Estado do programa:** Upload bloqueado se o programa está em estado `CANCELED`, `COMPLETED` ou `EXPIRED`
2. **Tamanho do arquivo:** `Content.Length` não pode exceder o máximo configurado
3. **Content-type:** Tipo MIME (lowercased) deve estar na whitelist configurada

Regra de startup: A aplicação lança `ServiceException` na inicialização se `AllowedContentTypes` não estiver configurado, prevenindo upload sem restrição de tipo.

4.9. Segurança WebSocket/SignalR

Três hubs SignalR estão disponíveis para comunicação em tempo real:

Hub	Endpoint	Propósito
`MessageHub`	`/hubs/messages`	Mensagens e notificações entre usuários
`TelemetryHub`	`/hubs/telemetry`	Dados de telemetria de dispositivos
`HeartbeatHub`	`/hubs/heartbeats`	Monitoramento de conectividade

Estado atual: Os hubs herdam a proteção HTTPS e CORS do servidor, mas **não possuem autenticação JWT dedicada**. A autenticação via token JWT em query string (`?access_token=`) está planejada e existe como código comentado no `ServiceCollectionExtensions`.

Tópicos de mensagens: Sete permissões possuem tópicos de mensagem correspondentes (`device_management`, `gateway_management`, `job_management`, `organization_approval`, `organization_management`, `product_management`, `user_management`). Ao adicionar/remover um usuário de um grupo, as subscrições de mensagens são gerenciadas automaticamente.

4.10. Logging e Monitoramento

Sentry:

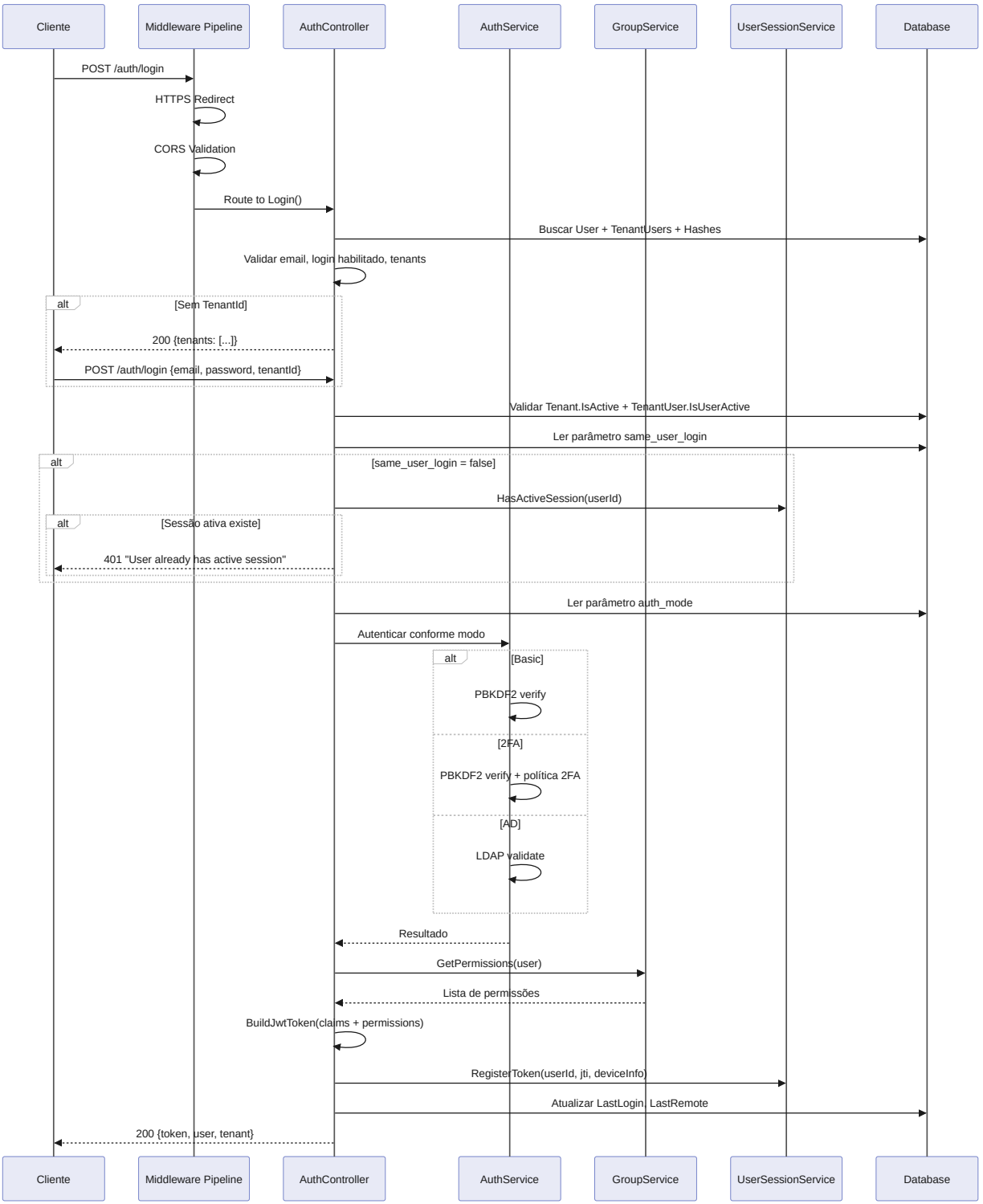
Configuração	Chave	Descrição
DSN	`Sentry:Dsn`	URL do projeto Sentry
Debug	`Sentry:Debug`	Modo debug (default: `false`)
Profiling	`Sentry:Profiling`	Profiling de performance (default: `false`)

- Sentry é **condicionalmente habilitado**: só ativa se `Sentry:Dsn` está configurado e não é o placeholder de deploy `#{SENTRY_DSN}#`
- Cada exceção capturada inclui tags: `tenantId`, `userId`, `errorMessage` e `requestPath`
- Profiling de performance é opcional via `AddProfilingIntegration()`

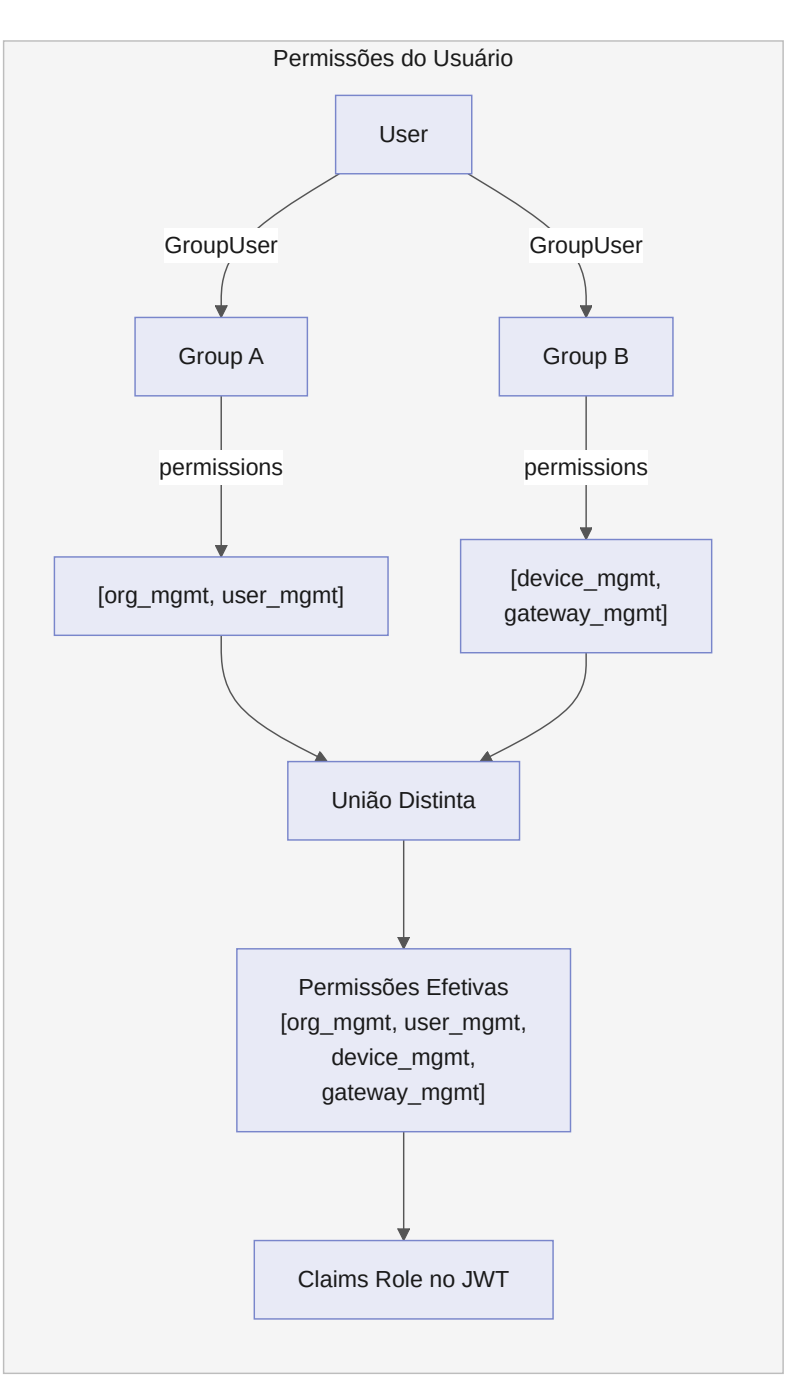
Contexto de segurança nos logs: Os provedores HTTP (`TenantProviderFromHttp`, `UserProviderFromHttp`) garantem que toda operação logada tenha contexto de tenant e usuário, permitindo rastreabilidade completa de ações.

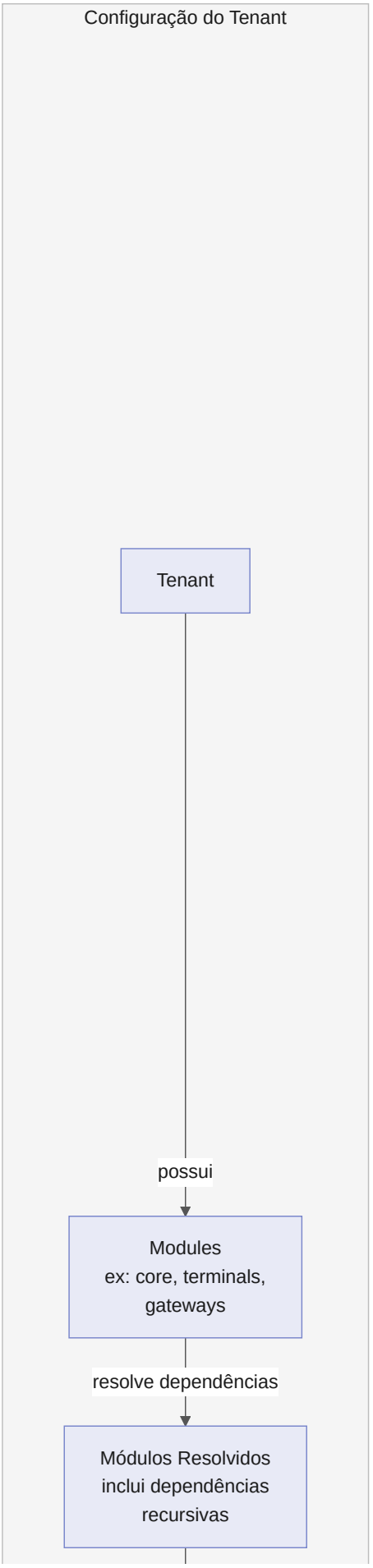
5. Fluxos de Segurança Integrados

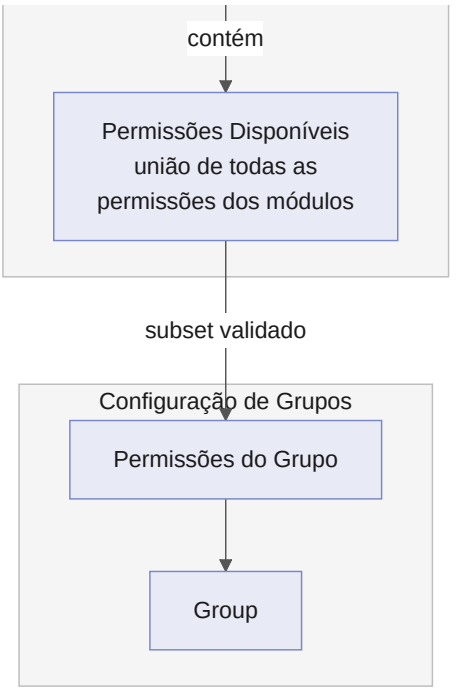
5.1. Fluxo Completo de Login



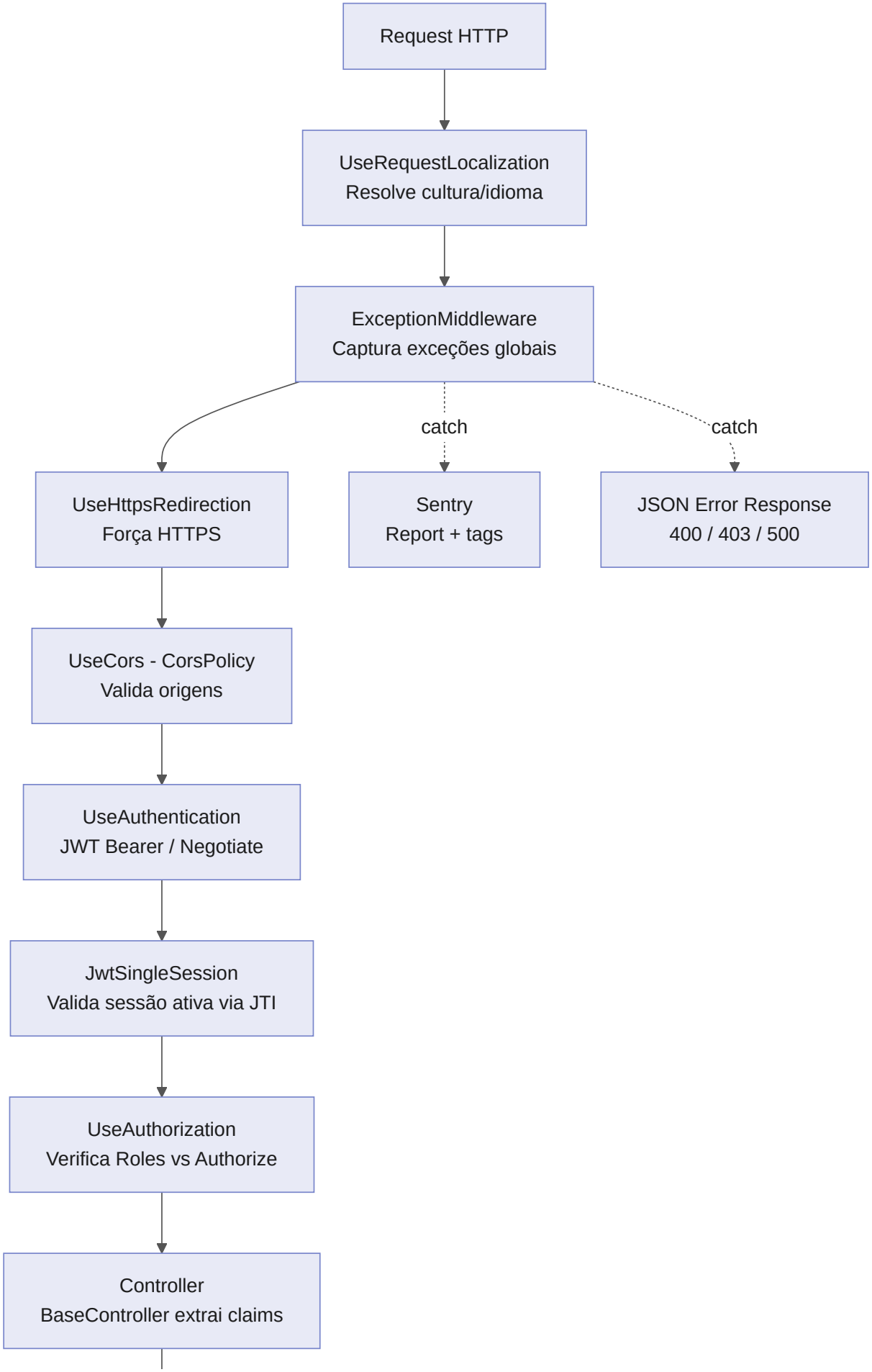
5.2. Resolução de Permissões

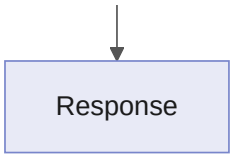

Permissões do Usuário
User
GroupUser
GroupUser
Group A
Group B
permissions
permissions
[org_mgmt, user_mgmt]
[device_mgmt, gateway_mgmt]
União Distinta
Permissões Efetivas [org_mgmt, user_mgmt, device_mgmt, gateway_mgmt]
Claims Role no JWT


Configuração do Tenant
Tenant
possui
Modules ex: core, terminals, gateways
resolve dependências
Módulos Resolvidos inclui dependências recursivas



5.3. Pipeline de Middleware


Request HTTP
UseRequestLocalization
Resolve cultura/idioma
ExceptionMiddleware
Captura exceções globais
catch
catch
UseHttpsRedirection
Força HTTPS
Sentry
Report + tags
JSON Error Response
400 / 403 / 500
UseCors - CorsPolicy
Valida origens
UseAuthentication
JWT Bearer / Negotiate
JwtSingleSession
Valida sessão ativa via JTI
UseAuthorization
Verifica Roles vs Authorize
Controller
BaseController extrai claims



6. Referência de Arquivos

Arquivo	Responsabilidade
`AutoloadV4.Api/Controllers/V1/AuthenticationController.cs`	Endpoints de login, logout, reset de senha
`AutoloadV4.Api/Services/AuthenticationService.cs`	Lógica dos 4 modos de autenticação
`AutoloadV4.Api/Helpers/ControllerHelper.cs`	Geração JWT, resolução de permissões
`AutoloadV4.Api/Controllers/BaseController.cs`	Extração de claims, propriedades base
`AutoloadV4.Api/Extensions/ServiceCollectionExtensions.cs`	Configuração JWT, CORS, pipeline de auth
`AutoloadV4.Api/Extensions/JwtSingleSessionExtensions.cs`	Validação de sessão única por JTI
`AutoloadV4.Api/Middleware/ExceptionMiddleware.cs`	Tratamento global de erros, Sentry
`AutoloadV4.Api/Program.cs`	Ordem do pipeline de middleware
`AutoloadV4.Domain.Commons/Helpers/ServicesHelper.cs`	Hashing PBKDF2, validação de senha, 2FA
`AutoloadV4.Domain.Commons/Services/UserService.cs`	Gestão de usuários, API keys, LDAP
`AutoloadV4.Domain.Commons/Services/UserSessionService.cs`	Sessões, single/multi login
`AutoloadV4.Domain.Commons/Services/GroupService.cs`	Grupos, permissões, org-scoped
`AutoloadV4.Domain.Commons/Services/PermissionService.cs`	Validação Tenant → Módulo → Permissão
`AutoloadV4.Domain.Commons/Services/ModuleService.cs`	Módulos e resolução de dependências
`AutoloadV4.Domain.Commons/Services/OrganizationService.cs`	IsPermitted(), hierarquia, HierarchyPath
`AutoloadV4.Domain.Commons/Services/AutoloadAuditTrailService.cs`	Auditoria global
`AutoloadV4.Domain.Commons/Services/TenantAuditTrailService.cs`	Auditoria por tenant
`AutoloadV4.Domain.Commons/Validations/*.cs`	Atributos de validação customizados
`AutoloadV4.Domain.Operations/Services/Programs/ProgramAttachmentService.cs`	Validação de upload de arquivos
`AutoloadV4.Domain.Operations/Resources/Permissions.json`	Definição das 55 permissões
`AutoloadV4.Domain.Operations/Resources/Modules.json`	Definição dos 13 módulos

7. Conclusão

O **Autoload V4** adota um modelo de **segurança em camadas (defesa em profundidade)** que protege a aplicação em múltiplos níveis. A autenticação suporta quatro modos — Basic, 2FA (TOTP), AD/LDAP e API Key — permitindo integração flexível com diferentes ambientes corporativos. O controle de acesso é implementado via **RBAC granular** com 55 permissões distribuídas em 13 módulos, sempre delimitadas pelo escopo da organização do usuário.

O isolamento multi-tenant é garantido pela estratégia **database-per-tenant**, eliminando qualquer possibilidade de vazamento de dados entre clientes. Toda ação relevante é registrada por um sistema de **auditoria dual**: logs globais para operações administrativas e logs por tenant para operações de negócio.

A arquitetura ainda incorpora boas práticas consolidadas de segurança: comunicação exclusiva via **HTTPS**, hashing de senhas com **PBKDF2**, política de **CORS** restritiva, **validação de entrada** em todas as camadas e prevenção de **SQL injection** por meio de queries parametrizadas e ORM.

Documento atualizado em: 18 de março de 2026
Versão: 1.0
Autor: Milton Kiefer de Albuquerque Mello
Validado por: Gustavo Schmal

VIII. Deploy e Ambientes

Procedimentos de deploy, ambientes e configuração.

1. Visão Geral de Deploy

O Autoload V4 opera em três ambientes principais, cada um com propósito, configuração e nível de acesso distintos.

1.1. Desenvolvimento (DEV)

Característica	Detalhe
Branch de origem	`develop`
Propósito	Desenvolvimento ativo e testes iniciais
Banco de dados	SQL Server local (`localhost\SQL2019LOCAL`) ou PostgreSQL local
MQTT Broker	`vbr01-dsv.autominddev.com.br:1883` (sem TLS)
Log Level	`Information` (verbose)
Swagger	Habilitado em `/swagger`
Storage	OCI bucket `autoload-dev-bucket` (região `sa-saopaulo-1`)
SMTP	Brevo relay (`smtp-relay.brevo.com:9025`)
Sentry	Opcional (DSN comentado por padrão)
CORS	`http://localhost:3000`

Principais diferenças:

- Log level mais detalhado (`Information` vs `Warning` em produção)
- JWT key simplificada para desenvolvimento
- Workers habilitados por padrão (JobExecution, MQTT)
- MQTT sem TLS para facilitar debugging
- Swagger acessível diretamente na raiz (`/`)

1.2. Homologação (HML)

Característica	Detalhe
Branch de origem	`release/*`
Propósito	Validação de funcionalidades antes de produção
Banco de dados	PostgreSQL dedicado (servidor HML)
MQTT Broker	Broker dedicado com TLS

Característica	Detalhe
Log Level	`Warning`
Swagger	Habilitado com restrição
Storage	OCI bucket dedicado HML
Sentry	Habilitado para rastreamento de erros
Build Tag	`release` (adicionada automaticamente pelo pipeline)

Principais diferenças:

- Configurações de produção com dados de teste
- TLS habilitado nas comunicações MQTT
- Sentry ativo para capturar erros pré-produção
- Builds recebem tag `release` para filtragem no Azure DevOps Releases

1.3. Produção (PRD)

Característica	Detalhe
Branch de origem	`master`
Propósito	Ambiente de operação real com clientes
Banco de dados	PostgreSQL em cluster de produção
MQTT Broker	Broker de produção com TLS obrigatório
Log Level	`Warning`
Swagger	Desabilitado ou restrito
Storage	OCI bucket de produção
Sentry	Habilitado com profiling
CORS	Restrito aos domínios de produção

Principais diferenças:

- Todas as comunicações utilizam TLS/HTTPS
- JWT key segura e rotacionada
- Swagger desabilitado ou com endpoint restrito
- Profiling do Sentry habilitado para métricas de performance
- Workers configurados conforme necessidade do terminal

1.4. Comparativo entre Ambientes

Aspecto	DEV	HML	PRD
Branch	`develop`	`release/*`	`master`

Aspecto	DEV	HML	PRD
Database Provider	SQL Server/PostgreSQL	PostgreSQL	PostgreSQL
MQTT TLS	Não	Sim	Sim
Log Level	Information	Warning	Warning
Swagger	Habilitado (`/`)	Restrito	Desabilitado
Sentry	Opcional	Habilitado	Habilitado + Profiling
CORS	`localhost:3000`	Domínio HML	Domínio PRD
JWT Key	Desenvolvimento	Segura	Segura + Rotacionada
Storage	OCI dev-bucket	OCI hml-bucket	OCI prd-bucket

---

2. Configuração por Ambiente

2.1. Mecanismo de Substituição de Variáveis

O Autoload V4 utiliza um padrão de **template placeholders** com formato `#{VARIAVEL}#` nos arquivos `appsettings.json`. No momento do deploy, o Azure DevOps Releases substitui esses placeholders pelos valores reais do ambiente.

Exemplo no `appsettings.json`:

```
{
  "Database": {
    "Provider": "#{DATABASE_PROVIDER}#",
    "Server": "#{DATABASE_SERVER}#",
    "Port": "#{DATABASE_PORT}#",
    "User": "#{DATABASE_USER}#",
    "Password": "#{DATABASE_PASSWORD}#"
  }
}
```

Após substituição (DEV):

```
{
  "Database": {
    "Provider": "postgresql",
    "Server": "vbr01-dsv.autominddev.com.br",
    "Port": "5432",
    "User": "autoload",
    "Password": "senha_real"
  }
}
```

2.2. Configuração do Frontend (Runtime)

O Frontend utiliza uma abordagem de **configuração em runtime** via arquivo `public/env.json`:

Desenvolvimento (`.env` / `.env.development`):

```
NEXT_PUBLIC_BASE_URL=http://localhost:5000/api
NEXT_PUBLIC_WS_BASE_URL=http://localhost:5000/hubs
NEXT_PUBLIC_API_KEY=dev-api-key
NEXT_PUBLIC_ENV=development
NEXT_PUBLIC_AGENT_URL=http://localhost:8000
NEXT_PUBLIC_SENTRY_ORG=automind-11
NEXT_PUBLIC_SENTRY_PROJECT=autoloadv4-frontend
```

Produção (`public/env.json` - template com substituição):

```
{
  "baseUrl": "#{BASE_URL}#",
  "wsBaseUrl": "#{WS_BASE_URL}#",
  "agentUrl": "#{AGENT_URL}#",
  "defaultTimezone": "#{DEFAULT_TIMEZONE}#",
  "environment": "#{ENVIRONMENT}#",
  "sentryDsn": "#{SENTRY_DSN}#",
  "sentryOrg": "#{SENTRY_ORG}#",
  "sentryProject": "#{SENTRY_PROJECT}#"
}
```

Lógica de carregamento (em `src/lib/config.ts`):

- Em **desenvolvimento** (`NODE_ENV === "development"`): usa `process.env` (variáveis do `.env`)
- Em **produção/outros**: faz `fetch("/env.json")` para carregar configuração em runtime
- Isso permite alterar configurações sem rebuild do frontend

2.3. Configuração do Kiosk (Runtime)

O Kiosk segue a mesma abordagem com `public/env.json`:

Desenvolvimento (`.env`):

```
VITE_API_URL=https://dev.autoload.cloud/api
```

Produção (`public/env.json`):

```
{
  "baseUrl": "#{BASE_URL}#"
}
```

O Kiosk é servido com base path `/kiosk/` (configurado no `vite.config.ts`).

2.4. Configuração do Backend

O Backend usa o padrão de Configuração do ASP.NET Core com camadas:

1. `appsettings.json` - Configuração base (templates `#{VAR}#` para ambientes remotos)
2. `appsettings.Development.json` - Sobrescreve valores para desenvolvimento local

3. `appsettings.devellop.json` - Configuração alternativa de desenvolvimento (connection string customizada)

Configurações críticas por ambiente:

Seção	Chaves Principais	Uso
`Database`	Provider, Server, Port, User, Password	Conexão com banco de dados
`MQTT`	Server, Port, User, Password, Protocol, UseTls	Broker de mensageria
`Jwt`	Key	Chave de assinatura JWT
`Cors`	Origins	Domínios permitidos
`Sentry`	Dsn, Debug, Profiling	Monitoramento de erros
`SMTP`	Server, Port, Options, User, Password	Envio de emails
`Hubs`	Messages, Telemetry	URLs dos hubs SignalR
`Workers`	JobExecutionWorkerEnabled, MQTTWorkerEnabled, ProgramWorkerEnabled	Ativação de workers
`StorageProvider`	Provider, Local/Oci config	Armazenamento de arquivos
`Logging`	LogLevel, File config, Context	Logs e observabilidade

2.5. Configuração do Edge Gateway

O Edge Gateway possui Configurações específicas para o terminal onde opera:

```
{
  "EdgeGateway": {
    "Id": "#{GATEWAY_ID}#",
    "TenantId": "#{TENANT_ID}#",
    "RpcPrefix": "#{RPC_PREFIX}#",
    "ReconnectInterval": "#{GATEWAY_RECONNECT_INTERVAL}#",
    "PoolingInterval": "#{GATEWAY_POOLING_INTERVAL}#"
  },
  "Telemetry": {
    "Interval": "#{TELEMETRY_INTERVAL}#",
    "ReconnectInterval": "#{TELEMETRY_RECONNECT_INTERVAL}#",
    "MaxLeaseDuration": "#{TELEMETRY_MAX_LEASE_DURATION}#"
  },
  "RPC": {
    "RequestTopic": "#{RPC_REQUEST}#",
    "ResponseTopic": "#{RPC_RESPONSE}#"
  },
  "MQTT": {
    "Server": "#{MQTT_SERVER}#",
    "Port": "#{MQTT_PORT}#",
    "User": "#{MQTT_USER}#",
    "Password": "#{MQTT_PASSWORD}#",
    "Protocol": "#{MQTT_PROTOCOL}#",
    "UseTls": "#{MQTT_USE_TLS}#"
```

```
  }
}
```

Cada gateway é **único por terminal/site** e possui:

- `GATEWAY_ID`: Identificador único do gateway
- `TENANT_ID`: Tenant ao qual pertence
- Tópicos MQTT específicos para RPC (Request/Response)
- Intervalos configurados para telemetria e reconexão

---

3. Arquitetura de Rede e Comunicação

3.1. Diagrama de Rede



3.2. Portas e Protocolos

Serviço	Porta (DEV)	Porta (PRD)	Protocolo
Backend API	5000	443 (HTTPS)	HTTP/HTTPS REST
Frontend Web	3000	443 (HTTPS)	HTTP/HTTPS
Kiosk	5173 (Vite) / 3000	443 (HTTPS)	HTTP/HTTPS
PostgreSQL	5432	5432	TCP
SQL Server	1433	1433	TCP
MQTT (sem TLS)	1883	-	MQTT
MQTT (com TLS)	-	8883	MQTT + TLS
MQTT WebSocket	9001	443	WSS
SignalR Hubs	5000/hubs	443/hubs	WebSocket/SSE
Swagger	5000/swagger	Desabilitado	HTTP
Sentry	-	Saída 443	HTTPS
SMTP (Brevo)	9025	587	SMTP/TLS

3.3. Comunicação entre Componentes

Frontend - Backend:

- REST API via HTTPS (`/api/*`)
- WebSocket via SignalR (`/hubs/*`)
- Autenticação via JWT Bearer token

Backend - Database:

- Entity Framework Core sobre TCP
- Dois contextos: `AutoloadContext` (dados globais) e `TenantContext` (dados por tenant)
- Multi-tenant com isolamento de dados

Backend - Edge Gateway:

- MQTT broker como intermediário
- Tópicos: `{TenantId}/gateways/{GatewayId}/[operação]`
- RPC (Request/Response) para comandos
- Telemetria publicada periodicamente

Edge Gateway - Dispositivos:

- Serial (RS-232/RS-485) para preset loaders, balanças, UCAs
- TCP/IP para OPC UA e ModBus TCP
- Drivers específicos por tipo de dispositivo

3.4. Tópicos MQTT

```
autoload/gateway/{gatewayId}/telemetry       # Telemetria do gateway
autoload/gateway/{gatewayId}/command          # Comandos para gateway
autoload/gateway/{gatewayId}/rpc/request      # RPC Request
autoload/gateway/{gatewayId}/rpc/response     # RPC Response
autoload/device/{deviceId}/telemetry          # Telemetria de dispositivos
autoload/device/{deviceId}/command            # Comandos para dispositivos
{tenantId}/gateways/{gatewayId}/startprocess  # Inicio de processo
```

3.5. Hubs SignalR

Hub	Endpoint	Finalidade
TelemetryHub	`/hubs/telemetry`	Telemetria de dispositivos e operações em tempo real
MessageHub	`/hubs/messages`	Notificações e mensagens em tempo real
HeartbeatHub	`/hubs/heartbeats`	Monitoramento de conectividade de gateways e dispositivos

---

4. Git Flow e Branching Strategy

4.1. Visão Geral

O Autoload V4 usa um modelo de branches organizado (Git Flow) para garantir que:

- O código em produção seja sempre estável
- Features novas sejam testadas antes de subirem para produção
- Bugs urgentes possam ser corrigidos rapidamente

Diagrama das Branches:

> **PAREI AQUI**

```
PRODUÇÃO (mais estável)
  |
  v
+---------+
| master  | -----> Código que esta rodando em produção
+---------+          Tag: v1.2.0, v1.2.1, v1.3.0...
    ^
    |
+------------+
| release/*  | ---> Código sendo testado para o próximo release
+------------+        Tag: v1.3.0-rc
    ^
    |
+---------+
| develop | -----> Código com todas as features prontas
+---------+          (ainda nao testado em HML)
    ^
    |
```

```
+------------------+
| feat/{PBI}-*     | ---> Seu código novo sendo desenvolvido
+------------------+        Ex: feat/12345-login-sso

DESENVOLVIMENTO (menos estável)
```

Branch Especial - Hotfix:

```
+---------+                    +---------+
| master  | ------------------> | master  |
+---------+                    +---------+
    |        +----------+          ^
    +------> | hotfix/* | ---------+
             +----------+
          (correção urgente)
```

4.2. Branches

master

O que é: A branch mais importante. Contém o código que está em PRODUÇÃO.

Regras:

- NUNCA faça commits diretamente na master
- Só recebe merges de `release/*` OU `hotfix/*`
- Cada merge gera uma tag de versão (ex: v1.2.0)

develop

O que é: A branch de integração. Todas as features novas vão para cá primeiro.

Regras:

- NUNCA faça commits diretamente na develop
- Só recebe merges de `feat/*`
- É a base para integrar features nas branches de release

feat/*

O que é: Sua branch de trabalho para desenvolver algo novo ou corrigir um bug não urgente.

Nomenclatura: `feat/{PBI-ID}-nome-descritivo`

- `{PBI-ID}` = Número do PBI (Product Backlog Item) no Azure DevOps
- `nome-descritivo` = Descrição curta do que será feito

Exemplos:

- `feat/12345-login-com-sso`
- `feat/12380-relatorio-de-vendas`
- `feat/12401-correcao-calculo-volume`

release/*

O que é: Branch para preparar uma nova versão para produção.

Nomenclatura: `release/X.Y.Z`

Exemplos:

- `release/1.3.0`
- `release/2.0.0`

hotfix/*

O que é: Branch para corrigir bugs URGENTES em produção.

Nomenclatura: `hotfix/X.Y.Z` (incrementando o último número da versão em produção)

Exemplos:

- Se produção está na v1.2.0: `hotfix/1.2.1`
- Se produção está na v1.2.1: `hotfix/1.2.2`

4.3. Tabela de Decisão

Situação	Branch	Base	Destino
Nova funcionalidade	`feat/{PBI}-descricao`	develop	develop (PR)
Correção de bug (não urgente)	`feat/{PBI}-descricao`	develop	develop (PR)
Preparar versão para produção	`release/X.Y.Z`	master	master (PR)
Bug URGENTE em produção	`hotfix/X.Y.Z`	master	master (PR) + backport develop

4.4. Fluxo: Desenvolver uma Feature

Quando usar: Sempre que for desenvolver algo novo ou corrigir um bug não-urgente.

Passo a Passo

1. Crie sua branch a partir da develop

```
# Atualize sua develop local
git checkout develop
git pull origin develop

# Crie sua feature branch (use o ID do PBI)
git checkout -b feat/12345-minha-feature
```

2. Desenvolva normalmente

```
# Faca seus commits
git add .
git commit -m "feat: implementa funcionalidade X"

# Faca mais commits conforme necessario
```

```
git add .
git commit -m "feat: adiciona validação Y"
```

3. Envie para o servidor

```
git push origin feat/12345-minha-feature
```

4. Abra um Pull Request (PR)

1. Vá ao Azure DevOps
2. Clique em "Pull Requests" > "New Pull Request"
3. **Source:** `feat/12345-minha-feature`
4. **Target:** `develop`
5. Preencha título e descrição
6. Aguarde a validação automática (build + testes)
7. Solicite revisão de código

5. Após aprovação

O PR será mergeado na develop. Sua feature agora faz parte do próximo release!

Diagrama Visual:

```
develop ──────────────────────────────────────> develop
         \                                    /
          \     feat/12345-minha-feature     /
           ──────────────────────────────────

                    (seus commits)
                          |
                          PR
                          |
                    [Build + Testes]
                          |
                    [Code Review OK]
                          |
                       Merge!
```

4.5. Fluxo: Criar um Release

Quando usar: Quando a develop está pronta para ir para produção.

Passo a Passo

1. Crie a branch de release a partir da master (ou ultima release)

```
# Atualize sua master local
git checkout master
git pull origin master

# Crie a release branch a partir da master
git checkout -b release/1.3.0
```

```
# Envie para o servidor
git push origin release/1.3.0
```

> **IMPORTANTE**
>
> A branch release deve ser criada a partir da **master** (ou última release), **NÃO** a partir da **develop**. Isso garante que o pipeline seja disparado corretamente quando o PR da develop for mergeado.

2. Abra PR da develop para a release

1. Vá ao Azure DevOps
2. Clique em "Pull Requests" > "New Pull Request"
3. **Source:** `develop`
4. **Target:** `release/1.3.0`
5. Título sugerido: "Release 1.3.0 - Features para produção"
6. Aguarde validação automática (build + testes)
7. Solicite revisão de código

3. Após o merge do PR

O pipeline automaticamente:

- Cria a tag `v1.3.0-rc` (Release Candidate)
- Gera o artefato para deploy em HML

4. Teste em HML

- O time de QA testa em HML
- Se encontrar bugs, corrija na própria branch release (via PR de uma branch de fix)
- Cada merge gera um novo artefato

5. Quando estiver OK, abra PR para master

1. Vá ao Azure DevOps
2. Clique em "Pull Requests" > "New Pull Request"
3. **Source:** `release/1.3.0`
4. **Target:** `master`
5. Aguarde validação e aprovação

6. Após o merge

O pipeline automaticamente:

- Cria a tag final `v1.3.0`
- Gera o artefato para deploy em PRODUÇÃO

Diagrama Visual:



```
           Tag: v1.3.0-rc
                |
           Deploy HML
                |
             Testes
                |
              [PR]
                |
              Merge
                v
master ──────────────────────────────────────> master
                                          Tag: v1.3.0
                                              |
                                          Deploy PRD
```

4.6. Fluxo: Corrigir Bug Urgente (Hotfix)

Quando usar:

- Bug CRÍTICO em produção
- Problema de segurança
- Sistema fora do ar ou com funcionalidade comprometida

> **IMPORTANTE**
>
> Hotfixes vão DIRETO para produção, sem passar por HML. Use com responsabilidade!

Passo a Passo

1. Identifique a versão atual em produção

```
# Veja qual tag esta em master
git checkout master
git pull origin master
git describe --tags
# Exemplo de saida: v1.2.0
```

2. Crie a branch de hotfix

```
# Incremente o ultimo numero da versao
# Se producao esta em v1.2.0, crie hotfix/1.2.1
git checkout -b hotfix/1.2.1

# Envie para o servidor
git push origin hotfix/1.2.1
```

3. Faça a correção

```
# Corrija o bug
git add .
git commit -m "fix: corrige problema critico X"
```

```
# Envie para o servidor
git push origin hotfix/1.2.1
```

4. Abra PR para master

1. Vá ao Azure DevOps
2. Clique em "Pull Requests" > "New Pull Request"
3. **Source:** `hotfix/1.2.1`
4. **Target:** `master`
5. ATENÇÃO: Revise com cuidado! Vai direto para produção!
6. Aguarde validação automática

5. Após o merge

O pipeline automaticamente:

- Cria a tag `v1.2.1`
- Gera o artefato para deploy em PRODUÇÃO

6. OBRIGATÓRIO: Backport para develop

> **NÃO ESQUEÇA ESTE PASSO!**
>
> O hotfix precisa ir para develop também, caso contrário a develop não terá a correção.

1. Vá ao Azure DevOps
2. Clique em "Pull Requests" > "New Pull Request"
3. **Source:** `hotfix/1.2.1`
4. **Target:** `develop`
5. Título sugerido: "Backport hotfix/1.2.1 para develop"
6. Aguarde validação e aprovação
7. Faça o merge do PR

Nota: Se houver conflitos, resolva-os localmente:

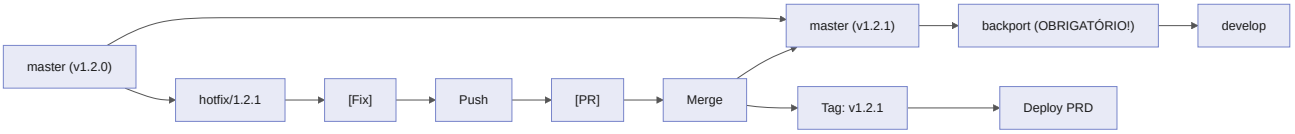
```
git checkout hotfix/1.2.1
git pull origin hotfix/1.2.1
git merge origin/develop
# Resolva os conflitos
git push origin hotfix/1.2.1
```

*7. Se houver release/ ativa**

Se existir uma branch release em andamento, faça o backport nela também:

1. Vá ao Azure DevOps
2. Clique em "Pull Requests" > "New Pull Request"
3. **Source:** `hotfix/1.2.1`
4. **Target:** `release/1.3.0`
5. Título sugerido: "Backport hotfix/1.2.1 para release/1.3.0"
6. Aguarde validação e aprovação
7. Faça o merge do PR

Diagrama Visual:



4.7. Convenções de Commits

Recomenda-se o uso de mensagens descritivas e objetivas:

```
feat: adicionar validação de peso no agendamento
fix: corrigir calculo de volume na telemetria
refactor: reorganizar services do dominio Operations
test: adicionar testes para AppointmentService
docs: atualizar documentacao de deploy
chore: atualizar dependencias do frontend
```

4.8. Convenções de Pull Requests

- **Título**: Breve e descritivo (ex: "feat: Adicionar filtro por status nos agendamentos")
- **Descrição**: Detalhar o que foi alterado e por quê
- **Reviewers**: Mínimo 1 aprovação
- **Checks obrigatórios**: Pipeline de validação (build + testes) deve passar
- **Branches alvo**: PRs só podem ser feitos para `develop`, `master`, `release/*` ou `hotfix/*`

Checklist antes de abrir um PR:

- ☐ Código compila localmente
- ☐ Testes passam localmente
- ☐ Branch está atualizada com a base (develop/master)
- ☐ Commits têm mensagens claras
- ☐ Não tem arquivos desnecessários (node_modules, bin, etc)

Checklist após merge de Hotfix:

- ☐ Fiz backport para develop
- ☐ Fiz backport para release/* (se existir)

4.9. Comandos Git Mais Usados

```
# Atualizar branch atual
git pull origin <branch-atual>

# Criar nova branch
git checkout -b <nome-da-branch>

# Enviar commits
git push origin <nome-da-branch>

# Ver em qual branch voce esta
git branch
```

```
# Mudar de branch
git checkout <nome-da-branch>

# Ver tags disponiveis
git tag -l

# Ver versao em producao
git checkout master
git pull origin master
git describe --tags
```

4.10. Perguntas Frequentes

1. Posso fazer commit direto na develop ou master?

NÃO! Sempre crie uma branch (feat, release ou hotfix) e use Pull Request.

2. Qual a diferença entre release e hotfix?

Aspecto	Release	Hotfix
Origem	develop	master
Passa por HML?	SIM	NÃO
Urgência	Normal	URGENTE
Backport	Não precisa	OBRIGATÓRIO

3. Preciso fazer backport de release?

Não. A release já veio da develop, então develop já tem o código.

4. Preciso fazer backport de hotfix?

SIM, SEMPRE! O hotfix foi feito a partir da master, então develop não tem o código.

5. Como sei qual versão está em produção?

```
git checkout master
git pull origin master
git describe --tags
```

6. Posso ter mais de uma release ao mesmo tempo?

Não é recomendado. Termine uma release antes de começar outra.

7. E se meu PR falhar no build?

1. Veja o log do erro no Azure DevOps
2. Corrija na sua branch
3. Faça push novamente
4. O pipeline vai executar de novo automaticamente

8. E se houver conflito no merge?

1. Atualize sua branch com a branch de destino:

```
git checkout sua-branch
git merge origin/develop    # ou master, dependendo do caso
```

2. Resolva os conflitos nos arquivos marcados
3. Faça commit e push
4. O PR será atualizado automaticamente

4.11. Repositórios do Projeto

Todos os módulos compartilham o mesmo repositório monorepo:

```
AUTOLOAD_V4/                    # Repositorio raiz
├── AUTOLOAD_V4_Backend/        # Subdiretorio do Backend
├── AUTOLOAD_V4_Frontend/       # Subdiretorio do Frontend
├── AUTOLOAD_V4_EdgeGateway/    # Subdiretorio do Edge Gateway
├── AUTOLOAD_V4_KIOSK/          # Subdiretorio do Kiosk
└── AUTOLOAD_V4_DevOps/         # Templates de pipelines (repositorio separado)
```

Cada módulo possui seus próprios pipelines (validate + artifact), mas todos seguem o mesmo Git Flow.

---

5. Pipelines de CI/CD

5.1. Visão Geral

O projeto utiliza **Azure Pipelines** com dois tipos de pipeline por modulo:

Pipeline	Arquivo	Trigger	Propósito
Validação	`azure-pipelines-validate.yml`	PRs abertos	Build + Testes (sem artefato)
Artefato	`azure-pipelines-artifact.yml`	Merge (push)	Build + Publicação de artefato
NuGet	`azure-pipelines-nuget.yml`	Push (Backend)	Publicação de pacotes internos

5.2. Pipeline de Validação (PR)

Executado automaticamente quando um PR é aberto para `develop`, `master`, `release/*` ou `hotfix/*`.

Backend (Windows):

```
trigger: none
pr:
  branches:
    include: [develop, master, release/*, hotfix/*]

steps:
  - Restore (NuGet + Azure Artifacts feed)
  - Test (**/*Test.csproj)
```

Frontend (Ubuntu):

```
trigger: none
pr:
  branches:
    include: [develop, master, release/*, hotfix/*]

steps:
  - Node.js 20.x
  - npm ci
  - npm run build
```

5.3. Pipeline de Artefato (Merge)

Executado após merge nas branches `develop`, `master`, `release/*` ou `hotfix/*`.

Backend:

```
trigger: [develop, master, release/*, hotfix/*]
pr: none

steps:
  - .NET 8.0 SDK
  - Restore (feed: f89bae47-037c-4526-ad13-86ddd4b5c199)
  - Build (--configuration Release)
  - Publish (zip)
  - Copy appsettings*.json e *.sql
  - Tag "release" (se branch release/*)
  - Publish Artifact: "drop-backend"
```

Frontend:

```
trigger: [develop, master, release/*, hotfix/*]
pr: none

steps:
  - Node.js 18.x
  - npm ci
  - npm run build
  - npm run postbuild
  - Copy .next/standalone
  - Tag "release" (se branch release/*)
  - Publish Artifact: "drop-frontend"
```

Edge Gateway:

```
trigger: [master, hotfix/*]
pr: none

steps:
  - .NET 8.0 SDK
  - NuGet Authenticate
  - Restore
  - Build
  - Publish Linux
  - Publish Artifact
```

Kiosk:

```
trigger: [develop, master, release/*, hotfix/*]
pr: none

steps:
  - Node.js 20.x
  - npm install
  - npm run build
  - Copy dist/**
  - Publish Artifact: "drop"
```

5.4. Pipeline NuGet (Pacotes Internos)

Publicação de pacotes NuGet privados (ex: `Automind.AutoloadV4.Logging`):

```
trigger: [main, develop]
paths: [AutoloadV4.Logging/**]

stages:
  CI:
    - Restore, Build, Test
    - Pack (.nupkg)
    - Publish artifact: "nuget-packages"

  Release:
    - Download artifact
    - Push to Azure Artifacts feed
```

5.5. Templates DevOps

Os pipelines utilizam templates compartilhados do repositório `AUTOLOAD_V4_DevOps`:

- `template-release-tag-create.yml` - Cria tags de release automaticamente
- `template-build-version-metadata.yml` - Gera metadados de versão (`version.json`)

5.6. Artefatos Gerados

Módulo	Nome do Artefato	Conteúdo
Backend	`drop-backend`	API publicada (zip) + appsettings + SQL files
Frontend	`drop-frontend`	Next.js standalone build
Edge Gateway	(default)	Worker Service binaries (Linux)
Kiosk	`drop`	Build estatico Vite (`dist/`)

---

6. Configuração do Ambiente Local (Setup)

6.1. Pré-requisitos

Software	Versão	Download
.NET SDK	8.0+	dotnet.microsoft.com
Node.js	20 ou superior	nodejs.org
PostgreSQL	14+	postgresql.org
Git	Última versão	git-scm.com

Verificar instalações:

```
dotnet --version    # Deve ser 8.0.x ou superior
node --version      # v20.x ou superior
psql --version      # 14.x ou superior
```

Ferramenta adicional:

```
dotnet tool install --global dotnet-ef
```

6.2. Configuração do Banco de Dados

O Autoload V4 cria automaticamente o banco de administração e o banco de tenant.

Criar banco PostgreSQL:

```
CREATE USER autoload WITH PASSWORD 'sua_senha_aqui';
CREATE DATABASE autoload OWNER autoload;
GRANT ALL PRIVILEGES ON DATABASE autoload TO autoload;
```

Configurar connection string:

Edite `AUTOLOAD_V4_Backend/AutoloadV4.Api/appsettings.develop.json`:

```
{
  "Database": {
    "Provider": "postgresql",
    "Server": "localhost",
    "Port": "5432",
    "User": "autoload",
    "Password": "sua_senha_aqui"
  },
  "Cors": {
    "Origins": "http://localhost:3000"
  }
}
```

> Para SQL Server, use `"Provider": "sqlserver"` e porta `1433`.

Providers disponíveis:

Provider	Descrição	Configuração adicional
`localdb`	SQL Server LocalDB (ideal para desenvolvimento no Visual Studio)	Nenhuma
`inmemory`	Banco em memória (para testes)	Nenhuma
`sqlserver`	SQL Server completo	Server, Port, User, Password
`postgresql`	PostgreSQL	Server, Port, User, Password

> Para `localdb` ou `inmemory`, não é necessário criar banco ou configurar servidor. Basta definir o provider no `appsettings.Development.json`.

Aplicar migrations:

```
cd AUTOLOAD_V4_Backend

dotnet ef database update `
  --project AutoloadV4.Infrastructure `
  --startup-project AutoloadV4.Api `
  --context AutoloadContext
```

6.3. Backend - Configuração e Execução

Instalar Azure Artifacts Credential Provider

O projeto usa pacotes NuGet privados do Azure Artifacts:

```
iex "& { $(irm https://aka.ms/install-artifacts-credprovider.ps1) }"
```

Restaurar pacotes e compilar

```
cd AUTOLOAD_V4_Backend

# Limpar cache (recomendado na primeira vez)
dotnet nuget locals all --clear

# Restaurar pacotes (abre pop-up de autenticação Azure DevOps na primeira vez)
dotnet restore

# Compilar
dotnet build
```

Executar

```
dotnet run --project AutoloadV4.Api/AutoloadV4.Api.csproj
```

URLs disponíveis:

- API: `http://localhost:5000`
- Swagger: `http://localhost:5000/swagger`

Configuração local com appsettings.Development.json

Para desenvolvimento local, adicione o arquivo `appsettings.Development.json` ao projeto `AutoloadV4.Api`:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Warning"
    }
  },
  "Cors": {
    "Origins": "http://localhost:3000"
  },
  "Jwt": {
    "Key": "abcdefghijklmnopqrstuvxywz123456"
  },
  "Database": {
    "Provider": "localdb"
  }
}
```

> Para usar `sqlserver` ou `postgresql`, adicione também `Server`, `Port`, `User` e `Password` dentro de `Database`.

Perfis de Inicialização (launchSettings.json)

O arquivo `Properties/launchSettings.json` define dois perfis de execução:

```
{
  "$schema": "http://json.schemastore.org/launchsettings.json",
  "profiles": {
```

```
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "applicationUrl": "http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "swagger": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:5001",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Perfil	URL	Descrição
`http`	`http://localhost:5000`	Perfil padrão (sem HTTPS)
`swagger`	`https://localhost:5001`	Abre o Swagger automaticamente no navegador

Usuário Administrador Automático

Ao iniciar o backend, o banco de dados `autoload` será criado automaticamente com um usuário administrador:

Campo	Valor
Email	`admin@autoload.com`
Senha	`pass`

Login via API:

```
POST /auth/login

{
  "email": "admin@autoload.com",
  "password": "pass"
}
```

Criar um novo assinante (tenant):

```
POST /tenants

{
  "TenantId": "<identificador do assinante>",
  "TenantName": "<nome do assinante>",
```

```
  "UserEmail": "<email do usuario administrador do assinante>",
  "UserName": "<nome do usuario administrador do assinante>"
}
```

Durante a criação do assinante, será criado:

- Banco de dados com o nome igual ao identificador do assinante
- Grupo de acesso "Administradores" com permissões de gestão de usuários e grupos
- Novo usuário com senha inicial `pass` associado ao grupo "Administradores"

6.4. Frontend - Configuração e Execução

Configurar variáveis de ambiente

Crie o arquivo `.env.development` na pasta `AUTOLOAD_V4_Frontend`:

```
NEXT_PUBLIC_BASE_URL=http://localhost:5000
NEXT_PUBLIC_WS_BASE_URL=http://localhost:5000/hubs
NEXT_PUBLIC_API_KEY=dev-api-key
NEXT_PUBLIC_ENV=development
```

Autenticar no Azure DevOps (NPM)

O Frontend usa pacotes privados `@automind/*` do projeto `AUTOLOAD_V4_Frontend_Components` no Azure DevOps.

Requisitos do Frontend:

- **Node.js**: Versão 20 ou superior
- **React**: Versão 18
- **Next.js**: Versão 14

```
cd AUTOLOAD_V4_Frontend

# Instalar ferramenta de autenticação (Windows)
npm install -g vsts-npm-auth

# Para usuários fora do Windows
npm install -g better-vsts-npm-auth

# Autenticar (Windows - abre navegador)
npm run auth
# ou: vsts-npm-auth -config .npmrc

# Autenticar (outras plataformas)
npm run auth:unix
# ou: better-vsts-npm-auth -config .npmrc
```

Instalar dependências e executar

```
npm install
npm run dev
```

URL: `http://localhost:3000`

6.5. Edge Gateway - Configuração e Execução (Opcional)

Necessário apenas para testar integração com equipamentos industriais.

Pré-requisitos adicionais

- Broker MQTT (Eclipse Mosquitto ou cloud)
- Simuladores (opcional): OPC UA Simulator, Modbus Simulator

Configuração

Crie `AUTOLOAD_V4_EdgeGateway/AutoloadV4.EdgeGateway.Worker/appsettings.Development.json`:

```
{
  "EdgeGateway": {
    "Id": "gateway-local-001",
    "TenantId": "seu_tenant_id",
    "RpcPrefix": "autoload",
    "ReconnectInterval": 5000,
    "PoolingInterval": 1000
  },
  "MQTT": {
    "Server": "localhost",
    "Port": 1883,
    "User": "autoload",
    "Password": "sua_senha_mqtt",
    "Protocol": "mqtt",
    "UseTls": false,
    "WebSocketPath": null
  },
  "Telemetry": {
    "Interval": 2000,
    "ReconnectInterval": 5000,
    "MaxLeaseDuration": 30000
  }
}
```

Instalar MQTT broker local

Opção A - Docker:

```
docker run -d --name mosquitto -p 1883:1883 -p 9001:9001 eclipse-mosquitto
```

Opção B - Windows Installer: Baixe o Mosquitto e instale.

Executar

```
cd AUTOLOAD_V4_EdgeGateway/AutoloadV4.EdgeGateway.Worker
dotnet run
```

6.6. Kiosk - Configuração e Execução (Opcional)

Configuração

Edite `.env` na pasta `AUTOLOAD_V4_KIOSK`:

```
VITE_API_URL=http://localhost:5000
```

Autenticar e executar

```
cd AUTOLOAD_V4_KIOSK
npm run auth
npm install
npm run dev
```

URL: `http://localhost:5173`

6.7. Ordem de Inicialização Recomendada

```
Terminal 1 - Backend:
  cd AUTOLOAD_V4_Backend
  dotnet run --project AutoloadV4.Api/AutoloadV4.Api.csproj

Terminal 2 - Frontend:
  cd AUTOLOAD_V4_Frontend
  npm run dev

Terminal 3 (Opcional) - Edge Gateway:
  cd AUTOLOAD_V4_EdgeGateway/AutoloadV4.EdgeGateway.Worker
  dotnet run

Terminal 4 (Opcional) - Kiosk:
  cd AUTOLOAD_V4_KIOSK
  npm run dev
```

6.8. Checklist de Verificação

```
Banco de Dados:
  [ ] PostgreSQL ou SQL Server instalado e rodando
  [ ] Banco 'autoload' criado
  [ ] Migrations aplicadas (tabelas criadas)
  [ ] Connection string configurada corretamente

Backend API:
  [ ] Azure Artifacts Credential Provider instalado
  [ ] Dependencias NuGet restauradas
  [ ] Compilacao sem erros
  [ ] API rodando em http://localhost:5000
  [ ] Swagger acessivel em http://localhost:5000/swagger
```

```
Frontend:
  [ ] Autenticacao Azure DevOps (npm) funcionando
  [ ] Dependencias npm instaladas
  [ ] Variáveis de ambiente configuradas (.env.development)
  [ ] Frontend rodando em http://localhost:3000
  [ ] Pagina de login carrega corretamente

Integracao Frontend-Backend:
  [ ] Frontend consegue chamar API (sem CORS errors)
  [ ] Login funciona e gera token JWT
  [ ] Dados carregam corretamente nas telas

Edge Gateway (Opcional):
  [ ] MQTT broker rodando
  [ ] Edge Gateway conecta ao broker
  [ ] Telemetria sendo enviada

Kiosk (Opcional):
  [ ] Kiosk rodando em http://localhost:5173
  [ ] Conecta ao backend
```

7. Variáveis de Ambiente

7.1. Backend API - Variáveis de Template (`#

Database

Variável	Descrição	Exemplo DEV	Exemplo PRD
`DATABASE_PROVIDER`	Provider do banco (postgresql, sqlserver)	`sqlserver`	`postgresql`
`DATABASE_SERVER`	Endereço do servidor	`localhost\SQL2019LOCAL`	`prd-db.autoload.cloud`
`DATABASE_PORT`	Porta do banco	`1433`	`5432`
`DATABASE_USER`	Usuário do banco	`sa`	`autoload_prd`
`DATABASE_PASSWORD`	Senha do banco	`dev_password`	`*****`

MQTT

Variável	Descrição	Exemplo DEV	Exemplo PRD
`MQTT_SERVER`	Endereço do broker MQTT	`vbr01-dsv.autominddev.com.br`	`mqtt.autoload.cloud`
`MQTT_PORT`	Porta do broker	`1883`	`8883`
`MQTT_USER`	Usuário MQTT	`autoload`	`autoload_prd`
`MQTT_PASSWORD`	Senha MQTT	`*****`	`*****`

Variável	Descrição	Exemplo DEV	Exemplo PRD
`MQTT_PROTOCOL`	Protocolo (mqtt ou ws)	`mqtt`	`mqtt`
`MQTT_USE_TLS`	Usar TLS	`false`	`true`
`MQTT_WEB_SOCKET_PATH`	Path para WebSocket	`null`	`/ws`

Autenticação e Segurança

Variável	Descrição	Exemplo DEV	Exemplo PRD
`JWT_KEY`	Chave de assinatura JWT (min 64 chars)	`1234...` (dev)	`chave_segura_rotacionada`
`CORS_ORIGIN`	Origens permitidas (CORS)	`http://localhost:3000`	`https://app.autoload.cloud`

SignalR Hubs

Variável	Descrição	Exemplo DEV
`HUBS_MESSAGES_URL`	URL do hub de mensagens	`/hubs/messages`
`HUBS_TELEMETRY_URL`	URL do hub de telemetria	`/hubs/telemetry`

Workers

Variável	Descrição	Valores
`WORKER_JOBS_ENABLED`	Ativar worker de jobs agendados	`true` / `false`
`WORKER_MQTT_ENABLED`	Ativar worker MQTT	`true` / `false`
`WORKER_PROGRAM_ENABLED`	Ativar worker de programas	`true` / `false`

SMTP (Email)

Variável	Descrição	Exemplo DEV
`SMTP_SERVER`	Servidor SMTP	`smtp-relay.brevo.com`
`SMTP_PORT`	Porta SMTP	`9025`
`SMTP_OPTIONS`	Opções de segurança	`Auto`
`SMTP_SENDER_NAME`	Nome do remetente	`Autoload`
`SMTP_SENDER_EMAIL`	Email do remetente	`noreply@dev.autoload.cloud`
`SMTP_USER`	Usuário SMTP	`*****`
`SMTP_PASSWORD`	Senha SMTP	`*****`

Sentry (Monitoramento)

Variável	Descrição	Exemplo
`SENTRY_DSN`	Data Source Name do Sentry	`https://xxx@sentry.io/xxx`
`SENTRY_DEBUG`	Modo debug do Sentry	`false`
`SENTRY_PROFILING`	Habilitar profiling	`true` (PRD), `false` (DEV)

Storage

Variável	Descrição	Exemplo
`STORAGE_PROVIDER`	Tipo de storage (Local, OCI)	`OCI`
`STORAGE_LOCAL_PATH`	Caminho local (se Local)	`C:\Storage\Attachments`
`STORAGE_OCI_TENANCY_OCID`	OCID do tenancy OCI	`ocid1.tenancy.oc1...`
`STORAGE_OCI_USER_OCID`	OCID do usuário OCI	`ocid1.user.oc1...`
`STORAGE_OCI_FINGERPRINT`	Fingerprint da chave OCI	`e7:07:f0:a3:...`
`STORAGE_OCI_PRIVATE_KEY`	Chave privada OCI (PEM)	`-----BEGIN PRIVATE KEY-----...`
`STORAGE_OCI_REGION`	Região OCI	`sa-saopaulo-1`
`STORAGE_OCI_NAMESPACE`	Namespace OCI	`grbdicjhnqcp`
`STORAGE_OCI_BUCKET_NAME`	Nome do bucket OCI	`autoload-dev-bucket`

Logging

Variável	Descrição	Exemplo
`LOGS_MAX_FILE_SIZE_MB`	Tamanho máximo do arquivo de log	`200`
`LOGS_RETENTION_DAYS`	Dias de retenção dos logs	`7`
`LOGS_CUSTOM_PATH`	Caminho customizado para logs	`null` (usa padrão)
`SITE_ID`	Identificador do site/terminal	`DEV_TERMINAL`
`ENVIRONMENT`	Nome do ambiente	`DEV`, `HML`, `PRD`
`VERSION`	Versão da aplicação	`4.0.1`

Swagger

Variável	Descrição	Exemplo
`SWAGGER_ENDPOINT`	Endpoint do Swagger	`/` (DEV), vazio (PRD)

7.2. Frontend - Variáveis de Ambiente

Build Time (`.env` / `.env.development`)

Variável	Descrição	Exemplo
`NEXT_PUBLIC_BASE_URL`	URL base da API	`http://localhost:5000/api`
`NEXT_PUBLIC_WS_BASE_URL`	URL base dos hubs WebSocket	`http://localhost:5000/hubs`
`NEXT_PUBLIC_API_KEY`	API key (se necessario)	`dev-api-key`
`NEXT_PUBLIC_ENV`	Nome do ambiente	`development`
`NEXT_PUBLIC_AGENT_URL`	URL do agente IA	`http://localhost:8000`
`NEXT_PUBLIC_SENTRY_DSN`	DSN do Sentry	`https://xxx@sentry.io/xxx`
`NEXT_PUBLIC_SENTRY_ORG`	Organização Sentry	`automind-11`
`NEXT_PUBLIC_SENTRY_PROJECT`	Projeto Sentry	`autoloadv4-frontend`

Runtime (`public/env.json`)

Variável	Descrição
`BASE_URL`	URL base da API
`WS_BASE_URL`	URL base dos hubs WebSocket
`AGENT_URL`	URL do agente IA
`DEFAULT_TIMEZONE`	Timezone padrão (ex: `America/Sao_Paulo`)
`ENVIRONMENT`	Nome do ambiente
`SENTRY_DSN`	DSN do Sentry
`SENTRY_ORG`	Organização Sentry
`SENTRY_PROJECT`	Projeto Sentry

7.3. Edge Gateway - Variáveis de Template

Variável	Descrição	Exemplo
`GATEWAY_ID`	ID único do gateway	`gateway-terminal-001`
`TENANT_ID`	ID do tenant	`uuid-do-tenant`
`RPC_PREFIX`	Prefixo dos tópicos RPC	`autoload`
`GATEWAY_RECONNECT_INTERVAL`	Intervalo de reconexão (ms)	`5000`

Variável	Descrição	Exemplo
`GATEWAY_POOLING_INTERVAL`	Intervalo de polling (ms)	`1000`
`TELEMETRY_INTERVAL`	Intervalo de telemetria (ms)	`2000`
`TELEMETRY_RECONNECT_INTERVAL`	Reconexao da telemetria (ms)	`5000`
`TELEMETRY_MAX_LEASE_DURATION`	Duração máxima de lease (ms)	`30000`
`RPC_REQUEST`	Tópico de request RPC	`{tenant}/gateways/{id}/rpc/request`
`RPC_RESPONSE`	Tópico de response RPC	`{tenant}/gateways/{id}/rpc/response`
`MQTT_RECONNECT_INTERVAL`	Reconexao MQTT (ms)	`5000`

7.4. Kiosk - Variáveis de Ambiente

Build Time (`.env`)

Variável	Descrição	Exemplo
`VITE_API_URL`	URL da API	`https://dev.autoload.cloud/api`

Runtime (`public/env.json`)

Variável	Descrição
`BASE_URL`	URL base da API

---

8. Configurações de Rede e Sistema

8.1. Registros de Pacotes Privados

NuGet (Backend / Edge Gateway)

Configurado em `nuget.config`:

```
<packageSources>
  <add key="nuget.org" value="https://api.nuget.org/v3/index.json" />
  <add key="automindautomacao"
       value="https://pkgs.dev.azure.com/automindautomacao/_packaging/automindautomacao/nuget/v3/index.json" />
</packageSources>

<packageSourceMapping>
  <packageSource key="automindautomacao">
    <package pattern="Automind.*" />
  </packageSource>
  <packageSource key="nuget.org">
```

```
    <package pattern="*" />
  </packageSource>
</packageSourceMapping>
```

Feed Azure Artifacts: `f89bae47-037c-4526-ad13-86ddd4b5c199`

Pacotes privados:

- `Automind.AutoloadV4.Logging`
- `Automind.Autoload.CorrecaoVolume`

NPM (Frontend / Kiosk)

Configurado em `.npmrc`:

```
@automind:registry=https://pkgs.dev.azure.com/automindautomacao/_packaging/automind-react-components/npm/registry/
always-auth=true
```

Pacotes privados: `@automind/*` (componentes React compartilhados)

8.2. Autenticação em Serviços Externos

Serviço	Tipo de Auth	Configuração
Azure Artifacts (NuGet)	Azure DevOps Credential Provider	`install-artifacts-credprovider.ps1`
Azure Artifacts (NPM)	`vsts-npm-auth`	`npm run auth`
PostgreSQL	Usuário/Senha	`appsettings.json`
MQTT Broker	Usuário/Senha + TLS	`appsettings.json`
OCI Object Storage	API Key + Private Key	`appsettings.json`
Sentry	DSN Token	`appsettings.json` / `env.json`
SMTP (Brevo)	Usuário/Senha	`appsettings.json`

8.3. Configuração de CORS

O Backend deve permitir requisições do Frontend conforme o ambiente:

Ambiente	CORS Origins
DEV	`http://localhost:3000`
HML	`https://hml.autoload.cloud`
PRD	`https://app.autoload.cloud`

Configurado em `appsettings.json` > `Cors.Origins`.

8.4. Build Output por Módulo

Módulo	Build Command	Output	Formato
Backend	`dotnet publish -c Release`	Binarios .NET	ZIP (IIS deploy)
Frontend	`npm run build` + `postbuild`	`.next/standalone`	Node.js standalone
Edge Gateway	`dotnet publish -c Release`	Binarios .NET	Worker Service
Kiosk	`npm run build`	`dist/`	Arquivos estáticos (base: `/kiosk/`)

8.5. Informações de Versao

O pipeline gera automaticamente um arquivo `version.json` via template `template-build-version-metadata.yml`:

```
{
  "version": "4.2.0",
  "branch": "release/v4.2.0",
  "commit": "abc123def",
  "buildId": "20260225.1",
  "buildDate": "2026-02-25T10:30:00Z"
}
```

O Frontend carrega esse arquivo em runtime via `loadVersion()` em `src/lib/config.ts`.

---

9. Processo de Deploy por Módulo

9.1. Backend API

1. **Build**: Pipeline gera artefato `drop-backend` (ZIP)
2. **Substituição**: Azure DevOps Release substitui placeholders `#{VAR}#` nos `appsettings.json`
3. **Deploy**: Publicação em IIS (Windows) ou container Docker
4. **Migrations**: Executadas via script SQL ou `dotnet ef database update`
5. **Verificação**: Health check endpoint `/api/health`

9.2. Frontend Web

1. **Build**: Pipeline gera artefato `drop-frontend` (Next.js standalone)
2. **Substituição**: Placeholders em `public/env.json` substituídos no release
3. **Deploy**: Copiado para servidor e executado via `node server.js`
4. **Verificação**: Acesso à página de login

9.3. Edge Gateway

1. **Build**: Pipeline gera artefato com binários Linux
2. **Substituição**: Placeholders no `appsettings.json` substituídos
3. **Deploy**: Instalado como Windows Service ou Linux Daemon no terminal
4. **Configuração**: Gateway ID e Tenant ID únicos por terminal
5. **Verificação**: Heartbeat via MQTT

9.4. Kiosk

1. **Build**: Pipeline gera artefato `drop` (arquivos estáticos)
2. **Substituição**: Placeholders em `public/env.json` substituídos
3. **Deploy**: Servido como site estático (base path `/kiosk/`)
4. **Verificação**: Acesso a interface de check-in

---

10. Troubleshooting

10.1 Erro NU1301

Não é possível carregar índice de serviço NuGet

```
iex "& { $(irm https://aka.ms/install-artifacts-credprovider.ps1) }"
dotnet nuget locals all --clear
dotnet restore --interactive
```

10.2 npm error 403 Forbidden

User '***' lacks permission to complete this action. You need to have 'ReadPackages'.

Esse erro é comum quando está iniciando o projeto pela primeira vez. Para solucioná-lo é necessário:

- Solicitar que um superior lhe conceda permissão `ReadPackages` para a biblioteca `react-components`
- Após ter permissão, acesse: dev.azure.com/automindautomacao
- Clique em `npm` e `Get the tools` para instalar o `vsts-npm-auth`
- Realize a instalação do Node.js e npm pelo site Download Node.js and npm ou apenas execute os seguintes comandos:

```
# Execute:
npm install -g npm
npm install -g vsts-npm-auth --registry https://registry.npmjs.com --always-auth false

# Por fim, execute npm install novamente:
npm install
```

10.3 Erro de Autenticação NPM

```
npm run auth
rm -r node_modules, package-lock.json
npm install
```

10.4 CORS Error (Frontend não conecta ao Backend)

Verifique se `Cors.Origins` no `appsettings` inclui a URL do Frontend. Reinicie o Backend após alterar.

10.5 Migrations não aplicam

```
cd AUTOLOAD_V4_Backend
dotnet ef database update `
  --project AutoloadV4.Infrastructure `
  --startup-project AutoloadV4.Api `
  --context AutoloadContext
```

10.6 Porta em uso

```
netstat -ano | findstr :5000
Stop-Process -Id [PID] -Force
```

11. Conclusão

O processo de deploy do **Autoload V4** foi projetado para garantir rastreabilidade, consistência e segurança em todas as etapas — desde o desenvolvimento local até a entrega em produção. A adoção do **Git Flow** como estratégia de branching, aliada a **pipelines automatizadas de CI/CD** no Azure DevOps, assegura que cada alteração passe por validação, build e aprovação antes de chegar aos ambientes produtivos.

A separação clara entre os ambientes **DEV**, **HML** e **PRD**, com variáveis e configurações isoladas por ambiente, reduz o risco de erros em produção e permite que testes sejam realizados de forma controlada. A utilização de **templates com placeholders** (`#{VAR}#`) nas pipelines garante que a mesma build possa ser promovida entre ambientes com segurança, alterando apenas as configurações específicas de cada destino.

A documentação de **variáveis de ambiente**, **configurações de rede**, **setup local** e **troubleshooting** reunidas neste guia visa facilitar a integração de novos membros à equipe e servir como referência contínua para operações do dia a dia, contribuindo para a padronização e eficiência do ciclo de vida de entrega do sistema.

Documento atualizado em: 05 de março de 2026
Versão: 1.0
Autor: Milton Kiefer de Albuquerque Mello
Validado por: ---
Última atualização tecnológica: .NET 8.0 / Next.js 14 / React 18

AUTOMIND

Simple·Smart·Reliable

automind_ai

automind

bit.ly/sobre-automind